

Méthodes numériques en actuariat avec R

Vincent Goulet



UNIVERSITÉ
LAVAL

Méthodes numériques en actuariat avec R

Vincent Goulet

Professeur titulaire

École d'actuariat, Université Laval

Avec la collaboration de

Laurent Caron



Vincent Goulet, 2022

© 2022 par Vincent Goulet. « Méthodes numériques en actuariat avec R » est mis à disposition sous licence **Attribution-Partage dans les mêmes conditions 4.0 International** de Creative Commons. En vertu de cette licence, vous êtes autorisé à :

- **partager** — copier, distribuer et communiquer le matériel par tous moyens et sous tous formats;
- **adapter** — remixer, transformer et créer à partir du matériel pour toute utilisation, y compris commerciale.

L'Offrant ne peut retirer les autorisations concédées par la licence tant que vous appliquez les termes de cette licence.

Selon les conditions suivantes :



Attribution — Vous devez créditer l'œuvre, intégrer un lien vers la licence et indiquer si des modifications ont été effectuées à l'œuvre. Vous devez indiquer ces informations par tous les moyens raisonnables, sans toutefois suggérer que l'Offrant vous soutient ou soutient la façon dont vous avez utilisé son œuvre.



Partage dans les mêmes conditions — Dans le cas où vous effectuez un remix, que vous transformez, ou créez à partir du matériel composant l'œuvre originale, vous devez diffuser l'œuvre modifiée dans les mêmes conditions, c'est-à-dire avec la même licence avec laquelle l'œuvre originale a été diffusée.

Code source

 [Voir sur GitLab](#)

Couverture

Deux papillons flambeaux (*Dryas iulia*) en train de boire les larmes de tortues (*Poecynemis expansa*) en Équateur.

Crédit photo : © amalavida.tv, **CC BY-SA 2.0**, via [Wikimedia Commons](#).

Introduction

Cet ouvrage propose à des étudiantes et étudiants en sciences actuarielles d'acquérir rapidement un ensemble de compétences en calcul numérique qui s'avèrent essentielles dans plusieurs facettes de leur cheminement académique et professionnel. En effet, tant la simulation stochastique que les méthodes numériques de résolution d'équations et de calcul d'intégrales deviennent des outils essentiels pour les scientifiques qui cherchent à résoudre des problèmes sans cesse plus complexes. Quant à l'algèbre linéaire, elle demeure un outil de modélisation puissant utilisé dans toutes les sphères des mathématiques et de la statistique.

L'ouvrage est divisé en trois parties dont le sommaire apparaît ci-dessous. J'ai tenté au mieux de tisser un fil conducteur tout au long de l'ouvrage : l'utilisation de l'ordinateur comme outil de calcul.

Simulation stochastique

La simulation stochastique est une technique utilisée dans un grand nombre de domaines. Pensons seulement aux simulations boursières qui font l'objet d'un concours annuel, aux voitures qui sont d'abord conçues sur ordinateur et soumises à des tests de collisions virtuels, ou encore aux prévisions météo qui ne sont en fait que les résultats de simulations de systèmes climatiques d'une grande complexité.

Toute simulation stochastique repose sur une source de nombres aléatoires de qualité. Comment en générer un grand nombre rapidement et, surtout, comment s'assurer que les nombres produits sont bien aléatoires ? C'est un sujet d'une grande importance, mais aussi fort complexe. Nous nous contenterons donc de l'effleurer en étudiant les techniques de base dans le [chapitre 1](#).

En sciences actuarielles, nous avons habituellement besoin de nombres aléatoires provenant d'une loi de probabilité non uniforme. Le [chapitre 2](#) pré-

sente quelques algorithmes pour transformer des nombres aléatoires uniformes en nombres non uniformes. Évidemment, des outils informatiques sont aujourd'hui disponibles pour générer facilement et rapidement des nombres aléatoires de diverses lois de probabilité. Nous passons en revue les fonctionnalités de R et d'Excel à cet égard.

La première partie de l'ouvrage se termine au [chapitre 3](#) par une application à première vue inusitée de la simulation, soit le calcul d'intégrales définies par la méthode dite « Monte Carlo ».

Analyse numérique

Les ordinateurs ne savent pas compter. Ou, en fait, très peu. Ils ne savent traiter que des 0 et des 1 et sont incapables de représenter tous les nombres réels — chose qu'un humain peut faire, du moins conceptuellement. Cela signifie qu'à peu près tout calcul effectué dans un ordinateur comporte une part d'erreur d'arrondi et de troncature. Comme on ne souhaite généralement pas que cette erreur devienne trop grande, il importe de connaître ses sources afin de la diminuer le plus possible. C'est, entre autres choses, l'objet du [chapitre 4](#).

Les procédures numériques pour résoudre des équations à une variable, optimiser une fonction ou calculer une intégrale définie sont aujourd'hui aisément accessibles dans une foule de logiciels à connotation mathématique ou même dans une simple calculatrice. Or, comment ces calculs sont-ils effectués, quels sont les algorithmes à l'œuvre en arrière-scène ? Le [chapitre 5](#) se penche sur les méthodes de base de résolution d'équations à une variable et le [chapitre 6](#) sur celles d'intégration numérique. Le [chapitre 5](#) présente également les principales fonctions d'optimisation disponibles dans Excel et dans R.

Algèbre linéaire

L'algèbre linéaire a le chic de surgir là où on ne l'attend pas nécessairement. Peut-être croyez-vous qu'en étudiant en sciences actuarielles, les notions de vecteurs et de matrices vous demeureront étrangères. Détrompez-vous.

Les matrices et leur algèbre permettent de représenter, de traiter et de résoudre efficacement de grands systèmes d'équations linéaires ou d'équations différentielles. La notion d'erreur quadratique moyenne s'apparente à la projection d'un vecteur dans un espace vectoriel. Les notions d'indépen-

dance stochastique et d'orthogonalité de vecteurs sont liées. On décrit le comportement d'une chaîne de Markov à l'aide d'une matrice de transition. Une classe de lois de probabilités requiert de calculer l'exponentielle d'une matrice. Ce sont là quelques exemples où l'algèbre linéaire joue un rôle en théorie des probabilités, en inférence statistique, en finance ou en théorie du risque.

Le [chapitre 7](#) revient sur les principales notions d'algèbre linéaire normalement étudiées au collège et qu'il est important de maîtriser dans ses études de premier cycle universitaire. Nous nous concentrons sur l'établissement de liens entre des éléments qui peuvent au premier chef sembler disjointes. Le [chapitre 8](#) construit sur le précédent pour introduire les concepts de valeurs propres, de vecteurs propres et de diagonalisation d'une matrice. Pour terminer sur des considérations numériques ayant jusque-là traversé l'ouvrage, le [chapitre 9](#) expose et compare très succinctement différentes stratégies utilisées pour résoudre des systèmes d'équations linéaires à l'aide d'un ordinateur.

Cette troisième partie de l'ouvrage diffère passablement des deux autres, autant dans sa nature que dans le format des activités d'apprentissage. La matière y est beaucoup plus mathématique et abstraite, il y a de nombreux exercices et les considérations informatiques sont réduites au minimum. Les chapitres [7](#) et [8](#) ne comportent que de courtes sections faisant la démonstration de fonctions R dédiées à l'algèbre linéaire.

Utilisation de l'ouvrage

La plupart des chapitres de cet ouvrage proposent un problème à résoudre au fil du texte. L'énoncé du problème, les indices en cours de chapitre et la solution complète se présentent dans des sections marquées des symboles ,  et .

L'étude de l'ouvrage implique des allers-retours entre le texte et le code R à la fin des chapitres. Ce code informatique et les commentaires qui l'accompagnent visent à enrichir vos apprentissages. Assurez-vous donc de lire attentivement tant les commentaires que le code, d'exécuter le code pas-à-pas et de bien comprendre ses effets.

Le code informatique est distribué avec l'ouvrage sous forme de fichiers de script. De plus, à chaque fichier `.R` correspond un fichier `.Rout` contenant les résultats de son évaluation non interactive.

Vous trouverez également en annexe un bref exposé sur la planification d'une simulation en R, des rappels sur la transformation de variables aléatoires, de même que les solutions complètes des exercices.

Fonctionnalités interactives

En consultation électronique, ce document se trouve enrichi de plusieurs fonctionnalités interactives.

- ▶ Intraliens du texte vers une ligne précise d'une section de code informatique et, en sens inverse, du numéro de la ligne vers le point de la référence dans le texte. Ces intraliens sont marqués par la couleur ■■■.
- ▶ Intraliens entre le numéro d'un exercice et sa solution, et vice versa. Ces intraliens sont aussi marqués par la couleur ■■■.
- ▶ Intraliens entre les citations dans le texte et leur entrée dans la bibliographie. Ces intraliens sont marqués par la couleur ■■■.
- ▶ Hyperliens vers des ressources externes marqués par le symbole ↗ et la couleur ■■■.
- ▶ Table des matières, liste des tableaux, liste des figures et liste des vidéos permettant d'accéder rapidement à des ressources du document.

Blocs signalétiques

Le document est parsemé de divers types de blocs signalétiques inspirés de [AsciiDoc](#) ↗ qui visent à attirer votre attention sur une notion. Vous pourrez rencontrer l'un ou l'autre des blocs suivants.



Astuce! Ces blocs contiennent un truc, une astuce, ou tout autre type d'information utile.



Attention! Vous risquez de vous brûler — métaphoriquement s'entend — si vous ne suivez pas les recommandations de ces blocs.



Important! Ces blocs contiennent les remarques les plus importantes. Veuillez à en tenir compte.



Ces blocs contiennent des remarques additionnelles sur la matière ou des informations amusantes, mais non essentielles.



Ces blocs contiennent des liens vers des vidéos dans ma [chaîne YouTube](#) ↗ dédiée à ce document de référence. Les vidéos sont répertoriées dans la liste des vidéos.



Ces blocs vous invitent à interrompre la lecture du texte pour passer à l'étude du code R des sections d'exemples.



Remarques spécifiques à macOS.

Document libre

Tout comme R et l'ensemble des outils présentés dans ce document, le projet « Méthodes numériques en actuariat avec R » s'inscrit dans le mouvement de l'**informatique libre** [↗](#). Vous pouvez accéder à l'ensemble du code source en format $\text{\LaTeX}$ en suivant le lien dans la page de copyright. Vous trouverez dans le fichier `README.md` toutes les informations utiles pour composer le document.

Votre contribution à l'amélioration du document est également la bienvenue; consultez le fichier `CONTRIBUTING.md` fourni avec ce document et voyez votre nom ajouté au fichier `COLLABORATEURS`.

Remerciements

Je tiens à souligner la précieuse collaboration de MM. Mathieu Boudreault, Sébastien Auclair et Louis-Philippe Pouliot lors de la rédaction des exercices et des solutions. Laurent Caron est le principal auteur des problèmes résolus au fil des chapitres.

Table des matières

Introduction **vii**

Table des matières **xiii**

Liste des tableaux **xix**

Liste des figures **xxi**

Liste des vidéos **xxv**

I Simulation stochastique 1

1 Génération de nombres aléatoires uniformes 3

- 1.1 Énoncé du problème 3
- 1.2 Pourquoi faire de la simulation ? 4
- 1.3 Indice pour le problème 5
- 1.4 Générateurs de nombres aléatoires 6
- 1.5 Congruence et modulo 7
- 1.6 Générateurs congruentiels linéaires 8
- 1.7 Indice pour le problème 10
- 1.8 Générateurs utilisés dans Excel et R 10
- 1.9 Indice pour le problème 12
- 1.10 Solution du problème 12
- 1.11 Code informatique 13
- 1.12 Exercices 15

2 Simulation de nombres aléatoires non uniformes 17

- 2.1 Énoncé du problème 18
- 2.2 Méthode de l'inverse 18

2.3	Indice pour le problème	24
2.4	Méthode acceptation-rejet	24
2.5	Indice pour le problème	30
2.6	Fonctions de simulation dans Excel et R	30
2.7	Modèles actuariels	34
2.8	Solution du problème	40
2.9	Code informatique	42
2.10	Exercices	50
3	Intégration Monte Carlo	57
3.1	Énoncé du problème	57
3.2	Contexte	58
3.3	Méthode Monte Carlo	58
3.4	Indice pour le problème	62
3.5	Intégrales multiples	63
3.6	Solution du problème	64
3.7	Code informatique	65
3.8	Exercices	68
II	Analyse numérique	69
4	Arithmétique des ordinateurs	71
4.1	Énoncé du problème	71
4.2	Les ordinateurs ne savent pas compter	71
4.3	Conversion de base	73
4.4	Unités de mesure	81
4.5	Représentation en virgule flottante	82
4.6	Éléments d'arithmétique en virgule flottante	88
4.7	Indice pour le problème	93
4.8	Codage de caractères	94
4.9	Solution du problème	95
4.10	Code informatique	96
4.11	Exercices	98
5	Résolution d'équations à une variable	103
5.1	Énoncé du problème	103
5.2	Mise en contexte	104
5.3	Indice pour le problème	105
5.4	Méthode de bisection	106

5.5	Indice pour le problème	110
5.6	Méthode du point fixe	110
5.7	Indice pour le problème	114
5.8	Méthode de Newton-Raphson	116
5.9	Indice pour le problème	124
5.10	Fonctions d'optimisation de Excel et de R	124
5.11	Indice pour le problème	127
5.12	Astuce Ripley	127
5.13	Outils additionnels	128
5.14	Solution du problème	128
5.15	Code informatique	129
5.16	Exercices	139
6	Intégration numérique	143
6.1	Énoncé du problème	143
6.2	Polynômes d'interpolation de Lagrange	144
6.3	Principes généraux d'intégration numérique	146
6.4	Méthode du point milieu	148
6.5	Méthode du trapèze	148
6.6	Méthode de Simpson	149
6.7	Méthode de Simpson 3/8	149
6.8	Solution du problème	151
6.9	Code informatique	153
6.10	Exercices	154
III	Algèbre linéaire	155
7	Notions fondamentales d'algèbre linéaire	157
7.1	Matrices	158
7.2	Arithmétique matricielle	160
7.3	Lien avec les systèmes d'équations linéaires	164
7.4	Lien avec les espaces vectoriels	171
7.5	Inverse d'une matrice	175
7.6	Déterminant	179
7.7	Rang	182
7.8	Unification des résultats	182
7.9	Code informatique	183
7.10	Exercices	185

8	Valeurs propres, vecteurs propres et diagonalisation	193
8.1	Énoncé du problème	194
8.2	Définitions	195
8.3	Indice pour le problème	197
8.4	Calcul des valeurs propres	198
8.5	Calcul des vecteurs propres	202
8.6	Diagonalisation	205
8.7	Indice pour le problème	211
8.8	Formes quadratiques	211
8.9	Solution du problème	214
8.10	Code informatique	215
8.11	Exercices	217
9	Méthodes de résolution de systèmes d'équations linéaires	221
9.1	Comparaison du nombre d'opérations	221
9.2	Décomposition LU	222
9.3	Exercices	226
A	Planification d'une simulation avec R	229
A.1	Contexte	229
A.2	Première approche : avec une boucle	230
A.3	Seconde approche : avec <code>sapply</code>	230
A.4	Variante de la seconde approche	234
A.5	Gestion des fichiers	234
A.6	Exécution en lot	235
A.7	Conclusion	236
B	Transformations de variables aléatoires	237
B.1	Technique de la fonction de répartition	237
B.2	Technique du changement de variable univariée	241
B.3	Technique du changement de variable multivariée	243
B.4	Technique de la fonction génératrice des moments	247
C	Solutions des exercices	249
	Chapitre 1	249
	Chapitre 2	251
	Chapitre 3	267
	Chapitre 4	269
	Chapitre 5	276
	Chapitre 6	293

Chapitre 7	296
Chapitre 8	313
Chapitre 9	326
Bibliographie	329

Liste des tableaux

- 2.1 Liste des fonctions relatives à des lois de probabilité depuis Excel 2010. 32
- 2.2 Lois de probabilité pour lesquelles il existe des fonctions dans le système R de base 33
- 4.1 Conversion d'un nombre décimal en binaire 77
- 4.2 Conversion d'un nombre binaire en décimal 79
- 4.3 Conversion d'un nombre décimal dans une base générale 80
- 4.4 Valeurs spéciales dans la norme IEEE 754 84
- 4.5 Représentation en virgule flottante simplifiée 89
- 4.6 Coût relatif de quelques opérations en virgule flottante 93
- 9.1 Nombre approximatif de multiplications et divisions pour résoudre le système d'équations à n équations et n inconnues $\mathbf{Ax} = \mathbf{b}$ 222
- C.1 Séquences des nombres en notation en complément à deux sur 8 bits 273
- C.2 Séquences des nombres en notation en complément à deux sur 16 bits 273
- C.3 Valeurs successives de la méthode de bisection pour l'exercice 5.3 282

Liste des figures

1.1	Enveloppe convexe d'un ensemble de points	4
2.1	Représentation graphique de la méthode de l'inverse	19
2.2	Illustration de la méthode de l'inverse pour une distribution discrète	22
2.3	Fonctions de densité de probabilité et de répartition d'une variable aléatoire mixte	23
2.4	Illustration de la méthode acceptation-rejet	26
2.5	Interprétation alternative de la méthode acceptation-rejet	27
2.6	Fonction de densité de probabilité d'une loi Bêta(3, 2) enveloppée d'un triangle	29
2.7	Simulation de points sur un disque par acceptation-rejet	31
2.8	Fonctions de densité de probabilité de deux log-normales et de leur mélange discret	36
2.9	Code d'une fonction pour simuler par la méthode d'acceptation-rejet des nombres uniformément sur un disque de rayon 1 centré à l'origine	41
2.10	Code d'une fonction pour simuler des nombres uniformément sur un disque de rayon 1 centré à l'origine en passant par les coordonnées polaires	42
2.11	Fonction de simulation d'une loi Bêta(α, β)	54
3.1	Approximation de l'aire sous une fonction par les deux sommes de Riemann R_n^- (gauche) et R_n^+ (droite)	59
3.2	Fonction $h(x)$ de l'exemple 3.1 et points choisis aléatoirement où la fonction est évaluée pour l'intégration Monte Carlo	62
3.3	Fonction $h(x)$ de l'exemple 3.2 et points choisis aléatoirement où la fonction est évaluée pour l'intégration Monte Carlo	65

- 4.1 Représentation schématique d'un nombre en double précision dans la norme IEEE 754. 83
- 5.1 Illustration de la méthode de bisection 107
- 5.2 Deux exemples de fonctions pour lesquelles on pourrait avoir $|f(x_n)| < \varepsilon$ même si $x_n \gg x^*$ ou $x_n \ll x^*$ 108
- 5.3 Exemples de graphiques permettant de faire la démonstration du théorème du point fixe 111
- 5.4 Illustrations de la méthode du point fixe pour une fonction décroissante et pour une fonction croissante 112
- 5.5 Graphiques des cinq fonctions de l'exemple 5.6 115
- 5.6 Illustration de la méthode de Newton-Raphson 117
- 5.7 Fonction $g(i) = i - f(i)/f'(i)$ où $f(i) = (1 - (1 + i)^{-10})/i - 8,2218$ pour $0,035 \leq i \leq 0,040$ et la droite $y = i$ 120
- 5.8 Fonction $g(x) = x - f(x)/f'(x)$, où $f(x) = x^3 + 4x^2 - 10$ 121
- 5.9 Fonction $f(x) = (4x - 7)/(x - 2)$ (ligne épaisse), asymptote et tangente en $x = 1,5$ 122
- 5.10 Fonction $g(x) = 4x^2 - 14x + 14$ 123
- 5.11 Illustration de la méthode de la sécante 125
- 6.1 Aire à calculer avec l'intégrale I 144
- 6.2 Procédures d'approximation de quatre méthodes d'intégration numérique 147
- 6.3 Approximation de $f(x)$ sur un intervalle 150
- 6.4 Comparaison des approximations de l'aire correspondant à l'intégrale I selon trois méthodes d'intégration numérique avec $n = 2$ 151
- 7.1 Solutions possibles de systèmes d'équations linéaires à deux et trois variables 166
- 8.1 Représentation matricielle d'un chiffre écrit à la main 195
- 8.2 Représentation matricielle peu lisible d'un chiffre écrit à la main 196
- 8.3 Illustrations de trois transformations d'un vecteur $\mathbf{x}$ correspondant à un produit matriciel $\mathbf{Ax}$. 196
- 8.4 Illustration de la relation entre une matrice, une valeur propre de celle-ci et le vecteur propre associé 197
- 8.5 Représentation des deux premières composantes principales des données *Optical Recognition of Handwritten Digits* 216
- A.1 Code pour la simulation utilisant une boucle for 231

- A.2 Définition de la fonction `simul1` 232
- A.3 Définitions des fonction `fun1` et `simul2` 233
- A.4 Définitions des fonction `fun2` et `simul3` 235

- C.1 Code de la fonction `rand` 249
- C.2 Paires de valeurs du générateur congruentiel multiplicatif avec $m = 2^{31} - 1$ et $a = 17$ 251
- C.3 Paires de valeurs du générateur congruentiel multiplicatif avec $m = 2^{31} - 1$ et $a = 85$ 252
- C.4 Démonstration graphique du fonctionnement de la transformation de Box-Muller 254
- C.5 Fonction de simulation d'une distribution $\text{Gamma}(\alpha, 1)$ avec $\alpha > 1$ 256
- C.6 Démonstration graphique du fonctionnement de la transformation de l'exercice 2.8 259
- C.7 Démonstration graphique du fonctionnement de la transformation de l'exercice 2.9 261
- C.8 Code R de la fonction `rbeta.ar2` 263
- C.9 Fonctions de l'exercice 5.2 277
- C.10 Fonction $g(x) = 2^{-x}$ dans $[0, 1]$ 284
- C.11 Fonctions de l'exercice 5.8 286
- C.12 Point de $y = x^2$ le plus près du point $(1, 0)$ 288
- C.13 Fonction R de calcul du taux de rendement interne d'une série de flux financiers 289
- C.14 Fonction R d'estimation du paramètre d'échelle d'une loi gamma par le maximum de vraisemblance 290
- C.15 Trois fonctions pour résoudre $a_{\overline{10}|i}^{(12)} = 8$ par la méthode du point fixe 292

Liste des vidéos

Le numéro indiqué à gauche est celui de la section dans laquelle se trouve le bloc signalétique.

- 2.4 [Méthode d'acceptation-rejet](#)  26
- 2.7 [Simulation de mélanges discrets](#)  37
- 4.2 [Arithmétique des ordinateurs](#)  73
- 5.12 [Astuce Ripley](#)  128
- 8.0 [Google et les vecteurs propres](#)  193

Première partie

Simulation stochastique

```
int getRandomNumber()  
{  
    return 4; // chosen by fair dice roll.  
              // guaranteed to be random.  
}
```

Tiré de [XKCD.com](https://xkcd.com)

1 Génération de nombres aléatoires uniformes

Objectifs du chapitre

- ▶ Identifier des applications actuarielles de la simulation stochastique.
- ▶ Énoncer les caractéristiques d'un bon générateur de nombres pseudo-aléatoires.
- ▶ Utiliser l'opérateur mathématique modulo.
- ▶ Générer des nombres pseudo-aléatoires à l'aide d'un générateur congruentiel linéaire.
- ▶ Établir les caractéristiques d'un générateur congruentiel linéaire, notamment sa période.
- ▶ Utiliser les générateurs de nombres aléatoires de Excel et R.

Ce chapitre traite de la simulation de nombres (pseudo) aléatoires distribués uniformément sur l'intervalle $(0, 1)$. La transformation de ces nombres uniformes en nombres provenant d'autres distributions statistiques fera l'objet du prochain chapitre.



1.1 Énoncé du problème

L'enveloppe convexe (*convex hull*) d'un ensemble de points dans le plan est l'ensemble des points qui, lorsque reliés entre eux, forment un polygone convexe contenant tous les autres points. La [figure 1.1](#) en présente un exemple.

Dans le problème des quatre points de [Sylvester \(1865\)](#), on cherche la probabilité que l'enveloppe convexe de quatre points placés dans une région R forme un triangle (voir l'article de [Weisstein](#) dans MathWorld pour plus de détails).

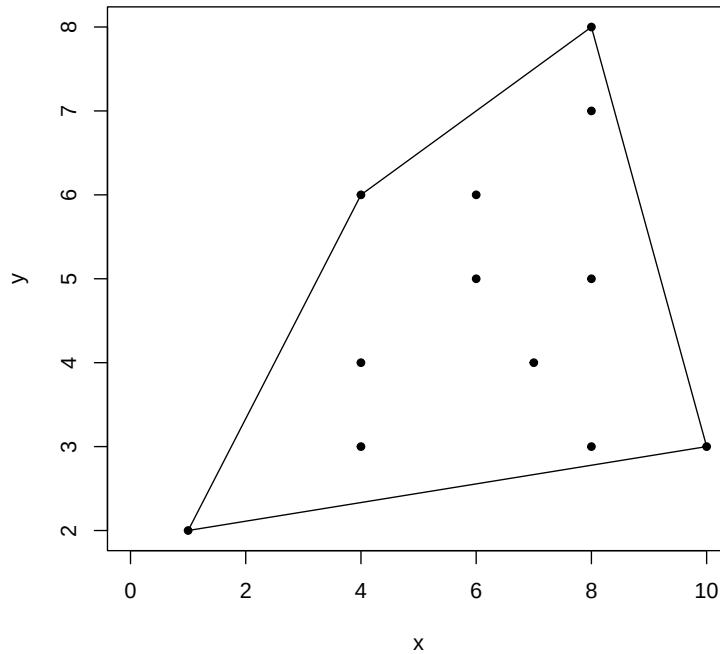


FIG. 1.1 – Enveloppe convexe d'un ensemble de points

Nous souhaitons vérifier que lorsque les quatre points sont distribués uniformément sur un carré, la probabilité que leur enveloppe convexe forme un triangle est $\frac{11}{36} \approx 0,30556$.

1.2 Pourquoi faire de la simulation ?

La simulation stochastique est une technique de plus en plus utilisée en actuariat comme dans la plupart des sciences appliquées, en génie, en finance, etc. Les modèles mathématiques et la simulation stochastique comportent plusieurs avantages par rapport à l'expérimentation directe, dont, entre autres :

- la simulation est non destructrice et peu coûteuse ;
- le système considéré n'a pas besoin d'exister ;

- ▶ la simulation est facile à répéter ;
- ▶ l'évolution dans la simulation peut être plus rapide que dans la réalité ;
- ▶ la simulation permet de considérer des modèles très complexes impossibles à traiter analytiquement.

En revanche, au rayon des inconvénients, on note :

- ▶ le coût (en temps et en argent) de modélisation et de programmation s'avère parfois important ;
- ▶ le temps d'exécution peut devenir excessif ;
- ▶ la simulation ne fournit que des estimations ;
- ▶ l'analyse statistique des résultats peut ne pas toujours être simple.

À la base, toute étude de simulation requiert une source de nombres aléatoires. Or, ces nombres aléatoires ne sont pas toujours faciles à obtenir — surtout en grande quantité — et la qualité de la source est primordiale pour que l'étude soit fiable. En effet, un générateur qui ne fournirait pas des nombres suffisamment aléatoires, ou alors qui se répètent trop rapidement, peut corrompre les résultats d'une étude jusqu'à rendre ses conclusions invalides.

Les nombres aléatoires sont également beaucoup utilisés en cryptographie. Ici encore, un générateur de mauvaise qualité peut avoir des conséquences fâcheuses. Par exemple, si la période du générateur est trop courte, il devient relativement facile de percer la sécurité d'un système en découvrant le mot de passe par une attaque en force.



1.3 Indice pour le problème

Nous pourrions effectuer la vérification demandée à l'aide de la simulation. L'idée consiste à générer les coordonnées de quatre points dans le plan et de vérifier si trois de ceux-ci forment l'enveloppe convexe — autrement dit, l'enveloppe est un triangle. En répétant cette procédure un grand nombre de fois, nous pourrions calculer la *proportion* d'enveloppes triangulaires et ainsi obtenir une estimation de la *probabilité* que l'enveloppe convexe de quatre points dans le plan forme un triangle.

Pour nous aider, la fonction `chull` de R calcule l'enveloppe convexe d'un ensemble de points. Plus précisément, la fonction retourne les indices des points faisant partie de l'enveloppe. Par exemple, les coordonnées cartésiennes des points de la [figure 1.1](#) sont :

```
> x <- c(8, 1, 7, 6, 4, 4, 8, 8, 4, 6, 8, 10)
> y <- c(8, 2, 4, 5, 3, 4, 5, 3, 6, 6, 7, 3)
```

Ceux formant l'enveloppe se trouvent aux positions suivantes des vecteurs de coordonnées :

```
> chull(x, y)
[1] 12 2 9 1
```

Comme vous pouvez facilement le confirmer visuellement, les points formant l'enveloppe convexe de l'ensemble de points de la [figure 1.1](#) sont donc : (10, 3), (1, 2), (4, 6) et (8, 8).

1.4 Générateurs de nombres aléatoires

Nous souhaitons obtenir des nombres issus d'une distribution uniforme sur un intervalle quelconque, en général (0, 1). Comment procéder ?

1. Nous pouvons utiliser les résultats de processus physiques aléatoires en apparence, par exemple :
 - ▶ le lancer d'une pièce de monnaie ou d'un dé ;
 - ▶ des nombres pris au hasard dans des tableaux de rapports ou dans un annuaire ;
 - ▶ la roulette ;
 - ▶ le bruit électronique (tableaux RAND) ;
 - ▶ les intervalles de temps dans un processus de décroissance radioactive sont considérés parfaitement aléatoires ; le site [HotBits](#)  fournit des nombres issus d'un tel processus.

L'utilisation de listes de nombres aléatoires ou de sources physiques est toutefois peu pratique avec un ordinateur, surtout lorsque l'on a besoin de milliers ou de millions de nombres aléatoires.

2. Une ancienne technique est celle des carrés centraux de von Neumann : on prend un nombre à quatre chiffres, on l'élève au carré, puis on extrait les quatre chiffres du milieu, et ainsi de suite. Par exemple :

$$8653^2 = \underline{74874409}$$

$$8744^2 = \underline{76457536}$$

$$4575^2 = \underline{20930625}$$

3. Nous pouvons construire des générateurs basés sur la suite de Fibonacci, des générateurs chaotiques, etc.

En fait, les générateurs couramment utilisés aujourd'hui dans les ordinateurs sont des évolutions des générateurs dits *congruentiels*. Ils sont particulièrement utiles parce qu'ils sont aisément *reproductibles*. De plus, nous pouvons généralement en connaître les propriétés — notamment la période — par une analyse mathématique poussée. Knuth (1997, section 3.1) fournit un exemple éloquent de l'importance de pouvoir démontrer mathématiquement les propriétés d'un générateur de nombres aléatoires. Cette référence de quelques pages seulement est fournie dans le site du cours; la lire avant d'aller plus loin.

C'est fait ? Bien. Intéressant, n'est-ce pas ?

Dans la suite, nous nous concentrerons sur les générateurs de nombres pseudo-aléatoires. En général, nous exigerons d'un générateur de ce type qu'il :

1. produise des nombres distribués approximativement uniformément ;
2. produise des nombres approximativement indépendants dans un bon nombre de dimensions ;
3. possède une période suffisamment longue (au moins 2^{60}) ;
4. soit facilement reproductible à partir d'un point de départ donné, mais qu'il soit autrement impossible à prédire.

1.5 Congruence et modulo

Les générateurs congruentiels utilisent l'arithmétique modulo. Une propriété de base de cette arithmétique est l'équivalence, ou congruence, modulo m .

Définition 1.1. Deux nombres a et b sont dits *équivalents*, ou *congruents*, modulo m si la différence entre a et b est un entier divisible par m . Mathématiquement,

$$a \equiv b \pmod{m} \iff \frac{a-b}{m} = k, \quad k \in \mathbb{Z}.$$

En d'autres mots, deux nombres sont équivalents modulo m si la distance entre eux est un multiple de m . La notion d'équivalence partitionne donc l'ensemble des nombres (ici, les réels).

Exemple 1.1. On a

1. $5 \equiv 14 \bmod 3$ car $\frac{14-5}{3} = 3$; 14 et 5 sont distants de 9, un multiple de 3;
2. $-1 \equiv 5 \bmod 3$ car $\frac{5+1}{3} = 2$; -1 et 5 sont distants de 6, un multiple de 3;
3. $0,33 \equiv 1,33 \bmod 1$; on notera que le calcul en modulo 1 équivaut à retourner la partie fractionnaire d'un nombre;
4. la minute dans l'heure est donnée en modulo 60 : 00h15, 1h15, 2h15, ...sont des heures équivalentes modulo 60.

□

De la définition de congruence découle celle de *réduction modulo* ou *résidu modulo* : si $a \equiv b \bmod m$ et $0 \leq a < m$, alors a est le résidu de la division de b par m , ou le résidu de b modulo m , c'est-à-dire

$$a = b \bmod m \quad \Leftrightarrow \quad a = b - \left\lfloor \frac{b}{m} \right\rfloor m,$$

où $\lfloor x \rfloor$ est le plus grand entier inférieur ou égal à x .



La plupart des langages de programmation et logiciels à connotation mathématique comportent un opérateur ou une fonction modulo. Dans R il s'agit de l'opérateur %%; dans Excel, de la fonction MOD().

1.6 Générateurs congruentiels linéaires

Dans un générateur congruentiel linéaire, tout nombre dans la suite générée détermine le nombre suivant par la formule

$$x_i = (ax_{i-1} + c) \bmod m,$$

où $0 \leq x_i < m$ et

- a est appelé le *multiplicateur*;
- c est appelé l'*incrément*;
- m est appelé le *module*;
- x_0 (un nombre quelconque) est l'*amorce* (« *seed* »).

Un générateur est dit *multiplicatif* lorsque l'incrément est $c = 0$, et *mixte* dans le cas contraire.

Pour obtenir des nombres uniformes sur $[0, 1]$ ou $(0, 1)$, il suffit de définir

$$u_i = \frac{x_i}{m}.$$



La méthode de génération des nombres est entièrement déterministe, c'est pourquoi il convient mieux de parler de nombres *pseudo*-aléatoires.

Un générateur congruentiel est forcément périodique puisqu'il ne peut prendre, au mieux, que les valeurs

- ▶ $0, 1, 2, \dots, m - 1$ pour un générateur mixte;
- ▶ $1, 2, \dots, m - 1$ pour un générateur multiplicatif.

C'est pourquoi nous cherchons donc à avoir la période la plus longue possible, tout en obtenant des suites en apparence aléatoires.

Pour les générateurs multiplicatifs ($c = 0$), la période maximale $m - 1$ est atteinte si :

- ▶ m est un nombre premier (on en choisira un grand);
- ▶ a est une *racine primitive* de m , c'est-à-dire que le plus petit entier k satisfaisant

$$1 = a^k \bmod m$$

est $k = m - 1$.

Des valeurs populaires sont $m = 2^{31} - 1$ (nombre premier de Mersenne) et $a = 7^5$.

Exemple 1.2. Soit un générateur congruentiel multiplicatif avec $a = 7$ et $m = 31$. Les quatre premiers nombres pseudo-aléatoires avec l'amorce $x_0 = 19$ sont :

$$\begin{aligned} (7 \times 19) \bmod 31 &= 133 \bmod 31 = 9 \rightarrow x_1 \\ (7 \times 9) \bmod 31 &= 63 \bmod 31 = 1 \rightarrow x_2 \\ (7 \times 1) \bmod 31 &= 7 \bmod 31 = 7 \rightarrow x_3 \\ (7 \times 7) \bmod 31 &= 49 \bmod 31 = 18 \rightarrow x_4. \end{aligned}$$

□



Étudiez les lignes 13–106 du fichier de script `generation.R` reproduit à la [section 1.11](#). Le code illustre l'effet des différents paramètres d'un générateur congruentiel sur la qualité des nombres pseudo-aléatoires produits. Il démontre également qu'un générateur apparemment de qualité en une dimension peut rapidement se révéler médiocre dans les dimensions supérieures. L'[exercice 1.3](#) reprend ces idées en trois dimensions avec un générateur longtemps considéré standard.



1.7 Indice pour le problème

Un générateur de nombres pseudo-aléatoires distribués uniformément sur $(0, 1)$ pourrait nous permettre de générer les coordonnées en abscisse et en ordonnée de quatre points sur un carré 1×1 .

1.8 Générateurs utilisés dans Excel et R

Avant d'utiliser pour quelque tâche moindrement importante un générateur de nombres aléatoires inclus dans un logiciel, il importe de s'assurer de la qualité de celui-ci. Vous trouverez en général relativement facilement l'information pertinente dans Internet.

Nous présentons ici, sans entrer dans les détails, les générateurs utilisés dans Excel et R.

1.8.1 Générateur d'Excel

La fonction à utiliser dans Microsoft Excel pour obtenir un nombre aléatoire dans l'intervalle $(0, 1)$ est `ALEA()` (dans la version française) ou `RAND()` (dans la version anglaise).

L'historique de Microsoft n'est pas reluisant lorsqu'il s'agit de mise en œuvre de générateurs de nombres aléatoires pour Excel. Au fil des ans, plusieurs articles scientifiques ont démontré les lacunes de la fonction `RAND()`.

Par exemple, dans les versions 2003 et 2007, Microsoft Excel utilisait le générateur de nombres aléatoires Whichmann-Hill. Ce générateur a longtemps été considéré comme le meilleur disponible, mais il a été supplanté ces dernières années. Microsoft prétendait que la période du générateur Whichmann-Hill est 10^{13} , mais omettait de tenir compte de littérature scientifique démontrant qu'elle est plutôt de $6,95 \times 10^{12} \approx 2^{43}$, une période désormais considérée trop courte.

De plus, la mise en œuvre du générateur Whichmann–Hill dans Excel 2003 avait le fâcheux défaut de pouvoir générer des nombres négatifs. Ce défaut a plus tard été corrigé par un *Service Pack* 1 de Office 2003 ainsi que dans la version 2007.

L'article de [McCullough et Heiser \(2008b\)](#) demeure une référence sur les lacunes observées dans la génération de nombres aléatoires et les procédures statistiques dans Excel 2007. De plus, [McCullough et Heiser \(2008a\)](#) démontrent que le générateur d'Excel ne saurait être véritablement celui de Whichmann–Hill. Les auteurs écrivent en conclusion :

Twice Microsoft has attempted to implement the dozen lines of code that define the Wichmann and Hill (1982) RNG¹, and twice Microsoft has failed, apparently not using standard methods for verifying that an RNG has been correctly implemented. Consequently, users of Excel's "rand" function have been using random numbers from an unknown and undocumented RNG of unknown period that is not known to pass any standard tests of randomness.

Les critiques relevées dans les articles ci-dessus ont poussé Microsoft à améliorer la précision des procédures statistiques depuis Excel 2010 ([Microsoft Office Blog, 2009](#)). Microsoft affirme également utiliser un nouveau générateur de nombres aléatoires pour la fonction `RAND()`. Seulement, plusieurs années après le lancement du produit, il demeure difficile d'obtenir des détails qui permettraient de dissiper tout doute sur la qualité des nombres aléatoires fournis par Excel.



Si vous programmez en VBA, peut-être serez-vous tenté d'utiliser la fonction `RND()` pour générer un nombre aléatoire. Or, le générateur utilisé par `RND()` est un simple générateur congruentiel linéaire. C'est tout à fait archaïque. De plus, le générateur utilise toujours la même amorce et, par conséquent, les suites de nombres aléatoires sont toujours les mêmes. Pour toute utilisation moindrement sérieuse de nombres aléatoires, vous devez donc à tout prix éviter la fonction `RND()` et lui préférer un appel à la fonction `RAND()` de Excel.²

1. *Random Number Generator*

2. L'information concernant le générateur utilisé par VBA se trouvait dans l'article 231847 de la base de connaissances Microsoft, mais ce lien est aujourd'hui obsolète. Vous trouverez l'information équivalente — avec une analyse exhaustive — dans une [entrée de blogue](#)  de

1.8.2 Générateurs de R

On obtient des nombres uniformes sur un intervalle quelconque avec la fonction `runif` dans R. La fonction `set.seed` permet de spécifier la valeur de l'amorce du générateur aléatoire, ce qui est utile si nous voulons répéter une simulation absolument à l'identique.

R offre la possibilité de choisir entre plusieurs générateurs de nombres aléatoires différents, ou encore de spécifier son propre générateur. Par défaut, R utilise le générateur Marsenne-Twister, considéré comme le plus avancé en ce moment. La période de ce générateur est $2^{19937} - 1$ (rien de moins!) et la distribution des nombres est uniforme dans 623 dimensions consécutives sur toute la période.

Pour de plus amples détails et les dernières informations sur les générateurs disponibles et la procédure de réglage de l'amorce, consulter les rubriques d'aide des fonctions `.Random.seed` et `set.seed`.



1.9 Indice pour le problème

Générer les coordonnées uniformes des points sera plus simple et plus rapide avec la fonction `runif` de R.



1.10 Solution du problème

En premier lieu, écrivons une fonction `is.triangle` pour déterminer si l'enveloppe convexe de quatre points forme un triangle. La fonction prendra en arguments un vecteur `x` des coordonnées en abscisse des quatre points et un vecteur `y` des coordonnées en ordonnée. La fonction retournera `TRUE` ou `FALSE` selon que l'enveloppe convexe est un triangle ou non.

```
> is.triangle <- function(x, y) 3 == length(chull(x, y))
> is.triangle(c(8, 1, 7, 10), c(8, 2, 4, 3))
[1] TRUE
> is.triangle(c(7, 5, 2, 1), c(9, 3, 10, 7))
[1] FALSE
```

L'expression suivante permet de placer quatre points au hasard dans un carré 1×1 , puis de vérifier si leur enveloppe convexe est un triangle.

la firme australienne de services informatiques Noesis.

```
> is.triangle(runif(4), runif(4))
```

Pour répéter cette expression un grand nombre de fois, nous aurons recours, tel qu'expliqué à l'[annexe A](#), à la fonction `replicate`. Puisque chaque expression exécutée par `replicate` retourne TRUE ou FALSE, le résultat sera un vecteur booléen. La valeur moyenne du vecteur booléen correspond donc à la proportion d'enveloppes convexes triangulaires au fil des 100 000 simulations.

```
> mean(replicate(1E5,
+             is.triangle(x = runif(4), y = runif(4))))
[1] 0.30323
```

La valeur ci-dessus est suffisamment proche de $\frac{11}{36} \approx 0,30556$ pour que nous jugions l'expérience concluante.

1.11 Code informatique

📄 Fichier d'accompagnement `generation.R`

```
12 ###
13 ### EFFET DES PARAMÈTRES D'UN GÉNÉRATEUR
14 ###
15
16 ## Définissons tout d'abord une petite fonction pour calculer
17 ## les valeurs successives d'un générateur congruentiel
18 ## linéaire général.
19 rand <- function(n, a, c, m, seed)
20 {
21   x <- numeric(n + 1)
22   x[1] <- seed
23   for (i in seq(n))
24     x[i + 1] <- (a * x[i] + c) %% m
25   x[-1]
26 }
27
28 ## Nous pouvons toujours obtenir une période maximale de m
29 ## avec un générateur congruentiel en posant a = c = 1, mais
30 ## l'aspect aléatoire de la suite de nombres obtenus en prend
31 ## alors pour son rhume...
32 rand(17, a = 1, c = 1, m = 16, seed = 0)
33
34 ## Avec un meilleur choix de multiplicateur et d'incrément, la
35 ## période est tout aussi longue, mais la suite davantage
```

```
36 ## aléatoire.
37 rand(17, a = 5, c = 1, m = 16, seed = 0)
38
39 ## Un tout petit changement et la période est beaucoup plus
40 ## courte. Même la valeur de l'amorce a une influence sur la
41 ## période.
42 rand(17, a = 5, c = 4, m = 16, seed = 0)
43 rand(17, a = 5, c = 4, m = 16, seed = 1)
44 rand(17, a = 5, c = 4, m = 16, seed = 2)
45
46 ## Le générateur multiplicatif de l'exemple 1.2 ne satisfait
47 ## pas les conditions pour que la période soit maximale (7
48 ## n'est pas une racine primitive de 31).
49 rand(32, a = 7, c = 0, m = 31, seed = 19)
50 length(unique(rand(32, a = 7, c = 0, m = 31, seed = 19)))
51
52 ## Avec a = 3, on atteint la période de 30 car 3 est une
53 ## racine primitive de 31.
54 length(unique(rand(32, a = 3, c = 0, m = 31, seed = 19)))
55
56 ####
57 #### ANALYSE DE LA QUALITÉ D'UN GÉNÉRATEUR
58 ####
59
60 ## Un générateur avec une période de 30. Les valeurs obtenues
61 ## semblent assez aléatoires.
62 (x <- rand(30, a = 3, c = 0, m = 31, seed = 19))
63
64 ## Un graphique des valeurs successives ne montre pas de
65 ## structure particulière.
66 par(pch = 19)          # disque plein comme symbole
67 plot(x)
68
69 ## Par contre, si nous traçons un graphique des paires de
70 ## valeurs successives (c'est-à-dire x[1] en fonction de x[2],
71 ## x[2] en fonction de x[3], etc.), un portrait dérangent
72 ## apparaît.
73 plot(head(x, -1), tail(x, -1))
74
75 ## Augmenter la valeur du multiplicateur améliore la
76 ## situation. Si nous utilisons plutôt a = 12 (une racine
77 ## primitive), alors la période est toujours de 30 et donc les
78 ## valeurs de la suite sont les mêmes que ci-dessus, dans un
79 ## ordre différent. Cependant, les valeurs sont mieux
80 ## distribuées.
```



```

81 x <- rand(30, a = 12, c = 0, m = 31, seed = 19)
82 plot(head(x, -1), tail(x, -1))
83
84 ## Illustrons par six autres graphiques comment le
85 ## choix des paramètres d'un générateur congruentiel
86 ## peut avoir une importance majeure sur la qualité des
87 ## nombres générés.
88 par(mfrow = c(3, 2)) # 6 graphiques sur 3 lignes, 2 colonnes
89
90 x <- rand(2048, a = 65, c = 1, m = 2048, seed = 0)
91 plot(tail(x, -1), head(x, -1), main = "a = 65, c = 1")
92
93 x <- rand(2048, a = 1365, c = 1, m = 2048, seed = 0)
94 plot(tail(x, -1), head(x, -1), main = "a = 1365, c = 1")
95
96 x <- rand(2048, a = 1229, c = 1, m = 2048, seed = 0)
97 plot(tail(x, -1), head(x, -1), main = "a = 1229, c = 1")
98
99 x <- rand(2048, a = 157, c = 1, m = 2048, seed = 0)
100 plot(tail(x, -1), head(x, -1), main = "a = 157, c = 1")
101
102 x <- rand(2048, a = 45, c = 0, m = 2048, seed = 1234)
103 plot(tail(x, -1), head(x, -1), main = "a = 45, c = 0")
104
105 x <- rand(2048, a = 43, c = 0, m = 2048, seed = 1234)
106 plot(tail(x, -1), head(x, -1), main = "a = 43, c = 0")

```

1.12 Exercices

L'exercice 1.3 requiert d'installer le paquetage `rgl`, disponible dans CRAN; Goulet (2020, chapitre 10) explique la procédure d'installation d'un paquetage R.



L'utilisation de `rgl` sous macOS requiert l'application `X11.app`, qui n'est plus livrée avec le système depuis la version 10.8 (Mountain Lion). Le projet XQuartz fournit un serveur et des bibliothèques client X11 compatibles avec les versions de macOS 10.6 et suivantes. Il suffit de télécharger l'application depuis le [site du projet](#) .

- 1.1 Calculer cinq nombres pseudo-aléatoires avec chacun des générateurs congruentiels ci-dessous. Dans tous les cas, $m = 64$. Choisir l'amorce.

- a) $a = 29, c = 17$
- b) $a = 9, c = 1$
- c) $a = 13, c = 0$
- d) $a = 11, c = 0$

- 1.2** Composer dans R une mise en œuvre du générateur congruentiel multiplicatif avec $m = 2^{13} - 1$ et $a = 17$. Générer une suite de 500 nombres pseudo-aléatoires, puis tracer le graphique des paires (x_i, x_{i+1}) . Sur combien de lignes les points sont-ils alignés ? Répéter avec $a = 85$.
- 1.3** Le générateur RANDU, qui a longtemps été le plus populaire générateur de nombres pseudo-aléatoires, est défini ainsi :

$$x_i = 65\,539x_{i-1} \bmod 2^{31}.$$

Les nombres aléatoires obtenus avec ce générateur présentent une excellente structure aléatoire en une et en deux dimensions. On peut toutefois démontrer mathématiquement qu'en trois dimensions, les triplets (x_i, x_{i+1}, x_{i+2}) se retrouvent sur quinze plans ou moins, rendant ainsi assez prévisible la valeur de x_{i+2} étant donné les deux autres valeurs. Contentons-nous, ici, de constater cet état de fait graphiquement.

- a) Générer une suite $\{x_i\}$ de longueur 20 002 avec le générateur RANDU et poser $u_i = 2^{-31}x_i$. Pour tous les triplets (u_i, u_{i+1}, u_{i+2}) , sélectionner les cas où $0,5 \leq u_{i+1} \leq 0,51$ et faire un graphique de u_{i+2} en fonction de u_i . Commenter le graphique obtenu.
- b) Générer une suite $\{x_i\}$ de longueur 1 002 avec le générateur RANDU. À l'aide de R, placer les triplets (u_i, u_{i+1}, u_{i+2}) dans un graphique en trois dimensions avec la fonction `plot3d` du paquetage `rgl`, puis faire pivoter le graphique pour voir les quinze plans sur lesquels se trouvent répartis les points.

2 Simulation de nombres aléatoires non uniformes

Objectifs du chapitre

- ▶ Développer un algorithme de simulation de nombres non uniformes à partir de la méthode de l'inverse.
- ▶ Développer un algorithme de simulation de nombres non uniformes à partir de la méthode d'acceptation-rejet.
- ▶ Calculer des nombres pseudo-aléatoires non uniformes en suivant un algorithme donné.
- ▶ Utiliser les outils de Excel et R pour la simulation de nombres non uniformes.

Habituellement, les applications de la simulation requièrent des nombres aléatoires provenant non pas d'une distribution $U(0, 1)$, mais plutôt d'une distribution avec fonction de répartition $F_X(x)$. Cela nécessite donc de transformer les nombres uniformes en des nombres provenant de distributions non-uniformes.

Il existe de très nombreux algorithmes pour générer des nombres aléatoires de différentes distributions; voir, par exemple, [Devroye \(1986\)](#). Nous n'en étudierons que deux en détail, soit la méthode de l'inverse et la méthode d'acceptation-rejet. Par ailleurs, la [section 2.7](#) présente des procédures de simulation pour les modèles actuariels les plus courants.

Plusieurs algorithmes de simulation de nombres non uniformes reposent sur des résultats de transformations de variables aléatoires. Par exemple :

- ▶ la somme de α exponentielles est une loi gamma avec paramètre de forme α entier;
- ▶ la loi géométrique est la partie entière de l'exponentielle;
- ▶ la loi F est un ratio de deux khi-carré; etc.

Le lecteur qui ne serait pas à l'aise avec les notions de transformation de variables aléatoires trouvera à l'[annexe B](#) des rappels sur les principales techniques étudiées dans les cours d'analyse probabiliste et d'analyse statistique.



2.1 Énoncé du problème

Au chapitre précédent, nous nous sommes penchés sur le problème des quatre points de Sylvester lorsque la région R est un carré.

Cette fois, nous devons vérifier par simulation que lorsque la région R est un disque, la probabilité que l'enveloppe convexe forme un triangle est $\frac{35}{12\pi^2} \approx 0,29552$.

2.2 Méthode de l'inverse

La méthode de l'inverse repose sur une idée toute simple, soit que nous pouvons transformer des nombres uniformes sur $(0, 1)$ en des nombres provenant de la distribution avec fonction de répartition $F_X(x)$ en utilisant le théorème suivant.

Théorème 2.1. Soit X une variable aléatoire avec fonction de répartition $F_X(x)$. Alors

$$F_X(X) \sim U(0, 1).$$

Démonstration. Soit la transformation $U = F_X(X)$. Alors,

$$\begin{aligned} F_U(u) &= \Pr[U \leq u] \\ &= \Pr[F_X(X) \leq u] \\ &= \Pr[X \leq F_X^{-1}(u)] \\ &= F_X(F_X^{-1}(u)) \\ &= u, \end{aligned}$$

d'où $U \sim U(0, 1)$. □

Par conséquent, si $U \sim U(0, 1)$, alors

$$F_X^{-1}(U) \sim X.$$

La fonction de répartition inverse, F_X^{-1} , est aussi appelée *fonction de quantile*.

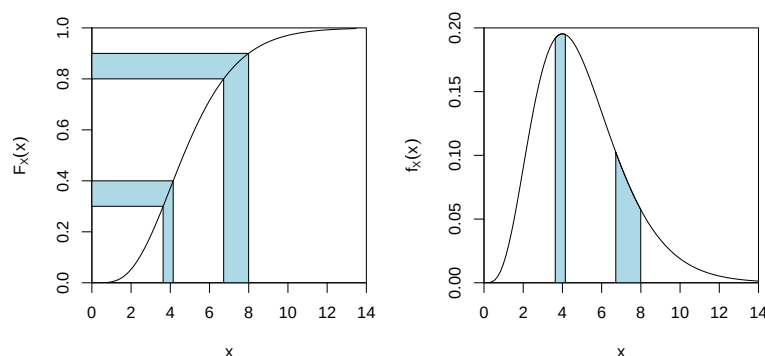


FIG. 2.1 – Représentation graphique de la méthode de l'inverse. À des intervalles égaux en ordonnée correspondent des intervalles différents en abscisse selon la forme de la distribution. La fonction de répartition de gauche correspond à la densité de droite.

Algorithme 2.1 (Méthode de l'inverse). *Soit X une variable aléatoire avec fonction de répartition $F_X(x)$. Les étapes ci-dessous permettent de générer une observation x de cette variable aléatoire.*

1. *Obtenir un nombre u d'une loi $U(0, 1)$.*
2. *Poser $x = F_X^{-1}(u)$.*

La méthode de l'inverse consiste à choisir un nombre uniformément sur l'axe des ordonnées d'une fonction de répartition (donc un nombre entre 0 et 1) et à trouver la valeur correspondante sur l'axe des abscisses telle que donnée par la fonction de quantile. Comme les valeurs en x seront plus concentrées là où la pente de la fonction de répartition est la plus grande, et vice versa, la distribution en abscisse ne sera pas uniforme. Voir la [figure 2.1](#) pour une représentation graphique de ce phénomène.

La méthode de l'inverse en est une bonne si la fonction de quantile est facile à calculer. S'il n'existe pas de forme explicite pour $F_X^{-1}(\cdot)$, résoudre numériquement

$$F_X(x) - u = 0$$

peut s'avérer aussi efficace que bien d'autres méthodes.



Dans Excel, il faut nécessairement utiliser la méthode de l'inverse. Plusieurs fonctions de quantiles sont disponibles; voir la [section 2.6](#).

2.2.1 Distributions continues

La méthode de l'inverse est, en principe du moins, simple à utiliser avec les distributions continues : il suffit de connaître la fonction de quantile et de l'appliquer à des nombres uniformes pour obtenir des nombres de la distribution souhaitée.

Dans les faits, il y a peu de lois de probabilité continues dont la fonction de répartition est simple à inverser (exponentielle, Pareto, Weibull). Il faut parfois utiliser d'autres méthodes.

Exemple 2.1. Nous voulons obtenir un échantillon aléatoire d'une distribution exponentielle de paramètre λ avec fonction de densité de probabilité

$$f(x) = \lambda e^{-\lambda x}, \quad x > 0$$

et fonction de répartition

$$F(x) = 1 - e^{-\lambda x}, \quad x > 0.$$

Or,

$$F^{-1}(u) = -\frac{1}{\lambda} \ln(1 - u),$$

donc

$$X = -\frac{1}{\lambda} \ln(1 - U) \sim \text{Exponentielle}(\lambda),$$

où $U \sim U(0, 1)$. En fait, puisque $U \sim U(0, 1) \Leftrightarrow 1 - U \sim U(0, 1)$, nous pouvons nous contenter de la relation

$$X = -\frac{1}{\lambda} \ln U \sim \text{Exponentielle}(\lambda).$$

Par conséquent, l'algorithme pour simuler des nombres provenant d'une exponentielle de paramètre λ est :

1. Obtenir un nombre u d'une $U(0, 1)$;
2. Poser $x = -\lambda^{-1} \ln u$.



Complétez cet exemple en étudiant les lignes 13-37 du fichier de script `simulation.R` reproduit à la [section 2.9](#).

□

2.2.2 Distributions discrètes

Nous pouvons aussi utiliser la méthode de l'inverse avec les distributions discrètes. Cependant, puisque la fonction de répartition comporte des sauts, son inverse n'existe pas formellement. Par conséquent, il n'existe pas de solution de $u = F_X(x)$ pour certaines valeurs de u , ou alors une infinité de solutions.

Supposons une distribution avec un saut en x_0 et

$$\begin{aligned} F_X(x_0^-) &= a \\ F_X(x_0) &= b > a. \end{aligned}$$

Si $a < u < b$, nous posons $x = x_0$. Ainsi, nous simulerons x_0 dans une proportion $b - a$ du temps, ce qui correspond à $\Pr[X = x_0]$. Voir la [figure 2.2](#) pour une illustration.

Que faire si $u = a$ ou $u = b$? Nous prenons la plus grande valeur de l'intervalle où $F_X(x)$ est constante :

$$\begin{aligned} u = a &\Rightarrow x = x_0 \\ u = b &\Rightarrow x = x_1. \end{aligned}$$

Il faut procéder ainsi parce que plusieurs générateurs produisent des nombres uniformes sur $[0, 1)$.

Exemple 2.2. Soit X une variable aléatoire avec fonction de densité de probabilité

$$f(x) = \begin{cases} 0,5, & 0 \leq x < 1 \\ 0, & 1 \leq x < 2 \\ 0,5, & 2 \leq x < 3. \end{cases}$$

Il s'agit d'une variable aléatoire *mixte* (en partie continue et en partie discrète) dont la fonction de répartition est

$$F(x) = \begin{cases} 0,5x, & 0 \leq x < 1 \\ 0,5, & 1 \leq x < 2 \\ 0,5x - 0,5, & 2 \leq x < 3. \end{cases}$$

Cette fonction est représentée à la [figure 2.3](#).

Un algorithme pour simuler des nombres aléatoires de cette distribution est donc :

1. Obtenir un nombre u d'une loi $U(0, 1)$.

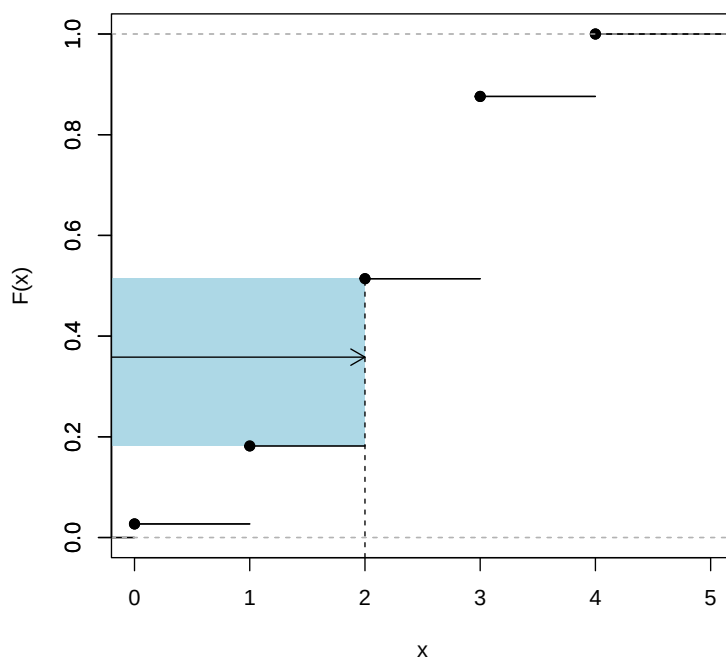


FIG. 2.2 - Illustration de la méthode de l'inverse pour une distribution discrète. Tout nombre uniforme tiré dans la zone colorée est converti en la même valeur, ce qui correspond à la probabilité d'obtenir cette valeur (la hauteur du saut).

2. Poser

$$x = \begin{cases} 2u, & \text{si } 0 \leq u < 0,5 \\ 2, & \text{si } u = 0,5 \\ 2u + 1 & \text{si } 0,5 < u < 1. \end{cases}$$

□

Exemple 2.3. Nous voulons simuler des observations d'une distribution binomiale de paramètres n et θ . Nous pourrions, pour chaque x simulé, faire n expériences de Bernoulli et compter le nombre de succès. Par la méthode de l'inverse pour la loi de Bernoulli, on a un succès si $u \leq \theta$. Par conséquent, un algorithme serait :

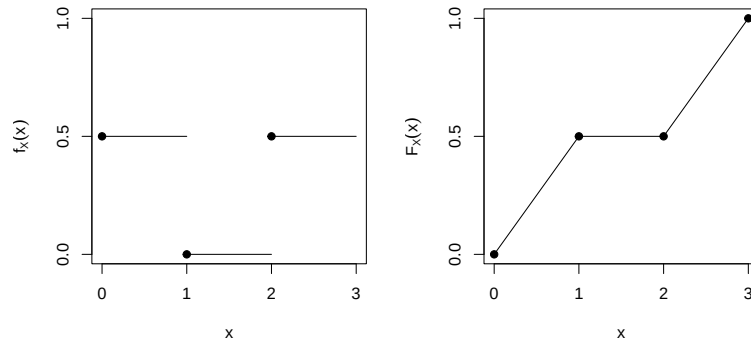


FIG. 2.3 - Fonction de densité de probabilité (gauche) et fonction de répartition (droite) de l'exemple 2.2

1. Simuler n nombres uniformes indépendants $u_1, \dots, u_n$ d'une loi $U(0, 1)$.
2. Poser

$$x = \sum_{i=1}^n I\{u_i \leq \theta\}.$$

Cette technique requiert toutefois de simuler n nombres uniformes pour chaque valeur de x .

Il est plus efficace d'utiliser la méthode de l'inverse directement avec la distribution binomiale. Par exemple, si $n = 4$ et $\theta = 0,5$, on a que

$$\Pr[X = x] = \begin{cases} 0,0625, & x = 0 \\ 0,25, & x = 1 \\ 0,375, & x = 2 \\ 0,25, & x = 3 \\ 0,0625, & x = 4 \end{cases}$$

et

$$\Pr[X \leq x] = \begin{cases} 0, & x < 0 \\ 0,0625, & 0 \leq x < 1 \\ 0,3125, & 1 \leq x < 2 \\ 0,6875, & 2 \leq x < 3 \\ 0,9375, & 3 \leq x < 4 \\ 1, & x \geq 4, \end{cases}$$

d'où l'algorithme suivant :

1. Simuler un nombre u d'une distribution $U(0, 1)$.
2. Déterminer x selon le tableau suivant :

u dans l'intervalle	Valeur de x
$[0, 0,0625)$	0
$[0,0625, 0,3125)$	1
$[0,3125, 0,6875)$	2
$[0,6875, 0,9375)$	3
$[0,9375, 1)$	4

□



La méthode de l'inverse pour les distributions discrètes à support fini est facile à mettre en œuvre dans Excel à l'aide de la fonction RECHERCHEV().



2.3 Indice pour le problème

Au cœur du problème que nous tentons de résoudre, il y a la simulation de points dans un cercle. L'interprétation géométrique est claire.

Il serait également possible d'en faire une interprétation probabiliste : nous voulons simuler des valeurs d'une distribution bidimensionnelle uniforme sur un disque — sans perte de généralité, de rayon 1 et centré à l'origine — dont la fonction de densité conjointe est

$$f_{XY}(x, y) = \frac{1}{\pi}, \quad -1 < x < 1, x^2 + y^2 < 1.$$

Cependant, les méthodes de simulation pour les distributions multidimensionnelles deviennent rapidement complexes, surtout lorsque les variables ne sont pas indépendantes, comme c'est le cas ici. Nous tâcherons donc de trouver une manière plus intuitive d'un point de vue géométrique pour simuler des points sur un disque.

2.4 Méthode acceptation-rejet

Supposons qu'il est compliqué de simuler des réalisations de la variable aléatoire X avec fonction de densité de probabilité $f_X(x)$. Si nous pouvons

trouver une variable aléatoire Y avec fonction de densité de probabilité $g_Y(x)$ pour laquelle la simulation est simple (uniforme, triangle, exponentielle, etc.) et qu'il est possible d'« envelopper » la densité f par un multiple de g , c'est-à-dire que

$$cg_Y(x) \geq f_X(x)$$

pour tout x , alors nous pouvons utiliser l'algorithme d'acceptation-rejet ci-dessous pour générer des observations de la variable aléatoire X .

Algorithme 2.2 (Méthode d'acceptation-rejet). *Soit X une variable aléatoire avec fonction de densité de probabilité $f_X(x)$ et Y une variable aléatoire avec fonction de densité de probabilité $g_Y(x)$. Les étapes ci-dessous permettent de générer un nombre x issu de la distribution f .*

1. Obtenir une réalisation y de distribution avec densité $g_Y(\cdot)$.
2. Obtenir un nombre u d'une $U(0, 1)$.
3. Si

$$u \leq \frac{f_X(y)}{cg_Y(y)},$$

poser $x = y$. Sinon, retourner à l'étape 1.



Puisque, par définition, l'aire sous les densités f et g vaut 1 dans les deux cas, la relation $cg_Y(x) \geq f_X(x)$ ne peut être vraie pour tout x que si $c > 1$.

L'idée de la méthode d'acceptation-rejet consiste à accepter la « bonne proportion » des réalisations de Y comme provenant de X . La principale difficulté avec la méthode d'acceptation-rejet consiste à trouver la fonction enveloppante $cg_Y(x)$.

Dans l'illustration de la [figure 2.4](#), la densité f à support fini est facilement enveloppée par un rectangle. Un nombre y simulé de cette distribution (à toutes fins pratiques une uniforme, ici) est accepté comme provenant de f dans une proportion correspondant au ratio entre la valeur de $f_X(y)$ (les segments pointillés dans la figure) et la valeur de $cg_Y(y)$ (les segments pleins). Vous constaterez aisément que certaines valeurs y seront acceptées plus souvent que d'autres en conformité avec la forme de la densité.

Une autre interprétation est possible. La méthode dit d'accepter la valeur y simulée de la densité g comme provenant de la densité f si

$$ucg_Y(y) \leq f_X(y),$$

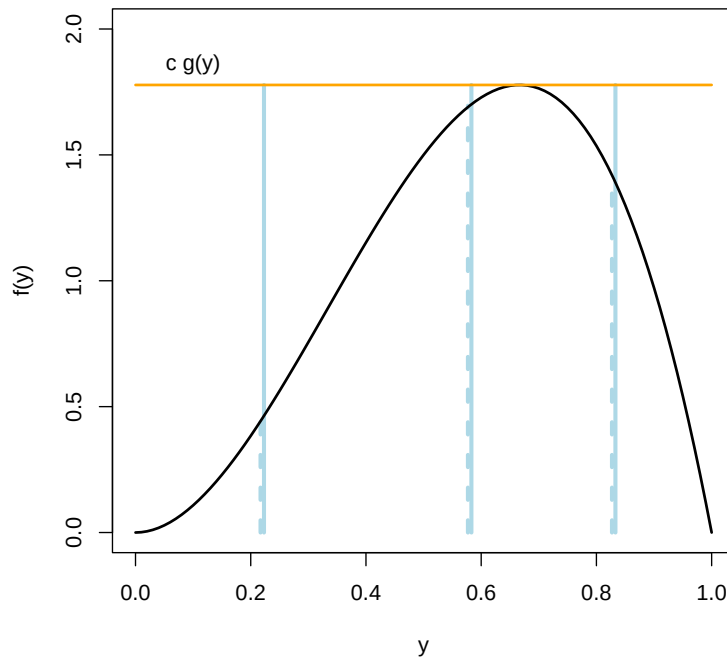


FIG. 2.4 - Illustration de la méthode acceptation-rejet

où u est un nombre issu d'une loi $U(0, 1)$. Graphiquement, cela signifie que nous acceptons la valeur y si le point $(y, u c g_Y(y))$ se trouve sous la courbe f en y et que nous la rejetons s'il se trouve entre f et l'enveloppe. La [figure 2.5](#) illustre cette interprétation.



Évidemment, plus l'enveloppe est près de $f_X(x)$, plus l'algorithme est performant puisque nous rejetons alors moins de valeurs.



Toujours perplexe devant la méthode d'acceptation-rejet ? La [vidéo sur le sujet](#) fournit une illustration graphique qui devrait vous aider à mieux comprendre la procédure. Visionnez-la avant d'aller plus loin.

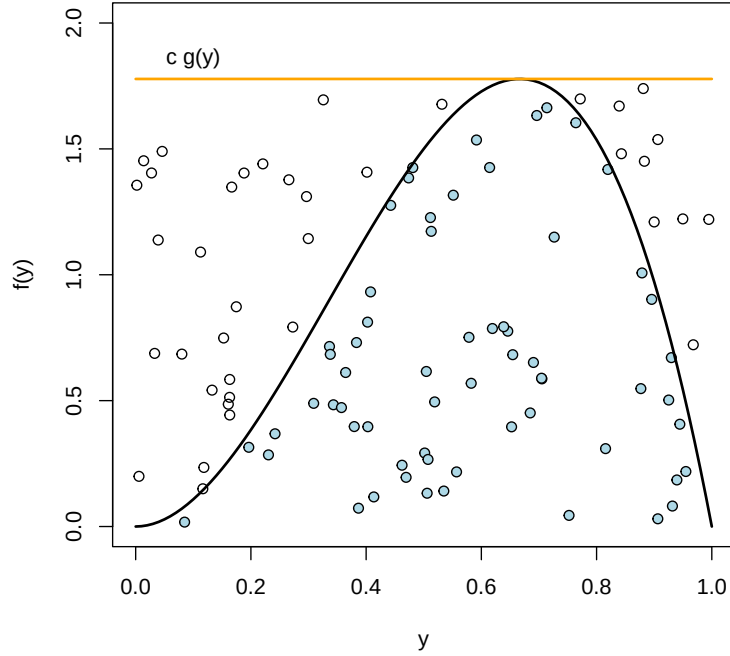


FIG. 2.5 – Interprétation alternative de la méthode acceptation-rejet. Chaque point représente un couple $(y, ucg_Y(y))$. On accepte les points pleins et on rejette les points vides.

Exemple 2.4. Soit $X \sim \text{Bêta}(3, 2)$. La densité de cette distribution est :

$$\begin{aligned} f_X(x) &= \frac{\Gamma(5)}{\Gamma(3)\Gamma(2)} x^{3-1}(1-x)^{2-1} \\ &= 12x^2(1-x), \quad 0 < x < 1. \end{aligned}$$

Elle est représentée aux figures 2.4 et 2.5. Nous pouvons facilement inscrire celle-ci dans un rectangle. Le mode de la densité se trouvant en $x = 2/3$, la hauteur du rectangle est $f(2/3) = 48/27 = 16/9$. Par conséquent,

$$cg_Y(x) = \frac{16}{9}, \quad 0 < x < 1.$$

Nous déduisons que $c = 16/9$ et que la densité g est une uniforme sur $(0, 1)$. Nous avons donc l'algorithme suivant :

1. Simuler deux nombres u_1 et u_2 d'une $U(0, 1)$.
2. Poser $y = u_1$.
3. Si

$$u_2 \leq \frac{f_X(y)}{16/9},$$

alors poser $x = y$. Sinon, retourner à l'étape 1.

L'aire du rectangle étant de $16/9$, nous pouvons nous attendre à rejeter une proportion de

$$\frac{16/9 - 1}{16/9} = \frac{7}{16} \approx 44 \%$$

des nombres u_1 simulés à l'étape 1 de l'algorithme ci-dessus. Si l'enveloppe était plus « serrée » autour de la densité, l'efficacité de l'algorithme en serait augmentée. $\square$

Exemple 2.5. Reprenons l'exemple 2.4 en tentant d'améliorer l'efficacité de l'algorithme. Il s'avère que nous pouvons inscrire la densité de la loi Bêta(3, 2) dans un triangle aux caractéristiques suivantes (voir la figure 2.6) :

1. l'arête gauche passe par $(0, 0)$, donc est de la forme $y = mx$. Cette droite étant tangente à $f(x)$, la pente m est telle que l'équation $mx = 12x^2(1 - x)$ a une seule racine autre que 0, d'où $y = 3x$;
2. l'arête droite passe par $(1, 0)$, donc est de la forme $y = mx + b$ avec $m + b = 0$. Comme la pente de cette droite est égale à la pente de $f(x)$ en $x = 1$, nous trouvons que $y = 12 - 12x$.

Ainsi,

$$cg_Y(x) = \begin{cases} 3x, & 0 < x < 0,8 \\ 12 - 12x, & 0,8 < x < 1. \end{cases}$$

Or, l'aire du triangle est

$$\frac{(1)cg_Y(0,8)}{2} = \frac{(1)(2,4)}{2} = 1,2,$$

d'où $c = 1,2$. Par conséquent,

$$g_Y(x) = \begin{cases} 2,5x, & 0 < x < 0,8 \\ 10 - 10x, & 0,8 < x < 1. \end{cases}$$

Pour simuler des observations de cette densité par la méthode de l'inverse, nous calculons la fonction de répartition correspondante,

$$G_Y(x) = \begin{cases} 1,25x^2, & 0 < x < 0,8 \\ -5x^2 + 10x - 4, & 0,8 < x < 1, \end{cases}$$

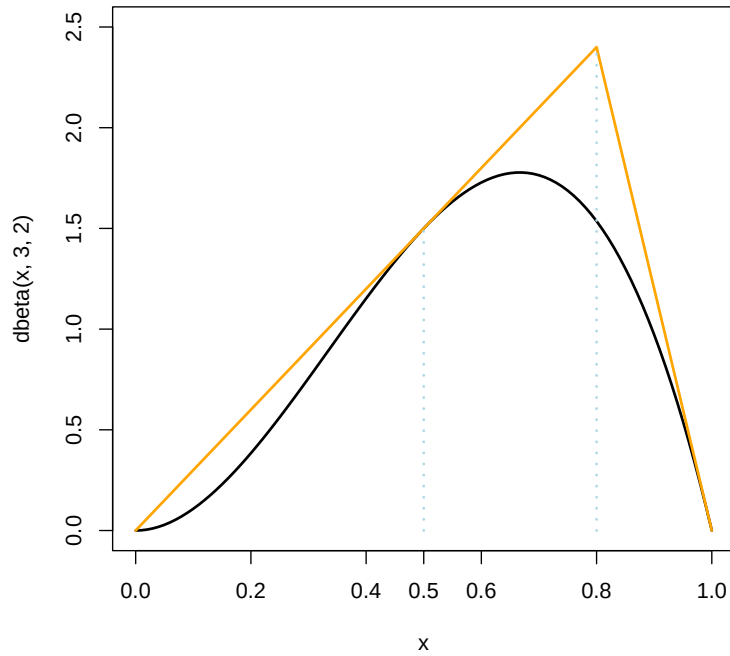


FIG. 2.6 - Fonction de densité de probabilité d'une loi Bêta(3, 2) enveloppée d'un triangle

d'où

$$G_Y^{-1}(y) = \begin{cases} \sqrt{0,8y}, & 0 < y < 0,8 \\ 1 - \sqrt{0,2 - 0,2y}, & 0,8 < y < 1. \end{cases}$$

Au final, nous obtenons l'algorithme suivant :

1. Simuler deux nombres u_1 et u_2 d'une $U(0, 1)$.
2. Poser $y = G_Y^{-1}(u_1)$.
3. Si

$$u_2 \leq \frac{f_X(y)}{1,2g_Y(y)} \Leftrightarrow 1,2g_Y(y)u_2 \leq f_X(y),$$

alors poser $x = y$. Sinon, retourner à l'étape 1.

L'aire du triangle étant de 1,2, nous pouvons maintenant nous attendre à ne rejeter que $0,2/1,2 \approx 17\%$ des nombres simulés, une amélioration importante par rapport à l'exemple 2.4.



Les lignes 40–78 du fichier de script `simulation.R` reproduit à la section 2.9 contiennent une mise en œuvre en R de l'algorithme ci-dessus. Étudiez ce code attentivement.

□

2.5 Indice pour le problème

L'interprétation géométrique de la méthode d'acceptation-rejet illustrée à la figure 2.5 nous met sur la piste d'une manière simple de générer des points distribués uniformément sur un disque.

En effet, il suffit de simuler des points uniformément sur un carré comme nous l'avons déjà fait au chapitre 1, puis de rejeter ceux qui ne se trouvent pas à l'intérieur du disque. La figure 2.7 illustre cette idée : nous accepterions les points pleins et nous rejeterions les points vides.

2.6 Fonctions de simulation dans Excel et R

Il est important de savoir simuler des valeurs d'une variable aléatoire quelconque à partir de la méthode de l'inverse ou d'un autre algorithme, surtout si la distribution de la variable aléatoire est peu usitée. Néanmoins, les différents outils statistiques fournissent en général la majorité des fonctions de simulation de variables aléatoires dont nous pouvons avoir besoin pour un usage quotidien.

2.6.1 Excel

Tel que mentionné à la section 2.2, il faut généralement utiliser la méthode de l'inverse pour simuler des observations de lois de probabilité dans Excel. Cette procédure est facilitée par le fait qu'il existe des fonctions Excel pour calculer la fonction de répartition inverse (ou fonction de quantile) des lois les plus courantes.

Ainsi, Excel fournit la fonction de densité de probabilité (lois continues) ou la fonction de masse de probabilité (lois discrètes) ou la fonction de répartition et, dans certains cas seulement, la fonction de quantile des lois de

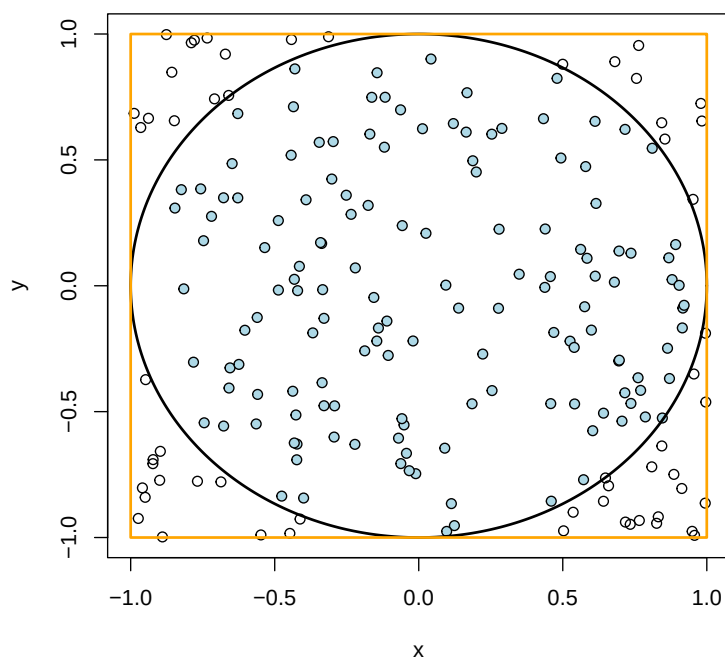


FIG. 2.7 – Simulation de points sur un disque par acceptation-rejet

probabilité présentées au [tableau 2.1](#). Les noms anglais des fonctions ont été modifiés pour être standardisés dans Excel 2010. Vous remarquerez que les traducteurs français ont omis de faire de même.

2.6.2 R

Un large éventail de fonctions donne directement accès aux caractéristiques de plusieurs lois de probabilité dans R. Pour chaque racine $\langle loi \rangle$, il existe quatre fonctions différentes :

1. $d\langle loi \rangle$ calcule la fonction de densité de probabilité (loi continue) ou la fonction de masse de probabilité (loi discrète);
2. $p\langle loi \rangle$ calcule la fonction de répartition;
3. $q\langle loi \rangle$ calcule la fonction de quantile;
4. $r\langle loi \rangle$ simule des observations de cette loi.

Loi	Fonctions Excel (nom anglais)	Fonctions Excel (nom français)
Bêta	BETA.DIST	LOI.BETA.N
	BETA.INV	BETA.INVERSE.N
Binomiale	BINOM.DIST	LOI.BINOMALE.N
	BINOM.INV	LOI.BINOMIALE.INVERSE
Binomiale nég.	NEGBINOM.DIST	LOI.BINOMIALE.NEG.N
Exponentielle	EXPON.DIST	LOI.EXPONENTIELLE.N
<i>F</i> (Fisher)	F.DIST	LOI.F.N
	F.INV	INVERSE.LOI.F.N
Gamma	GAMMA.DIST	LOI.GAMMA.N
	GAMMA.INV	LOI.GAMMA.INVERSE.N
Hypergéom.	HYPGEOM.DIST	LOI.HYPERGEOMETRIQUE.N
Khi carré	CHISQ.DIST	LOI.KHIDEUX
	CHISQ.INV	LOI.KHIDEUX.INVERSE
Log-normale	LOGNORM.DIST	LOI.LOGNORMALE.N
	LOGNORM.INV	LOI.LOGNORMALE.INVERSE.N
Normale	NORM.DIST	LOI.NORMALE.N
	NORM.INV	LOI.NORMALE.INVERSE.N
	NORM.S.DIST	LOI.NORMALE.STANDARD.N
	NORM.S.INV	LOI.NORMALE.STANDARD.INVERSE.N
Poisson	POISSON.DIST	LOI.POISSON.N
<i>t</i> (Student)	T.DIST	LOI.STUDENT.N
	T.INV	LOI.STUDENT.INVERSE.N
Weibull	WEIBULL.DIST	LOI.WEIBULL.N

TAB. 2.1 – Liste des fonctions relatives à des lois de probabilité depuis Excel 2010.

Les différentes lois de probabilité disponibles dans le système R de base, leur racine et le nom de leurs paramètres sont rassemblés au [tableau 2.2](#). Des paquetages fournissent des fonctions pour d'autres lois dont, entre autres, **actuar** (Dutang et collab., 2008) et **SuppDists** (Wheeler, 2013).

Toutes les fonctions du [tableau 2.2](#) sont vectorielles, c'est-à-dire qu'elles acceptent en argument un vecteur de points où la fonction (de densité, de répartition ou de quantile) doit être évaluée et même un vecteur de paramètres. Par exemple, l'expression ci-dessous retourne la probabilité que des lois de Poisson de paramètre 1, 4 et 10 prennent les valeurs 3, 0 et 8, dans

Loi de probabilité	Racine dans R	Noms des paramètres
Bêta	beta	shape1, shape2
Binomiale	binom	size, prob
Binomiale négative	nbinom	size, prob ou mu
Cauchy	cauchy	location, scale
Exponentielle	exp	rate
F (Fisher)	f	df1, df2
Gamma	gamma	shape, rate ou scale
Géométrique	geom	prob
Hypergéométrique	hyper	m, n, k
Khi carré	chisq	df
Logistique	logis	location, scale
Log-normale	lnorm	meanlog, sdlog
Normale	norm	mean, sd
Poisson	pois	lambda
t (Student)	t	df
Uniforme	unif	min, max
Weibull	weibull	shape, scale
Wilcoxon	wilcox	m, n

TAB. 2.2 – Loïs de probabilité pour lesquelles il existe des fonctions dans le système R de base

l'ordre.

```
> dpois(c(3, 0, 8), lambda = c(1, 4, 10))
[1] 0.06131324 0.01831564 0.11259903
```

Le premier argument de toutes les fonctions de simulation est la quantité de nombres aléatoires désirée. Ainsi, l'expression suivante retourne trois nombres aléatoires issus de distributions de Poisson de paramètre 1, 4 et 10, respectivement.

```
> rpois(3, lambda = c(1, 4, 10))
[1] 1 1 9
```

Évidemment, spécifier un vecteur comme premier argument d'une fonction de simulation n'a pas tellement de sens, mais, si c'est fait, R retournera une quantité de nombres aléatoires égale à la *longueur* du vecteur (sans égard aux valeurs contenues dans le vecteur).

Autre joueur important dans l'équipe de simulation de R, la fonction `sample` permet de générer des nombres d'une distribution discrète quelconque. Sa syntaxe est

```
sample(x, size, replace = FALSE, prob = NULL)},
```

- ▶ `x` est un vecteur des valeurs possibles de l'échantillon à simuler (le support de la distribution). Si `x` est un nombre unique supérieur ou égal à 1, le support de la distribution est `1:x`;
- ▶ `size` est la quantité de nombres à simuler, par défaut le nombre d'éléments dans l'argument `x`;
- ▶ `prob` est un vecteur de probabilités associées à chaque valeur de `x`, par défaut `1/length(x)`;
- ▶ `replace` détermine si l'échantillonnage s'effectue avec ou sans remise.

En analyse de données, la fonction `sample` est particulièrement utile pour choisir des éléments d'un vecteur ou des lignes d'un tableau au hasard. Le support de la « distribution » dans de tels cas est alors `1, 2, ..., n`, où `n` est la longueur du vecteur ou le nombre de lignes du tableau de données.



Le code des lignes 81–165 du fichier de script `simulation.R` reproduit à la section 2.9 contient des exemples plus détaillés d'utilisation des fonctions mentionnées ci-dessus.

2.7 Modèles actuariels

Cette section traite de techniques de simulation pour des distributions couramment utilisées dans la modélisation des risques en sciences actuarielles.

2.7.1 Mélanges discrets

Les *mélanges discrets* (ou *finis*; *mixture* en anglais) sont utiles pour décrire des phénomènes considérés comme provenant de la combinaison de plusieurs phénomènes qui surviennent chacun avec une probabilité connue d'avance. Par exemple, si nous estimons qu'il y a 90 % de chances que la distribution du nombre de personnes dans une file d'attente soit une Poisson de paramètre 2 et 10 % de chances qu'il s'agisse plutôt d'une Poisson de paramètre 10, alors la fonction de masse de probabilité de la distribution du

nombre de personnes dans la file d'attente en tout temps est :

$$\Pr[X = x] = 0,9 \times \frac{2^x e^{-2}}{x!} + 0,1 \times \frac{10^x e^{-10}}{x!}.$$

Cette fonction de masse de probabilité est un mélange discret de deux lois de Poisson.

Définition 2.1. La variable aléatoire Y est un mélange discret à k points des variables aléatoires $X_1, X_2, \dots, X_k$ si sa fonction de répartition est

$$F_Y(x) = p_1 F_{X_1}(x) + p_2 F_{X_2}(x) + \dots + p_k F_{X_k}(x),$$

où $p_j \in [0, 1]$ pour tout $j = 1, \dots, k$ et $\sum_{j=1}^k p_j = 1$.

Dans la suite, nous ne considérerons que les mélanges de $k = 2$ distributions. Lorsque celles-ci sont continues avec fonctions de densité de probabilité f_1 et f_2 , la densité du mélange peut s'écrire

$$f(x) = p f_1(x) + (1 - p) f_2(x).$$

Exemple 2.6. Les mélanges discrets sont très souvent utilisés pour créer de nouvelles distributions aux caractéristiques particulières que l'on ne retrouve pas chez les distributions d'usage courant.

La [figure 2.8](#) montre un mélange de deux distributions log-normales qui résulte en une fonction de densité de probabilité bimodale ayant une queue similaire à celle d'une log-normale. $\square$

La conception d'un algorithme de simulation d'un mélange discret à deux distributions nécessite de bien comprendre que chaque observation provient de l'une *ou* de l'autre des distributions. Reste, à chaque fois, à déterminer à partir de quelle distribution simuler.

Algorithme 2.3 (Mélange discret de deux distributions). Soit $F(x) = p F_1(x) + (1 - p) F_2(x)$ la fonction de répartition d'un mélange discret des distributions avec fonctions de répartition F_1 et F_2 . Les étapes ci-dessous permettent de générer un nombre aléatoire x de la distribution F .

1. Obtenir un nombre u d'une loi $U(0, 1)$.
2. Si $u \leq p$, obtenir x de la distribution F_1 , sinon obtenir x de F_2 .

Vous aurez reconnu dans le test $u \leq p$, où u provient d'une loi $U(0, 1)$, le cœur de la procédure de simulation d'une loi de Bernoulli mentionnée à

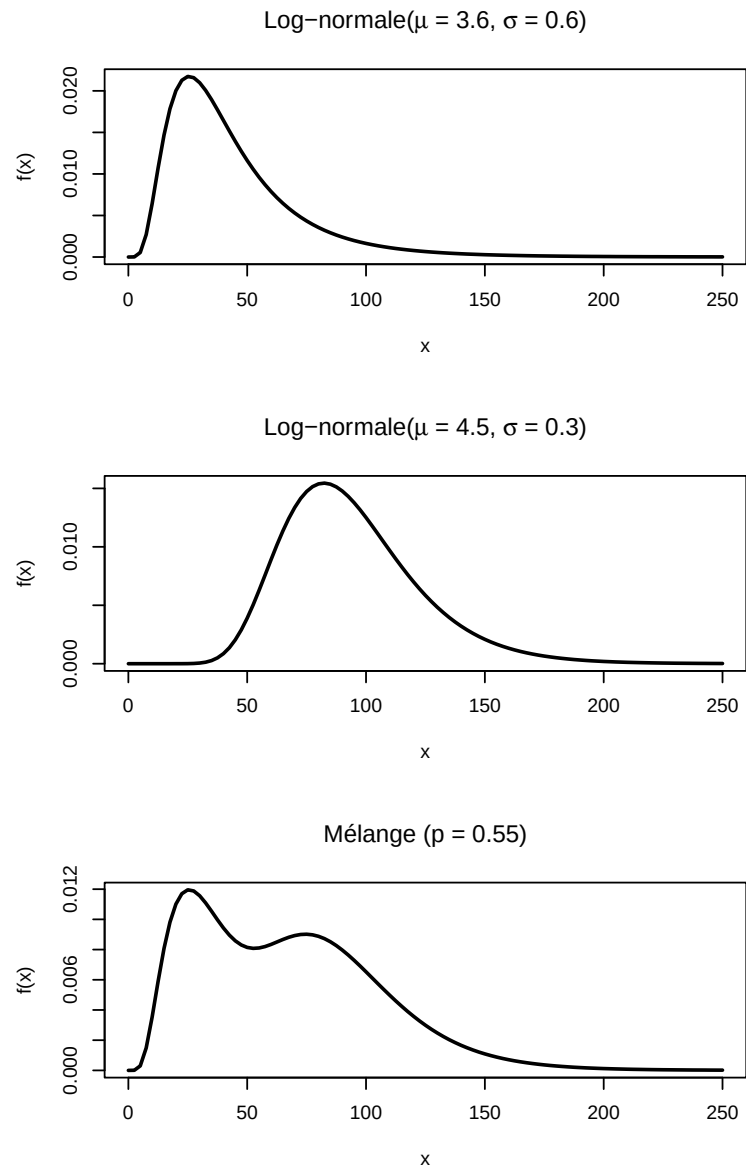


FIG. 2.8 - Fonctions de densité de probabilité de deux log-normales et de leur mélange discret

l'**exemple 2.3**. L'algorithme 2.3 revient donc à simuler un nombre de la distribution F_1 si $u \leq p$ et aucun nombre de la distribution F_2 ou, au contraire, aucun nombre de F_1 si $u > p$ et un nombre de F_2 . Cette observation joue un rôle important pour concevoir une procédure de simulation de $n > 1$ valeurs d'un mélange.

La fonction `rmixture` du paquetage **actuar** offre une interface conviviale pour obtenir des nombres aléatoires de mélanges discrets.



La vidéo sur la **simulation de mélanges discrets** [🔗](#) propose une représentation visuelle de l'**algorithme 2.3**, en plus d'expliquer une erreur commise très fréquemment.

2.7.2 Mélanges continus et modèles hiérarchiques

Nous pouvons étendre le concept de mélange discret à une quantité non dénombrable de distributions. Il en résulte ce que l'on nomme un *mélange continu*. La fonction de densité de probabilité $u(\theta)$ dans la définition ci-dessous joue le rôle des probabilités discrètes p_j de la **définition 2.1**.

Définition 2.2. Soit X une variable aléatoire dont la distribution dépend d'un paramètre θ . On considère ce paramètre comme une réalisation d'une variable aléatoire Θ avec fonction de densité $u(\theta)$. La fonction de densité de la variable aléatoire $X|\Theta = \theta$ est $f(x|\theta)$. Par la loi des probabilités totales, la fonction de densité marginale (ou non conditionnelle) de la variable aléatoire X est

$$f_X(x) = \int_{\Omega} f(x|\theta)u(\theta) d\theta.$$

Cette distribution est un mélange continu des distributions de $X|\Theta$ et Θ .

Les mélanges continus sont beaucoup utilisés en analyse bayésienne et dans certaines disciplines de l'actuariat. Plusieurs lois de probabilité sont uniquement définies en tant que mélanges.

L'algorithme ci-dessous, très simple, permet d'obtenir des observations aléatoires d'un mélange de distributions.

Algorithme 2.4 (Mélange continu de deux distributions). Soit $f(x)$ la fonction de densité de probabilité du mélange continu des distributions avec densités $f(x|\theta)$ et $u(\theta)$. Les étapes ci-dessous permettent de générer un nombre aléatoire x de la densité f_X .

1. Obtenir un nombre θ issu de la distribution avec densité $u(\theta)$.
2. Obtenir x de la distribution avec densité $f(x|\theta)$.

Pour simuler plusieurs observations d'un mélange, il faut répéter les deux étapes de l'algorithme 2.4 pour *chaque* observation. Le paramètre de mélange (θ) change donc à chaque fois. C'est essentiel, autrement il n'y aurait pas de mélange et nous obtiendrions simplement un échantillon de la distribution $f(x|\theta)$ pour une valeur de θ quelconque.

La simulation de mélanges continus s'avère très simple à réaliser en R grâce à la nature vectorielle des fonctions de simulation.

Exemple 2.7. Soit X une variable aléatoire issue du mélange $X|\Theta = \theta \sim \text{Poisson}(\theta)$ et $\Theta \sim \text{Gamma}(5, 4)$. Les deux expressions R ci-dessous permettent de générer 1 000 observations de X .

```
> theta <- rgamma(1000, 5, 4)
> x <- rpois(1000, theta)
```

Il est tout à fait raisonnable, ici, d'écrire le tout en une seule expression. Cela fait d'ailleurs bien ressortir le fait que le paramètre de la loi de Poisson est lui-même issu d'une simulation.

```
> x <- rpois(1000, rgamma(1000, 5, 4))
```

□

Il est possible de répéter la procédure de mélange plus d'une fois, ce qui donne lieu à ce que nous nommerons des *modèles hiérarchiques*¹. La généralisation de l'algorithme 2.4 pour les modèles hiérarchiques devrait aller de soi.

Exemple 2.8. Soit le modèle hiérarchique suivant où les variables aléatoires Θ et Λ représentent les paramètres de mélange :

$$\begin{aligned} X|\Lambda, \Theta &\sim \text{Poisson}(\Lambda) \\ \Lambda|\Theta &\sim \text{Gamma}(3, \Theta) \\ \Theta &\sim \text{Gamma}(2, 2). \end{aligned}$$

L'expression R suivante permet de générer n observations de la variable aléatoire X .

```
> rpois(n, rgamma(n, 3, rgamma(n, 2, 2)))
```

□

1. En sciences actuarielles, cette terminologie est surtout employée dans les contextes de structures de classification des risques sous forme d'arbre. Dans de tels cas, le nombre de valeurs à simuler pourrait différer à chaque niveau de l'arbre.



Étudiez les lignes 168–281 du fichier de script `simulation.R` reproduit à la section 2.9. Vous y trouverez une procédure de simulation d'un mélange discret, des exemples d'utilisation de la fonction `rmixture`, ainsi que des exemples de mélanges continus.

2.7.3 Distributions composées

Les distributions composées jouent un rôle central dans la modélisation des risques en actuariat puisqu'elles permettent de représenter séparément l'effet de la fréquence et de la sévérité des sinistres dans un portefeuille d'assurance.

Définition 2.3. Soit N une variable aléatoire discrète avec fonction de répartition $P_N(x)$ et $X_1, X_2, \dots$ des variables aléatoires mutuellement indépendantes, identiquement distribuées avec fonction de répartition $F_X(x)$ et indépendantes de la variable aléatoire N . La variable aléatoire $S = X_1 + \dots + X_N$ — avec, par convention, $S = 0$ lorsque $N = 0$ — a une distribution composée. Sa fonction de répartition est

$$F_S(x) = \sum_{n=0}^{\infty} F_X^{*n}(x) p_N(n),$$

où p_N est la fonction de masse de probabilité de N et F_X^{*n} est la fonction de répartition de la n^{e} convolution de X .

Dans l'appellation de la distribution de la variable aléatoire S , l'adjectif « composée » est accolé au nom de la distribution de la variable aléatoire N . Par exemple, si la distribution de N est une loi de Poisson, alors la distribution de S est une Poisson composée. C'est d'ailleurs là le cas le plus fréquent — et de loin — en modélisation des risques d'assurance.

L'algorithme suivant décrit la procédure de simulation d'observations d'une distribution composée.

Algorithme 2.5 (Distribution composée). Soit $S = X_1 + \dots + X_N$ une distribution composée. Les étapes ci-dessous permettent de générer une observation s de la variable aléatoire S .

1. Obtenir un nombre n de la distribution avec fonction de répartition P_N .
2. Obtenir des nombres $x_1, \dots, x_n$ de la distribution avec fonction de répartition F_X .
3. Poser $s = x_1 + \dots + x_n$.

Contrairement à la simulation des mélanges continus, il n'est pas possible, en R, de vectoriser la procédure de simulation de l'algorithme 2.5. Il faut avoir recours à une boucle ou, mieux, à une fonction d'application.

La fonction `rcompound` du paquetage **actuar** permet de générer facilement et intuitivement des nombres aléatoires d'une distribution composée quelconque. La fonction `rcomppois` offre une interface simplifiée pour le cas le plus courant de la Poisson composée. Enfin, bien que le sujet se trouve hors de la portée du présent document, mentionnons que la très générale fonction `rcomphierarc` permet de générer des observations de modèles hiérarchiques composés.



Le code du fichier de script `simulation.R` reproduit à la [section 2.9](#) fournit, aux lignes 283–348, deux mises en œuvre de l'algorithme 2.5 ainsi que des exemples d'utilisation des fonctions `rcomppois` et `rcompound`.



L'[annexe A](#) propose quelques trucs pour bien planifier une étude de simulation en R.

⚡ 2.8 Solution du problème

Nous adoptons la stratégie expliquée à l'astuce de la [page 30](#). La fonction R de la [figure 2.9](#) génère des points distribués uniformément sur un disque par la méthode d'acceptation-rejet en simulant d'abord des points sur un carré.

Avec en mains cette fonction, il devient simple de vérifier par simulation la probabilité $\frac{35}{12\pi^2} \approx 0,29552$ en procédant comme au [chapitre 1](#) :

```
> mean(replicate(1E5, 3 == length(chull(sim.disque(4)))))
[1] 0.29423
```

Une autre stratégie de simulation se révèle toutefois plus efficace. Elle repose sur la simulation des coordonnées polaires (rayon et angle) des points sur le disque, puis de leur transformation en coordonnées cartésiennes. Si un point du disque unité centré à l'origine se trouve à un rayon $r < 1$ du centre et à un angle θ de l'horizontale, alors ses coordonnées cartésiennes

```

sim.disque <- function(n)
{
  ## La fonction retourne les coordonnées des points
  ## dans une matrice de deux colonnes. Nous créons
  ## d'abord un contenant.
  X <- matrix(NA, n, 2)

  ## Remplissage de la matrice.
  i <- 1
  repeat
  {
    ## Simulation de coordonnées dans un carré
    ## 2 x 2 centré à l'origine.
    x <- runif(2, -1, 1)

    ## Si les coordonnées sont dans le disque...
    if (sum(x^2) < 1)
    {
      ## ... la paire est ajoutée à la matrice 'X'.
      X[i, ] <- x

      ## Il faut cesser après avoir accepté 'n' paires.
      if (n < (i <- i + 1))
        break
    }
  }
  X
}

```

FIG. 2.9 – Code d’une fonction pour simuler par la méthode d’acceptation-rejet des nombres uniformément sur un disque de rayon 1 centré à l’origine

sont

$$x = \sqrt{r} \cos \theta$$

$$y = \sqrt{r} \sin \theta.$$

La [figure 2.10](#) propose une mise en œuvre de cette méthode en R. La démonstration de la validité de cette méthode fait quant à elle l’objet de l’[exercice 2.8](#).

```

sim.disque2 <- function(n)
{
  ## Simulation des coordonnées polaires
  r <- runif(n)                # rayon
  angle <- runif(n, 0, 2 * pi) # angle

  ## Transformation en coordonnées cartésiennes
  sqrt(r) * cbind(cos(angle), sin(angle))
}

```

FIG. 2.10 – Code d'une fonction pour simuler des nombres uniformément sur un disque de rayon 1 centré à l'origine en passant par les coordonnées polaires

Parce qu'elle ne requiert aucune boucle, cette seconde méthode est *beaucoup* plus rapide que la méthode d'acceptation-rejet :

```

> system.time(sim.disque(1E5))
utilisateur      système      écoulé
      0.316       0.039       0.357
> system.time(sim.disque2(1E5))
utilisateur      système      écoulé
      0.009       0.000       0.010

```

2.9 Code informatique

📄 Fichier d'accompagnement `simulation.R`

```

12 ###
13 ### MÉTHODE DE L'INVERSE
14 ###
15
16 ## Simulation d'un échantillon aléatoire d'une distribution
17 ## exponentielle par la méthode de l'inverse.
18 lambda <- 5
19 x <- -log(runif(1000))/lambda
20
21 ## Pour faire une petite vérification, nous allons tracer
22 ## l'histogramme de l'échantillon et y superposer la véritable
23 ## densité d'une exponentielle de paramètre lambda. Les deux

```

```
24 ## graphiques devraient concorder.
25
26 ## Tracé de l'histogramme. Il faut spécifier l'option 'prob =
27 ## TRUE' pour que l'axe des ordonnées soit gradué en
28 ## probabilités plutôt qu'en nombre de données. Sinon, le
29 ## graphique de la densité que nous allons ajouter dans un
30 ## moment n'apparaîtra pas sur le graphique.
31 hist(x, prob = TRUE) # histogramme gradué en probabilités
32
33 ## Ajoutons maintenant la densité à l'aide de la très utile
34 ## fonction 'curve' qui permet de tracer une fonction f(x)
35 ## quelconque. Avec l'option 'add = TRUE', le graphique est
36 ## ajouté au graphique existant.
37 curve(dexp(x, rate = lambda), add = TRUE)
38
39 ###
40 ### MÉTHODE ACCEPTATION-REJET
41 ###
42
43 ## Nous effectuons une mise en oeuvre de l'algorithme
44 ## d'acceptation-rejet pour simuler des observations d'une
45 ## distribution Bêta(3, 2). La procédure est intrinsèquement
46 ## itérative, alors nous devons utiliser une boucle. Il y a
47 ## diverses manières de faire, j'ai ici utilisé une boucle
48 ## 'repeat'; une autre mise en oeuvre est présentée dans les
49 ## exercices.
50 ##
51 ## Remarquez que le vecteur contenant les résultats est
52 ## initialisé au début de la fonction pour éviter le Syndrome
53 ## de la plaque à biscuits expliqué dans «Programmer avec R»
54 ## du même auteur.
55 rbeta.ar <- function(n)
56 {
57     x <- numeric(n)      # initialisation du contenant
58     g <- function(x)     # fonction enveloppante
59         ifelse(x < 0.8, 2.5 * x, 10 - 10 * x)
60     Ginv <- function(x)  # l'inverse de son intégrale
61         ifelse(x < 0.8, sqrt(0.8 * x), 1 - sqrt(0.2 - 0.2 * x))
62
63     i <- 0                # initialisation du compteur
64     repeat
65     {
66         y <- Ginv(runif(1))
67         if (1.2 * g(y) * runif(1) <= dbeta(y, 3, 2))
68             x[i <- i + 1] <- y # assignation et incrément
```

```

69         if (i > n)
70             break          # sortir de la boucle repeat
71     }
72     x                      # retourner x
73 }
74
75 ## Vérification empirique pour voir si ça fonctionne.
76 x <- rbeta.ar(1000)
77 hist(x, prob = TRUE)
78 curve(dbeta(x, 3, 2), add = TRUE)
79
80 ###
81 ### FONCTIONS DE SIMULATION DANS R
82 ###
83
84 ## La fonction de base pour simuler des nombres uniformes est
85 ## 'runif'.
86 runif(10)                  # sur (0, 1) par défaut
87 runif(10, 2, 5)           # sur un autre intervalle
88 2 + 3 * runif(10)         # équivalent, moins lisible
89
90 ## R est livré avec plusieurs générateurs de nombres
91 ## aléatoires. Nous pouvons en changer avec la fonction
92 ## 'RNGkind'.
93 RNGkind("Wichmann-Hill")  # générateur de Excel
94 runif(10)                 # rien de particulier à voir
95 RNGkind("default")        # retour au générateur par défaut
96
97 ## La fonction 'set.seed' est très utile pour spécifier
98 ## l'amorce d'un générateur. Si deux simulations sont
99 ## effectuées avec la même amorce, nous obtiendrons exactement
100 ## les mêmes nombres aléatoires et, donc, les mêmes résultats.
101 ## Très utile pour répéter une simulation à l'identique.
102 set.seed(1)               # valeur sans importance
103 runif(5)                  # 5 nombres aléatoires
104 runif(5)                  # 5 autres nombres
105 set.seed(1)               # réinitialisation de l'amorce
106 runif(5)                  # les mêmes 5 nombres que ci-dessus
107
108 ## Plutôt que de devoir utiliser la méthode de l'inverse ou un
109 ## autre algorithme de simulation pour obtenir des nombres
110 ## aléatoires d'une loi de probabilité non uniforme, R fournit
111 ## des fonctions de simulation pour bon nombre de lois. Toutes
112 ## ces fonctions sont vectorielles. Ci-dessous, P == Poisson
113 ## et G == Gamma pour économiser sur la notation.

```

```
114 n <- 10                                # taille des échantillons
115 rbinom(n, 5, 0.3)                       # Binomiale(5, 0,3)
116 rbinom(n, 1, 0.3)                       # Bernoulli(0,3)
117 rnorm(n)                               # Normale(0, 1)
118 rnorm(n, 2, 5)                         # Normale(2, 25)
119 rpois(n, c(2, 5))                      # P(2), P(5), P(2), ..., P(5)
120 rgamma(n, 3, 2:11)                     # G(3, 2), G(3, 3), ..., G(3, 11)
121 rgamma(n, 11:2, 2:11)                  # G(11, 2), G(10, 3), ..., G(2, 11)
122
123 ## La fonction 'sample' sert à simuler un échantillon d'une
124 ## distribution discrète quelconque. Le premier argument est
125 ## le support de la distribution et le second, la taille de
126 ## l'échantillon. Par défaut, l'échantillonnage se fait sans
127 ## remise et avec des probabilités égales sur tout le support.
128 sample(1:49, 7)                         # numéros pour le 7/49
129 sample(1:10, 10)                       # permutation des nombres de 1 à 10
130 sample(1:10)                           # idem, plus simple
131 sample(10)                             # idem, encore plus simple!
132
133 ## Échantillonnage avec remise.
134 sample(1:10, 10, replace = TRUE)
135
136 ## Distribution de probabilités non uniforme.
137 x <- sample(c(0, 2, 5), 1000, replace = TRUE,
138             prob = c(0.2, 0.5, 0.3))
139 table(x)                               # tableau de fréquences
140
141 ## Insérons des données manquantes de manière aléatoire dans
142 ## le vecteur précédent. Pour ce faire, nous allons tirer
143 ## des positions dans le vecteur avec 'sample' pour ensuite
144 ## remplacer les valeurs à ces positions par 'NA'. Le support
145 ## de la «distribution» dans 'sample' est 1, 2, ..., 1000, une
146 ## suite que l'on génère facilement avec 'seq_along'.
147 x[sample(seq_along(x), 50)] <- NA # 50 données manquantes
148 sum(is.na(x))                         # vérification
149 table(x)                             # comparer avec ci-dessus
150
151 ## Le vecteur 'state.name' fourni avec R contient les noms des
152 ## cinquante États américains.
153 state.name
154
155 ## Sélection de 10 États au hasard en combinant indiciage et
156 ## fonction 'sample'.
157 state.name[sample(seq_along(state.name), 10)]
```

```
158
159 ## Le tableau de données 'quakes' fourni avec R contient les
160 ## positions de 1000 secousses sismiques de magnitude
161 ## supérieure à 4 autour des Fidji.
162 quakes
163
164 ## Sélection de 10 évènements au hasard.
165 quakes[sample(seq.int(nrow(quakes)), 10), ]
166
167 ###
168 ### MODÈLES ACTUARIELS
169 ###
170
171 ## Le paquetage actuar contient plusieurs fonctions de
172 ## simulation de modèles actuariels. Nous allons fournir des
173 ## exemples d'utilisation de ces fonctions dans la suite.
174 library(actuar)           # charger le paquetage
175
176 ## MÉLANGES DISCRETS
177
178 ## La clé pour simuler facilement un mélange discret en R
179 ## consiste à simuler toutes les observations de la première
180 ## loi, puis toutes celles de la seconde loi. Il suffit
181 ## ensuite d'en rebrasser l'ordre aléatoirement au cas où
182 ## l'ordre des observations simulées joue un rôle dans les
183 ## calculs (si ce n'est pas le cas, vous pouvez omettre cette
184 ## dernière étape).
185 ##
186 ## Reste à déterminer le nombre d'observations qui provient de
187 ## chaque loi. Pour chaque observation, la probabilité qu'elle
188 ## provienne de la première loi est p. Le nombre
189 ## d'observations provenant de la première loi suit donc une
190 ## distribution binomiale de paramètres n et p, où n est le
191 ## nombre total d'observations à simuler.
192 ##
193 ## Voici un exemple de simulation d'observations du mélange
194 ## discret de deux lois log-normales présenté dans le
195 ## chapitre. Les paramètres de la première loi sont 3,6 et
196 ## 0,6; ceux de la seconde loi sont 4,5 et 0,3. Le paramètre de
197 ## mélange est p = 0,55.
198 n <- 10000                # taille de l'échantillon
199 n1 <- rbinom(1, n, 0.55)  # quantité provenant de la loi 1
200 x <- c(rlnorm(n1, 3.6, 0.6),      # observations de la loi 1
201       rlnorm(n - n1, 4.5, 0.3)) # observations de la loi 2
202 x <- sample(x)             # rebrasser les observations
```



```
203 hist(x, prob = TRUE) # histogramme
204 curve(0.55 * dlnorm(x, 3.6, 0.6) +
205       0.45 * dlnorm(x, 4.5, 0.3),
206       add = TRUE, lwd = 2, col = "red3") # densité théorique
207
208 ## La fonction 'rmixture' du paquetage actuar offre une
209 ## interface conviviale pour simuler des observations de
210 ## mélanges discrets (avec un nombre quelconque de
211 ## distributions).
212 ##
213 ## La fonction compte trois arguments:
214 ##
215 ## 'n': nombre d'observations à simuler;
216 ## 'probs': vecteur de poids relatif de chaque modèle dans le
217 ##          mélange (normalisé pour sommer à 1);
218 ## 'models': vecteur d'expressions contenant les modèles
219 ##           formant le mélange.
220 ##
221 ## La méthode de formulation des modèles est commune à toutes
222 ## les fonctions de simulation de actuar. Il s'agit de
223 ## fournir, sous forme d'expression non évaluée, des appels à
224 ## des fonctions de simulation en omettant le nombre de
225 ## valeurs à simuler.
226 ##
227 ## Pour illustrer, reprenons l'exemple ci-dessus avec
228 ## 'rmixture'.
229 x <- rmixture(10000, probs = c(0.55, 0.45),
230             models = expression(rlnorm(3.6, 0.6),
231                                rlnorm(4.5, 0.3)))
232
233 ## Vérifions la validité de la fonction en superposant à
234 ## l'histogramme de l'échantillon la densité théorique du
235 ## mélange.
236 hist(x, prob = TRUE) # histogramme
237 curve(0.55 * dlnorm(x, 3.6, 0.6) +
238       0.45 * dlnorm(x, 4.5, 0.3),
239       add = TRUE, lwd = 2, col = "red3") # densité théorique
240
241 ## Simulation d'un mélange de deux exponentielles (de moyennes
242 ## 1/3 et 1/7) avec poids égaux. Le vecteur de poids est
243 ## recyclé automatiquement par 'rmixture'.
244 rmixture(10, 0.5, expression(rexp(3), rexp(7)))
245
246 ## MÉLANGES CONTINUS
247
```

```

248 ## La simulation des mélanges continus est simple à faire en R
249 ## puisque les fonctions r<loi> sont vectorielles. Il suffit
250 ## de simuler autant de paramètres de mélange que nous
251 ## souhaitons obtenir d'observations de la distribution
252 ## marginale.
253 ##
254 ## Reprenons ici l'exemple du texte, soit:
255 ##
256 ## X|Theta = theta ~ Poisson(theta)
257 ## Theta ~ Gamma(5, 4)
258 ##
259 ## D'abord en deux étapes.
260 theta <- rgamma(1000, 5, 4) # 1000 paramètres de mélange...
261 x <- rpois(1000, theta) # ... pour 1000 lois de Poisson
262
263 ## Nous pouvons écrire le tout en une seule expression.
264 x <- rpois(1000, rgamma(1000, 5, 4))
265
266 ## On peut démontrer (faites-le en exercice!) que la
267 ## distribution non conditionnelle de X est une binomiale
268 ## négative de paramètres 5 et  $4/(4 + 1) = 0,8$ .
269 ##
270 ## Vérifions ce résultat empiriquement en calculant d'abord le
271 ## tableau de fréquences des observations de l'échantillon
272 ## avec la fonction 'table', puis en traçant le graphique des
273 ## résultats. Il existe une méthode de 'plot' pour les
274 ## tableaux de fréquences.
275 (p <- table(x)) # tableau de fréquences
276 plot(p/length(x)) # graphique (fréquences relatives)
277
278 ## Ajoutons au graphique les masses de probabilités
279 ## théoriques.
280 (xu <- unique(x)) # valeurs distinctes de x
281 points(xu, dnbinom(xu, 5, 0.8), pch = 21, bg = "red3")
282
283 ## DISTRIBUTIONS COMPOSÉES
284
285 ## La simulation des distributions composées  $S = X_1 + \dots +$ 
286 ##  $X_N$  est une procédure intrinsèquement itérative puisque
287 ## pour chaque observation de la variable aléatoire S, il faut
288 ## d'abord connaître le nombre d'observations à simuler de la
289 ## variable aléatoire X, nombre obtenu de la distribution de
290 ## la variable aléatoire N.
291 ##
292 ## Nous présentons deux techniques «manuelles» avant

```

```
293 ## d'illustrer les fonctions 'rcomppois' et 'rcompound' du
294 ## paquetage actuar.
295 ##
296 ## Considérons le cas d'une Poisson composée. Le paramètre de
297 ## Poisson vaut 2 et la distribution de la variable aléatoire
298 ## X est une Gamma(2, 1).
299 ##
300 ## Une première approche repose sur la fonction 'sapply'. Elle
301 ## applique pour chaque observation de la loi de Poisson une
302 ## fonction anonyme qui, elle, génère les observations x_1,
303 ## ..., x_n et calcule la somme s = x_1 + ... + x_n.
304 sapply(rpois(10, 2), function(n) sum(rgamma(n, 2, 1)))
305
306 ## Une approche alternative consiste à générer les
307 ## observations de la loi de Poisson et toutes les
308 ## observations de la loi gamma, à séparer ces dernières en
309 ## catégories à l'aide de facteurs et à effectuer les sommes
310 ## par catégorie. Il faut explicitement tenir compte, dans
311 ## cette procédure, des cas où N = 0.
312 s <- numeric(10)           # contenant avec des 0
313 N <- rpois(10, 2)           # échantillon loi de Poisson
314 x <- rgamma(sum(N), 2, 1)    # échantillon loi gamma
315 f <- rep.int(seq_len(10), N) # facteurs
316 s[which(N != 0)] <- tapply(x, f, sum) # sommes par catégorie
317 s # résultats
318
319 ## La seconde approche est moins intuitive que la première,
320 ## mais elle s'avère plus simple à programmer de manière
321 ## générale et, de plus, elle est un peu plus rapide.
322 n <- 1e6
323 system.time(sapply(rpois(n, 2), function(n) sum(rgamma(n, 2, 1))))
324 system.time({
325   s <- numeric(n)
326   N <- rpois(n, 2)
327   x <- rgamma(sum(N), 2, 1)
328   f <- rep.int(seq_len(n), N)
329   s[which(N != 0)] <- tapply(x, f, sum);
330 s})
331
332 ## C'est l'approche utilisée par les fonctions 'rcomppois' et
333 ## 'rcompound' de actuar.
334 rcomppois
335
336 ## Nous pouvons simuler une Poisson composée de manière plus
337 ## intuitive avec la fonction 'rcomppois'. Nous spécifions le
```

```

338 ## modèle de variable aléatoire X comme dans 'rmixture'.
339 ## Puisqu'il n'y a qu'un modèle à spécifier, il n'est pas
340 ## nécessaire de le placer dans 'expression'.
341 rcomppois(10, lambda = 2, model.sev = rgamma(2, 1))
342
343 ## La fonction 'rcompound' permet de simuler des distributions
344 ## composées générales où la distribution de fréquence est
345 ## autre que la Poisson. Voici un exemple de binomiale
346 ## négative composée.
347 rcompound(10, model.freq = rnbinom(2, 0.2),
348           model.sev = rgamma(2, 1))

```

2.10 Exercices

2.1 La transformation de Box-Muller est populaire pour simuler des nombres normaux à partir de nombres uniformes. Soit $U_1 \sim U(0,1)$ et $U_2 \sim U(0,1)$ deux variables aléatoires indépendantes et

$$X_1 = (-2 \log U_1)^{1/2} \cos(2\pi U_2)$$

$$X_2 = (-2 \log U_1)^{1/2} \sin(2\pi U_2).$$

a) Vérifier de manière heuristique que la transformation ci-dessus est bijective de $\{(u_1, u_2); 0 < u_1 < 1, 0 < u_2 < 1\}$ à $\{(x_1, x_2); -\infty < x_1 < \infty, -\infty < x_2 < \infty\}$, c'est-à-dire qu'elle associe à un point (u_1, u_2) un et un seul point (x_1, x_2) .

b) Démontrer que la transformation inverse est

$$U_1 = e^{-(X_1^2 + X_2^2)/2}$$

$$U_2 = \frac{1}{2\pi} \arctan \frac{X_2}{X_1}.$$

c) Démontrer que X_1 et X_2 sont deux variables aléatoires indépendantes chacune distribuée selon une $N(0, 1)$.

d) Vérifier empiriquement la validité de ces formules à l'aide de Excel ou de R. Dans R, il est possible de transformer les nombres uniformes obtenus avec la fonction `runif` en nombres normaux sans même utiliser de boucles grâce à la fonction `outer` et deux fonctions anonymes.

2.2 La distribution de Laplace, ou double exponentielle, est définie comme la différence entre deux distributions exponentielles identiques et indépendantes. Sa fonction de densité de probabilité est

$$f(x) = \frac{\lambda}{2} e^{-\lambda|x|}, \quad -\infty < x < \infty.$$

Proposer une ou plusieurs façons de simuler des nombres issus de cette distribution.

- 2.3** En ouverture de chapitre, nous mentionnons qu'une façon de simuler des nombres issus d'une loi géométrique consiste à extraire la partie entière de nombres aléatoires provenant d'une loi exponentielle.

Démontrer que si la loi de la variable aléatoire X est une exponentielle, alors la loi de $\lfloor X \rfloor$ est une géométrique. Déterminer le paramètre de la loi géométrique en fonction du paramètre de la loi exponentielle.

- 2.4** Réaliser une mise en œuvre informatique de l'algorithme de simulation suivant. Il s'agit d'un algorithme pour simuler des observations d'une loi $\text{Gamma}(\alpha, 1)$, où $\alpha > 1$.

1. Générer u_1 et u_2 indépendamment d'une loi $U(0, 1)$ et poser

$$v = \frac{(\alpha - \frac{1}{6\alpha})u_1}{(\alpha - 1)u_2}.$$

2. Si

$$\frac{2(u_2 - 1)}{\alpha - 1} + v + \frac{1}{v} \leq 2,$$

alors retourner le nombre $x = (\alpha - 1)v$. Sinon, si

$$\frac{2 \log u_2}{\alpha - 1} - \log v + v \leq 1,$$

alors retourner le nombre $x = (\alpha - 1)v$.

3. Répéter au besoin la procédure depuis l'étape 1.

Faire les vérifications empiriques usuelles de la validité de l'algorithme.

- 2.5** En utilisant le résultat de l'exercice précédent, quelle procédure pourriez-vous suivre pour simuler des nombres d'une loi $\text{Gamma}(\alpha, \lambda)$ où $\alpha > 1$?

- 2.6** a) Démontrer que si $X|\Theta \sim \text{Exponentielle}(\Theta)$ et $\Theta \sim \text{Gamma}(\alpha, \lambda)$, alors $X \sim \text{Pareto}(\alpha, \lambda)$. La fonction de densité de probabilité d'une loi de Pareto est

$$f(x) = \frac{\alpha \lambda^\alpha}{(x + \lambda)^{\alpha+1}}, \quad x > 0.$$

- b) Utiliser le résultat ci-dessus pour proposer un algorithme de simulation de nombres issus d'une loi de Pareto. Faire la mise en œuvre informatique de cet algorithme et les vérifications d'usage de sa validité.

2.7 La fonction de densité de probabilité de la loi de Pareto translatée est

$$f(x) = \frac{\alpha \lambda^\alpha}{x^{\alpha+1}}, \quad x > \lambda.$$

Simuler trois valeurs d'une telle distribution avec $\alpha = 2$ et $\lambda = 1\,000$ à l'aide de la méthode de l'inverse et du générateur congruentiel linéaire suivant :

$$x_n = (65x_{n-1} + 1) \bmod 2\,048.$$

Utiliser une amorce de 12.

2.8 Soit U_1 et U_2 deux variables aléatoires indépendantes uniformément distribuées sur l'intervalle $(0, 1)$ et soit la transformation

$$X_1 = \sqrt{U_1} \cos(2\pi U_2)$$

$$X_2 = \sqrt{U_1} \sin(2\pi U_2).$$

Démontrer que la distribution conjointe de X_1 et X_2 est uniforme sur le disque de rayon de 1 centré en $(x_1, x_2) = (0, 0)$. À quoi ce résultat peut-il servir ?

2.9 a) Soit $Y_1 \sim \text{Gamma}(\alpha, 1)$ et $Y_2 \sim \text{Gamma}(\beta, 1)$ deux variables aléatoires indépendantes. Démontrer que

$$X = \frac{Y_1}{Y_1 + Y_2} \sim \text{Bêta}(\alpha, \beta).$$

b) Utiliser le résultat en a) pour proposer un algorithme de simulation d'observations d'une loi Bêta(α, λ).

c) Faire la mise en œuvre informatique de l'algorithme en b) ainsi que les vérifications d'usage.

2.10 a) Dans la méthode d'acceptation-rejet, un nombre y tiré d'une variable aléatoire Y avec fonction de densité de probabilité $g_Y(\cdot)$ est accepté comme réalisation d'une variable aléatoire X avec fonction de densité de probabilité $f_X(\cdot)$ si

$$U \leq \frac{f_X(y)}{c g_Y(y)},$$

où $U \sim U(0, 1)$. Calculer la probabilité d'accepter une valeur lors de toute itération de la méthode d'acceptation-rejet, c'est-à-dire

$$\Pr \left[U \leq \frac{f_X(Y)}{c g_Y(Y)} \right].$$

Astuce : utiliser la loi des probabilités totales en conditionnant sur $Y = y$.

- b) Déterminer la distribution du nombre d'essais avant d'accepter un nombre y dans la méthode d'acceptation-rejet.
- c) Déterminer le nombre moyen d'essais avant d'accepter un nombre y dans la méthode d'acceptation-rejet.

2.11 Soit X une variable aléatoire continue définie sur l'intervalle (a, b) , où a et b sont des nombres réels. Pour simuler des observations de cette variable aléatoire par la méthode d'acceptation-rejet, il est toujours possible d'inscrire la fonction de densité de probabilité de X dans un rectangle de hauteur M , où M est la valeur de la densité au mode de X .

- a) Énoncer l'algorithme d'acceptation-rejet découlant d'une telle procédure.
- b) Calculer l'*efficacité* de l'algorithme en a), soit la probabilité d'accepter une valeur lors d'une itération de l'algorithme.

2.12 Effectuer les calculs ci-dessous dans le contexte de la simulation d'observations d'une loi Bêta(3, 2) à l'aide de la méthode d'acceptation-rejet.

- a) Calculer l'efficacité de l'algorithme développé à l'[exercice 2.11](#) et à l'[exemple 2.4](#) pour le cas présent.
- b) Calculer l'efficacité de l'algorithme développé dans l'[exemple 2.5](#), où nous avons déterminé que

$$f_X(x) \leq \begin{cases} 3x, & 0 < x < 0,8 \\ 12 - 12x, & 0,8 \leq x < 1. \end{cases}$$

- c) Faire la mise en œuvre informatique de l'algorithme le plus efficace entre celui de la partie a) et celui de la partie b). Vérifier la fonction en superposant l'histogramme d'un grand échantillon obtenu avec cette fonction et la vraie fonction de densité de la loi bêta.

2.13 La fonction R de la [figure 2.11](#) permet de simuler des observations de la distribution Bêta(α, β).

- a) Identifier le type d'algorithme utilisé dans cette fonction.
- b) Vous disposez des valeurs suivantes, obtenues dans R .

```
> set.seed(12345)
> runif(10)
[1] 0.72 0.88 0.76 0.89 0.46 0.17 0.33 0.51 0.73
[10] 0.99
```

```

simul <- function(n, alpha, beta)
{
  xmax <- (alpha - 1)/(alpha + beta - 2)
  M <- dbeta(xmax, alpha, beta)
  x <- numeric(n)
  i <- 0
  repeat
  {
    u <- runif(1)
    if (M * runif(1) <= dbeta(u, alpha, beta))
      x[i <- i + 1] <- u
    if (i == n)
      break
  }
  x
}

```

FIG. 2.11 – Fonction de simulation d'une loi Bêta(α, β) pour l'exercice 2.13

Évaluer le résultat des expressions ci-dessous.

```

> set.seed(12345)
> simul(2, alpha = 2, beta = 3)

```

2.14 a) Démontrer que, si $0 < \alpha < 1$,

$$x^{\alpha-1}e^{-x} \leq \begin{cases} x^{\alpha-1}, & 0 \leq x \leq 1 \\ e^{-x}, & x > 1. \end{cases}$$

b) Développer un algorithme d'acceptation-rejet pour simuler des observations d'une loi Gamma($\alpha, 1$), $0 < \alpha < 1$ à partir du résultat en a).

2.15 Soit l'inégalité suivante, valide pour $\alpha \geq 1$:

$$x^{\alpha-1}e^{-x} \leq \alpha^{\alpha-1}e^{-x/\alpha+1-\alpha}, \quad x > 0.$$

Utiliser cette inégalité pour justifier l'algorithme d'acceptation-rejet ci-dessous servant à simuler des observations d'une loi Gamma($\alpha, 1$) avec $\alpha \geq 1$.

1. Simuler deux observations indépendantes v_1 et v_2 d'une loi exponentielle de moyenne 1.
2. Si $v_2 < (\alpha - 1)(v_1 - \ln v_1 - 1)$, poser $x = \alpha v_1$. Sinon, retourner à l'étape 1.

2.16 Dans la mise en œuvre de l'algorithme 2.3 de simulation d'un mélange discret présentée à la section 2.9, nous générons un nombre d'une distribution binomiale pour déterminer la quantité de nombres à simuler de la première distribution du mélange (et, conséquemment, de celle de la seconde distribution). Généraliser le concept pour simuler des observations d'un mélange discret de $k > 2$ distributions. *Astuce* : consulter le code source de la fonction `rmixture`.

2.17 Écrire des expressions R pour simuler des observations des modèles ci-dessous d'abord sans, puis avec, les fonctions dédiées du paquetage **actuar**, le cas échéant.

- a) Variable aléatoire dont la fonction de densité est $f(x) = 0,3f_1(x) + 0,7f_2(x)$, où f_1 est la densité d'une distribution exponentielle de moyenne 1/2 et f_2 est la densité d'une distribution gamma de paramètre $\alpha = 2$ et de moyenne 6.
- b) Mélange $f(x) = 0,5647I\{x = 0\} + 0,4353(2^x e^{-2}/x!)$, où $I\{\mathcal{A}\}$ est une fonction indicatrice qui vaut 1 lorsque la condition $\mathcal{A}$ est vraie et 0 sinon.
- c) Variable aléatoire X définie ainsi : $X|\Theta = \theta \sim \text{Poisson}(\theta)$, $\Theta \sim \text{Inverse gaussienne}(2, 1)$. La fonction de densité de probabilité de la distribution inverse gaussienne de moyenne μ et de dispersion ϕ est

$$f(x) = \left(\frac{1}{2\pi\phi x^3} \right)^{1/2} \exp \left(-\frac{(x - \mu)^2}{2\mu^2\phi x} \right), \quad x \geq 0, \mu > 0, \phi > 0.$$

Astuce : la fonction `rinvgauss` du paquetage **actuar** permet de simuler des observations de la distribution inverse gaussienne.

- d) Distribution composée $S = X_1 + \dots + X_N$ avec $N \sim \text{Poisson}(2)$ et $X \sim \text{Log-normale}(7, 1)$.
- e) Binomiale négative composée où les paramètres de la binomiale négative sont $r = 2$ et $p = 0,3$, et la distribution de la variable aléatoire X est une log-normale de paramètres $(7, 1)$.

Réponses

2.3 $p = 1 - e^{-\lambda}$

2.7 1 271, 2 172, 1 137

2.10 a) $1/c$ b) Géométrique($1/c$) commençant à 1 c) c

2.11 a) $1/(M(b - a))$

2.12 a) $9/16$ b) $4/5$

2.13 b) (0,46, 0,33)

3 Intégration Monte Carlo

Objectifs du chapitre

- ▶ Définir le principe du calcul de la valeur d'une intégrale définie à l'aide de nombres aléatoires.
- ▶ Calculer la valeur d'une intégrale définie par la méthode Monte Carlo.

On appelle simulation Monte Carlo (ou méthode de Monte Carlo) toute méthode consistant à résoudre une expression mathématique à l'aide de nombres aléatoires. Par extension, l'appellation est souvent utilisée pour référer à toute utilisation de la simulation.

L'une des utilisations de la simulation Monte Carlo la plus répandue est le calcul d'intégrales, principalement pour des dimensions supérieures à un. Nous ne présentons ici que l'idée générale.



3.1 Énoncé du problème

L'intégrale de Gauss

$$\int e^{-x^2} dx$$

intervient dans plusieurs disciplines des mathématiques. En théorie des probabilités, on la retrouve à la base de la définition de la loi normale, ou loi gaussienne. Sans primitive, elle sert souvent, en calcul intégral, comme exemple d'intégrale définie qu'il faut résoudre par la transformation du domaine en coordonnées polaires.

Nous savons que

$$G = \int_{-\infty}^{\infty} e^{-x^2} dx = \sqrt{\pi}.$$

Comment pourrions-nous vérifier ce résultat à l'aide de la simulation ?

3.2 Contexte

Nous souhaitons calculer l'intégrale

$$\theta = \int_a^b h(x) dx, \quad (3.1)$$

mais celle-ci est trop compliquée pour être évaluée explicitement. Nous pourrions d'abord faire deux approximations de l'aire sous la fonction h à l'aide des sommes de Riemann

$$R_n^- = \sum_{k=0}^{n-1} h(a + k\Delta x) \Delta x$$

et

$$R_n^+ = \sum_{k=1}^n h(a + k\Delta x) \Delta x,$$

où $\Delta x = (b - a)/n$ et n est grand. Voir la [figure 3.1](#) pour une illustration de ces deux aires.

Par la suite, une estimation numérique de l'intégrale serait la moyenne des deux sommes de Riemann :

$$\hat{\theta}_n = \frac{R_n^- + R_n^+}{2}.$$

Cette procédure devient toutefois rapidement compliquée pour les intégrales multiples à plusieurs dimensions. Il y a éventuellement beaucoup trop de points à évaluer.

3.3 Méthode Monte Carlo

L'idée de l'intégration Monte Carlo consiste à évaluer la fonction à intégrer en des points choisis aléatoirement, puis à s'en remettre à la Loi des grands nombres pour estimer l'intégrale.

Nous exprimons tout d'abord la fonction $h(x)$ sous la forme

$$h(x) = g(x)f(x),$$

où $f(x)$ est une densité sur (a, b) . Ainsi, par définition de l'espérance :

$$\begin{aligned} \theta &= \int_a^b g(x)f(x) dx \\ &= E[g(X)], \end{aligned}$$

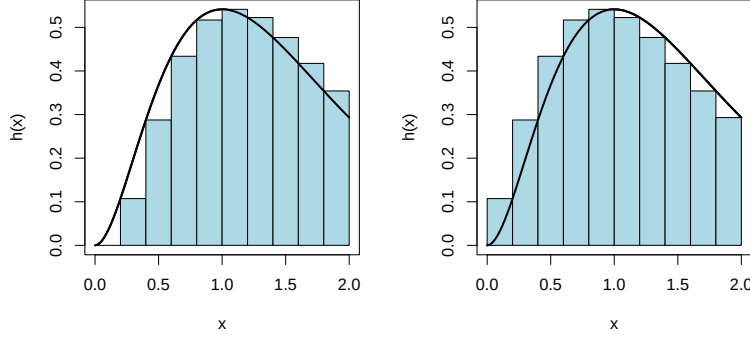


FIG. 3.1 – Approximation de l'aire sous une fonction par les deux sommes de Riemann R_n^- (gauche) et R_n^+ (droite)

où X est la variable aléatoire avec fonction de densité de probabilité $f(x)$. Par la suite, si $x_1, \dots, x_n$ sont des observations simulées de la densité $f(x)$, alors

$$\hat{\theta}_n = \frac{1}{n} \sum_{i=1}^n g(x_i) \quad (3.2)$$

constitue une estimation de l'intégrale θ . Par la Loi des grands nombres, $\hat{\theta}_n \xrightarrow{n \rightarrow \infty} \theta$.

Par souci de simplicité et parce qu'il est facile d'obtenir un échantillon aléatoire d'une loi uniforme, nous posons en général $X \sim U(a, b)$, soit

$$f(x) = \frac{1}{b-a}, \quad a < x < b.$$

Dans ce cas, $g(x) = (b-a)h(x)$ et l'intégrale de départ (3.1) se réécrit

$$\begin{aligned} \theta &= (b-a) \int_a^b h(x) \frac{1}{b-a} dx \\ &= (b-a)E[h(X)]. \end{aligned} \quad (3.3)$$

L'estimation (3.2) de l'intégrale devient alors

$$\hat{\theta}_n = \frac{b-a}{n} \sum_{i=1}^n h(x_i),$$

où $x_1, \dots, x_n$ est un échantillon aléatoire d'une loi $U(a, b)$.

Vous remarquerez qu'il est équivalent d'effectuer le changement de variable

$$\begin{aligned} u &= \frac{x-a}{b-a} & \Leftrightarrow & & x &= a + (b-a)u \\ du &= \frac{dx}{b-a} & & & dx &= (b-a) du \end{aligned}$$

dans l'intégrale (3.3). On obtient alors

$$\begin{aligned} \theta &= (b-a) \int_0^1 h(a + (b-a)u) (1) du \\ &= (b-a) E[h(a + (b-a)U)], \end{aligned}$$

où $U \sim U(0, 1)$. Une estimation de l'intégrale est donc

$$\hat{\theta}_n = \frac{b-a}{n} \sum_{i=1}^n h(a + (b-a)u_i),$$

où $u_1, \dots, u_n$ est un échantillon aléatoire d'une loi $U(0, 1)$. Nous devons utiliser la technique du changement de variable lorsque le domaine est infini.

Exemple 3.1. Soit l'intégrale

$$\theta = \int_2^5 x^{11/5} e^{-x/10} dx.$$

Parce que l'exposant de x n'est pas un entier, nous ne pouvons évaluer cette intégrale par parties. Cependant, nous remarquons que la fonction à intégrer est, à une constante près, la densité d'une loi gamma :

$$\begin{aligned} \theta &= \int_2^5 x^{11/5} e^{-x/10} dx \\ &= \frac{\Gamma\left(\frac{16}{5}\right)}{\left(\frac{1}{10}\right)^{16/5}} \int_2^5 \frac{\left(\frac{1}{10}\right)^{16/5}}{\Gamma\left(\frac{16}{5}\right)} x^{16/5-1} e^{-x/10} dx \\ &= \frac{\Gamma\left(\frac{16}{5}\right)}{\left(\frac{1}{10}\right)^{16/5}} \left[G\left(5; \frac{16}{5}, \frac{1}{10}\right) - G\left(2; \frac{16}{5}, \frac{1}{10}\right) \right], \end{aligned}$$

où $G(x; \alpha, \lambda)$ est la fonction de répartition de la loi $\text{Gamma}(\alpha, \lambda)$. R nous fournit la valeur exacte de l'intégrale.

```
> gamma(3.2) * diff(pgamma(c(2, 5), 3.2, 0.1)) / 0.1^3.2
[1] 34.51998
```

Pour utiliser la méthode Monte Carlo, nous posons

$$\theta = 3 \int_2^5 x^{11/5} e^{-x/10} \left(\frac{1}{3}\right) dx.$$

Si $\{x_1, \dots, x_n\}$ est un échantillon aléatoire d'une loi $U(2, 5)$, alors

$$\hat{\theta}_n = \frac{3}{n} \sum_{i=1}^n x_i^{11/5} e^{-x_i/10}$$

est une estimation de θ . Nous obtenons les résultats suivants avec R (voir aussi la [figure 3.2](#)).

```
> f <- function(x) x^2.2 * exp(-x/10)
> x <- runif(1e2, 2, 5)
> 3 * mean(f(x))
[1] 33.52802
> x <- runif(1e3, 2, 5)
> 3 * mean(f(x))
[1] 34.83705
> x <- runif(1e4, 2, 5)
> 3 * mean(f(x))
[1] 34.26627
> x <- runif(1e5, 2, 5)
> 3 * mean(f(x))
[1] 34.58616
> x <- runif(1e6, 2, 5)
> 3 * mean(f(x))
[1] 34.5068
```



Étudiez le code R pour effectuer les calculs et les graphiques ci-dessus aux lignes 13-47 du fichier de script `montecarlo.R` reproduit à la [section 3.7](#).

□

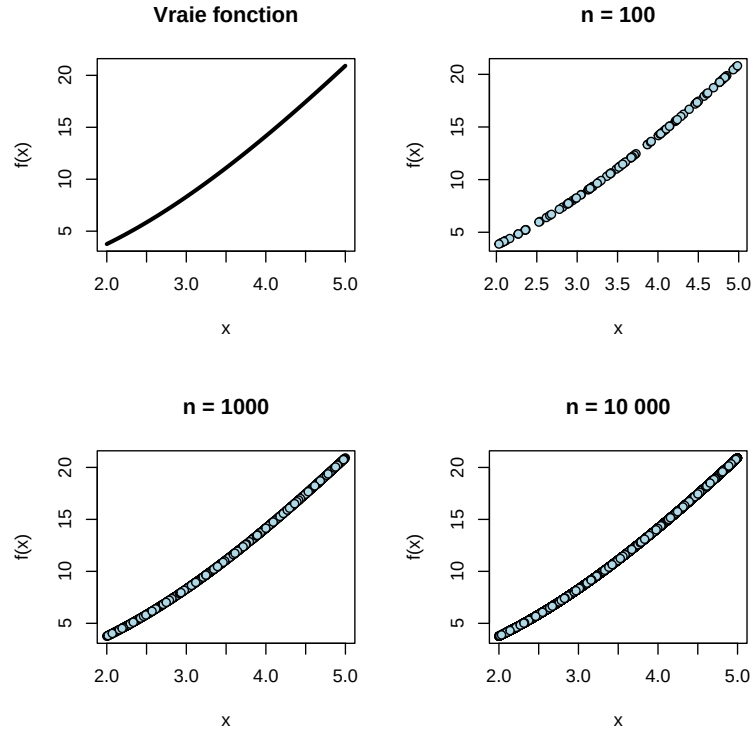


FIG. 3.2 – Fonction $h(x)$ de l'exemple 3.1 et points choisis aléatoirement où la fonction est évaluée pour l'intégration Monte Carlo



3.4 Indice pour le problème

La présentation ci-dessus repose sur un domaine d'intégration fini. Or, celui de l'intégrale à calculer est infini. Pour utiliser des nombres aléatoires uniformes dans la méthode Monte-Carlo, il faut ramener le domaine d'intégration infini à un domaine fini par le biais d'un changement de variable.

Tout d'abord, par symétrie de l'intégrande,

$$\begin{aligned}
 G &= \int_{-\infty}^{\infty} e^{-x^2} dx \\
 &= 2 \int_0^{\infty} e^{-x^2} dx \\
 &= 2 \left(\int_0^1 e^{-x^2} dx + \int_1^{\infty} e^{-x^2} dx \right).
 \end{aligned}$$

Ensuite, plusieurs changements de variable sont possibles pour transformer le domaine de la seconde intégrale de droite. Nous choisissons le changement de variable $u = x^{-1}$. Le domaine d'intégration subit donc la transformation $(1, \infty) \mapsto (1, 0)$. Ainsi,

$$\begin{aligned} G &= 2 \left(\int_0^1 e^{-x^2} dx + \int_1^0 e^{-u^{-2}} (-u^{-2}) du \right) \\ &= 2 \left(\int_0^1 e^{-x^2} dx + \int_0^1 e^{-u^{-2}} u^{-2} du \right) \\ &= 2 \int_0^1 (e^{-x^2} + e^{-x^{-2}} x^{-2}) dx. \end{aligned}$$

Exprimée ainsi, l'intégrale se prête à la méthode d'intégration Monte Carlo.

3.5 Intégrales multiples

La procédure à suivre avec les intégrales multiples est la même qu'avec les intégrales simples, sauf que nous tirons des points uniformément dans autant de dimensions que nécessaire.

Exemple 3.2. Soit l'intégrale

$$\theta = \int_0^{5/4} \int_0^{5/4} \sqrt{4 - x^2 - y^2} dy dx.$$

Posons :

$$\begin{aligned} \theta &= \int_0^{5/4} \int_0^{5/4} \sqrt{4 - x^2 - y^2} dy dx \\ &= \frac{25}{16} \int_0^{5/4} \int_0^{5/4} \sqrt{4 - x^2 - y^2} \frac{1}{\left(\frac{5}{4}\right) \left(\frac{5}{4}\right)} dy dx \\ &= \frac{25}{16} E[\sqrt{4 - X^2 - Y^2}], \end{aligned}$$

où X et Y sont des variables aléatoires indépendantes distribuées uniformément sur l'intervalle $(0, \frac{5}{4})$. Autrement dit, la densité conjointe de X et Y est uniforme sur $(0, \frac{5}{4}) \times (0, \frac{5}{4})$. Par conséquent, une estimation de θ calculée à partir d'un échantillon $\{(x_1, y_1), \dots, (x_n, y_n)\}$ tiré de cette loi conjointe est

$$\hat{\theta}_n = \frac{25}{16n} \sum_{i=1}^n \sqrt{4 - x_i^2 - y_i^2}.$$

Voici quelques résultats obtenus avec R (voir aussi la [figure 3.3](#)).

```

> u <- runif(1e2, 0, 1.25)
> v <- runif(1e2, 0, 1.25)
> mean(sqrt(4 - u^2 - v^2)) * 1.25^2
[1] 2.624937
> u <- runif(1e3, 0, 1.25)
> v <- runif(1e3, 0, 1.25)
> mean(sqrt(4 - u^2 - v^2)) * 1.25^2
[1] 2.646307
> u <- runif(1e4, 0, 1.25)
> v <- runif(1e4, 0, 1.25)
> mean(sqrt(4 - u^2 - v^2)) * 1.25^2
[1] 2.66722

```



Étudiez le code R pour effectuer les calculs et les graphiques ci-dessus aux lignes [50-110](#) du fichier de script `montecarlo.R` reproduit à la [section 3.7](#).

□

⚡ 3.6 Solution du problème

En posant

$$\begin{aligned}\theta &= 2 \int_0^1 (e^{-x^2} + e^{-x^{-2}} x^{-2}) dx \\ &= 2E[e^{-U^2} + e^{-U^{-2}} U^{-2}],\end{aligned}$$

où $U \sim U(0, 1)$, nous avons réécrit l'intégrale de Gauss comme l'espérance d'une transformation d'une variable aléatoire uniforme. Par la Loi des grands nombres, une bonne estimation de θ est

$$\hat{\theta}_n = \frac{2}{n} \sum_{i=1}^n (e^{-u_i^2} + e^{-u_i^{-2}} u_i^{-2})$$

pour n grand et où $u_1, \dots, u_n$ sont des nombres aléatoires distribués uniformément sur l'intervalle $(0, 1)$.

Le calcul est ensuite très simple à faire avec R.

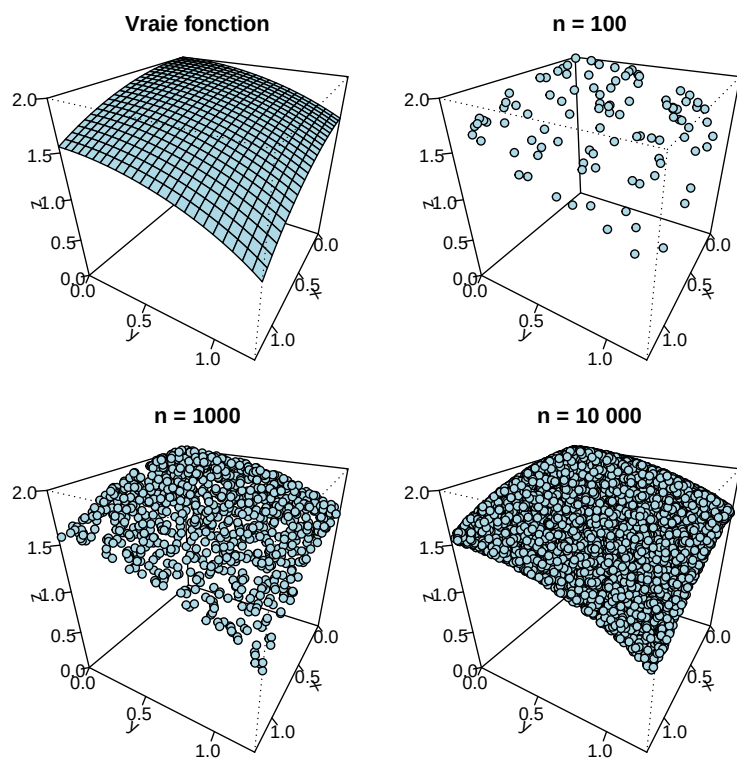


FIG. 3.3 – Fonction $h(x)$ de l'exemple 3.2 et points choisis aléatoirement où la fonction est évaluée pour l'intégration Monte Carlo

```
> u <- runif(1E5)
> 2 * (mean(exp(-u^2) + exp(-u^(-2)) * u^(-2)))
[1] 1.773378
```

Ce résultat est suffisamment près de $\sqrt{\pi} = 1,772454$ pour juger l'expérience concluante.

3.7 Code informatique

📄 Fichier d'accompagnement montecarlo.R

```
12 ###
13 ### INTÉGRALE SIMPLE
14 ###
```

```

15
16 ## Estimations successives de l'intégrale par la méthode Monte
17 ## Carlo avec des échantillons de plus en plus grands.
18 f <- function(x) x^2.2 * exp(-x/10)
19 x <- runif(1e2, 2, 5)
20 3 * mean(f(x))
21 x <- runif(1e3, 2, 5)
22 3 * mean(f(x))
23 x <- runif(1e4, 2, 5)
24 3 * mean(f(x))
25 x <- runif(1e5, 2, 5)
26 3 * mean(f(x))
27 x <- runif(1e6, 2, 5)
28 3 * mean(f(x))
29
30 ## Traçons des graphiques de la vraie fonction à intégrer et
31 ## de trois évaluations par la méthode Monte Carlo.
32 op <- par(mfrow = c(2, 2)) # 4 graphiques en grille 2 x 2
33 curve(f(x), xlim = c(2, 5), lwd = 2, main = "Vraie fonction")
34
35 x <- runif(1e2, 2, 5)
36 plot(x, f(x), main = "n = 100",
37       pch = 21, bg = "lightblue")
38
39 x <- runif(1e3, 2, 5)
40 plot(x, f(x), main = "n = 1000",
41       pch = 21, bg = "lightblue")
42
43 x <- runif(1e4, 2, 5)
44 plot(x, f(x), main = "n = 10 000",
45       pch = 21, bg = "lightblue")
46
47 par(op) # revenir aux paramètres par défaut
48
49 ###
50 ### INTÉGRALE DOUBLE
51 ###
52
53 ## Estimations successives de l'intégrale double par la
54 ## méthode Monte Carlo avec des échantillons de plus en plus
55 ## grands.
56 u <- runif(1e2, 0, 1.25)
57 v <- runif(1e2, 0, 1.25)
58 mean(sqrt(4 - u^2 - v^2)) * 1.25^2
59 u <- runif(1e3, 0, 1.25)

```

```
60 v <- runif(1e3, 0, 1.25)
61 mean(sqrt(4 - u^2 - v^2)) * 1.25^2
62 u <- runif(1e4, 0, 1.25)
63 v <- runif(1e4, 0, 1.25)
64 mean(sqrt(4 - u^2 - v^2)) * 1.25^2
65
66 ## Graphiques de la vraie fonction et des points où celle-ci
67 ## est évaluée avec la méthode Monte Carlo. Pour faire un
68 ## graphique en trois dimensions, nous pouvons utiliser la
69 ## fonction 'persp'.
70 op <- par(mfrow = c(2, 2), mar = c(1, 1, 2, 1))
71 f <- function(x, y) sqrt(4 - x^2 - y^2)
72 x <- seq(0, 1.25, length = 25)
73 y <- seq(0, 1.25, length = 25)
74 persp(x, y, outer(x, y, f), main = "Vraie fonction",
75       zlim = c(0, 2), theta = 120, phi = 30, col = "lightblue",
76       zlab = "z", ticktype = "detailed")
77
78 ## Pour faire les trois autres graphiques, nous allons d'abord
79 ## créer des graphiques en trois dimensions vides, puis y
80 ## ajouter des points avec la fonction 'points'. La fonction
81 ## 'trans3d' sert à convertir les coordonnées des points dans
82 ## la projection utilisée par 'persp'.
83 u <- runif(1e2, 0, 1.25)
84 v <- runif(1e2, 0, 1.25)
85 res <- persp(x, y, matrix(NA, length(x), length(y)),
86             main = "n = 100",
87             zlim = c(0, 2), theta = 120, phi = 30,
88             zlab = "z", ticktype = "detailed")
89 points(trans3d(u, v, f(u, v), pm = res),
90       pch = 21, bg = "lightblue")
91
92 u <- runif(1e3, 0, 1.25)
93 v <- runif(1e3, 0, 1.25)
94 res <- persp(x, y, matrix(NA, length(x), length(y)),
95             main = "n = 1000",
96             zlim = c(0, 2), theta = 120, phi = 30,
97             zlab = "z", ticktype = "detailed")
98 points(trans3d(u, v, f(u, v), pm = res),
99       pch = 21, bg = "lightblue")
100
101 u <- runif(1e4, 0, 1.25)
102 v <- runif(1e4, 0, 1.25)
103 res <- persp(x, y, matrix(NA, length(x), length(y)),
104             main = "n = 10 000",
```

```

105         zlim = c(0, 2), theta = 120, phi = 30,
106         zlab = "z", ticktype = "detailed")
107 points(trans3d(u, v, f(u, v), pm = res),
108        pch = 21, bg = "lightblue")
109
110 par(op)                                # revenir aux paramètres par défaut

```

3.8 Exercices

3.1 Évaluer l'intégrale

$$\int_0^1 \ln(5x + 4) dx$$

exactement ainsi qu'à l'aide de l'intégration Monte Carlo. Comparer les réponses.

3.2 Évaluer l'intégrale

$$\int_0^1 \int_0^1 e^{2xy} \ln(3x + y^2) dx dy$$

à l'aide de l'intégration Monte Carlo. Comparer la réponse obtenue avec la vraie valeur, 1,203758, obtenue à l'aide de Maple.

3.3 Soit l'intégrale

$$\theta = \int_0^\infty x^2 \sin(\pi x) e^{-x/2} dx.$$

- Évaluer cette intégrale par Monte Carlo en effectuant un changement de variable.
- Évaluer cette intégrale par Monte Carlo par échantillonnage direct d'une loi de probabilité appropriée.

3.4 Soit $X_1, \dots, X_{25}$ un échantillon aléatoire de taille 25 d'une distribution $N(0, 1)$ et soit $X_{(1)} \leq \dots \leq X_{(25)}$ les statistiques d'ordre de cet échantillon, c'est-à-dire les données de l'échantillon triées en ordre croissant. Estimer $E[X_{(5)}]$ par intégration Monte Carlo. Comparer la réponse obtenue avec la vraie espérance, $-0,90501$.

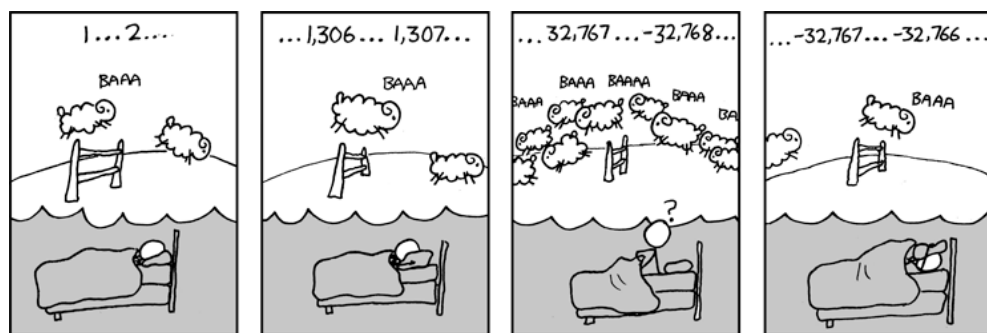
Réponses

3.1 Valeur exacte : 1,845969

3.3 Valeur exacte : $-0,055292$

Deuxième partie

Analyse numérique



Tiré de XKCD.com

4 Arithmétique des ordinateurs

Objectifs du chapitre

- ▶ Concevoir l'effet de la représentation interne des nombres dans un ordinateur sur la précision des calculs.
- ▶ Convertir un nombre décimal vers une base quelconque, et vice versa.
- ▶ Exprimer un nombre décimal dans la représentation interne d'un ordinateur selon la norme IEEE 754.
- ▶ Planifier ses calculs avec un ordinateur de manière à diminuer l'impact des erreurs d'arrondis et de troncature et optimiser le temps de calcul.



4.1 Énoncé du problème

Les étudiants d'actuariat utilisent pour la plupart la calculatrice TI-30XS MultiView de Texas Instruments puisque c'est un modèle agréé pour les examens des organismes professionnels.

Peut-on se fier aux résultats de cette calculatrice ? Effectuons un calcul simple pour une valeur de x donnée :

$$(x - 1)(x + 1) - x^2 + 1.$$

Le résultat devrait bien entendu être égal à zéro, et ce, peu importe la valeur de x .

Si l'on fait le calcul avec $x = 10^6$, le résultat est bien 0.

Si l'on fait le calcul avec $x = 10^8$, le résultat est maintenant de... 1.

Que s'est-il passé ? Doit-on douter de la qualité de la calculatrice ?

4.2 Les ordinateurs ne savent pas compter

Le type de bogue le plus fréquemment rapporté dans les forums de discussion de R a trait au fait que le logiciel ne retourne pas le bon résultat lors

d'opérations arithmétiques simples.



Étudiez le code des lignes 13-42 du fichier de script `arithmetique_ordinateurs.R` reproduit à la [section 4.10](#). Il contient des exemples de calculs simples que R n'arrive apparemment pas à effectuer correctement. Les exemples sont tous tirés de véritables messages envoyés à la liste de discussion [r-help](#).

On devine que les créateurs de R n'ont pas négligé une fonctionnalité aussi fondamentale pour un langage mathématique que le calcul arithmétique. Les supposées erreurs ci-dessus relèvent toutes de la représentation interne des nombres dans un ordinateur et d'erreurs d'arrondi inhérentes aux opérations arithmétiques avec ces nombres. En effet, un des plus célèbres principes de programmation proposés par [Kernighan et Plauger \(1978\)](#) est :

10,0 fois 0,1 ne donne jamais vraiment 1,0.

D'ailleurs, les auteurs des faux rapports de bogues dans `r-help` sont invariablement renvoyés à l'[entrée 7.31](#) de la foire aux questions de R.

Quiconque utilise un ordinateur pour le calcul scientifique devrait connaître les grands principes du fonctionnement interne d'un ordinateur afin, entre autres :

- ▶ d'éviter les erreurs d'arrondi et de troncature ;
- ▶ d'éviter les dépassements et soupassements de capacité (*overflow* et *underflow*) ;
- ▶ d'optimiser le code informatique.

Ce chapitre se penche sur les grands principes de l'arithmétique des ordinateurs. Quoique, en fait, les ordinateurs ne savent pas faire d'arithmétique à proprement parler. Ils ne connaissent que deux états : ouvert (1) et fermé (0). Nous en profiterons donc d'abord pour réviser la conversion des nombres décimaux vers, et de, n'importe quelle base. Après avoir, à la [section 4.4](#), introduit les principales unités de mesure en informatique, nous expliquerons la représentation interne des nombres dans un ordinateur en nous concentrant sur la double précision. Nous pourrons ainsi justifier que les ordinateurs ne peuvent représenter qu'un sous-ensemble des nombres réels, d'où les erreurs d'arrondi et de troncature. La [section 4.6](#) explique comment ces erreurs surviennent. On y donne également quelques pistes pour éviter ou pour diminuer l'impact de ces erreurs. Enfin, le chapitre se

clôt par un survol d'un sujet quelque peu périphérique : la représentation interne des caractères.



Avant d'entamer la lecture du chapitre, je vous recommande de visionner une vidéo d'introduction qui offre un survol du sujet de l'**arithmétique des ordinateurs** [↗](#).

4.3 Conversion de base

La *base*, dans un système de numération, est le nombre de symboles (habituellement les chiffres) qui pourront servir à exprimer des nombres. L'humain s'est habitué à compter en base 10, le système *décimal*, où les symboles sont 0, 1, ..., 9. De nos jours, la grande majorité des ordinateurs travaillent toutefois en base 2, le système *binaire*. Les autres systèmes d'usage courant, principalement en informatique, sont l'*octal* (base 8) et l'*hexadécimal* (base 16). Dans ce dernier système, les symboles utilisés pour les nombres 10–15 sont généralement les lettres A–F. (C'est pourquoi l'on retrouve ces six lettres sur le pavé des calculatrices scientifiques.)

Cette section étudie la conversion des nombres entre la base 10 et une autre base quelconque.

4.3.1 Notation et définitions

De manière générale, soit x un nombre (entier pour le moment) dans la base de numération b composé de m chiffres ou symboles, c'est-à-dire

$$x = x_{m-1}x_{m-2} \cdots x_1x_0,$$

où $0 \leq x_i \leq b - 1$. On a donc

$$x = \sum_{i=0}^{m-1} x_i b^i. \quad (4.1)$$

Lorsque le contexte ne permet pas de déterminer avec certitude la base d'un nombre, celle-ci est identifiée en indice du nombre par un nombre décimal. Par exemple, 10011_2 est le nombre binaire 10011.

Exemple 4.1. Soit le nombre décimal 348. Selon la notation ci-dessus, on a $x_0 = 8$, $x_1 = 4$, $x_2 = 3$ et $b = 10$. En effet,

$$348 = 3 \times 10^2 + 4 \times 10^1 + 8 \times 10^0.$$

Ce nombre a les représentations suivantes dans d'autres bases. En binaire :

$$\begin{aligned} 101011100_2 &= 1 \times 2^8 + 0 \times 2^7 + 1 \times 2^6 \\ &\quad + 0 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 \\ &\quad + 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0. \end{aligned}$$

En octal :

$$534_8 = 5 \times 8^2 + 3 \times 8^1 + 4 \times 8^0.$$

En hexadécimal :

$$15C_{16} = 1 \times 16^2 + 5 \times 16^1 + 12 \times 16^0.$$

Des représentations ci-dessus, l'hexadécimale est la plus compacte : elle permet de représenter avec un seul symbole un nombre binaire comptant jusqu'à quatre chiffres. C'est, entre autres, pourquoi c'est une représentation populaire en informatique. $\square$

Dans un ordinateur réel (par opposition à théorique), l'espace disponible pour stocker un nombre est fini, c'est-à-dire que $m < \infty$. Le plus grand nombre que l'on peut représenter avec m chiffres ou symboles en base b est

$$\begin{aligned} x_{\max} &= \sum_{i=0}^{m-1} (b-1)b^i \\ &= (b-1) \sum_{i=0}^{m-1} b^i \\ &= (b-1) \left(\frac{b^m - 1}{b - 1} \right) \\ &= b^m - 1. \end{aligned}$$

Par exemple, le plus grand nombre décimal représentable avec $m = 4$ symboles est

$$x_{\max} = 9\,999 = 10\,000 - 1 = 10^4 - 1,$$

alors que le plus grand nombre binaire est seulement

$$x_{\max} = 1111 = 10000 - 1 = 2^4 - 1 = 15_{10}.$$

Par une extension naturelle de ce qui précède, un nombre composé de $m \geq 0$ symboles dans sa partie entière et $n \geq 0$ symboles dans sa partie

fractionnaire est représenté en base b comme

$$\begin{aligned} x &= x_{m-1}x_{m-2} \cdots x_1x_0, x_{-1}x_{-2} \cdots x_{-n} \\ &= \sum_{i=-n}^{m-1} x_i b^i. \end{aligned}$$

Le symbole qui sépare les parties entière et fractionnaire du nombre est la *séparation fractionnaire* (une virgule en français, un point en anglais).

4.3.2 Conversion vers une base quelconque

Avant de discuter de la conversion de nombres décimaux vers une base quelconque, il convient de définir les notions de *quotient* et de *reste* d'une division.

Le quotient est la partie entière de la division de deux entiers a et d ; sa représentation mathématique habituelle est

$$q = \left\lfloor \frac{a}{d} \right\rfloor, \quad (4.2)$$

où $\lfloor x \rfloor$ est la fonction qui retourne le plus grand entier inférieur ou égal à x . Le reste de la division est simplement la valeur

$$r = a - d \left\lfloor \frac{a}{d} \right\rfloor. \quad (4.3)$$

Évidemment, on a $r \in \{0, 1, \dots, d-1\}$. Le reste est le résultat de l'opération modulo, notée $r = a \bmod d$.

On remarquera que le premier chiffre en partant de la droite d'un entier décimal est le reste de la division de ce nombre par 10, que le second chiffre est le reste de la division par 10 du quotient de la division précédente, et ainsi de suite.

Exemple 4.2. Cet exemple semblera plutôt artificiel dans la mesure où nous sommes habitués à la représentation des nombres en base 10. Néanmoins, il permet d'illustrer les propos du paragraphe précédent. Soit le nombre 2 547. Dans ce nombre :

$$\begin{array}{ll} 7 \text{ est le reste de } \frac{2\,547}{10} & 7 = 2\,547 - 10 \left\lfloor \frac{2\,547}{10} \right\rfloor \\ 4 \text{ est le reste de } \frac{254}{10} & 4 = 254 - 10 \left\lfloor \frac{254}{10} \right\rfloor \end{array}$$

$$\begin{array}{ll}
 5 \text{ est le reste de } \frac{25}{10} & 5 = 25 - 10 \left\lfloor \frac{25}{10} \right\rfloor \\
 2 \text{ est le reste de } \frac{2}{10} & 2 = 2 - 10 \left\lfloor \frac{2}{10} \right\rfloor.
 \end{array}$$

□

La conversion d'un nombre décimal en une base b implique simplement de diviser par b plutôt que par 10 et de déterminer le symbole dans la base b correspondant au reste de chaque division.

L'algorithme suivant reprend ces idées de manière plus formelle et en ajoutant le traitement de la partie fractionnaire. Nous démontrerons plus loin qu'il n'est pas réellement nécessaire de savoir effectuer la conversion de la partie fractionnaire d'un nombre réel.

Algorithme 4.1 (Conversion de la base 10 vers la base b). *Soit x un nombre réel en base 10.*

1. Poser $i \leftarrow 0$ et $v \leftarrow \lfloor x \rfloor$.
2. Répéter les étapes suivantes jusqu'à ce que $v = 0$:
 - a) Poser $d_i \leftarrow v \bmod b$ et trouver x_i , le symbole dans la base b correspondant à d_i ;
 - b) Poser $v \leftarrow \lfloor v/b \rfloor$;
 - c) Poser $i \leftarrow i + 1$.
3. Poser $i \leftarrow 1$ et $v \leftarrow x - \lfloor x \rfloor$.
4. Répéter les étapes suivantes jusqu'à ce que $v = 0$ ou que $i = n$ (le nombre voulu ou maximal de chiffres après la séparation fractionnaire) :
 - a) Poser $d_{-i} \leftarrow \lfloor bv \rfloor$ et trouver x_{-i} , le symbole dans la base b correspondant à d_{-i} ;
 - b) Poser $v \leftarrow bv - d_{-i}$;
 - c) Poser $i \leftarrow i + 1$.
5. Retourner

$$x_b = x_{m-1}x_{m-2} \cdots x_1x_0, x_{-1}x_{-2} \cdots x_{-n}.$$

Exemple 4.3. Soit le nombre décimal 23,31 que l'on convertit en binaire (base 2) avec un maximum de cinq chiffres après la séparation fractionnaire ($n = 5$). Le [tableau 4.1](#) montre le processus en suivant les étapes de l'[algorithme 4.1](#). On obtient le résultat final en combinant la dernière colonne du tableau 4.1 (a) lue de bas en haut avec la dernière colonne du tableau 4.1 (b)

TAB. 4.1 – Conversion d'un nombre décimal en binaire

(a) partie entière					(b) partie fractionnaire				
i	v	$\lfloor v/2 \rfloor$	$v \bmod 2$	x_i	i	v	$2v$	$\lfloor 2v \rfloor$	x_{-i}
0	23	11	1	1	1	0,31	0,62	0	0
1	11	5	1	1	2	0,62	1,24	1	1
2	5	2	1	1	3	0,24	0,48	0	0
3	2	1	0	0	4	0,48	0,96	0	0
4	1	0	1	1	5	0,96	1,92	1	1

lue de haut en bas. Ainsi, la représentation binaire du 23,31 limitée à cinq chiffres après la virgule est 10111,01001.

□



Tel que mentionné ci-dessus, il est tout à fait possible de se passer de la conversion de la partie fractionnaire. Pour convertir le nombre réel x en base b avec un maximum de n chiffres dans la partie fractionnaire, il suffit de :

1. multiplier x par b^n ;
2. convertir l'entier $\lfloor xb^n \rfloor$ en base b ;
3. déplacer la virgule de n positions vers la gauche.

Exemple 4.4. Soit de nouveau la conversion de 23,31 en binaire avec une partie fractionnaire d'au plus cinq chiffres. En suivant les étapes de la remarque ci-dessus, on a :

1. $23,31 \times 2^5 = 23,31 \times 32 = 745,92$;
2. la représentation binaire de $745 = 2^9 + 2^7 + 2^6 + 2^5 + 2^3 + 1$ est 1011101001 ;
3. enfin, en déplaçant la virgule de cinq positions vers la gauche, on obtient 10111,01001, comme à l'exemple 4.3.

□

On tire une conclusion intéressante de la procédure ci-dessus. S'il existe un entier $k \leq n$ tel que la partie fractionnaire de x est un multiple de $1/b^k$, alors la multiplication de x par b^k résultera en un entier. Par conséquent, la représentation de x en base b aura une partie fractionnaire de longueur

finie. Autrement dit, pour obtenir une représentation binaire finie, la partie fractionnaire d'un nombre réel doit être un multiple de $\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \dots$. Ce n'était pas le cas à l'exemple 4.4 (la partie fractionnaire étant 0,31), d'où la nécessité de limiter le nombre de chiffres après la virgule.

4.3.3 Conversion en décimal

La conversion d'un nombre en base b vers la base 10 repose essentiellement sur la définition (4.1), mais avec chaque symbole x_i remplacé pour son équivalent décimal. Un algorithme de conversion est le suivant.

Algorithme 4.2 (Conversion de la base b vers la base 10). Soit x

$$x_b = x_{m-1}x_{m-2} \cdots x_1x_0, x_{-1}x_{-2} \cdots x_{-n}$$

un nombre réel en base b .

1. Poser $x \leftarrow 0$ et $y \leftarrow 0$.
2. Pour $i = m - 1, m - 2, \dots, 0$, faire les étapes suivantes :
 - a) Trouver d_i , le nombre décimal correspondant au symbole x_i ;
 - b) Poser $x \leftarrow xb + d_i$.
3. Pour $i = -n, -n + 1, \dots, -1$, faire les étapes suivantes :
 - a) Trouver d_i , le nombre décimal correspondant au symbole x_i ;
 - b) Poser $y \leftarrow (y + d_i)/b$.
4. Retourner $x + y$.

On peut, ici aussi, aisément éviter de convertir la partie fractionnaire. Il suffit de déplacer la virgule de n positions vers la droite dans le nombre x_b de manière à obtenir un entier, de convertir ce nombre en décimal avec les étapes 1 et 2 de l'algorithme 4.2 et, finalement, de diviser le nombre obtenu par b^n .

Exemple 4.5. On convertit le nombre binaire 101011,011 en décimal. Le tableau 4.2 illustre le processus de conversion selon l'algorithme 4.2 (le chiffre en surbrillance est celui traité lors de chaque étape de l'algorithme). Le résultat final est 43,375.

De manière équivalente, mais plus simple, on peut déplacer la virgule de trois positions vers la droite pour obtenir l'entier binaire 101011011. Ce nombre est $2^8 + 2^6 + 2^4 + 2^3 + 2 + 1 = 347$ en décimal. Enfin, $347 \div 2^3 = 43,375$. $\square$

TAB. 4.2 – Conversion d'un nombre binaire en décimal

(a) partie entière			(b) partie fractionnaire		
i	d_i	x	i	d_{-i}	y
5	101011,011	1	3	101011,011	0,5
4	101011,011	2	2	101011,011	0,75
3	101011,011	5	1	101011,011	0,375
2	101011,011	10			
1	101011,011	21			
0	101011,011	43			

Exemple 4.6. Soit le nombre hexadécimal AC2,3D8. On peut le convertir en base 10 à partir de la définition :

$$\begin{aligned}
 \text{AC2,3D8} &= 10 \times 16^2 + 12 \times 16 + 2 + 3 \times 16^{-1} + 13 \times 16^{-2} + 8 \times 16^{-3} \\
 &= 2\,754,240\,234\,375.
 \end{aligned}$$

□

4.3.4 Conversion avec des bases générales

Il existe de nombreuses bases de numération d'usage courant où chaque symbole est (potentiellement) exprimé dans une base différente. On a qu'à penser à l'heure ou à l'ensemble du système de mesure impérial. Or, il peut s'avérer utile de savoir convertir de et vers une base quelconque. Le matériel de cette section se retrouve dans très peu d'ouvrages d'analyse numérique ou de statistique numérique. Pourtant, il existe quelques applications intéressantes de la conversion de et vers des bases générales, comme nous le verrons.

On peut généraliser la notion de nombre présentée dans les sections précédentes à une collection de « symboles », chacun dans une base différente. On restreint la discussion aux entiers sans perte de généralité. On a donc

$$x = x_{m-1}x_{m-2} \cdots x_1x_0,$$

où x_{m-1} est un nombre en base b_{m-1} , x_{m-2} est un nombre en base b_{m-2} , etc. Nous dirons que le nombre x est exprimé en base $[b_{m-1} \ b_{m-2} \ \dots \ b_0]$. On a alors

$$x = \sum_{i=0}^{m-1} x_i \prod_{j=-1}^{i-1} b_j, \quad (4.4)$$

TAB. 4.3 – Conversion d'un nombre décimal dans une base générale

i	v	b_i	$\lfloor v/b_i \rfloor$	$v \bmod b_i$	x_i
0	91 492	60	1 524	52	52
1	1 524	60	25	24	24
2	25	24	1	1	1
3	1	365	0	1	1

avec $b_{-1} = 1$.

Les algorithmes de conversion 4.1 et 4.2 demeurent essentiellement valides ici. Il suffit de remplacer chaque mention de b par b_i .

Exemple 4.7. Le jour de l'année et l'heure du jour est un « nombre » exprimé en base [365 24 60 60]. En utilisant directement l'équation (4.4), le nombre de secondes correspondant à 1 jour, 2 heures, 33 minutes et 20 secondes est :

$$\begin{aligned} 1 \text{ j } 2 \text{ h } 33 \text{ min } 20 \text{ sec} &= 1 \times (24)(60)(60) + 2 \times (60)(60) + 33 \times 60 + 20 \\ &= 95\,600 \text{ secondes.} \end{aligned}$$

Si l'on fait la conversion en sens inverse, le nombre de jours, d'heures, de minutes et de secondes correspondant à 91 492 secondes est, en utilisant l'algorithme 4.1 avec la base [365 24 60 60] : 1 jour, 1 heure, 24 minutes et 52 secondes (voir le tableau 4.3 pour les calculs). $\square$

Les bases générales sont particulièrement utiles pour assigner ou extraire les éléments d'une matrice ou d'un tableau dans un ordre non séquentiel. Typiquement, l'index de l'élément à traiter est le résultat d'un calcul. L'exemple suivant illustre cette idée.

Exemple 4.8. Soit A une matrice 4×5 que l'on suppose remplie en ordre lexicographique (par ligne). Il est simple de déterminer, ici, que le 14^e élément est a_{34} . Or, on observe que la conversion du nombre $14 - 1 = 13$ dans la base [4 5] donne :

$$13 \div 5 = 2 \text{ reste } 3 \quad \Rightarrow \quad x_0 = 3$$

$$2 \div 4 = 0 \text{ reste } 2 \quad \Rightarrow \quad x_1 = 2,$$

soit le « nombre » (2, 3). En additionnant 1 à ce résultat, on obtient précisément la position du 14^e élément dans la matrice. Les opérations -1 et $+1$

sont rendues nécessaires par le fait que les lignes et les colonnes sont numérotées à partir de 1, alors que les systèmes de numération débutent à 0. $\square$



La fonction `arrayInd` de R permet de faire exactement ce qui est décrit dans l'exemple précédent, soit retourner les coordonnées du i^{e} élément d'un tableau. Consultez le code des lignes 45-65 du fichier de script `arithmetique_ordinateurs.R` reproduit à la [section 4.10](#) pour quelques exemples d'utilisation.

4.4 Unités de mesure

Les ordinateurs que nous utilisons couramment fonctionnent en base 2. La plus petite quantité d'information qu'un ordinateur peut traiter est un *bit* (compression de l'anglais *binary digit*). En informatique, un bit est égal à $\boxed{0}$ ou à $\boxed{1}$. Le symbole du bit est b.

Un *octet* (*byte*) est un groupe de huit bits. Son symbole est o ou B. L'octet est l'unité de mesure la plus fréquemment utilisée en informatique, en grande partie parce que le codage d'un caractère dans la plupart des langues occidentales requiert un octet; voir la [section 4.8](#). Les termes pour de plus grandes quantités de bits ou d'octets utilisent généralement les préfixes usuels du système international d'unités. Par exemple :

- ▶ 1 kilooctet (ko) est $2^{10} = 1\,024$ octets;
- ▶ 1 mégaoctet (Mo) est $2^{20} = 1\,048\,576$ octets;
- ▶ 1 gigaoctet (Go) est $2^{30} = 1\,073\,741\,824$ octets.

On voit que cette pratique fort répandue entraîne des distorsions par rapport aux définitions usuelles de kilo, méga, giga, etc., et que cette distorsion s'amplifie au fur et à mesure que l'on grimpe dans l'échelle des tailles d'objets. Pour cette raison, on a défini les préfixes kibi, mébi, gibi, etc., mais ceux-ci demeurent peu utilisés à ce jour.

Dans l'industrie de l'informatique, on joue beaucoup avec les unités pour montrer son produit sous un jour favorable. Deux exemples :

1. Les fournisseurs d'accès Internet expriment la vitesse de téléchargement en Mb/sec, alors que la taille des fichiers est généralement affichée en octets. Il faut donc diviser par 8 la vitesse annoncée pour avoir une idée plus juste du temps requis pour télécharger un fichier.

2. Les fabricants de disques durs divisent la capacité réelle de leurs disques par 10^9 pour l'exprimer en gigaoctets. Ainsi, un disque dont la capacité annoncée est de 100 Go ne peut contenir, en fait, que $100 \times 10^9 \div 2^{30} = 93,132$ Go de données. Cette confusion serait éliminée si les fabricants affichaient plutôt une capacité de 93,132 gibioctets. Moins vendeur...

4.5 Représentation en virgule flottante

Cette section décrit la manière standard de représenter les nombres réels dans les ordinateurs d'usage courant. Il est utile de connaître les grandes lignes afin de comprendre pourquoi les nombres réels n'ont pas tous une représentation exacte dans un ordinateur et comment surviennent et se propagent les erreurs d'arrondi. L'étude du sujet est également intéressante en soi pour constater comment les ingénieurs et les informaticiens sont parvenus à stocker un maximum d'information dans un espace limité.

Tel que mentionné précédemment, la capacité de stockage d'un ordinateur, bien que vaste de nos jours, n'en demeure pas moins limitée. Par conséquent, les nombres y sont représentés par un ensemble de m bits. Habituellement, m est un multiple de 2 et sa valeur détermine le type de nombre. Par exemple, des définitions usuelles sont $m = 2^4 = 16$ pour un entier, $m = 2^5 = 32$ pour un nombre réel en simple précision (*float* dans plusieurs langages de programmation) et $m = 2^6 = 64$ pour un nombre réel en double précision (*double*).

Il existe deux grandes façons de représenter les nombres (réels) à l'aide de m bits.

Virgule fixe Dans cette représentation, la position de la virgule dans le nombre est prédéterminée et, par conséquent, n'a pas besoin d'être conservée en mémoire. On réserve alors m_e positions pour la partie entière et m_f pour la partie fractionnaire (avec $m = m_e + m_f$). La représentation en virgule fixe est particulièrement utile dans les applications financières.

Virgule flottante Dans la représentation des nombres en virgule flottante, celle-ci peut être placée n'importe où dans le nombre. L'étendue de cette représentation est beaucoup plus grande que la virgule fixe, mais ceci au détriment de la précision puisque quelques bits doivent être réservés pour stocker la position de la virgule dans le nombre.

La représentation en virgule flottante est celle utilisée dans les applications scientifiques et celle sur laquelle nous nous concentrons dans la suite.

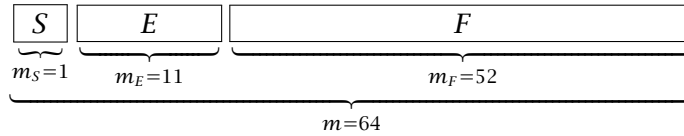


FIG. 4.1 – Représentation schématique d'un nombre en double précision dans la norme IEEE 754.

La représentation en virgule flottante est analogue à la notation scientifique. Le nombre réel x est entièrement défini par un bit de signe S , un exposant positif E et une mantisse M tel que

$$x = (-1)^S \times B^{E-e} \times M, \quad (4.5)$$

où B est la base de la représentation et e est un entier prédéterminé appelé le décalage, ou le biais, de l'exposant. Le recours au décalage facilite les calculs internes et évite de devoir réserver un autre bit pour le signe de l'exposant en le stockant sous forme non signée (c'est-à-dire sous forme d'entier positif).

La norme IEEE 754 définit la manière standard de représenter les nombres en virgule flottante dans les ordinateurs (IEEE, 2003; Wikipedia, 2012, pour une excellente présentation). En premier lieu, la norme stipule que la base dans la représentation est $B = 2$. Cela est implicite et n'est pas stocké où que ce soit dans l'ordinateur. En second lieu, la norme suppose que, pour les nombres dits *normalisés*, la mantisse est toujours de la forme $M = 1, F$, où F est un entier binaire. Le bit à gauche de la virgule est appelé le *bit caché* puisqu'il est lui aussi implicite et non stocké dans l'ordinateur.

Nous décrivons plus en détail la norme pour les nombres réels en *double précision* puisque c'est le type avec lequel R travaille toujours¹. Tout d'abord, un nombre en double précision est stocké dans un mot de $m = 64$ bits (8 octets) divisé ainsi de gauche à droite : $m_S = 1$ bit pour le signe, $m_E = 11$ bits pour l'exposant et $m_F = 52$ bits pour la partie fractionnaire de la mantisse. Le premier bit du mot utilisé pour le signe est appelé le *bit fort*. Voir la figure 4.1 pour une représentation schématique.

On remarque que le bit caché confère à la mantisse une longueur de mot effective (ou précision) de 53 bits.

1. Il est possible de créer des vrais entiers dans R en ajoutant un suffixe L à un nombre entier. Ainsi, 1L est un nombre entier dans le sens où `is.integer(1L)` est TRUE. Cela est relativement peu utilisé en programmation normale.

TAB. 4.4 – Valeurs spéciales dans la norme IEEE 754

S	E	F	Représentation binaire			Valeur spéciale
0	0	0	0	000000000000	00 ... 00	0
1	0	0	1	000000000000	00 ... 00	-0
0	2 047	0	0	111111111111	00 ... 00	$+\infty$
1	2 047	0	1	111111111111	00 ... 00	$-\infty$
0 ou 1	2 047	$F \neq 0$	S	111111111111	F	NaN ^a

^a Not a number, par exemple $\frac{0}{0}$ ou $\frac{\infty}{\infty}$.

Le décalage est $e = 2^{m_E-1} - 1 = 2^{10} - 1 = 1\,023$. Avec une longueur de mot de 11 bits, les valeurs possibles de E vont de 0 à $2^{11} - 1 = 2\,047$. Cependant, les valeurs 0 et 2 047 sont réservées pour des usages spéciaux ; voir le [tableau 4.4](#). Par conséquent, les valeurs possibles de l'exposant (une fois décalé) pour les nombres en double précision vont de $-1\,022$ à $1\,023$.

Exemple 4.9. Tel qu'expliqué à la [section 4.3.2](#), le nombre $\frac{1}{2}$ possède une représentation binaire exacte :

$$\begin{aligned}\frac{1}{2} &= 2^{-1} \\ &= 0,1_2 \\ &= (-1)^0 \times 2^{1\,022-1\,023} \times 1,0.\end{aligned}$$

Puisque la représentation de l'exposant est $1\,022_{10} = 1111111110_2$, la représentation interne de $\frac{1}{2}$ selon la norme IEEE 754 est

$$\boxed{0} \boxed{01111111110} \boxed{00 \dots 00}.$$

Si nous devons multiplier ce nombre par 2, la valeur obtenue serait exactement 1.

En revanche, le nombre $\frac{1}{10}$ n'a pas une représentation binaire finie. En utilisant l'[algorithme 4.1](#), on trouve (le côté droit de l'égalité étant en binaire) :

$$\begin{aligned}\frac{1}{10} &= 0,0001100110011001100110011001 \dots \\ &= 2^{-4} \times 1,1001100110011001 \dots \\ &= (-1)^0 \times 2^{1\,019-1\,023} \times 1,1001100110011001 \dots\end{aligned}$$

Or, $1\,019_{10} = 1111111011_2$, d'où la représentation interne finie de $\frac{1}{10}$ est

$$\boxed{0} \boxed{01111111011} \boxed{1001 \dots 1001}.$$

Si l'on reconvertit ce nombre en décimal, on obtient

$$2^{-4} \times \left(1 + \sum_{k=0}^{12} (2^{-4k-1} + 2^{-4k-4}) \right),$$

un nombre près de, mais pas exactement égal à, $\frac{1}{10}$ (le programme de calcul en multiprécision bc donne 0,0999999999999999166). Dès lors, la multiplication de ce nombre par 10 ne donnera pas 1. Voilà qui justifie le principe de programmation énoncé à la [page 72](#). $\square$



Vous pouvez déduire de l'exemple précédent que les représentations des puissances positives et négatives de 2 sont exactes en binaire (dans les limites des valeurs représentables). Par conséquent, le produit de puissances opposées ($2 \times \frac{1}{2}$, $4 \times \frac{1}{4}$, $8 \times \frac{1}{8}$, etc.) donnera toujours exactement 1.

Les principales caractéristiques des nombres en double précision sont les suivantes.

1. Soit $x_{\max}$ le plus grand nombre représentable. Parce que l'exposant $E = 2047$ est réservé, on a

$$\begin{aligned} x_{\max} &= \boxed{0} \boxed{111111111110} \boxed{11 \dots 11} \\ &= (-1)^0 \times 2^{1023} \times 1,11 \dots 11 \\ &= 2^{1023} \times (2 - 2^{-52}) \\ &= 1,797\,693 \times 10^{308}. \end{aligned}$$

2. Soit $x_{\min}$ le plus petit nombre normalisé représentable. Parce que l'exposant $E = 0$ est réservé, on a

$$\begin{aligned} x_{\min} &= \boxed{0} \boxed{000000000001} \boxed{00 \dots 00} \\ &= (-1)^0 \times 2^{-1022} \times 1,00 \dots 00 \\ &= 2^{-1022} \\ &= 2,225\,074 \times 10^{-308}. \end{aligned}$$

Afin de rendre la transition vers 0 moins abrupte, la norme IEEE 754 définit également des nombres *dénormalisés*. Ceux-ci sont identifiés par $E = 0$ et une partie fractionnaire F non nulle. Cependant, par définition,

les nombres dénormalisés ont $E - e = -1\,022$ et leur bit caché est 0. Par conséquent, le plus petit nombre dénormalisé est stocké sous la forme

$$\boxed{0} \boxed{000000000000} \boxed{00 \dots 01}$$

et sa valeur est

$$\begin{aligned} x_{\min} &= (-1)^0 \times 2^{-1\,022} \times 0,00 \dots 01 \\ &= 2^{-1\,022} \times 2^{-52} \\ &= 2^{-1\,074} \\ &= 4,940\,656 \times 10^{-324}. \end{aligned}$$

3. On a la même étendue pour les nombres négatifs, soit

$$[-1,797\,693 \times 10^{308}, -2,225\,074 \times 10^{-308}]$$

ou

$$[-1,797\,693 \times 10^{308}, -4,940\,656 \times 10^{-324}].$$

en considérant les nombres dénormalisés.

4. Soit ε la plus petite valeur tel que $1 + \varepsilon \neq 1$ dans la représentation en virgule flottante. Cette valeur est appelée l'*epsilon de la machine* ou la *précision de la machine*. Or, puisque

$$1 = (-1)^0 \times 2^0 \times 1,00 \dots 00,$$

le nombre suivant est

$$(-1)^0 \times 2^0 \times 1,00 \dots 01.$$

Par conséquent, on a $\varepsilon = 2^{-52} = 2,220\,446 \times 10^{-16}$.

5. Tout nombre x représente en fait un intervalle de $\mathbb{R}$. Par exemple, tout nombre dans l'intervalle $[1, 1 + \varepsilon)$ est représenté par le nombre 1 dans l'ordinateur.
6. On a un ensemble fini de nombres pour représenter $\mathbb{R}$. Or, cet ensemble est plus dense près de 0 que pour les grandes valeurs. En effet, le nombre suivant $x_{\min}$ est

$$\begin{aligned} x_{\min}^+ &= (1 + \varepsilon) \times 2^{-1\,022} \\ &= x_{\min} + \varepsilon \times 2^{-1\,022} \\ &= x_{\min} + 2^{-1\,074}, \end{aligned}$$

alors que le nombre précédant $x_{\max}$ est

$$\begin{aligned}x_{\max}^- &= (2 - 2^{-52} - \varepsilon) \times 2^{1023} \\&= x_{\max} - \varepsilon \times 2^{1023} \\&= x_{\max} - 2^{971}.\end{aligned}$$

L'écart entre les deux plus petits nombres représentables est donc de $2^{-1074} \approx 10^{-324}$, alors que celui entre les deux plus grands est de $2^{971} \approx 10^{292}$!

Tout calcul impliquant un nombre $x > x_{\max}$ ($x < -x_{\max}$) entraîne un *dépassement de capacité*. Habituellement, cela entraîne l'arrêt immédiat des calculs et un résultat de $+\infty$ ($-\infty$). À l'autre bout du spectre, un calcul impliquant un nombre $x < x_{\min}$ entraîne un *souppassement de capacité* et le résultat est habituellement considéré égal à 0.

R conserve dans une liste nommée `.Machine` les valeurs mentionnées ci-dessus — ainsi que plusieurs autres — pour l'architecture de l'ordinateur courant. Par exemple, on confirme les valeurs de ε , $x_{\min}$, $x_{\max}$, B , m_f et m_E , dans l'ordre, pour l'architecture x86 :

```
> .Machine[c(1, 3:6, 11)]
$double.eps
[1] 2.220446e-16

$double.xmin
[1] 2.225074e-308

$double.xmax
[1] 1.797693e+308

$double.base
[1] 2

$double.digits
[1] 53

$double.exponent
[1] 11
```



Le 1^{er} décembre 2014, YouTube a annoncé que la vidéo de la chanson *Gangnam Style* avait « brisé » son compteur en dépassant le nombre maximal de visionnements admissible, soit 2 147 483 647. Cette limite, bien inférieure à celle des nombres réels en double précision, provient de la représentation des nombres sous forme d'entier signé sur 32 bits. Voir l'[exercice 4.4](#) pour les détails.

4.6 Éléments d'arithmétique en virgule flottante

La section précédente expliquait pourquoi les nombres stockés dans un ordinateur ne sont pas toujours ceux que l'on croit — ou que l'on voudrait. Puisque l'ordinateur ne peut représenter tous les nombres réels, toute opération arithmétique avec des nombres en virgule flottante implique une erreur d'arrondi ou de troncature (selon l'architecture de l'ordinateur) dont il importe de tenir compte lors de la mise en oeuvre de certains algorithmes. Cette section présente quelques grands principes d'arithmétique en virgule flottante ainsi que de bons usages pour minimiser les erreurs d'arrondi. Consulter les ouvrages [Monahan \(2001\)](#); [Burden et Faires \(1988\)](#); [Knuth \(1997\)](#), entre autres, pour une discussion plus complète de ce vaste sujet.

Aux fins de cette section, on note $\text{fl}(x)$ la représentation en virgule flottante du nombre x et l'on définit la représentation simplifiée

$$\text{fl}(x) = (-1)^S \times 10^E \times 0,F, \quad (4.6)$$

où F compte cinq chiffres significatifs, dont le dernier est arrondi. Le [tableau 4.5](#) fournit quelques exemples.



Dans la norme IEEE 754, les règles d'arrondi de base sont : arrondir à la valeur la plus près (0,14 devient 0,1 et 0,16 devient 0,2); arrondir un nombre exactement à mi-chemin au nombre avec un dernier chiffre significatif pair ou nul (0,05 devient 0,0 alors que 0,15 devient 0,2).

4.6.1 Erreurs d'arrondi ou de troncature

Lors de l'addition et de la soustraction de nombres en virgule flottante, on rend les exposants égaux en déplaçant les bits de la mantisse du plus petit nombre vers la droite. Il en résulte une perte de bits significatifs et donc une *erreur d'arrondi*. Dans les cas extrêmes où les deux opérandes

TAB. 4.5 – Représentation en virgule flottante simplifiée

x	$\text{fl}(x)$
2,5	$0,25000 \times 10^1$
-42,182	$-0,42182 \times 10^2$
0,214356	$0,21436 \times 10^0$

sont de grandeur très différente, le plus petit opérande devient nul suite à la perte de tous ses bits significatifs.

La situation pour la multiplication et la division est quelque peu différente, mais la source d'erreur d'arrondi demeure la même.

À toute fin pratique, tout calcul fait naître de l'erreur d'arrondi et celle-ci augmente avec le nombre d'opérations. Les quelques principes de programmation ci-dessous, lorsque suivis par le programmeur prudent et consciencieux, permettent d'atténuer l'impact de l'erreur d'arrondi.

- (P1) L'addition et la soustraction en virgule flottante ne sont pas associatives. Additionner les nombres en ordre croissant de grandeur afin d'accumuler des bits significatifs.
- (P2) Éviter de soustraire un petit nombre d'un grand, ou alors le faire le plus tard possible dans la chaîne des calculs.
- (P3) Pour calculer une somme alternée, additionner tous les termes positifs et tous les termes négatifs, puis faire la soustraction.
- (P4) Multiplier et diviser des nombres d'un même ordre de grandeur. Si x et y sont presque égaux, le calcul x/y sera plus précis en virgule flottante que $y^{-1}x$.

Les exemples ci-dessous illustrent ces principes.

Exemple 4.10. Soit $x = 7$, $y = 4$ et $z = 100\,000$. Dans la représentation simplifiée (4.6), on a $\text{fl}(x) = 0,70000 \times 10^1$, $\text{fl}(y) = 0,40000 \times 10^1$ et $\text{fl}(z) = 0,10000 \times 10^6$. Or, $x + y + z = 100\,011$ et

$$\begin{aligned}
 [\text{fl}(x) + \text{fl}(y)] + \text{fl}(z) &= (0,70000 \times 10^1 + 0,40000 \times 10^1) + 0,10000 \times 10^6 \\
 &= 0,11000 \times 10^2 + 0,10000 \times 10^6 \\
 &= 0,00001 \times 10^6 + 0,10000 \times 10^6 \\
 &= 0,10001 \times 10^6,
 \end{aligned}$$

soit un résultat raisonnablement précis. Cependant,

$$\begin{aligned}\text{fl}(x) + [\text{fl}(y) + \text{fl}(z)] &= 0,70000 \times 10^1 + (0,00000 \times 10^6 + 0,10000 \times 10^6) \\ &= 0,00000 \times 10^6 + 0,10000 \times 10^6 \\ &= 0,10000 \times 10^6.\end{aligned}$$

En effectuant les opérations dans un mauvais ordre, on obtient $x + y + z = z$ même si $x \neq 0$ et $y \neq 0$. $\square$

Exemple 4.11. Soit $x = 7$, $y = 100\,006$ et $z = 100\,002$. On a $z - y - x = -11$. Or,

$$\begin{aligned}\text{fl}(z) - [\text{fl}(y) + \text{fl}(x)] &= 0,10000 \times 10^6 - (0,10000 \times 10^6 + 0,00000 \times 10^6) \\ &= 0,00000 \times 10^0,\end{aligned}$$

alors que

$$\begin{aligned}[\text{fl}(z) - \text{fl}(y)] - \text{fl}(x) &= (0,10000 \times 10^6 - 0,10000 \times 10^6) - 0,70000 \times 10^1 \\ &= 0,00000 \times 10^1 - 0,70000 \times 10^1 \\ &= -0,70000 \times 10^1.\end{aligned}$$

$\square$

L'exemple suivant est adapté de [Monahan \(2001\)](#).

Exemple 4.12. L'évaluation numérique de probabilités loin dans les queues d'une fonction de répartition peut s'avérer imprécise selon la technique employée. Par exemple, la fonction de répartition de la distribution logistique est

$$F(x) = (1 + e^{-x})^{-1}.$$

La vraie valeur de $\Pr[X > 6]$ est $1 - F(6) = 1 - (1 + e^{-6})^{-1} = 0,002\,472\,623$. L'évaluation de cette probabilité dans notre représentation en virgule flottante avec la formule ci-dessus est

$$\begin{aligned}1 - [1 \div (1 + \text{fl}(e^{-6}))] &= 1 - [1 \div (1 + 0,24788 \times 10^{-2})] \\ &= 1 - [0,10000 \times 10^1 \div 0,10025 \times 10^1] \\ &= 1 - 0,99751 \times 10^0 \\ &= 0,10000 \times 10^1 - 0,09975 \times 10^1 \\ &= 0,25000 \times 10^{-2}.\end{aligned}$$

(Nous n'avons pas écrit les entiers en virgule flottante ci-dessus pour alléger la notation.) Toutefois, si l'on utilise plutôt la formule équivalente

$$\begin{aligned} 1 - F(6) &= \frac{e^{-6}}{1 + e^{-6}} \\ &= \frac{1}{1 + e^6} \end{aligned}$$

dans laquelle l'opération de soustraction entre deux nombres presque égaux est déjà effectuée, on obtient le résultat bien plus précis

$$\begin{aligned} 1 \div (1 + \text{fl}(e^6)) &= 1 \div (1 + 0,40343 \times 10^3) \\ &= 1 \div (0,00100 \times 10^3 + 0,40343 \times 10^3) \\ &= 1 \div 0,40443 \times 10^3 \\ &= 0,24726 \times 10^{-2}. \end{aligned}$$

□



Toutes les fonctions de calcul de fonction de répartition de R permettent d'évaluer plus précisément les probabilités $1 - F(x)$ pour une grande valeur de x en spécifiant l'option `lower.tail = FALSE`.

Les deux principes ci-dessous permettent d'éviter des dépassements ou des sous-passements de capacité et d'améliorer la précision des calculs.

(P5) Chercher des formules mathématiques équivalentes évitant de devoir traiter des très grands ou des très petits nombres.

(P6) Travailler en échelle logarithmique. Par exemple, le produit de deux grands nombres x et y dépassera la capacité plus tard et demeurera précis plus longtemps s'il est calculé sous la forme $e^{\log x + \log y}$.

Exemple 4.13. Soit $X_1, \dots, X_n$ un échantillon aléatoire tiré d'une distribution de Pareto translatée, dont la fonction de répartition est

$$F(x; \mu, \alpha) = 1 - \left(\frac{\mu}{x}\right)^\alpha, \quad x > \mu.$$

On peut démontrer que l'estimateur du maximum de vraisemblance de α est

$$\hat{\alpha} = \frac{n}{\ln(X_1 \cdots X_n / X_{(1)}^n)},$$

où $X_{(1)} = \min(X_1, \dots, X_n)$. Soit x un objet R contenant un échantillon de 1 000 valeurs d'une distribution Pareto translatée de moyenne 5 000 (le contenu de cet objet n'est pas affiché ici pour des raisons évidentes). Le calcul de l'estimateur $\hat{\alpha}$ directement par la formule entraîne rapidement un dépassement de capacité, tant lors du produit $X_1 \cdots X_n$ que lors de l'opération $X_{(1)}^n$:

```
> prod(x)
[1] Inf
> min(x)^length(x)
[1] Inf
```

Le résultat est donc NaN :

```
> length(x)/log(prod(x)/min(x)^length(x))
[1] NaN
```

Selon la grandeur des données dans l'échantillon, la formule équivalente

$$\hat{\alpha} = \frac{n}{\ln \prod_{i=1}^n X_i / X_{(1)}}$$

peut éviter le dépassement de capacité. Elle n'est toutefois d'aucun secours dans le présent exemple :

```
> length(x)/log(prod(x/min(x)))
[1] 0
```

On remarquera de plus que cette approche nécessite un grand nombre de multiplications (voir la [section 4.6.2](#)), en plus d'ouvrir la porte à des erreurs d'arrondi.

Pour obtenir une réponse on utilisera plutôt une autre formule algébriquement équivalente :

$$\hat{\alpha} = \frac{n}{\sum_{i=1}^n \ln X_i - n \ln X_{(1)}}.$$

On obtient alors

```
> length(x)/(sum(log(x)) - length(x) * log(min(x)))
[1] 1.40843
```

Cet exemple illustre combien des calculs *algébriquement* équivalents ne sont pas nécessairement *numériquement* équivalents. $\square$

TAB. 4.6 – Coût relatif de quelques opérations en virgule flottante

Opération arithmétique	Coût relatif
Addition et soustraction	1,0
Multiplication	1,3
Division	3,0
Racine carrée	4,0
Logarithme	15,4

4.6.2 Coût des opérations

Les ordinateurs modernes disposent d'une unité de calcul en virgule flottante (FPU) intégrée au processeur. Cette unité est évidemment très optimisée et, par conséquent, elle accélère beaucoup le calcul en virgule flottante. Néanmoins, certaines opérations sont plus coûteuses à réaliser que d'autres en termes de temps de calcul. À titre indicatif, on trouve au [tableau 4.6](#) le coût relatif approximatif de quelques opérations en virgule flottante.

À titre d'exemple, considérons le calcul d'une simple moyenne arithmétique

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i.$$

Si l'on effectue le calcul tel qu'écrit ci-dessus, cela requiert $n - 1$ additions et une division. En revanche, le calcul algébriquement équivalent

$$\bar{x} = \sum_{i=1}^n \frac{x_i}{n},$$

requiert $n - 1$ divisions et $n - 1$ additions. Pour n grand, utiliser la première approche plutôt que la seconde fera une différence.



4.7 Indice pour le problème

La calculatrice TI-30XS MultiView conserve 13 chiffres significatifs en mémoire pour les calculs. Nous pourrions tenter d'expliquer les résultats du calcul de $(x - 1)(x + 1) - x^2 + 1$ à l'aide de la représentation simplifiée (4.6) en utilisant une mantisse F comptant 13 chiffres significatifs.

4.8 Codage de caractères

Pendant que l'on discute de la représentation interne des nombres dans un ordinateur, on peut toucher un mot de la représentation interne, ou le codage, des caractères. Ce sujet demeure une cible mouvante puisque de nouveaux standards apparaissent encore après de nombreuses années de systèmes incompatibles et basés sur la langue anglaise.

Peu importe le système retenu, les caractères doivent être codés en binaire dans l'ordinateur. La partie la plus difficile consiste à créer un système standard permettant aux ordinateurs et autres appareils numériques de communiquer entre eux. Le premier standard d'usage courant fut le Code américain normalisé pour l'échange d'information, mieux connu sous son acronyme ASCII ([ANSI, 1986](#)). La norme ASCII définit des codes numériques pour 128 caractères, soit 95 affichables (lettres, chiffres, symboles divers et l'espace) et 33 non affichables (essentiellement des caractères de contrôle tels que saut de ligne, retour de chariot ou espacement arrière).

À titre d'exemple, les lettres majuscules A-Z occupent les cases 65-90 (1000001-1011010 en binaire) alors que les lettres minuscules a-z occupent les cases 97-122 (1100001-1111010 en binaire). On constate que les versions majuscule et minuscule d'une même lettre ne diffèrent, dans leur représentation binaire, que par leur second bit le plus significatif. Le changement de casse d'une lettre ou d'un mot est donc une opération très simple et rapide.

La représentation ASCII ne requiert en soi que sept bits. Néanmoins, on a rapidement codé les caractères sur un octet (huit bits) puisqu'il s'agit du type de donnée natif des ordinateurs depuis les années 1970. L'ajout d'un bit a créé de l'espace pour 128 caractères additionnels (cases 128-255). Une myriade de systèmes de codage différents sont alors apparus pour supporter les caractères des langues autres que l'anglais (les caractères accentués, par exemple) ainsi que d'autres symboles graphiques. La norme ISO 8859-1, ou Latin 1 ([ISO, 1998](#)), a fini par s'imposer comme l'un des standards les plus répandus. Ces listes de codes fixes se révèlent toutefois problématiques lors de l'apparition d'un nouveau symbole. Par exemple, pour faire de la place pour le symbole de l'euro à la fin des années 1990, il a fallu retirer un symbole de ISO 8859-1. Avec quelques autres changements, la nouvelle liste de code est devenue ISO 8859-15. De plus, comment doit-on traiter les langues CJC (chinois, japonais, coréen) qui comptent des milliers de symboles différents ?

Depuis le début des années 1990, la mondialisation de l'informatique a suscité un effort concerté de création d'un système de codage standard couvrant la presque totalité des systèmes d'écriture du monde. Le système

devait aussi permettre la composition de textes en plusieurs langues, par exemple en français et dans une écriture de droite à gauche comme l'hébreu ou l'arabe. Le standard ainsi créé est Unicode ([Unicode Consortium, 2007](#)). Le plus populaire système de codage capable de représenter l'ensemble des caractères Unicode est *Unicode Transformation Format* 8 bits, ou UTF-8 ([Unicode Consortium, 2007](#), section 3.9). Dans l'UTF-8, chaque caractère est codé sur une suite d'un à quatre octets et les premiers 128 caractères sont identiques à la norme ASCII. L'UTF-8 est le système de codage par défaut dans macOS et les plus récentes distributions GNU/Linux.

Le système R supporte les caractères multi-octets de manière assez exhaustive. Sans doute aidés en cela par le fait qu'il s'agit d'un projet international, les développeurs de R ont mis beaucoup d'effort pour assurer l'internationalisation et la localisation du logiciel ; voir [Ripley \(2005\)](#).

⚡ 4.9 Solution du problème

Nous calculons $(x - 1)(x + 1) - x^2 + 1$ avec, en premier lieu, $x = 10^6$. Dans la représentation interne de la calculatrice, ce nombre est stocké sous la forme :

$$\text{fl}(x) = 0,10000000000000 \times 10^7.$$

Aux fins des calculs, le nombre 1 sera, quant à lui, traité comme

$$\text{fl}(1) = 0,0000001000000 \times 10^7.$$

Nous avons donc

$$\begin{aligned} & [\text{fl}(x) - \text{fl}(1)][\text{fl}(x) + \text{fl}(1)] - \text{fl}(x)^2 + \text{fl}(1) \\ &= (0,0999999000000 \times 10^7)(0,1000001000000 \times 10^7) \\ &\quad - 0,1000000000000 \times 10^{13} + 0,0000001000000 \times 10^7 \\ &= 0,0999999999999 \times 10^{13} - 0,1000000000000 \times 10^{13} \\ &\quad + 0,0000000000001 \times 10^{13} \\ &= -0,0000000000001 \times 10^{13} + 0,0000000000001 \times 10^{13} \\ &= 0, \end{aligned}$$

ce qui est le résultat attendu. On remarque que dans les opérations ci-dessus, le nombre de chiffres significatifs nécessaire ne dépasse jamais la capacité de 13 de la calculatrice.

Répetons ces calculs avec, cette fois, la valeur $x = 10^8$ et les représentations internes

$$\text{fl}(x) = 0,1000000000000 \times 10^9$$

$$\text{fl}(1) = 0,0000000010000 \times 10^9.$$

Nous avons

$$\begin{aligned} & [\text{fl}(x) - \text{fl}(1)][\text{fl}(x) + \text{fl}(1)] - \text{fl}(x)^2 + \text{fl}(1) \\ &= (0,0999999990000 \times 10^9)(0,1000000010000 \times 10^9) \\ &\quad - 0,1000000000000 \times 10^{17} + 0,0000000010000 \times 10^9 \\ &= 0,1000000000000 \times 10^{17} - 0,1000000000000 \times 10^{17} \\ &\quad + 0,0000000010000 \times 10^9 \\ &= 0 + 0,0000000010000 \times 10^9 \\ &= 1. \end{aligned}$$

Là où le calcul de la calculatrice s'écarte de son équivalent mathématique, c'est lors du passage de la première à la seconde égalité. Le produit des deux nombres entre parenthèses doit être arrondi à 13 décimales et il en résulte un nombre numériquement équivalent à $\text{fl}(x)^2$.

Afin de prévenir ce genre d'erreurs, il conviendra donc de respecter les principes de la [section 4.6](#).

4.10 Code informatique

✓ Fichier d'accompagnement arithmetique_ordinateurs.R

```

12 ###
13 ### EXEMPLES DE CALCULS EN APPARENCE ERRONÉS
14 ###
15
16 ## Opérateurs d'addition et de soustraction ne respectant pas les
17 ## règles d'arithmétique de base.
18 1.2 + 1.4 + 2.8           # 5.4
19 1.2 + 1.4 + 2.8 == 5.4    # non?!?
20 2.8 + 1.2 + 1.4           # encore 5.4
21 2.8 + 1.2 + 1.4 == 5.4    # donc addition non commutative?
22 2.6 - 1.4 - 1.2           # devrait donner 0
23 2.6 - 1.5 - 1.1           # correct cette fois-ci
24
25 ## Division donnant parfois des résultats erronés.
```

```
26 0.2/0.1 == 2          # ok
27 (1.2 - 1.0)/0.1 == 2  # pourtant équivalent
28 0.3/0.1 == 3          # fonctionnait pourtant avec 0.2
29
30 ## Distance plus grande entre 3,2 et 3,15 qu'entre 3,10 et
31 ## 3,15.
32 3.2 - 3.15 > 0.05      # écart supérieur à 0,05...
33 3.15 - 3.1 < 0.05      # et cette fois inférieur à 0,05
34
35 ## Valeurs inexactes dans les suites de nombres générées avec
36 ## 'seq', qui sont générées algébriquement (par additions ou
37 ## soustractions successives).
38 (a <- seq(0.9, 0.95, by = 0.01))
39 a == 0.94              # 0.94 n'est pas dans le vecteur!
40 (b <- c(0.90, 0.91, 0.92, 0.93, 0.94, 0.95))
41 b == 0.94              # mais ici, oui!
42 a - b                  # remarquer le 5e élément
43
44 ###
45 ### CONVERSION DANS DES BASES GÉNÉRALES
46 ###
47
48 ## La fonction 'arrayInd', dont la syntaxe (simplifiée) est
49 ##
50 ##     arrayInd(ind, dim)
51 ##
52 ## retourne les coordonnées des éléments aux positions 'ind'
53 ## dans un tableau de dimensions 'dim'. Par exemple, le 14e
54 ## élément d'une matrice 4 x 5 remplie par colonne (ordre R)
55 ## est en position (2, 4).
56 arrayInd(14, c(4, 5))  # comparer avec l'exemple 10.7
57
58 ## C'est toujours un peu plus compliqué avec les tableaux. Par
59 ## exemple, un tableau 3 x 4 x 5 doit être vu comme cinq
60 ## matrices 3 x 4 placées les unes derrière les autres. Où se
61 ## trouve le 'i'ème élément?
62 arrayInd(8, 3:5)        # élément (2, 3) de 1ère matrice
63 arrayInd(13, 3:5)        # élément (1, 1) de 2e matrice
64 arrayInd(59, 3:5)        # élément (2, 4) de 5e matrice
65 arrayInd(c(8, 13, 59), 3:5) # en un seul appel
```

4.11 Exercices

4.1 Convertir les nombres décimaux suivants en base 6, puis en binaire.

- a) 119
- b) 343
- c) 96
- d) 43

4.2 Convertir les nombres hexadécimaux suivants en nombres décimaux.

- a) A1B
- b) 12A
- c) B41
- d) BAFFE

4.3 a) Utiliser l'algorithme de conversion des nombres en base b vers la base 10 et les idées de l'exemple 4.8 pour trouver une formule générale donnant la position de l'élément a_{ijk} d'un tableau de dimensions $I \times J \times K$ dans l'ordre de la liste des éléments du tableau. Utiliser l'ordre lexicographique où le tableau est rempli dans l'ordre

$$a_{111}, a_{112}, \dots, a_{11K}, a_{121}, a_{122}, \dots$$

b) Répéter la partie a) en utilisant l'ordre R où le tableau est plutôt rempli dans l'ordre $a_{111}, a_{211}, \dots, a_{I11}, a_{121}, a_{221}, \dots$.

4.4 La plupart des langages de programmation — dont R, voir la note au bas de la page 83 — permettent de définir des nombres entiers « stricts », par opposition à des nombres réels qui s'avèrent simplement être entiers. La représentation de ces nombres entiers est complètement différente de celle de la norme IEEE 754, sauf en ce qui a trait au bit fort, qui sert toujours à stocker le signe du nombre.

Les nombres entiers sont stockés dans une représentation dite « en complément à deux » (Wikipedia, 2013). Dans cette notation :

- i) un entier positif est représenté en binaire de manière usuelle, avec le bit fort égal à 0;
- ii) pour un entier négatif, on inverse les bits de la représentation de sa valeur absolue et on ajoute 1 au résultat.

Le bit fort est automatiquement mis à 1 (pour représenter un nombre négatif) par l'opération d'inversion. Les deux opérations ci-dessus sont équivalentes à stocker un nombre négatif x en tant que $2^n - |x|$.

Par exemple, la représentation sur 8 bits du nombre 5 est

0	0000101
---	---------

.

Pour représenter -5 , on inverse ces bits et on ajoute 1 :

1	1111011
---	---------

.

On remarque que $11111011_2 = 251_{10} = 2^8 - 5$.

- a) L'un des avantages de la notation en complément à deux est qu'elle évite d'avoir des représentations pour $+0$ et -0 . Pour des entiers de 8 bits,

0	0000000
---	---------

 correspond tout naturellement à 0. À quel nombre correspond

1	0000000
---	---------

 selon cette notation?
- b) Un autre avantage, plus important que celui mentionné en a), de la notation en complément à deux est qu'elle permet d'effectuer les opérations arithmétiques en binaire naturellement. Pour constater ce fait, effectuer directement sur 8 bits l'opération $15 - 5$ en utilisant les notations suivantes :

- i) la notation naturelle où le bit fort représente le signe et les sept autres bits pour représentent la valeur absolue du nombre ;
- ii) la notation en complément à deux.

Dans les deux cas, ignorer les éventuels dépassements de capacité vers la gauche.

- c) C'est maintenant le temps d'expliquer le trait d'humour dans le strip de XKCD.com au début du document. Pour ce faire, convertir en décimal les représentations en complément à deux sur 8 bits des nombres de

00000000

 à

11111111

. Répéter l'exercice sur 16 bits. (En fait, convertir non pas toutes les représentations, mais bien seulement quelques valeurs clés qui permettront d'observer la séquence des nombres décimaux correspondants.)
- d) Plusieurs langages de programmation permettent de choisir parmi divers types de structures de données pour représenter des nombres réels ou des nombres entiers. Nous avons déjà mentionné les types `float` et `double` pour les nombres réels en simple et en double précision. Cette nomenclature du Fortran se perpétue dans le C et ses

nombreux dérivés. Le C dispose aussi des types `int` et `long` pour représenter des nombres entiers, positifs ou négatifs. Le type `int` utilise 16 bits d'espace mémoire et le type `long`, 32 bits. Tous deux ont recours à la notation en complément à deux. Déterminer l'ensemble des entiers représentables pour chacun des types de données `int` et `long`.

4.5 La norme IEEE 754 pour les nombres en virgule flottante (S, E, F) en simple précision est la suivante :

- ▶ longueur totale de $m = 32$ bits;
- ▶ 1 bit pour le signe S (valeur de 0 pour un nombre positif);
- ▶ 8 bits pour l'exposant E , avec un biais de 127;
- ▶ 23 bits pour la partie fractionnaire F .

Un nombre x est donc représenté comme

$$x = (-1)^S \times 2^{E-127} \times 1.F.$$

Trouver les valeurs ε , $x_{\max}$ et $x_{\min}$ pour les nombres en simple précision. Comparer les résultats avec les limites du type `Single` en VBA.

4.6 Représenter les nombres suivants comme des nombres en virgule flottante en simple précision selon la norme IEEE 754.

- a) $-1\,234$
- b) 55
- c) $8\,191$
- d) -10
- e) $\frac{2}{3}$
- f) $\frac{1}{100}$

4.7 Les nombres ci-dessous sont représentés en format binaire selon la norme IEEE 754 pour les nombres en simple précision. Convertir ces nombres en décimal.

- a)

0	00111101	100100001000000000000000
---	----------	--------------------------
- b)

1	00111101	100100001000000000000000
---	----------	--------------------------
- c)

0	10000100	100100001000000000000000
---	----------	--------------------------
- d)

1	10000100	100100001000000000000000
---	----------	--------------------------

4.8 Trouver, pour les nombres des parties a) et c) de l'exercice 4.7, le nombre suivant et le nombre précédent en représentation binaire.

Réponses

La notation x_b signifie que le nombre x est en base b . On omet généralement b pour les nombres en base 10.

4.1 a) $315_6, 1110111_2$

b) $1331_6, 101010111_2$

c) $240_6, 1100000_2$

d) $111_6, 101011_2$

4.2 a) 2 587 b) 298 c) 2 881 d) 765 950

4.3 a) $k + K(j - 1 + J(i - 1))$

b) $i + I(j - 1 + J(k - 1))$

4.4 a) -128

b) i) -20 ii) 10

d) Type Byte : $\{0, \dots, 255\}$; type Integer : $\{-32\,768, \dots, 32\,767\}$;

type Long : $\{-2\,147\,483\,648, \dots, 2\,147\,483\,647\}$

4.5 $\varepsilon = 2^{-23} = 1,192 \times 10^{-7}$, $x_{\max} = (2 - 2^{-23}) \times 2^{127} = 3,403 \times 10^{38}$,
 $x_{\min} = 2^{-126} = 1,175 \times 10^{-38}$ (nombre normalisé) ou $x_{\min} = 2^{-149} =$
 $1,401 \times 10^{-45}$ (nombre dénormalisé)

4.6 a)

1	10001001	001101001000000000000000
---	----------	--------------------------

b)

0	10000100	101110000000000000000000
---	----------	--------------------------

c)

0	10001011	111111111111000000000000
---	----------	--------------------------

d)

1	10000010	010000000000000000000000
---	----------	--------------------------

e)

0	01111110	01010101010101010101010
---	----------	-------------------------

f)

0	01111000	01000111101011100001010
---	----------	-------------------------

4.7 a) $2,120\,229\,346 \times 10^{-20}$ b) $-2,120\,229\,346 \times 10^{-20}$ c) 50,0625

d) -50,0625

4.8 a) $2,120\,229\,508 \times 10^{-20}$ et $2,120\,229\,185 \times 10^{-20}$

b) 50,062 503 815 et 50,062 496 185

5 Résolution d'équations à une variable

Objectifs du chapitre

- ▶ Résoudre une équation à une variable à l'aide des méthodes de la bisection, du point fixe et de Newton-Raphson.
- ▶ Faire un choix parmi l'une des méthodes ci-dessus pour résoudre un problème donné.
- ▶ Déterminer si une fonction admet un point fixe dans un intervalle donné.
- ▶ Utiliser les fonctions de résolution d'équations et d'optimisation de Excel et de R.



5.1 Énoncé du problème

Les fonctions décrivant les heures de lever et de coucher du soleil à Québec pour le mois de janvier 2016 sont représentées par $L(x)$ et $C(x)$, respectivement :

$$L(x) = e^{-0.001431x+2,016340}$$
$$C(x) = e^{0,001312x+2,778538},$$

où x est le jour du mois (un entier). La fonction du nombre d'heures d'ensoleillement pour une journée est donc $H(x) = C(x) - L(x)$.

Pendant quel jour du mois les Québécois bénéficient-ils de 9 heures d'ensoleillement, à une minute près ?

5.2 Mise en contexte

Ce chapitre passe en revue les méthodes les plus populaires pour résoudre numériquement une équation à une variable. Ce problème, simple seulement en apparence, survient dans une foule de domaines. Débutons par deux exemples, tirés des mathématiques financières et de l'inférence statistique.

Exemple 5.1. Un investisseur est disposé à payer k \$ aujourd'hui en retour d'une série de n versements de 1 \$, le premier dans un an. Pour déterminer le taux de rendement qu'il exige, on doit trouver la valeur de i tel que $a_{\overline{n}|i} = k$ ou, énoncé autrement, la valeur de i tel que

$$f(i) = \frac{1 - (1 + i)^{-n}}{i} - k = 0.$$

□

Exemple 5.2. On a un échantillon aléatoire $X_1, \dots, X_n$ provenant d'une distribution binomiale négative avec fonction de masse de probabilité

$$\Pr[X = x] = \binom{x + r - 1}{r - 1} p^r (1 - p)^x, \quad x = 0, 1, \dots,$$

où p est supposé connu. On souhaite trouver l'estimateur du maximum de vraisemblance du paramètre r .

La fonction de vraisemblance est

$$\begin{aligned} L(r) &= \prod_{i=1}^n \Pr[X = x_i] \\ &= \prod_{i=1}^n \binom{x_i + r - 1}{r - 1} p^r (1 - p)^{x_i} \end{aligned}$$

et la fonction de log-vraisemblance est

$$\begin{aligned} l(r) &= \sum_{i=1}^n [\ln \Gamma(x_i + r) - \ln \Gamma(r) - \ln \Gamma(x_i + 1) \\ &\quad + r \ln p + x_i \ln(1 - p)]. \end{aligned}$$

Par conséquent, l'estimateur du maximum de vraisemblance $\hat{r}$ est la solution de

$$f(r) = \frac{d}{dr} l(r) = \sum_{i=1}^n (\Psi(x_i + r) - \Psi(r) + \ln p) = 0,$$

où $\Psi(x) = \Gamma'(x)/\Gamma(x)$ est la *fonction digamma*.

□

Le problème sur lequel nous allons nous pencher consiste donc à trouver la racine d'une fonction $f(x)$, c'est-à-dire la solution de $f(x) = 0$.



5.3 Indice pour le problème

Nous cherchons à déterminer pour quelle valeur de $x \in \{1, 2, \dots, 31\}$ la fonction $H(x)$ vaut 9 ou, de manière équivalente, la racine de l'équation $H(x) - 9 = 0$.

Une technique d'essai-erreur intuitive suivrait probablement à peu près les étapes suivantes :

1. vérifier qu'il existe bel et bien une racine dans l'intervalle $[1, 31]$ en évaluant le nombre d'heures d'ensoleillement aux bornes de l'intervalle, puis déterminer si la fonction est croissante ou décroissante sur l'intervalle (elle est croissante dans le cas présent);
2. essayer une première valeur pour x et calculer la valeur de $H(x)$;
3. si le nombre d'heures d'ensoleillement est trop élevé, diminuer la valeur de x et si, à l'inverse, le nombre d'heures d'ensoleillement est trop faible, augmenter la valeur de x ;
4. répéter les deux étapes précédentes jusqu'à trouver la bonne valeur de x au degré de précision désiré.

Voici une mise en œuvre simple en R. Définissons d'abord une fonction pour calculer le nombre d'heures d'ensoleillement $H(x)$.

```
> H <- function(x)
+ {
+   exp(0.001312 * x + 2.778538) -
+   exp(-0.001431 * x + 2.016340)
+ }
```

Calculons les valeurs au début et à la fin de l'intervalle.

```
> H(1)
[1] 8.616558
> H(31)
[1] 9.578721
```

Selon les valeurs à $x = 1$ et $x = 31$, le jour où l'ensoleillement total vaut 9 semble se situer environ à la mi-mois. Essayons donc $x = 15$.

```
> H(15)
[1] 9.064084
```

La valeur est supérieure à 9. Le jour recherché se trouve donc entre 1 et 15. Essayons $x = 10$.

```
> H(10)
[1] 8.903965
```

La valeur est inférieure à 9. Le jour recherché se trouve donc entre 10 et 15. Essayons $x = 13$.

```
> H(13)
[1] 8.999997
```

Nous avons trouvé la bonne valeur à une minute près.

5.4 Méthode de bisection

Toutes les méthodes numériques de résolution d'une équation qui seront étudiées dans ce chapitre reposent sur une procédure systématique d'essais et d'erreurs. La procédure — ou algorithme — utilisée déterminera l'efficacité de la méthode. À ce chapitre, la méthode de bisection est la plus simple et la plus intuitive méthode de résolution, mais aussi la moins efficace.

Soit f une fonction continue sur un intervalle $[a, b]$ choisi de telle sorte que $f(a)$ et $f(b)$ sont de signes opposés. Par le théorème de la valeur intermédiaire, il existe un point x^* tel que $f(x^*) = 0$. En d'autres mots, si une fonction continue est d'un côté de l'axe des abscisses au début d'un intervalle et de l'autre côté à l'autre extrémité de l'intervalle, alors elle a forcément croisé l'axe quelque part dans l'intervalle.

L'idée de la méthode de bisection consiste à trouver la solution x^* par essais successifs en utilisant le point milieu d'intervalles de plus en plus petits, mais que l'on sait toujours contenir le point cherché. L'algorithme 5.1 systématise cette idée et la figure 5.1 l'illustre pour quelques itérations de l'algorithme.

Algorithme 5.1 (Méthode de la bisection). *Soit f une fonction continue sur l'intervalle $[a, b]$ où $f(a)$ et $f(b)$ sont de signes opposés. On cherche x^* tel que $f(x^*) = 0$ en un maximum de $N_{\max}$ itérations.*

1. Poser $n = 1$.
2. Répéter les étapes suivantes.

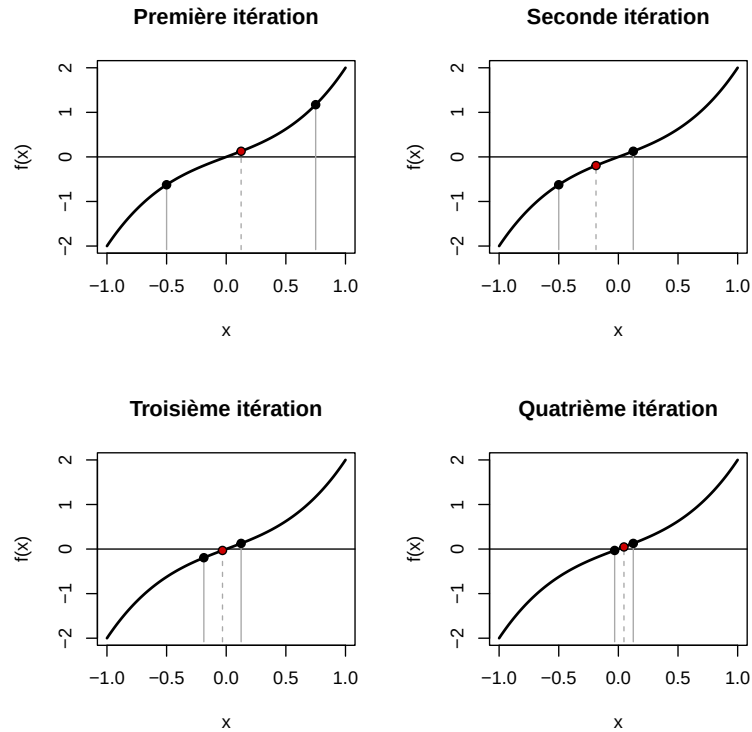


FIG. 5.1 - Illustration de la méthode de bisection. À chaque itération, les points noirs reliés par des segments verticaux en ligne pleine identifient les bornes de l'intervalle dans lequel on sait que se trouve la solution de l'équation $f(x) = 0$. Le point rouge relié par un segment en pointillé est l'essai de l'itération, c'est-à-dire le point milieu de l'intervalle. On remarquera comment ce point devient la borne inférieure ou supérieure de l'intervalle à l'itération suivante.

2.1. Poser $x_n = (a + b)/2$.

2.2. Si $f(a)f(x_n) > 0$, poser $a = x_n$, sinon poser $b = x_n$.

2.3. Si $|x_n - x_{n-1}|/|x_n| < \varepsilon$, poser $x^* = x_n$ et terminer.

2.4. Si $n \geq N_{\max}$, terminer sans convergence.

2.5. Poser $n = n + 1$.

L'application de l'algorithme 5.1 génère une suite $\{x_n\}$, $n = 1, 2, \dots$ qui, si tout se passe bien, converge vers la valeur x^* .

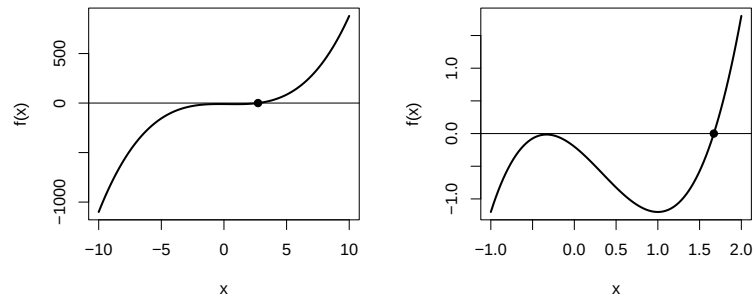


FIG. 5.2 – Deux exemples de fonctions pour lesquelles on pourrait avoir $|f(x_n)| < \varepsilon$ même si $x_n \gg x^*$ ou $x_n \ll x^*$. Le point identifie la véritable racine dans chaque graphique.



À l'étape 2.2 de l'algorithme, le produit $f(a)f(x_n)$ permet de déterminer si l'on se trouve du même côté de l'axe des abscisses aux points a et x_n (produit de deux valeurs négatives ou de deux valeurs positives) ou de part et d'autre de l'axe (produit de valeurs de signes opposés).

L'étape 2.3 de l'algorithme permet de valider si la procédure a convergé vers une réponse. La valeur de ε sera donc en général « petite ». Ici, nous avons utilisé l'erreur relative comme critère d'arrêt. Il s'agit en général du meilleur choix. En effet, un critère tel que $|f(x_n)| < \varepsilon$ peut être satisfait même si $x_n \gg x^*$ ou $x_n \ll x^*$ (voir la [figure 5.2](#) pour deux exemples). Quant à l'erreur absolue $|x_n - x_{n-1}| < \varepsilon$, l'inégalité peut être satisfaite même si la suite $\{x_n\}$ diverge.

Le code informatique de la [section 5.15](#) contient la définition d'une fonction `bisection` dont les arguments sont :

1. la fonction $f(x)$;
2. la valeur de a ;
3. la valeur de b ;
4. la valeur de ε (10^{-6} par défaut);
5. le nombre maximal d'itérations $N_{\max}$ (100 par défaut);

6. une valeur booléenne indiquant si les calculs intermédiaires doivent ou non être affichés à l'écran : si l'argument est TRUE, les valeurs de a , b , x_n et $f(x_n)$ sont affichées à chaque itération.

La fonction retourne une liste dont l'élément `$root` contient la réponse et l'élément `$nb.iter` le nombre d'itérations nécessaire pour obtenir cette réponse.



Étudiez attentivement les lignes 13–57 du fichier de script `resolution_equations.R` reproduit à la section 5.15. Nous aurons recours à la fonction `bisection` dans les exemples qui suivent.

Exemple 5.3. On cherche la racine de $f(x) = x^3 + 4x^2 - 10$ dans l'intervalle $[1, 2]$. On peut vérifier que la fonction f n'a qu'une seule racine dans cet intervalle et que, de plus, $f(1) = -5$ et $f(2) = 14$. Ainsi, $a = 1$ et $b = 2$ constituent des valeurs de départ adéquates.

Après 17 itérations de la fonction `bisection`, la réponse obtenue est, à neuf décimales, 1,365 226 746. L'erreur d'approximation est au maximum de

$$|x^* - x_{17}| < |b_{18} - a_{18}| = |1,365234 - 1,365227| = 0,000007$$

et, puisque $|a_{18}| < |x^*|$,

$$\frac{|x^* - x_{17}|}{|x^*|} < \frac{|b_{18} - a_{18}|}{|a_{18}|} = 5,13 \times 10^{-6}.$$

La réponse est donc exacte à au moins cinq décimales près.



Vérifiez les résultats de cet exemple à l'aide du code des lignes 59–73 du fichier de script `resolution_equations.R`.

□

Exemple 5.4. La valeur actuelle d'une série de dix paiements de fin d'année est 8,2218. Pour déterminer le taux d'intérêt (ou taux de rendement, selon le point de vue), on cherche i tel que $f(i) = a_{\overline{10}|i} - 8,2218 = 0$.



Complétez cet exemple en exécutant les lignes 75–90 du fichier de script `resolution_equations.R`.

□



5.5 Indice pour le problème

La méthode de bisection est en fait une systématisation de la technique d'essai-erreur intuitive présentée précédemment.

On utilise l'[algorithme 5.1](#) avec $f(x) = H(x) - 9$, $a = 1$ et $b = 31$.

5.6 Méthode du point fixe

La solution de l'équation

$$g(x) = x$$

est un *point fixe* de la fonction g . Il s'agit du point où les fonctions $g(x)$ et $y = x$ sont égales, c'est-à-dire là où la fonction g croise la droite à 45° dans le plan.

Tout problème de recherche de racine peut être exprimé comme un problème de point fixe. Par exemple, en définissant

$$g(x) = x - f(x),$$

les problèmes de trouver x tel que $g(x) = x$ ou tel que $f(x) = 0$ sont équivalents.

La méthode du point fixe repose sur le théorème du même nom. Celui-ci fournit les conditions garantissant, d'une part, l'existence d'un point fixe pour une fonction g dans un intervalle donné et, d'autre part, l'unicité de ce point fixe. Un point fixe unique peut néanmoins exister sans que ces conditions ne soient remplies.

Théorème 5.1 (Théorème du point fixe). *Soit g une fonction continue sur un intervalle $[a, b]$.*

1. *Si $g(x) \in [a, b]$ pour tout $x \in [a, b]$, alors g a un point fixe dans $[a, b]$.*
2. *Si, de plus, $g'(x)$ existe sur (a, b) et qu'il existe une constante k tel que*

$$|g'(x)| \leq k < 1$$

pour tout $x \in (a, b)$, alors g a un point fixe unique dans $[a, b]$.

Démonstration. Le théorème compte deux volets. Nous en faisons une démonstration simplifiée ne reposant que sur des arguments graphiques.

1. Le premier volet garantit l'existence d'un point fixe sur $[a, b]$ si l'image de la fonction g se trouve aussi dans un intervalle $[a, b]$. Cette paire d'intervalles engendre le carré en pointillé que l'on retrouve dans les graphiques de la [figure 5.3](#).

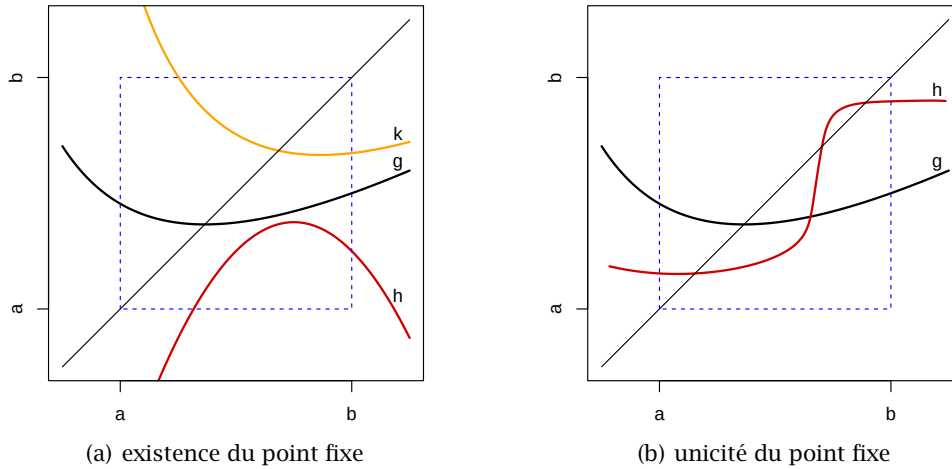


FIG. 5.3 – Exemples de graphiques permettant de faire la démonstration du théorème du point fixe

En définitive, le premier volet du théorème établit que si la fonction continue g entre dans l'intervalle $[a, b]$ par la gauche du carré ($g(x) \in [a, b]$) et en ressort par la droite (idem), alors elle doit forcément croiser la droite $y = x$. Observons le graphique (a) de la [figure 5.3](#) :

- ▶ $g(x) \in [a, b]$ pour tout $x \in [a, b]$, donc un point fixe existe nécessairement ;
 - ▶ $h(x) \notin [a, b]$ pour tout $x \in [a, b]$, donc la fonction n'a pas nécessairement de point fixe dans $[a, b]$;
 - ▶ $k(x) \notin [a, b]$ pour tout $x \in [a, b]$, mais un point fixe demeure néanmoins possible.
2. Le second volet garantit l'unicité du point fixe si la pente de la fonction sur (a, b) n'excède jamais 1. Cette condition empêche la fonction de croiser la droite $y = x$ une première fois et de la croiser de nouveau suite à une forte augmentation. Comparons les deux fonctions du graphique (b) de la [figure 5.3](#) :
- ▶ $g'(x) < 1$ pour tout $x \in (a, b)$, donc $g(x)$ croise $y = x$ en un seul point et le point fixe est unique ;
 - ▶ $h(x) \in [a, b]$ pour tout $x \in [a, b]$, mais $h'(x) > 1$ pour $x \in (a, b)$, ce qui ouvre la porte à des points fixes multiples, tel qu'illustré.

□

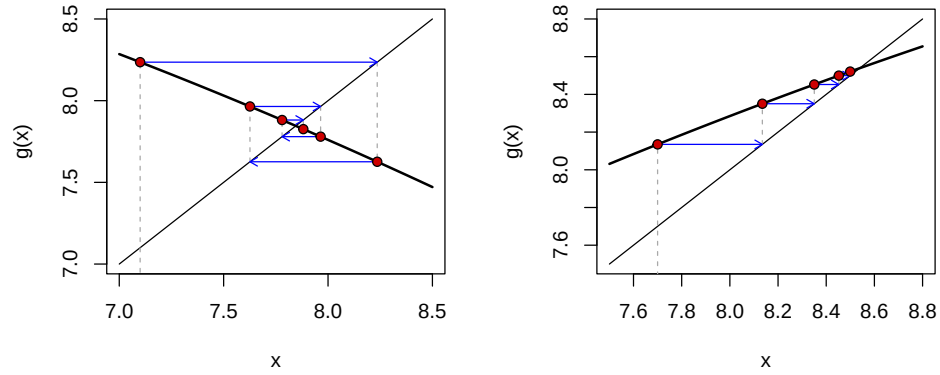


FIG. 5.4 - Illustrations de la méthode du point fixe pour une fonction décroissante et pour une fonction croissante. Les points rouges identifient les essais successifs, le premier étant, ici, le point le plus à gauche dans les graphiques. Les flèches illustrent comment est déterminé l'essai suivant. La relation $x_n = g(x_{n-1})$ implique que l'abscisse d'un essai correspond à la projection de l'ordonnée de l'essai précédent sur la droite $y = x$.

Le théorème suivant, donné sans preuve, établit véritablement la procédure qui sera utilisée par la méthode numérique pour trouver le point fixe d'une fonction.

Théorème 5.2. *Si les deux conditions du [théorème 5.1](#) sont satisfaites, alors la série $\{x_n = g(x_{n-1})\}$ converge vers un point fixe unique dans $[a, b]$.*

Tel que mentionné au début de la [section 5.4](#), toutes les méthodes numériques — et la méthode du point fixe ne fait pas exception — procèdent par essais et erreurs, mais de manière systématique. Le [théorème 5.2](#) nous dit que pour trouver un point fixe, le meilleur essai x_n à faire est la valeur de la fonction g à l'essai précédent, $g(x_{n-1})$. La [figure 5.4](#) illustre comment la suite des essais successifs peut converger vers le point fixe.

Nous pouvons dès lors composer un algorithme de résolution par la méthode du point fixe.

Algorithme 5.2 (Méthode du point fixe). *Soit g une fonction et x_0 une valeur de départ. On cherche x^* tel que $g(x^*) = x^*$ en un maximum de $N_{\max}$ itérations.*

1. Poser $n = 1$.
2. Répéter les étapes suivantes.
 - a) Poser $x_n = g(x_{n-1})$.
 - b) Si $|x_n - x_{n-1}|/|x_n| < \varepsilon$, alors poser $x^* = x_n$ et terminer.
 - c) Si $n \geq N_{\max}$, terminer sans convergence.
 - d) Poser $n = n + 1$.



Comme on peut le déduire des illustrations de la [figure 5.4](#), la rapidité de la convergence est fonction de $g'(x)$: plus la pente est *faible*, plus la convergence est *rapide*, et vice versa.

Le code informatique de la [section 5.15](#) définit une fonction `pointfixe` similaire à la fonction `bissection` présentée à la [section 5.4](#). Ses arguments sont d'ailleurs les mêmes, sinon que les points a et b sont remplacés par une valeur de départ x_0 .



Étudiez attentivement les lignes [93-149](#) du fichier de script `resolution_equations.R` reproduit à la [section 5.15](#). Nous aurons recours à la fonction `pointfixe` dans les exemples qui suivent.

Exemple 5.5. On répète l'[exemple 5.4](#) à l'aide de la méthode du point fixe, soit trouver le taux d'intérêt i tel que $a_{\overline{10}|i} = 8,2218$. On a déjà déterminé à l'[exemple 5.4](#) que le taux d'intérêt se trouve dans l'intervalle $[0,035, 0,040]$. On peut exprimer le problème sous forme de point fixe ainsi : trouver la valeur de i tel que

$$g(i) = \frac{1 - (1 + i)^{-10}}{8,2218} = i.$$



Exécutez le code des lignes [151-170](#) du fichier de script `resolution_equations.R` pour calculer le point fixe de la fonction $g(i)$ ci-dessus.

Il faut 34 itérations pour obtenir la convergence avec un critère identique à celui utilisé avec la bissection. Pour ce type de problème, la méthode du point fixe n'est pas la plus efficace. $\square$

Exemple 5.6. On répète l'exemple 5.3 à l'aide de la méthode du point fixe, à savoir : trouver la racine de $f(x) = x^3 + 4x^2 - 10 = 0$ dans l'intervalle $[1, 2]$.

Il y a plusieurs façons d'exprimer le problème de recherche de racine en termes de point fixe. Examinons cinq différentes fonctions toutes algébriquement équivalentes :

$$g_1(x) = x - x^3 - 4x^2 + 10$$

$$g_2(x) = \left(\frac{10}{x} - 4x\right)^{1/2}$$

$$g_3(x) = \frac{1}{2}(10 - x^3)^{1/2}$$

$$g_4(x) = \left(\frac{10}{4 + x}\right)^{1/2}$$

$$g_5(x) = x - \frac{x^3 + 4x^2 - 10}{3x^2 + 8x}.$$

Il est laissé en exercice de vérifier que, dans chacun des cas, la racine de $f(x)$ est le point fixe de $g_i(x)$, $i = 1, \dots, 5$.

Les graphiques des cinq fonctions ci-dessus se trouvent à la figure 5.5. L'examen de ces graphiques permet de déterminer, avant de faire quelque calcul que ce soit, pour quelles fonctions la procédure itérative du point fixe convergera ainsi que les taux de convergence relatifs, le cas échéant.



Complétez cet exemple en exécutant les lignes 172-204 du fichier de script `resolution_equations.R`.

□



5.7 Indice pour le problème

Pour utiliser l'algorithme du point fixe 5.2, il faut exprimer notre problème de recherche du jour du mois où l'ensoleillement est de 9 heures sous la forme $g(x) = x$.

Nous pourrions simplement réécrire l'équation de départ $H(x) = 9$ sous la forme $H(x) - 9 + x = x$. Seulement, la pente de cette fonction est très près de 1 et la méthode du point fixe serait donc très lente.

Isolons donc plutôt une valeur de x dans l'équation de départ

$$e^{0,001312x+2,778538} - e^{-0,001431x+2,016340} = 9.$$

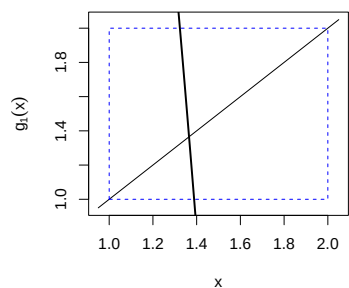
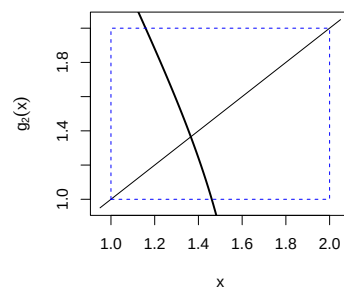
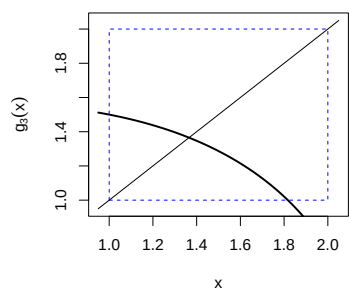
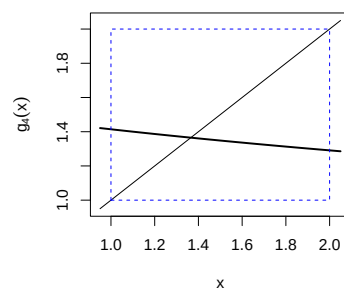
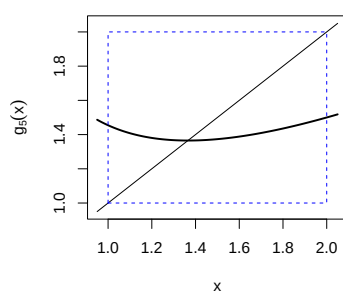
(a) fonction $g_1(x)$ (b) fonction $g_2(x)$ (c) fonction $g_3(x)$ (d) fonction $g_4(x)$ (e) fonction $g_5(x)$

FIG. 5.5 – Graphiques des cinq fonctions de l'exemple 5.6

En isolant le x dans la première exponentielle, cette équation se réécrit

$$x = \frac{\ln(9 + e^{-0,001431x+2,016340}) - 2,778538}{0,001312}.$$

Nous pourrions donc utiliser l'algorithme du point fixe avec la fonction

$$g(x) = \frac{\ln(9 + e^{-0,001431x+2,016340}) - 2,778538}{0,001312}.$$

On peut vérifier graphiquement que la pente de cette fonction est beaucoup plus faible que celle de $H(x) - 9 + x$.

5.8 Méthode de Newton–Raphson

La méthode de Newton–Raphson est l'une des plus populaires et puissantes méthodes numériques de résolution d'équations à une variable. Les algorithmes plus élaborés qui existent aujourd'hui sont d'ailleurs souvent basés sur cet algorithme.

L'étude de la méthode de Newton–Raphson nous ramène au problème de recherche de la racine x^* d'une fonction $f(x)$, c'est-à-dire la solution de $f(x) = 0$. Il y a plusieurs manières d'introduire cette méthode de résolution. Nous privilégierons une approche graphique.

Dans la méthode de Newton–Raphson, les essais successifs sont déterminés en utilisant les points où la tangente de la fonction f à l'essai précédent croise l'axe des abscisses; voir la [figure 5.6](#) pour une illustration.

Une première justification mathématique de la méthode va comme suit. Soit $\tilde{x}$ un point « près » de la racine x^* et tel que $f(\tilde{x}) \neq 0$. Comme on peut le voir à la [figure 5.6](#), la tangente de f en $\tilde{x}$ croise l'abscisse en un point $\hat{x}$ « près » de x^* . Ainsi, le point $\hat{x}$ est tel que

$$\frac{f(\tilde{x}) - 0}{\tilde{x} - \hat{x}} = f'(\tilde{x}),$$

d'où

$$\begin{aligned} x^* &\simeq \hat{x} \\ &= \tilde{x} - \frac{f(\tilde{x})}{f'(\tilde{x})}. \end{aligned}$$

On peut également justifier la formule précédente à l'aide du développement de Taylor de $f(x)$ autour de $\tilde{x}$:

$$f(x) = f(\tilde{x}) + (x - \tilde{x})f'(\tilde{x}) + \frac{(x - \tilde{x})^2}{2}f''(\tilde{x}) + \dots$$

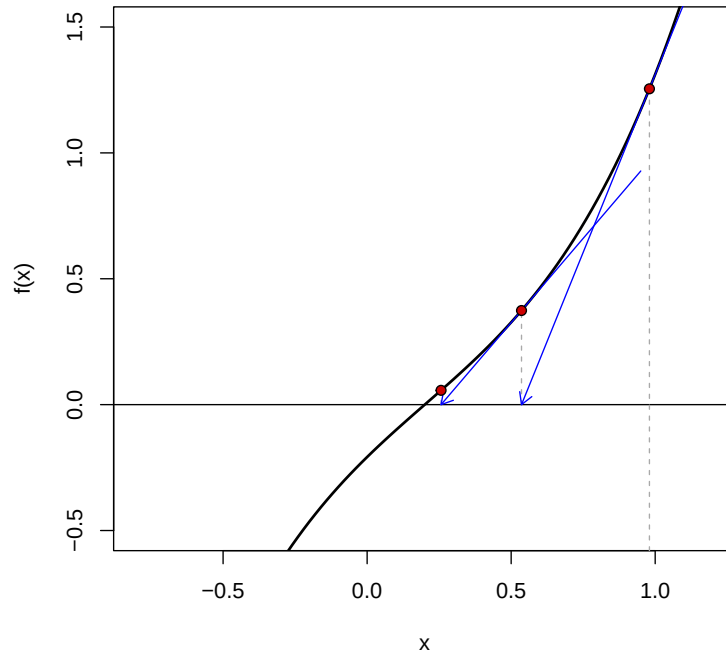


FIG. 5.6 - Illustration de la méthode de Newton-Raphson. Les points rouges identifient les essais successifs, le premier étant, ici, le point le plus à droite dans le graphique. Les flèches illustrent comment est déterminé l'essai suivant. On trace la tangente en un point ; le prochain essai est la valeur en abscisse où cette tangente croise l'axe.

Toujours en supposant que $\tilde{x}$ est près de x^* , on a

$$f(x^*) = 0 = f(\tilde{x}) + (x^* - \tilde{x})f'(\tilde{x}) + \frac{(x^* - \tilde{x})^2}{2}f''(\tilde{x}) + \dots,$$

où l'on peut considérer les termes de puissance 2 et plus dans le développement comme négligeables. On obtient alors l'approximation

$$x^* \simeq \tilde{x} - \frac{f(\tilde{x})}{f'(\tilde{x})}.$$

Tel qu'illustré à la [figure 5.6](#), on peut répéter la procédure ci-dessus si le point $\hat{x}$ est trop éloigné de la racine x^* . On utilise alors $\hat{x}$ comme nouveau

point où l'on calcule la tangente, et ainsi de suite jusqu'à l'obtention d'une « bonne » approximation.

Formellement, soit f une fonction différentiable sur $[a, b]$ et $x^* \in [a, b]$. Alors x^* peut être obtenu comme le point de convergence de la série $\{x_n\}$ définie par

$$x_n = x_{n-1} - \frac{f(x_{n-1})}{f'(x_{n-1})},$$

avec x_0 une valeur de départ quelconque. L'algorithme 5.3 systématise cette procédure.

Algorithme 5.3 (Méthode de Newton-Raphson). *Soit f une fonction continue et différentiable sur l'intervalle $[a, b]$. On cherche x^* tel que $f(x^*) = 0$ en un maximum de $N_{\max}$ itérations avec une valeur de départ x_0 .*

1. Poser $n = 1$.
2. Répéter les étapes suivantes.
 - a) Poser $x_n = x_{n-1} - f(x_{n-1})/f'(x_{n-1})$.
 - b) Si $|x_n - x_{n-1}|/|x_n| < \varepsilon$, alors poser $x^* = x_n$ et terminer.
 - c) Si $n \geq N_{\max}$, terminer sans convergence.
 - d) Poser $n = n + 1$.



L'examen attentif de la méthode de Newton-Raphson révèle qu'il s'agit en fait d'une procédure de point fixe avec

$$g(x) = x - \frac{f(x)}{f'(x)}.$$

La méthode de Newton-Raphson nous fournit donc la « bonne » fonction g à utiliser dans la méthode du point fixe pour trouver la racine de la fonction f .

On peut démontrer que si f est doublement différentiable sur $[a, b]$ et que $x^* \in [a, b]$ est tel que $f(x^*) = 0$ et $f'(x^*) \neq 0$, alors il existe un $\delta > 0$ tel que la série $\{x_n\}$ converge vers x^* pour tout $x_0 \in [x^* - \delta, x^* + \delta]$. Autrement dit :

- les hypothèses du [théorème 5.1](#) sont satisfaites;
- la méthode de Newton-Raphson fonctionne toujours avec un « bon » choix de valeur de départ x_0 .

Le code informatique de la [section 5.15](#) définit une fonction `nr` similaire à la fonction `pointfixe` présentée à la [section 5.6](#), sauf qu'on y a ajouté un argument pour la dérivée de la fonction $f(x)$.



Étudiez attentivement les lignes [207-244](#) du fichier de script `resolution_equations.R` reproduit à la [section 5.15](#). Nous aurons recours à la fonction `nr` dans les exemples qui suivent.

Exemple 5.7. Reprenons l'[exemple 5.5](#) de calcul d'un taux de rendement, cette fois à l'aide de la méthode de Newton-Raphson. Vous savez peut-être de votre cours de mathématiques financières que la méthode de Newton-Raphson est la plus efficace pour résoudre ce genre de problème. La [figure 5.7](#) illustre pourquoi elle est plus efficace que la méthode du point fixe : la fonction

$$g(i) = i - \frac{f(i)}{f'(i)}$$

avec $f(i) = a_{\overline{n}|i} - k$ est beaucoup plus plate que la fonction

$$g(i) = \frac{1 - v^n}{k}$$

utilisée à l'[exemple 5.5](#).



Exécutez le code des lignes [246-267](#) du fichier de script `resolution_equations.R` pour calculer la racine de $f(i) = a_{\overline{n}|i} - k$.

La méthode de Newton-Raphson ne requiert donc que 2 itérations pour obtenir un résultat équivalent à celui obtenu par la méthode du point fixe avec la fonction g de l'[exemple 5.5](#) en 34 itérations. $\square$

Exemple 5.8. Revisitons l'[exemple 5.3](#) où nous recherchions la racine de la fonction $f(x) = x^3 + 4x^2 - 10$ dans l'intervalle $[1, 2]$. On a $f'(x) = 3x^2 + 8x$, d'où $f'(x) \neq 0$ pour tout $x \in [1, 2]$. La méthode de Newton-Raphson dicte en définitive de rechercher le point fixe de la fonction

$$\begin{aligned} g(x) &= x - \frac{f(x)}{f'(x)} \\ &= x - \frac{x^3 + 4x^2 - 10}{3x^2 + 8x}. \end{aligned}$$

Celle-ci est présentée à la [figure 5.8](#). On constate que la procédure itérative de Newton-Raphson convergera rapidement vers une racine unique dans l'intervalle $[1, 2]$. En effet :

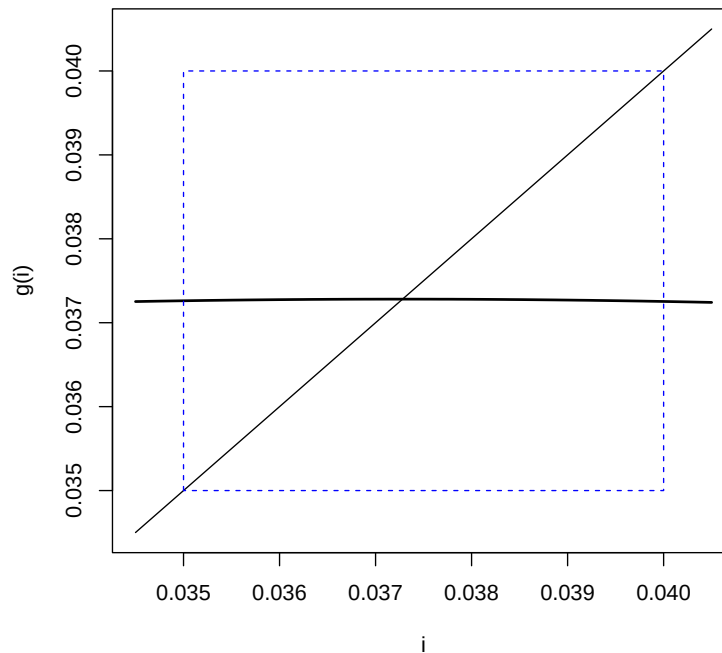


FIG. 5.7 - Fonction $g(i) = i - f(i)/f'(i)$ où $f(i) = (1 - (1 + i)^{-10})/i - 8,2218$ pour $0,035 \leq i \leq 0,040$ (ligne épaisse) et la droite $y = i$ (ligne mince)

```
> f <- function(x) x^3 + 4*x^2 - 10
> fp <- function(x) 3*x^2 + 8 * x
> nr(f, fp, start = 1.5)
$root
[1] 1.36523

$nb.iter
[1] 3
```

Si vous êtes attentif, vous aurez remarqué que la fonction g utilisée ici était la fonction g_5 dans l'exemple 5.6, celle pour laquelle la convergence de la méthode du point fixe était la plus rapide. C'est précisément ce que mentionnait la remarque suivant l'algorithme 5.3, à la page 118. $\square$

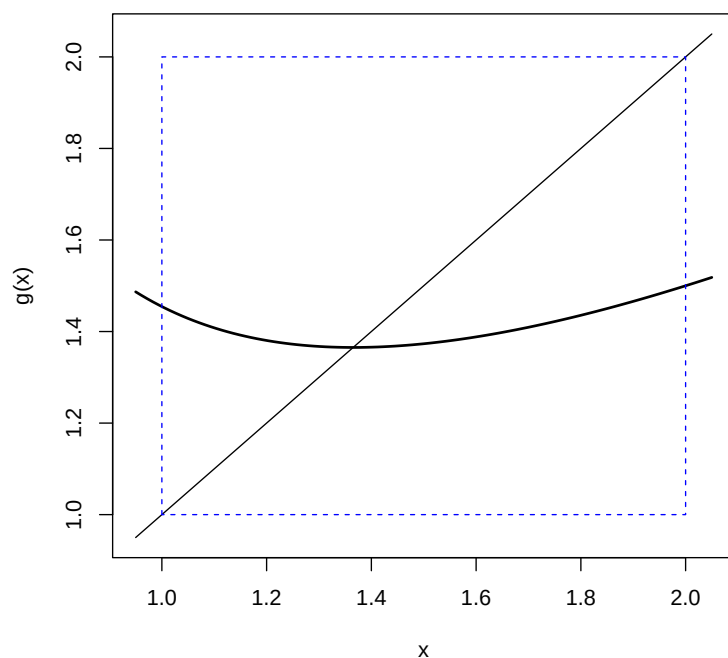


FIG. 5.8 - Fonction $g(x) = x - f(x)/f'(x)$, où $f(x) = x^3 + 4x^2 - 10$

Exemple 5.9. Cet exemple illustre l'importance d'utiliser, pour certaines fonctions, une valeur de départ près de la racine. La fonction $f(x) = (4x - 7)/(x - 2)$ a une racine en $x = 1,75$ et une asymptote en $x = 2$, ce qui cause quelques soucis. (Voir la [figure 5.9](#) pour un graphique de cette fonction.)

On peut tenter de trouver numériquement la racine de la fonction $f(x)$ avec l'algorithme de Newton-Raphson en utilisant

$$f(x) = \frac{4x - 7}{x - 2}$$

et

$$f'(x) = -\frac{1}{(x - 2)^2},$$

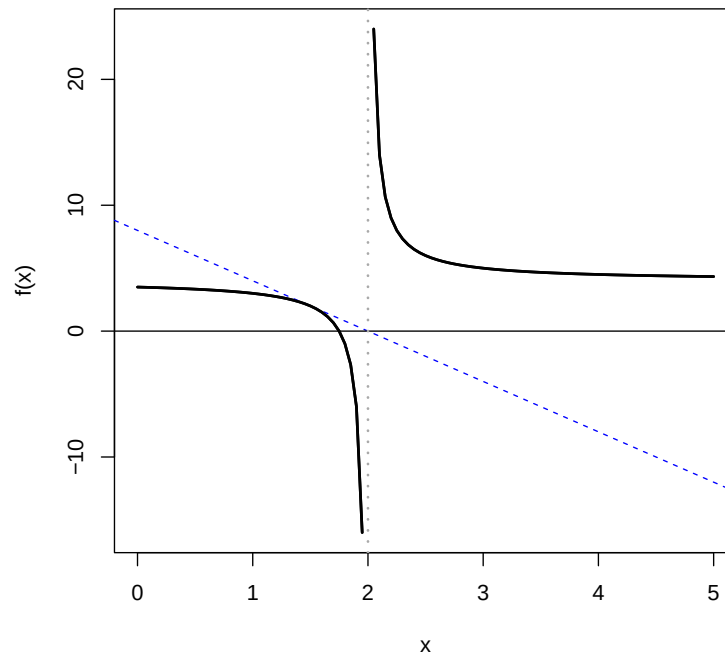


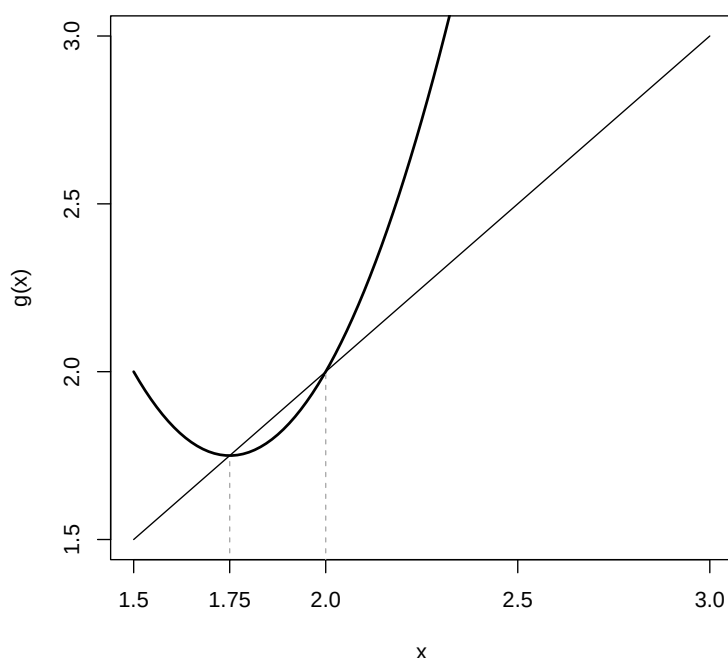
FIG. 5.9 - Fonction $f(x) = (4x - 7)/(x - 2)$ (ligne épaisse), asymptote (ligne pointillée) et tangente en $x = 1,5$ (ligne traitillée)

ou encore directement par l'algorithme du point fixe avec

$$\begin{aligned} g(x) &= x - \frac{f(x)}{f'(x)} \\ &= 4x^2 - 14x + 14. \end{aligned}$$

Le graphique de la fonction $g(x)$ ci-dessus se trouve à la [figure 5.10](#). On notera toutefois que le point fixe en $x = 2$ n'est pas une solution acceptable dans le problème d'origine.

On constate également dans cette dernière figure que les hypothèses pour l'existence et l'unicité d'un point fixe — et, donc, pour la convergence de la méthode de Newton-Raphson — ne sont pas rencontrées. La valeur de départ utilisée aura donc un impact sur la réponse obtenue.

FIG. 5.10 - Fonction $g(x) = 4x^2 - 14x + 14$ 

Complétez cet exemple en exécutant les lignes 269-302 du fichier de script `resolution_equations.R`.

□

Le principal inconvénient de la méthode de Newton-Raphson demeure le fait de devoir connaître la dérivée de la fonction f . Lorsque celle-ci s'avère difficile ou inefficace à calculer, on peut utiliser la *méthode de la sécante*. Cette méthode repose sur la sécante entre les deux essais précédents plutôt que sur la tangente en un essai pour déterminer la valeur de l'essai suivant ; voir la figure 5.11. Les valeurs des essais successifs sont alors données par :

$$x_n = x_{n-1} - \frac{f(x_{n-1})(x_{n-1} - x_{n-2})}{f(x_{n-1}) - f(x_{n-2})},$$

avec x_0, x_1 des valeurs de départ.

5.9 Indice pour le problème

Nous revenons au problème de départ exprimé sous la forme $f(x) = H(x) - 9 = 0$. La fonction des heures d'ensoleillement H étant simple à dériver, la méthode de Newton-Raphson est simple à appliquer. En effet,

$$\begin{aligned} f(x) &= H(x) - 9 \\ &= e^{0,001312x+2,778538} - e^{-0,001431x+2,016340} - 9, \end{aligned}$$

d'où

$$f'(x) = 0,001312e^{0,001312x+2,778538} + 0,001431e^{-0,001431x+2,016340}.$$

Il ne reste qu'à appliquer l'[algorithme 5.3](#) avec les fonctions $f(x)$ et $f'(x)$ ainsi qu'une valeur de départ $x_0 \in \{1, 2, \dots, 31\}$.

5.10 Fonctions d'optimisation de Excel et de R

Les méthodes de bisection, du point fixe, de Newton-Raphson et autres permettent de résoudre des équations à une variable de la forme $f(x) = 0$ ou $g(x) = x$. Il existe également des versions de ces méthodes pour les systèmes à plusieurs variables comme

$$\begin{aligned} f_1(x_1, x_2, x_3) &= 0 \\ f_2(x_1, x_2, x_3) &= 0 \\ f_3(x_1, x_2, x_3) &= 0. \end{aligned}$$

De tels systèmes d'équations surviennent plus souvent qu'autrement lors de l'optimisation d'une fonction. Par exemple, en recherchant le maximum ou le minimum d'une fonction $f(x, y)$, on souhaitera résoudre le système d'équations

$$\begin{aligned} \frac{\partial}{\partial x} f(x, y) &= 0 \\ \frac{\partial}{\partial y} f(x, y) &= 0. \end{aligned}$$

La grande majorité des suites logicielles comportent des outils d'optimisation de fonctions. Ce document passe en revue les fonctions disponibles dans Excel et dans R.

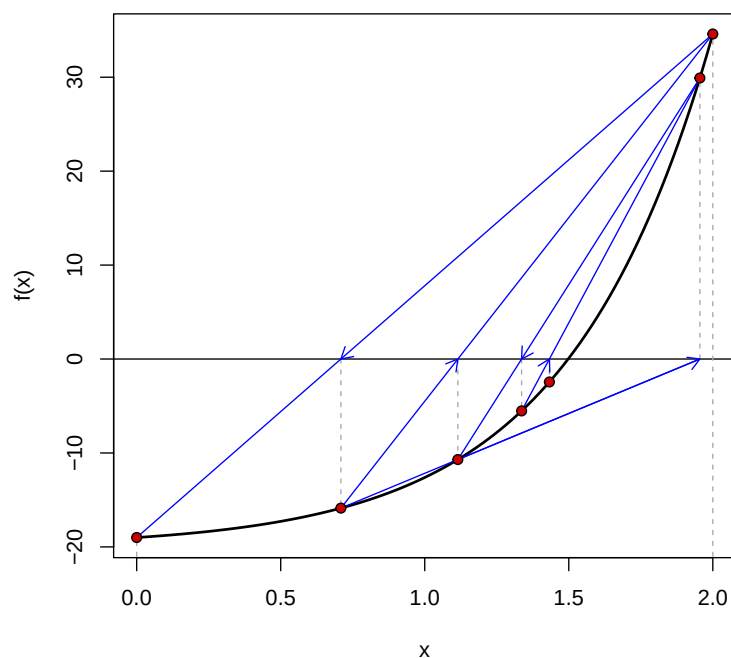


FIG. 5.11 – Illustration de la méthode de la sécante. Cette méthode requiert deux essais initiaux. Dans le graphique, il s'agit, dans l'ordre, du point à l'extrême gauche et celui à l'extrême droite. Les flèches illustrent comment est déterminé l'essai suivant. On trace la sécante entre deux points et le prochain essai est la valeur en abscisse où cette sécante croise l'axe.

5.10.1 Solveur de Excel

Le principal outil d'optimisation utilisé dans Excel est le Solveur. La rubrique d'aide de cet outil est complète et on trouvera plusieurs exemples dans le classeur SOLVAMP.XLS livré avec Excel.

Le reste de cette section est dévolu à des fonctions d'optimisation de R.

5.10.2 Fonction uniroot

La fonction `uniroot` recherche la racine d'une fonction dans un intervalle. C'est donc la fonction de base pour trouver la solution (unique) de

l'équation $f(x) = 0$ dans un intervalle déterminé.

5.10.3 Fonction `optimize`

La fonction `optimize` recherche le minimum local (par défaut) ou le maximum local d'une fonction dans un intervalle donné.

5.10.4 Fonction `nlm`

La fonction `nlm` minimise une fonction non linéaire sur un nombre arbitraire de paramètres.

5.10.5 Fonction `nlminb`

La fonction `nlminb` est similaire à `nlm`, sauf qu'elle permet de spécifier des bornes inférieure ou supérieure pour les paramètres. Attention, toutefois : les arguments de la fonction ne sont ni les mêmes, ni dans le même ordre que ceux de `nlm`.

5.10.6 Fonction `optim`

La fonction `optim` est l'outil d'optimisation tout usage de R. À ce titre, la fonction est souvent utilisée par d'autres fonctions. Elle permet de choisir parmi plusieurs algorithmes d'optimisation différents et, selon l'algorithme choisi, de fixer des seuils minimum ou maximum aux paramètres à optimiser.

5.10.7 `polyroot`

En terminant, un mot sur `polyroot()`, qui n'est pas à proprement parler une fonction d'optimisation, mais qui pourrait être utilisée dans ce contexte. La fonction `polyroot` calcule toutes les racines (complexes) du polynôme $\sum_{i=0}^n a_i x^i$. Le premier argument est le vecteur des coefficients $a_0, a_1, \dots, a_n$, dans cet ordre.



Prenez le temps d'étudier attentivement le code des lignes 305-428 du fichier de script `resolution_equations.R` reproduit à la section 5.15. Vous y trouverez des exemples d'utilisation de chacune des fonctions d'optimisation de R mentionnées ci-dessus.



5.11 Indice pour le problème

La fonction `uniroot` permet de trouver la solution à l'équation $f(x) = 0$ dans un intervalle déterminé. Nous pourrions donc l'utiliser dans la solution en fin de chapitre pour confirmer les autres résultats.

5.12 Astuce Ripley

J'ai appris le truc suivant dans une publication de Brian Ripley — un important développeur de R — dans les forums de discussion de R. Il m'a été utile à de nombreuses reprises, alors je le dissémine.

Une application statistique fréquente de l'optimisation est la maximisation numérique d'une fonction de vraisemblance ou, plus communément, la minimisation de la log-vraisemblance négative

$$-l(\theta) = - \sum_{i=1}^n \ln f(x_i; \theta).$$

Les fonctions d'optimisation sont d'ailleurs illustrées dans ce contexte dans le code informatique de la [section 5.15](#).

En actuariat, nous utilisons principalement des lois de probabilité dont les paramètres sont strictement positifs. Or, en pratique, il n'est pas rare que les fonctions d'optimisation s'égarent dans les valeurs négatives des paramètres. La fonction de densité n'étant pas définie, la log-vraisemblance vaut alors NaN et cela peut faire complètement dérailler la procédure d'optimisation ou, à tout le moins, susciter des doutes sur la validité de la réponse.

Afin de remédier à ce problème, l'Astuce Ripley™ propose d'estimer non pas les paramètres de la loi eux-mêmes, mais plutôt leurs logarithmes. Si l'on définit $\tilde{\theta} = \ln \theta$, alors on peut écrire la fonction de log-vraisemblance ci-dessus sous la forme

$$-l(\tilde{\theta}) = - \sum_{i=1}^n \ln f(x_i; e^{\tilde{\theta}}).$$

Dès lors, $\tilde{\theta}$ (qui peut représenter un ou plusieurs paramètres) demeure valide sur tout l'axe des réels, ce qui permet d'éviter bien des soucis de nature numérique lors de la minimisation de $-l(\tilde{\theta})$.

Évidemment, le résultat de l'optimisation est l'estimateur du maximum de vraisemblance de $\tilde{\theta}$. Il faudra donc veiller à faire la transformation inverse pour retrouver l'estimateur de θ .



Vous trouverez un exemple d'utilisation de l'Astuce Ripley aux lignes 431-458 du fichier de script `resolution_equations.R` reproduit à la [section 5.15](#).



Si l'exemple informatique n'est pas suffisant, la vidéo explicative sur l'[Astuce Ripley](#) vous permettra de comprendre le fonctionnement, au fond tout simple, de la technique.

5.13 Outils additionnels

Les paquetages disponibles sur CRAN fournissent plusieurs autres outils d'optimisation pour R. Pour un bon résumé des options disponibles, consulter la [CRAN Task View](#) consacrée à l'optimisation.



5.14 Solution du problème

Nous résolvons avec R notre problème consistant à trouver la valeur de x tel que $H(x) = 9$ avec les méthodes numériques étudiées dans ce chapitre. Les fonctions `bissection`, `pointfixe` et `nr` ci-dessous sont celles tirées du code informatique de la [section 5.15](#).

Méthode de bisection : résolution de $H(x) - 9 = 0$.

```
> H <- function(x)
+   exp(0.001312 * x + 2.778538) -
+   exp(-0.001431 * x + 2.016340)
> bissection(FUN = function(x) H(x) - 9,
+           lower = 1, upper = 31)
$root
[1] 13.00008

$nb.iter
[1] 22
```

Méthode du point fixe : résolution de $g(x) = x$ avec la fonction g de l'encadré de la [page 116](#).

```
> g <- function(x)
+   (log(9 + exp(-0.001431*x+2.016340)) - 2.778538) /
```

```
+      0.001312
> pointfixe(g, start = 15)
$fixed.point
[1] 13.00008

$nb.iter
[1] 18
```

Méthode de Newton-Raphson : résolution de $f(x) = H(x) - 9 = 0$ avec les fonctions f et f' de l'encadré de la [page 124](#).

```
> f <- function(x) H(x) - 9
> fp <- function(x)
+      0.001312 * exp(0.001312 * x + 2.778538) +
+      0.001431 * exp(-0.001431 * x + 2.016340)
> nr(f, fp, start = 15)
$root
[1] 13.00008

$nb.iter
[1] 2
```

Vérification avec la fonction `uniroot`.

```
> uniroot(f, c(1, 31))$root
[1] 13.00008
```

5.15 Code informatique

✓ Fichier d'accompagnement `resolution_equations.R`

```
12 ###
13 ### MÉTHODE DE BISSECTION
14 ###
15
16 ## Fonction pour trouver la solution de 'FUN'(x) = 0 par la
17 ## méthode de bisection, où 'FUN' est une fonction continue
18 ## sur l'intervalle ['lower', 'upper'].
19 bisection <- function(FUN, lower, upper, TOL = 1E-6,
20                      MAX.ITER = 100, echo = FALSE)
21 {
22     ## Cas triviaux où une borne est la solution
```

```
23     if (identical(FUN(lower), 0))
24         return(lower)
25     if (identical(FUN(upper), 0))
26         return(upper)
27
28     ## Bornes de départ inadéquates
29     if (FUN(lower) * FUN(upper) > 0)
30         stop('FUN(lower) and FUN(upper) must be of opposite signs')
31
32     x <- lower
33     i <- 1
34
35     repeat
36     {
37         xt <- x
38         x <- (lower + upper)/2
39         fx <- FUN(x)
40
41         if (echo)
42             print(c(lower, upper, x, fx))
43
44         if (abs(x - xt)/abs(x) < TOL)
45             break
46
47         if (MAX.ITER < (i <- i + 1))
48             stop('Maximum number of iterations reached
49                 without convergence')
50
51         if (FUN(lower) * fx > 0)
52             lower <- x
53         else
54             upper <- x
55     }
56     list(root = x, nb.iter = i)
57 }
58
59 ## RACINE D'UN POLYNÔME
60 ##
61 ## La fonction dont on cherche la racine
62 f <- function(x) x^3 + 4*x^2 - 10
63
64 ## Un graphique de la fonction permet de vérifier qu'elle ne
65 ## possède qu'une seule racine sur un intervalle donné ainsi
66 ## que des valeurs de départ pour l'algorithme.
67 curve(f(x), xlim = c(0, 3), lwd = 2) # graphique sur [0, 3]
```

```
68 abline(h = 0)                                # axe des abscisses
69
70 ## Résolution. Nous utilisons a = 1 et b = 2 comme valeurs de
71 ## départ. Nous pourrions choisir des valeurs de départ plus
72 ## près de la racine, ce qui accélérerait la résolution.
73 bisection(f, lower = 1, upper = 2, TOL = 1E-5, echo = TRUE)
74
75 ## VALEUR ACTUELLE
76 ##
77 ## La fonction dont on cherche la racine
78 f <- function(x) (1 - (1 + x)^(-10))/x - 8.2218
79
80 ## Graphique de la fonction sur un intervalle "raisonnable"
81 ## dans lequel on peut pressentir que se trouvera la réponse.
82 ## Ici, on y va pour un taux d'intérêt se trouvant entre 1 %
83 ## et 10 %.
84 curve(f(x), xlim = c(0.01, 0.10), lwd = 2)
85 abline(h = 0)
86
87 ## Le graphique permet de déterminer à l'oeil que la racine se
88 ## trouve entre 3 % et 4 %. Résolution avec la fonction
89 ## 'bisection'.
90 bisection(f, lower = 0.03, upper = 0.04)
91
92 ###
93 ### MÉTHODE DU POINT FIXE
94 ###
95
96 ## Fonction pour trouver la solution de 'FUN'(x) = x par la
97 ## méthode du point fixe à partir d'un essai initial 'start'.
98 ##
99 ## Je dois expliquer une petite subtilité dans la mise en
100 ## œuvre ci-dessous.
101 ##
102 ## Si l'argument 'echo' est 'TRUE', la fonction doit afficher
103 ## à l'écran la plus récente valeur de la variable 'x'. Sinon,
104 ## rien n'est affiché. La première opération de la boucle
105 ## 'repeat' serait donc
106 ##
107 ##     print(xt <- x)
108 ##
109 ## ou simplement
110 ##
111 ##     xt <- x
112 ##
```

```
113 ## selon que 'echo' est 'TRUE' ou 'FALSE', dans l'ordre.
114 ##
115 ## Or, il est inutile de tester la valeur de 'echo' à chaque
116 ## itération de la boucle, puisque la valeur de l'objet ne
117 ## change pas! J'utilise donc un petit truc: sauvegarder dans
118 ## un objet ('expr', ici) l'expression à évaluer dans la
119 ## boucle. Oui, c'est possible de faire ça en R! C'est
120 ## justement le rôle de la fonction 'expression'. On évalue
121 ## ensuite l'expression stockée dans l'objet avec la fonction
122 ## 'eval'.
123 pointfixe <- function(FUN, start, TOL = 1E-6, MAX.ITER = 100,
124                       echo = FALSE)
125 {
126   x <- start
127
128   if (echo)
129     expr <- expression(print(xt <- x))
130   else
131     expr <- expression(xt <- x)
132
133   i <- 0
134
135   repeat
136   {
137     eval(expr)
138
139     x <- FUN(xt)
140
141     if (abs(x - xt)/abs(x) < TOL)
142       break
143
144     if (MAX.ITER < (i <- i + 1))
145       stop('Maximum number of iterations reached
146            without convergence')
147   }
148   list(fixed.point = x, nb.iter = i)
149 }
150
151 ## VALEUR ACTUELLE (bis)
152 ##
153 ## La fonction dont on cherche le point fixe.
154 g <- function(x) (1 - (1 + x)^(-10))/8.2218
155
156 ## Un graphique de la fonction permet de vérifier que les
157 ## conditions pour qu'il existe un point fixe unique dans
```

```

158 ## l'intervalle [0,035, 0,040] sont satisfaites. Cependant, la
159 ## forme de la courbe (pente près de 1) nous indique que la
160 ## convergence sera relativement lente.
161 f <- function(x) x          # pour tracer la droite y = x
162 lim <- c(0.035, 0.040)     # intervalle [a, b]
163 curve(g, xlim = lim, ylim = lim, lwd = 2,
164        xlab = "i", ylab = "g(i)")
165 curve(f, add = TRUE)
166 polygon(rep(lim, each = 2), c(lim, rev(lim)),
167         lty = "dashed", border = "blue")
168
169 ## Résolution avec la fonction 'pointfixe'.
170 pointfixe(g, start = 0.0375, echo = TRUE)
171
172 ## POLYNÔME (bis)
173 ##
174 ## Nous cherchons la racine de  $f(x) = x^3 + 4 * x^2 - 10$  en
175 ## exprimant le problème sous forme de point fixe. Les cinq
176 ## fonctions ci-dessous sont toutes algébriquement
177 ## équivalentes, c'est-à-dire que  $g(x) = x$  lorsque  $f(x) = 0$ .
178 g1 <- function(x) x - x^3 - 4 * x^2 + 10
179 g2 <- function(x) sqrt(10/x - 4*x)
180 g3 <- function(x) sqrt(10 - x^3)/2
181 g4 <- function(x) sqrt(10/(4 + x))
182 g5 <- function(x) x - (x^3 + 4*x^2 - 10)/(3*x^2 + 8*x)
183
184 ## Si les fonctions sont algébriquement équivalentes, elles ne
185 ## le sont pas numériquement devant la méthode du point fixe.
186 ## Ainsi, avec la fonction 'g1', la procédure diverge.
187 ## (Remarque: la fonction 'pointfixe' ne prévoit pas de message
188 ## d'erreur pour ce cas. Qu'ajouteriez-vous à la fonction?)
189 pointfixe(g1, 1.5)
190 pointfixe(g1, 1.5, echo = TRUE) # plus évident ainsi
191
192 ## Avec la fonction 'g2', la procédure s'arrête lorsqu'il faut
193 ## calculer la racine carrée d'un nombre négatif.
194 pointfixe(g2, 1.5)
195
196 ## Avec les trois autres fonctions, la méthode du point fixe
197 ## est extrêmement rapide et précise. Une analyse rapide des
198 ## graphiques fournis dans le chapitre nous permettrait de
199 ## déterminer avec quelle fonction la convergence serait la
200 ## plus rapide. En effet, c'est la fonction 'g5' qui a la
201 ## pente la plus faible près de son point fixe.
202 pointfixe(g3, 1.5)

```

```

203 pointfixe(g4, 1.5)
204 pointfixe(g5, 1.5)
205
206 ###
207 ### MÉTHODE DE NEWTON-RAPHSON
208 ###
209
210 ## Fonction pour trouver la solution de 'FUN'(x) = x par la
211 ## méthode de Newton-Raphson à partir de sa dérivée 'FUNp' et
212 ## d'un essai initial 'start'.
213 ##
214 ## On ajoute une amélioration par rapport aux fonctions
215 ## 'bissection' et 'pointfixe', soit la possibilité de passer
216 ## des arguments additionnels aux fonctions 'FUN' et 'FUNp'
217 ## via l'argument '...'
218 nr <- function(FUN, FUNp, start, TOL = 1E-6,
219                MAX.ITER = 100, echo = FALSE, ...)
220 {
221   x <- start
222
223   if (echo)
224     expr <- expression(print(xt <- x))
225   else
226     expr <- expression(xt <- x)
227
228   i <- 0
229
230   repeat
231   {
232     eval(expr)
233
234     x <- xt - FUN(xt, ...)/FUNp(xt, ...)
235
236     if (abs(x - xt)/abs(x) < TOL)
237       break
238
239     if (MAX.ITER < (i <- i + 1))
240       stop('Maximum number of iterations reached
241            without convergence')
242   }
243   list(root = x, nb.iter = i)
244 }
245
246 ### VALEUR ACTUELLE (ter)
247 ###

```



```
248 ## Nous effectuons de nouveau le calcul d'un taux de
249 ## rendement, cette fois avec la méthode de Newton-Raphson. La
250 ## convergence devrait être plus rapide qu'avec la méthode du
251 ## point fixe puisque la fonction  $g(i) = i - f(i)/f'(i)$  est
252 ## plus plate que celle utilisée avec la méthode du point
253 ## fixe. Premièrement, la fonction dont on cherche la racine.
254 f <- function(i) (1 - (1 + i)^(-10))/i - 8.2218
255
256 ## Sa dérivée.
257 fp <- function(i) (10 * i * (1 + i)^(-11) + (1 + i)^(-10) - 1)/i^2
258
259 ## Résolution avec la fonction 'nr'.
260 nr(f, fp, start = 0.0375, echo = TRUE)
261
262 ## À noter que si la fonction g est définie adéquatement, nous
263 ## pouvons de manière tout aussi équivalente utiliser la
264 ## fonction 'pointfixe' pour effectuer les itérations de la
265 ## méthode de Newton-Raphson.
266 g <- function(i) i - f(i)/fp(i)
267 pointfixe(g, 0.0375, echo = TRUE)
268
269 ## CAS PATHOLOGIQUE
270 ##
271 ## Les fonctions f(x), f'(x) et g(x) définies dans le texte de
272 ## l'exemple.
273 f <- function(x) ifelse(x == 2, NA, (4 * x - 7)/(x - 2))
274 fp <- function(x) ifelse(x == 2, NA, -1/(x - 2)^2)
275 g <- function(x) 4 * x^2 - 14 * x + 14
276
277 ## Vérifions que, étant donné la forme des fonctions 'f' et
278 ## 'g', la valeur de départ utilisée dans les méthodes de
279 ## Newton-Raphson ou du point fixe a un impact sur la réponse
280 ## obtenue. Avec une valeur de départ 'start' = 1,625 < 1,75,
281 ## la convergence se fait vers la bonne racine.
282 nr(f, fp, 1.625, echo = TRUE)
283 pointfixe(g, 1.625)
284
285 ## Nous pouvons vérifier sur le graphique de la fonction 'g'
286 ## qu'avec une valeur de départ entre 1,75 et 2, la procédure
287 ## itérative convergera aussi vers la bonne réponse.
288 nr(f, fp, 1.875, echo = TRUE)
289 pointfixe(g, 1.875, echo=TRUE)
290
291 ## La tangente en x = 1,5 tracée sur le graphique de la
292 ## fonction 'f' montre que 'start' = 1,5 constitue un mauvais
```

```

293 ## choix car  $g(1,5) = 2$ . La procédure itérative converge donc
294 ## vers une valeur non admissible.
295 nr(f, fp, 1.5)           # division par 0
296 pointfixe(g, 1.5)       # point fixe non admissible
297
298 ## Pour toute valeur de départ supérieure à 2, la procédure
299 ## itérative diverge. On peut vérifier ce fait dans les
300 ## graphiques de 'f' et de 'g'.
301 nr(f, fp, 3, echo = TRUE)
302 pointfixe(g, 3, echo = TRUE)
303
304 ###
305 #### FONCTIONS D'OPTIMISATION DE R
306 ####
307
308 ## FONCTION 'uniroot'
309 ##
310 ## La fonction 'uniroot' recherche la racine d'une fonction
311 ## 'f' dans un intervalle spécifié soit comme une paire de
312 ## valeurs dans un argument 'interval', soit via des arguments
313 ## 'lower' et 'upper'.
314 ##
315 ## On calcule la solution de l'équation  $x - 2^{(-x)} = 0$  dans
316 ## l'intervalle  $[0, 1]$ .
317 f <- function(x) x - 2(-x)      # fonction
318 uniroot(f, c(0, 1))             # appel simple
319 uniroot(f, lower = 0, upper = 1) # équivalent
320
321 ## On peut aussi utiliser 'uniroot' avec une fonction anonyme.
322 uniroot(function(x) x - 2(-x), lower = 0, upper = 1)
323
324 ## FONCTION 'optimize'
325 ##
326 ## On cherche le maximum local de la densité d'une loi bêta
327 ## dans l'intervalle (0, 1), son domaine de définition. (Ce
328 ## problème est facile à résoudre explicitement.)
329 ##
330 ## Les arguments de 'optimize' sont essentiellement les mêmes
331 ## que ceux de 'uniroot'. Ici, on utilise aussi l'argument
332 ## '...' pour passer les paramètres de la loi bêta à 'dbeta'.
333 ##
334 ## Par défaut, la fonction recherche un minimum. Il faut donc
335 ## lui indiquer de rechercher plutôt un maximum.
336 optimize(dbeta, interval = c(0, 1), maximum = TRUE,
337          shape1 = 3, shape2 = 2)

```

```
338
339 ## On pourrait aussi avoir recours à une fonction auxiliaire.
340 ## Moins élégant et moins flexible.
341 f <- function(x) dbeta(x, 3, 2)
342 optimize(f, lower = 0, upper = 1, maximum = TRUE)
343
344 ## FONCTION 'nlm'
345 ##
346 ## Pour la suite, nous allons donner des exemples
347 ## d'utilisation des fonctions d'optimisation dans un contexte
348 ## d'estimation des paramètres d'une loi gamma par la méthode
349 ## du maximum de vraisemblance.
350 ##
351 ## On commence par se donner un échantillon aléatoire de la
352 ## loi. Évidemment, pour ce faire nous devons connaître les
353 ## paramètres de la loi. C'est un exemple fictif.
354 set.seed(1) # toujours le même échantillon
355 x <- rgamma(10, 5, 2)
356
357 ## Les estimateurs du maximum de vraisemblance des paramètres
358 ## 'shape' et 'rate' de la loi gamma sont les valeurs qui
359 ## maximisent la fonction de vraisemblance
360 ##
361 ## prod(dgamma(x, shape, rate))
362 ##
363 ## ou, de manière équivalente, qui minimisent la fonction de
364 ## log-vraisemblance négative
365 ##
366 ## -sum(log(dgamma(x, shape, rate))).
367 ##
368 ## On remarquera au passage que les fonctions de calcul de
369 ## densités de lois de probabilité dans R ont un argument
370 ## 'log' qui, lorsque TRUE, retourne la valeur du logarithme
371 ## (naturel) de la densité de manière plus précise qu'en
372 ## prenant le logarithme après coup. Ainsi, pour faire le
373 ## calcul ci-dessus, on optera plutôt, pour l'expression
374 ##
375 ## -sum(dgamma(x, shape, rate, log = TRUE))
376 ##
377 ## La fonction 'nlm' suppose que la fonction à optimiser,
378 ## passée en premier argument, a elle-même comme premier
379 ## argument le vecteur 'p' des paramètres à optimiser. Le
380 ## second argument de 'nlm' est un vecteur de valeurs de
381 ## départ, une pour chaque paramètre.
382 ##
```

```

383 ## Ainsi, pour trouver les estimateurs du maximum de
384 ## vraisemblance avec la fonction 'nlm' pour l'échantillon
385 ## ci-dessus, on doit d'abord définir une fonction auxiliaire
386 ## conforme aux attentes de 'nlm' pour calculer la fonction de
387 ## log-vraisemblance (à un signe près).
388 f <- fonction(p, x) -sum(dgamma(x, p[1], p[2], log = TRUE))
389
390 ## L'appel de 'nlm' est ensuite tout simple. Remarquer comment
391 ## on passe notre échantillon aléatoire (contenu dans l'objet
392 ## 'x') comme second argument à 'f' via l'argument '...' de
393 ## 'nlm'. Le fait que l'argument de 'f' et l'objet contenant
394 ## les valeurs portent le même nom est sans importance. R sait
395 ## faire la différence entre l'un et l'autre.
396 nlm(f, c(1, 1), x = x)
397
398 ## FONCTION 'nlminb'
399 ##
400 ## L'utilisation de la fonction 'nlminb' peut s'avérer
401 ## intéressante dans notre contexte puisque l'on sait que les
402 ## paramètres d'une loi gamma sont strictement positifs.
403 nlminb(c(1, 1), f, x = x, lower = 0, upper = Inf)
404
405 #### FONCTION 'optim'
406 ##
407 ## La fonction 'optim' est très puissante, mais requiert aussi
408 ## une bonne dose de prudence. Ses principaux arguments sont:
409 ##
410 ## par: un vecteur contenant les valeurs initiales des
411 ## paramètres;
412 ## fn: la fonction à minimiser. Le premier argument de fn
413 ## doit être le vecteur des paramètres.
414 ##
415 ## Comme pour les autres fonctions étudiées ci-dessus, on peut
416 ## passer des arguments à 'fn' (les données, par exemple) par
417 ## le biais de l'argument '...' de 'optim'.
418 optim(c(1, 1), f, x = x)
419
420 ## FONCTION 'polyroot'
421 ##
422 ## Racines du polynôme  $x^3 + 4x^2 - 10$ . Les réponses sont
423 ## données sous forme de nombre complexe. Utiliser les
424 ## fonctions 'Re' et 'Im' pour extraire les parties réelles et
425 ## imaginaires des nombres, respectivement.
426 polyroot(c(-10, 0, 4, 1)) # racines
427 Re(polyroot(c(-10, 0, 4, 1))) # parties réelles

```

```

428 Im(polyroot(c(-10, 0, 4, 1))) # parties imaginaires
429
430 ###
431 ### ASTUCE RIPLEY
432 ###
433
434 ## L'optimisation avec la fonction 'nlm', ci-dessus, a généré
435 ## des avertissements? C'est parce que la fonction
436 ## d'optimisation s'est égarée dans les valeurs négatives,
437 ## alors que les paramètres d'une gamma sont strictement
438 ## positifs. Cela arrive souvent en pratique et cela peut
439 ## faire complètement dérailler la procédure d'optimisation
440 ## (c'est-à-dire: pas de convergence).
441 ##
442 ## L'Astuce Ripley consiste à remédier à ce problème en
443 ## estimant plutôt les logarithmes des paramètres. Pour ce
444 ## faire, il s'agit de réécrire la log-vraisemblance comme une
445 ## fonction du logarithme des paramètres, mais de la calculer
446 ## avec les véritables paramètres.
447 f2 <- fonction(logp, x)
448 {
449     p <- exp(logp)          # retour aux paramètres originaux
450     -sum(dgamma(x, p[1], p[2], log = TRUE))
451 }
452 nlm(f2, c(0, 0), x = x)
453
454 ## Les valeurs obtenues ci-dessus sont toutefois les
455 ## estimateurs des logarithmes des paramètres de la loi gamma.
456 ## On retrouve les estimateurs des paramètres en prenant
457 ## l'exponentielle des réponses.
458 exp(nlm(f2, c(0, 0), x = x)$estimate)

```

5.16 Exercices

- 5.1** En vous basant sur les fonctions `bissection`, `pointfixe` et `nr` présentées dans le code de la [section 5.15](#), écrire une fonction R pour effectuer les calculs de l'algorithme de la sécante. Outre les arguments communs à toutes les fonctions que sont le niveau de tolérance ε , le nombre maximal d'itérations $N_{\max}$ et une valeur booléenne spécifiant si les valeurs successives des itérations doivent être affichées à l'écran, la fonction doit compter les arguments $f(x)$, x_0 et x_1 .

- 5.2** Trouver la solution des équations suivantes par les méthodes de bisection, de Newton-Raphson et de la sécante.
- a) $x^3 - 2x^2 - 5 = 0$ pour $1 \leq x \leq 4$
 - b) $x^3 + 3x^2 - 1 = 0$ pour $-4 \leq x \leq 0$
 - c) $x - 2^{-x} = 0$ pour $0 \leq x \leq 1$
 - d) $e^x + 2^{-x} + 2 \cos x - 6 = 0$ pour $1 \leq x \leq 2$
 - e) $e^x - x^2 + 3x - 2 = 0$ pour $0 \leq x \leq 1$
- 5.3** Déterminer la valeur numérique de $\sqrt{2}$ à l'aide de la méthode de bisection dans l'intervalle $[0, 2]$ avec dix itérations. Comparer avec la vraie valeur.
- 5.4** Soit $\{x_n\}$ une suite définie par

$$x_n = \sum_{k=1}^n \frac{1}{k}.$$

Démontrer que $\lim_{n \rightarrow \infty} (x_n - x_{n-1}) = 0$, mais que la suite diverge néanmoins. Ceci illustre que l'erreur absolue peut être un mauvais critère d'arrêt dans les méthodes numériques.

- 5.5** Soit la fonction $g(x) = 2^{-x}$ sur l'intervalle $[0, 1]$.
- a) Vérifier si les hypothèses du [théorème 5.1](#) quant à l'existence et l'unicité d'un point fixe dans $[0, 1]$ sont satisfaites.
 - b) Déterminer graphiquement l'existence et l'unicité d'un point fixe de $g(x)$ dans $[0, 1]$ puis, le cas échéant, calculer ce point fixe.
- 5.6** Vérifier que les cinq fonctions $g_1, \dots, g_5$ de l'[exemple 5.6](#) ont toutes un point fixe en x^* lorsque $f(x^*) = 0$, où $f(x) = x^3 + 4x^2 - 10$.
- 5.7** Soit $g(x) = 4x^2 - 14x + 14$. Pour quels intervalles de valeurs de départ la procédure de point fixe converge-t-elle et diverge-t-elle ?
- 5.8** Les trois fonctions ci-dessous sont toutes des candidates pour faire l'approximation, par la méthode du point fixe, de $\sqrt[3]{21}$:

$$g_1(x) = \frac{20x + 21x^{-2}}{21}$$

$$g_2(x) = x - \frac{x^3 - 21}{3x^2}$$

$$g_3(x) = x - \frac{x^4 - 21x}{x^2 - 21}.$$

Classer ces fonctions en ordre décroissant de vitesse de convergence de l'algorithme du point fixe. *Astuce* : comparer les valeurs des dérivées autour du point fixe.

- 5.9** Démontrer, à l'aide du théorème du point fixe, que la fonction $g(x) = 2^{-x}$ possède un point fixe unique dans l'intervalle $[\frac{1}{3}, 1]$. Calculer par la suite ce point fixe à l'aide de la fonction `pointfixe`.
- 5.10** Vérifier graphiquement que le [théorème 5.1](#) demeure valide si la condition $|g'(x)| \leq k < 1$ est remplacée par $g'(x) \leq k < 1$.
- 5.11** Utiliser la méthode de Newton-Raphson pour trouver le point sur la courbe $y = x^2$ le plus près du point $(1, 0)$. *Astuce* : minimiser la distance entre le point $(1, 0)$ et le point (x, x^2) .
- 5.12** Le taux de rendement interne d'une série de flux financiers $\{CF_t\}$ est le taux i tel que

$$\sum_{t=0}^n \frac{CF_t}{(1+i)^t} = 0.$$

Écrire une fonction R permettant de calculer le taux de rendement interne d'une série de flux financiers quelconque à l'aide de la méthode de Newton-Raphson. Les arguments de la fonction sont un vecteur `CF` et un scalaire `erreur.max`. Au moins un élément de `CF` doit être négatif pour représenter une sortie de fonds. Dans tous les cas, utiliser $i = 0,05$ comme valeur de départ.

- 5.13** a) Composer une fonction R pour estimer par la méthode du maximum de vraisemblance le paramètre θ d'une loi gamma de paramètre de forme $\alpha = 3$ et de paramètre d'échelle $\theta = 1/\lambda$ — de moyenne 3θ , donc — à partir d'un échantillon aléatoire $x_1, \dots, x_n$. *Astuce* : maximiser la fonction de log-vraisemblance plutôt que la fonction de vraisemblance.
- b) Simuler 20 observations d'une loi gamma avec paramètre de forme $\alpha = 3$ et moyenne 3 000, puis estimer le paramètre d'échelle θ par le maximum de vraisemblance.

- 5.14** À l'aide de la méthode du point fixe, trouver le taux d'intérêt i tel que

$$a_{10|i}^{(12)} = \frac{1 - (1+i)^{-10}}{i^{(12)}} = 8,$$

où

$$\left(1 + \frac{i^{(12)}}{12}\right)^{12} = 1 + i.$$

Comparer les résultats obtenus avec la méthode de Newton-Raphson.

5.15 Trouver la solution des équations suivantes à l'aide d'une fonction R appropriée.

- a) $x^3 - 2x^2 - 5 = 0$ pour $1 \leq x \leq 4$
- b) $x^3 + 3x^2 - 1 = 0$ pour $-4 \leq x \leq 0$
- c) $x - 2^{-x} = 0$ pour $0 \leq x \leq 1$
- d) $e^x + 2^{-x} + 2 \cos x - 6 = 0$ pour $1 \leq x \leq 2$
- e) $e^x - x^2 + 3x - 2 = 0$ pour $0 \leq x \leq 1$

5.16 La fonction de densité de probabilité et la fonction de répartition de la loi de Pareto de paramètres α et λ sont, respectivement,

$$f(x) = \frac{\alpha \lambda^\alpha}{(x + \lambda)^{\alpha+1}}$$

et

$$F(x) = 1 - \left(\frac{\lambda}{x + \lambda} \right)^\alpha.$$

Calculer les estimateurs du maximum de vraisemblance des paramètres de la Pareto à partir d'un échantillon aléatoire obtenu par simulation avec la commande

```
| > x <- lambda * (runif(100)^(-1/alpha) - 1)
```

pour des valeurs de α et λ choisies.

Réponses

5.7 Converge pour $x_0 \in [1,5,2]$; diverge pour $x_0 \leq 1,5$ et $x_0 > 2$

5.8 $g_2, g_1; g_3$ ne converge pas

5.11 0,589755

5.14 0,0470806

6 Intégration numérique

Objectifs du chapitre

- ▶ Avoir une connaissance générale de ce qu'est un polynôme d'interpolation de Lagrange et comment il peut servir dans le contexte de l'intégration numérique.
- ▶ Calculer la valeur approximative d'une intégrale définie à l'aide des méthodes du point milieu, du trapèze, de Simpson et de Simpson 3/8.
- ▶ Développer les formules d'approximation composées de chacune des méthodes d'intégration numériques présentées dans le chapitre.

Il n'est pas rare de devoir calculer l'intégrale d'une fonction qui n'admet pas de primitive sous forme explicite ou dont la primitive est très difficile à calculer. Dans de tels cas, l'intégration numérique permet d'obtenir une approximation — parfois excellente — de l'intégrale recherchée.

Les méthodes d'approximation qui seront étudiées dans ce chapitre reposent toutes sur le remplacement de la fonction à intégrer par une « bonne » approximation sur un intervalle donné. Cette approximation devra évidemment être simple à intégrer, autrement aucun gain n'est réalisé. Or, quelles fonctions sont simples à intégrer sinon les polynômes ?

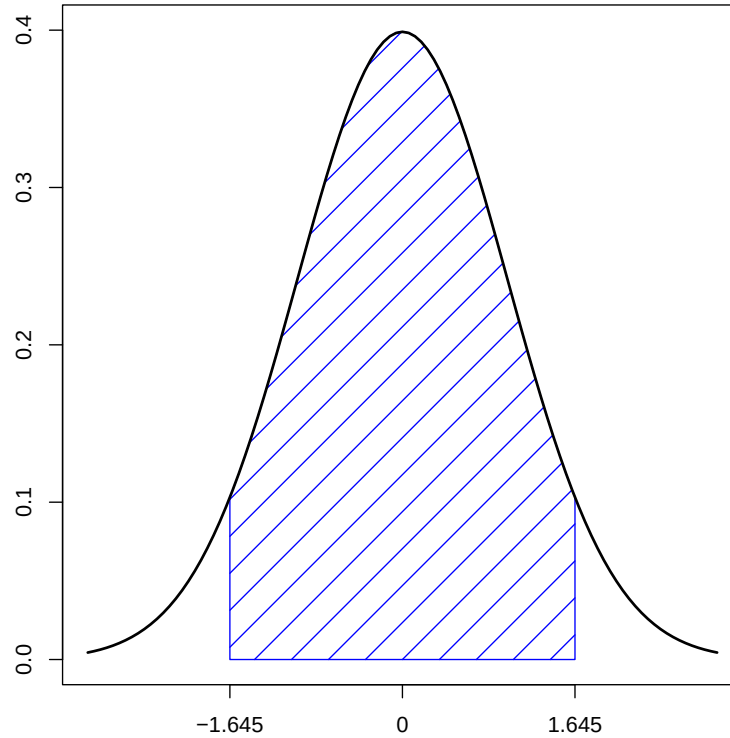


6.1 Énoncé du problème

Au chapitre 3, nous avons tiré profit de la méthode Monte Carlo pour calculer l'intégrale de Gauss sur tout son domaine sans passer par les coordonnées polaires. Nous reprenons ce contexte ici.

Cette fois, nous cherchons à calculer l'intégrale

$$I = \int_{-1,645}^{1,645} \frac{1}{\sqrt{2\pi}} e^{-x^2/2} dx.$$

FIG. 6.1 – Aire à calculer avec l'intégrale I

Il s'agit bien sûr d'un résultat connu : nous pouvons reconnaître l'expression de la probabilité sous la densité d'une loi normale centrée réduite entre ses 5^e et 95^e centiles ; voir la [figure 6.1](#). Par conséquent, nous savons que la valeur de l'intégrale est 0,90. Cela nous donnera un point de référence.

6.2 Polynômes d'interpolation de Lagrange

Un résultat connu en analyse mathématique sous le nom de Théorème d'approximation de Weierstrass établit que l'on peut faire l'approximation de toute fonction continue par un polynôme de degré suffisant. Formellement, soit

$$P_n(x) = a_0 + a_1x + \cdots + a_{n-1}x^{n-1} + a_nx^n$$

un polynôme de degré n . Si f est une fonction continue sur un intervalle $[a, b]$, alors pour tout $\varepsilon > 0$ il existe un polynôme $P(x)$ tel que

$$|f(x) - P(x)| < \varepsilon$$

pour tout $x \in [a, b]$.

La notion d'approximation d'une fonction par un polynôme est beaucoup utilisée en analyse numérique puisque les polynômes sont simples à dériver et à intégrer, et que leurs dérivées et intégrales sont elles-mêmes des polynômes.

Un type de polynôme d'approximation que vous connaissez déjà est le polynôme (ou développement) de Taylor autour d'un point x_0 . Cependant, les polynômes de Taylor sont conçus pour être précis autour de x_0 et non sur tout un intervalle. Dans le contexte de l'intégration numérique, où nous voudrions remplacer la fonction à intégrer par un polynôme, nous aurons besoin d'une bonne approximation sur tout le domaine d'intégration.

C'est là qu'interviennent les polynômes d'interpolation de Lagrange. Soit $x_0, x_1, \dots, x_n$ un ensemble de $n + 1$ points distincts et f une fonction qui passe par ces points (ou *nœuds*). Le polynôme d'interpolation de Lagrange de degré n de la fonction f est

$$P_n(x) = f(x_0)L_0(x) + f(x_1)L_1(x) + \dots + f(x_n)L_n(x) = \sum_{k=0}^n f(x_k)L_k(x),$$

où

$$\begin{aligned} L_k(x) &= \frac{(x - x_0)(x - x_1) \dots (x - x_{k-1})(x - x_{k+1}) \dots (x - x_n)}{(x_k - x_0)(x_k - x_1) \dots (x_k - x_{k-1})(x_k - x_{k+1}) \dots (x_k - x_n)} \\ &= \prod_{\substack{i=0 \\ i \neq k}}^n \frac{(x - x_i)}{(x_k - x_i)}. \end{aligned}$$

Remarquer que le terme $(x - x_k)$ n'apparaît pas au numérateur de la fonction $L_k(x)$.

On peut démontrer (voir, par exemple, [Burden et Faires, 2011](#)) que le polynôme $P(x)$ peut être aussi près que l'on veut de la fonction $f(x)$.

Exemple 6.1. Le polynôme d'interpolation de Lagrange de premier degré de la fonction f est

$$\begin{aligned} P_1(x) &= f(x_0)L_0(x) + f(x_1)L_1(x) \\ &= f(x_0) \frac{(x - x_1)}{(x_0 - x_1)} + f(x_1) \frac{(x - x_0)}{(x_1 - x_0)}. \end{aligned}$$

Le polynôme de second degré est, quant à lui :

$$\begin{aligned} P_2(x) &= f(x_0)L_0(x) + f(x_1)L_1(x) + f(x_2)L_2(x) \\ &= f(x_0) \frac{(x-x_1)(x-x_2)}{(x_0-x_1)(x_0-x_2)} + f(x_1) \frac{(x-x_0)(x-x_2)}{(x_1-x_0)(x_1-x_2)} \\ &\quad + f(x_2) \frac{(x-x_0)(x-x_1)}{(x_2-x_0)(x_2-x_1)}. \end{aligned}$$

Supposons que $f(x) = 1/x$ et que les polynômes d'interpolation passent par les nœuds $x_0 = 2$, $x_1 = 2,75$ et $x_2 = 4$. On a alors

$$\begin{aligned} P_1(x) &= \frac{1}{2} \frac{(x-2,75)}{(2-2,75)} + \frac{1}{2,75} \frac{(x-2)}{(2,75-2)} \\ &= -\frac{2}{3}(x-2,75) + \frac{16}{33}(x-2) \\ &= -\frac{6}{33}x + \frac{171}{198} \end{aligned}$$

et

$$\begin{aligned} P_2(x) &= \frac{1}{2} \frac{(x-2,75)(x-4)}{(2-2,75)(2-4)} + \frac{1}{2,75} \frac{(x-2)(x-4)}{(2,75-2)(2,75-4)} \\ &\quad + \frac{1}{4} \frac{(x-2)(x-2,75)}{(4-2)(4-2,75)} \\ &= \frac{1}{3}(x-2,75)(x-4) - \frac{64}{165}(x-2)(x-4) + \frac{1}{10}(x-2)(x-2,75) \\ &= \frac{1}{22}x^2 - \frac{35}{88}x + \frac{49}{44}. \end{aligned}$$

Des approximations de $f(3) = 1/3$ sont donc $P_1(3) \approx 0,31818$ et $P_2(3) \approx 0,32955$. $\square$



La fonction `poly.calc` du paquetage **polynom** permet de calculer le polynôme d'interpolation de Lagrange de degré n dans $\mathbb{R}$. Étudiez et exécutez les lignes 13-36 du fichier de script `integration_numerique.R` reproduit à la section 5.15 pour vérifier les calculs de l'exemple 6.1.

6.3 Principes généraux d'intégration numérique

Supposons que l'on cherche à calculer $\int_a^b f(x) dx$. Toutes les méthodes d'intégration numérique reposent, en premier lieu, sur le découpage du domaine (a, b) en n intervalles. Cela permet d'évaluer l'intégrale comme une

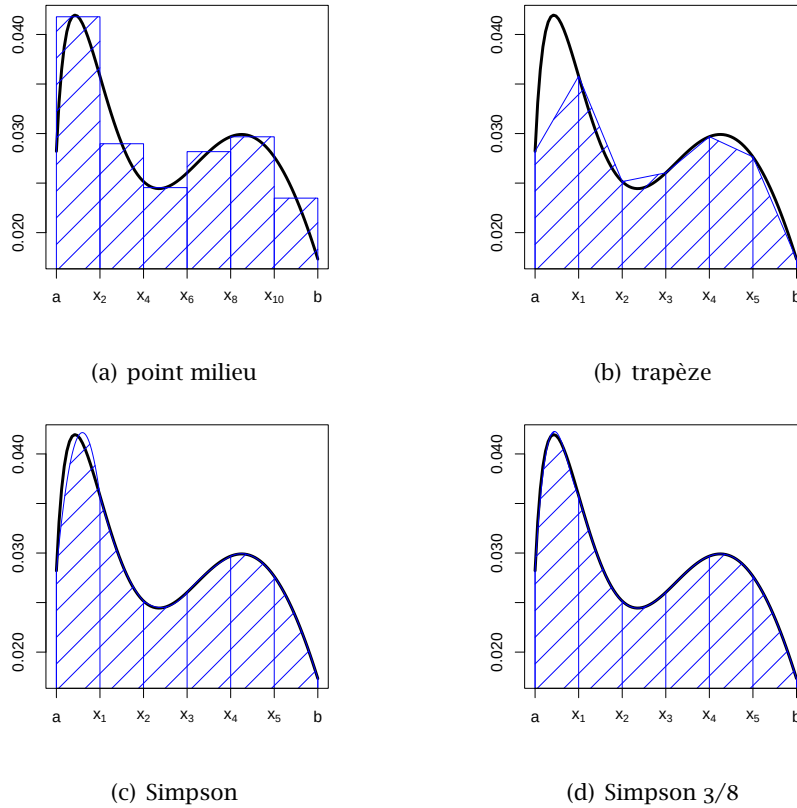


FIG. 6.2 – Procédures d'approximation de quatre méthodes d'intégration numérique

somme d'intégrales sur chacun de ces intervalles :

$$\int_a^b f(x) dx = \sum_{j=0}^{n-1} \int_{x_j}^{x_{j+1}} f(x) dx.$$

Les diverses méthodes d'intégration numérique diffèrent par la suite essentiellement par l'approximation de la fonction $f(x)$ sur l'intervalle (x_j, x_{j+1}) utilisée afin de rendre l'intégrale $\int_{x_j}^{x_{j+1}} f(x) dx$ simple à calculer.

La [figure 6.2](#) illustre les procédures d'approximation de quatre méthodes courantes d'intégration numérique.

Les formules d'intégration numérique présentées dans la suite découlent de la procédure suivante : le domaine d'intégration (a, b) est découpé en n intervalles de longueur égale. Chacun de ces intervalles est à son tour divisé

en m sous-intervalles de longueur égale, pour un total de nm points. On a alors

$$\begin{aligned} \int_a^b f(x) dx &= \int_{x_0}^{x_m} f(x) dx + \int_{x_m}^{x_{2m}} f(x) dx + \cdots + \int_{x_{(n-1)m}}^{x_{nm}} f(x) dx \\ &= \sum_{j=0}^{n-1} \int_{x_{jm}}^{x_{(j+1)m}} f(x) dx, \end{aligned}$$

où $x_0 = a$ et $x_{nm} = b$. L'approximation numérique se trouve dans l'évaluation de l'intégrale du côté droit de la dernière équation. Pour toutes les méthodes étudiées dans ce chapitre sauf la méthode du point milieu, la fonction f sera remplacée par un polynôme d'interpolation de Lagrange.

Afin de ne pas alourdir inutilement la notation, les formules d'approximation de l'intégrale

$$\int_{x_{jm}}^{x_{(j+1)m}} f(x) dx$$

seront présentées pour le cas $j = 0$ seulement dans les sections suivantes.

6.4 Méthode du point milieu

La méthode du point milieu est la plus simple et la plus intuitive méthode d'intégration numérique. Les intervalles sont divisés en $m = 2$ parties et la valeur de la fonction f sur l'intervalle (x_0, x_2) est estimée par $f(x_1)$ (figure 6.3(a)). Ainsi, on a l'approximation

$$\int_{x_0}^{x_2} f(x) dx \approx 2hf(x_1), \quad (6.1)$$

où $h = x_2 - x_1 = x_1 - x_0$.

La formule composée pour l'approximation par la méthode du point milieu de $\int_a^b f(x) dx$ avec n intervalles est

$$\int_a^b f(x) dx \approx 2h \sum_{j=0}^{n-1} f(x_{2j+1}) \quad (6.2)$$

où $h = (b - a)/(2n)$ et $x_j = a + jh$.

6.5 Méthode du trapèze

Les trois prochaines méthodes d'intégration numérique sont basées sur l'approximation de la fonction f sur un intervalle par un polynôme d'interpolation de Lagrange de degré m .

La méthode du trapèze utilise un polynôme du premier degré ($m = 1$) pour faire l'approximation de la valeur de $f(x)$ sur l'intervalle (x_0, x_1) , ce qui est équivalent à une simple interpolation linéaire (figure 6.3(b)). On a donc

$$\int_{x_0}^{x_1} f(x) dx \approx \frac{h}{2} [f(x_0) + f(x_1)], \quad (6.3)$$

où $h = x_1 - x_0$. Contrairement à ce que peut laisser croire la figure 6.2, cette méthode est généralement plus précise que la méthode du point milieu.

La formule composée pour l'approximation de $\int_a^b f(x) dx$ par la méthode du trapèze avec n intervalles est

$$\int_a^b f(x) dx \approx \frac{h}{2} \left[f(a) + f(b) + 2 \sum_{j=1}^{n-1} f(x_j) \right], \quad (6.4)$$

où $h = (b - a)/n$ et $x_j = a + jh$.

6.6 Méthode de Simpson

La méthode d'approximation de Simpson est la plus usuelle des méthodes d'intégration numérique, encore que pas nécessairement la plus précise. La fonction $f(x)$ est remplacée, sur l'intervalle (x_0, x_2) , par un polynôme d'interpolation de Lagrange du second degré (figure 6.3(c)). On a donc $m = 2$ et on peut démontrer que

$$\int_{x_0}^{x_2} f(x) dx \approx \frac{h}{3} [f(x_0) + 4f(x_1) + f(x_2)], \quad (6.5)$$

où $h = x_2 - x_1 = x_1 - x_0$. Cette méthode d'approximation numérique s'avère exacte pour les fonctions polynomiales de degré trois ou moins.

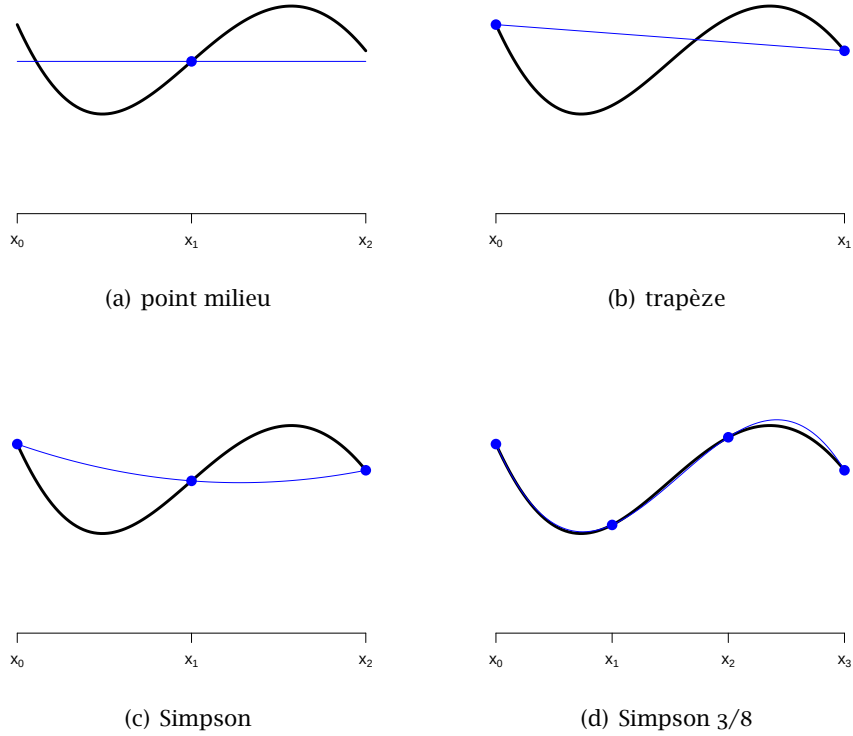
La formule composée pour l'approximation par la méthode de Simpson de $\int_a^b f(x) dx$ avec n intervalles est

$$\int_a^b f(x) dx \approx \frac{h}{3} \left[f(a) + 2 \sum_{j=1}^{n-1} f(x_{2j}) + 4 \sum_{j=1}^n f(x_{2j-1}) + f(b) \right], \quad (6.6)$$

où $h = (b - a)/(2n)$ et $x_j = a + jh$.

6.7 Méthode de Simpson 3/8

La méthode de Simpson 3/8 constitue une extension de la méthode de Simpson où la fonction f est remplacée par un polynôme d'interpolation de

FIG. 6.3 - Approximation de $f(x)$ sur un intervalle

degré $m = 3$ (figure 6.3(d)). On peut alors démontrer que

$$\int_{x_0}^{x_3} f(x) dx \approx \frac{3h}{8} [f(x_0) + 3f(x_1) + 3f(x_2) + f(x_3)], \quad (6.7)$$

où $h = x_3 - x_2 = x_2 - x_1 = x_1 - x_0$. La dérivation de la formule composée d'approximation de $\int_a^b f(x) dx$ avec n intervalles est laissée en exercice.

Consulter [Burden et Faires \(2011\)](#) pour de plus amples détails sur les polynômes d'interpolation de Lagrange et les méthodes d'intégration numérique présentées ci-dessus.

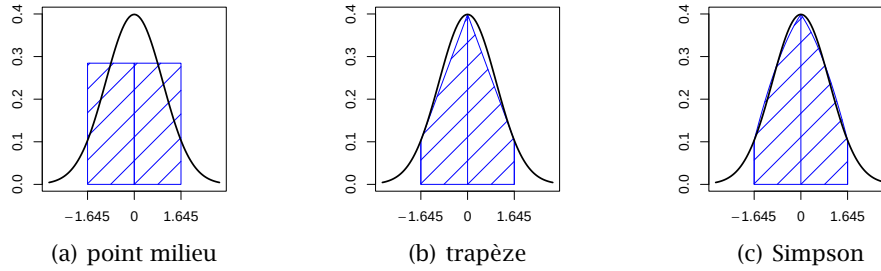


FIG. 6.4 – Comparaison des approximations de l'aire correspondant à l'intégrale I selon trois méthodes d'intégration numérique avec $n = 2$



Il n'y a pas de fonction dans la distribution de base de R pour le calcul d'intégrales avec précisément l'une ou l'autre des méthodes mentionnées dans ce chapitre. Cependant, la fonction `integrate` réalise à peu près la même chose avec un algorithme différent. Étudiez les lignes 39–51 du fichier de script `integration_numerique.R` reproduit à la section 6.9 pour en apprendre un peu plus sur cette fonction.

⚡ 6.8 Solution du problème

Nous allons calculer l'intégrale

$$I = \int_{-1.645}^{1.645} \frac{1}{\sqrt{2\pi}} e^{-x^2/2} dx$$

à l'aide de trois méthodes d'intégration numérique : point milieu, trapèze et Simpson. Dans tous les cas, pour simplifier les calculs, nous allons utiliser $n = 2$ intervalles. Il va donc sans dire que l'approximation sera relativement grossière. La figure 6.4 fournit une comparaison graphique des trois approximations.

Dans tous les cas, nous avons $a = -1.645$, $b = 1.645$ et

$$f(x) = \phi(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}.$$

Tout d'abord, avec la méthode du point milieu, nous avons :

$$h = \frac{b - a}{2n} = \frac{1.645 + 1.645}{4} = 0.8225$$

$$x_j = a + jh = -1.645 + 0.8225j$$

et donc

$$\begin{aligned}
 I &\approx 2h \sum_{j=0}^{n-1} f(x_{2j+1}) \\
 &= 2(0,8225)[f(x_1) + f(x_3)] \\
 &= 1,645[f(-0,8225) + f(0,8225)] \\
 &= 0,9358462.
 \end{aligned}$$

Avec la méthode du trapèze, maintenant :

$$\begin{aligned}
 h &= \frac{b-a}{n} = \frac{1,645 + 1,645}{2} = 1,645 \\
 x_j &= a + jh = -1,645 + 1,645j,
 \end{aligned}$$

d'où

$$\begin{aligned}
 I &\approx \frac{h}{2} \left[f(a) + f(b) + 2 \sum_{j=1}^{n-1} f(x_j) \right] \\
 &= \frac{1,645}{2} [f(a) + f(b) + 2f(x_1)] \\
 &= 0,8225[f(-1,645) + f(1,645) + 2f(0)] \\
 &= 0,8258773.
 \end{aligned}$$

Enfin, avec la méthode de Simpson, nous avons :

$$\begin{aligned}
 h &= \frac{b-a}{2n} = \frac{1,645 + 1,645}{4} = 0,8225 \\
 x_j &= a + jh = -1,645 + 0,8225j
 \end{aligned}$$

et

$$\begin{aligned}
 I &\approx \frac{h}{3} \left[f(a) + 2 \sum_{j=1}^{n-1} f(x_{2j}) + 4 \sum_{j=1}^n f(x_{2j-1}) + f(b) \right] \\
 &= \frac{0,8225}{3} [f(a) + 2f(x_2) + 4(f(x_1) + f(x_3)) + f(b)] \\
 &= \frac{0,8225}{3} [f(-1,645) + 2f(0) \\
 &\quad + 4(f(-0,8225) + f(0,8225)) + f(1,645)] \\
 &= 0,8991899.
 \end{aligned}$$

6.9 Code informatique

📄 Fichier d'accompagnement `integration_numerique.R`

```

12 ###
13 ### POLYNÔMES D'INTERPOLATION DE LAGRANGE
14 ###
15
16 ## La fonction 'poly.calc' du package 'polynom' permet de
17 ## calculer le polynôme d'interpolation de Lagrange passant
18 ## par un ensemble de points (x, y), où x et y sont des
19 ## vecteurs de même longueur. Le package n'est pas livré avec
20 ## R, il faut donc l'installer depuis CRAN, puis le charger en
21 ## mémoire. [Décommenter la ligne ci-dessous pour installer
22 ## (une seule fois!) le package.]
23 #install.packages("polynom", repos = "https://cran.ca.r-project.org")
24 library(polynom)
25
26 ## Soit  $f(x) = 1/x$ . Calculons les coefficients des polynômes
27 ## d'interpolation de Lagrange de degré 1 et 2 pour cette
28 ## fonction.
29 x <- c(2, 2.75, 4)                # noeuds de f
30 (P1 <- poly.calc(x[1:2], 1/x[1:2])) # polynôme de degré 1
31 (P2 <- poly.calc(x, 1/x))          # polynôme de degré 2
32
33 ## Pour calculer la valeur des polynômes en un point, il faut
34 ## utiliser la fonction 'predict'.
35 predict(P1, 3)                    # tel qu'obtenu dans le texte
36 predict(P2, 3)                    # idem
37
38 ###
39 ### MÉTHODES D'INTÉGRATION NUMÉRIQUE
40 ###
41
42 ## Pas de démonstration en R des méthodes étudiées dans le
43 ## chapitre. Il est laissé en exercice d'écrire des fonctions
44 ## 'pointmilieu', 'trapeze', 'simpson' et 'simpson38'.
45 ## Cependant, il y a dans R une fonction 'integrate' qui
46 ## permet d'intégrer numériquement une fonction 'f' entre des
47 ## bornes 'lower' et 'upper'.
48 integrate(sin, 0, 2)              # intégrale de sin(x)
49 f <- function(x) x^2 * exp(-x)   # une autre fonction
50 integrate(f, 0, 1)               # intégrale sur [0, 1]
51 integrate(dnorm, -1.645, 1.645)  # exemple du chapitre

```

6.10 Exercices

6.1 À partir des formules d'approximation de $\int_{x_0}^{x_m} f(x) dx$, $m = 1, 2, 3$, développer les formules composées pour les méthodes d'approximation suivantes.

- a) Trapèze ($m = 1$)
- b) Simpson ($m = 2$)
- c) Simpson 3/8 ($m = 3$)

6.2 Évaluer numériquement les intégrales suivantes avec les méthodes du point milieu, du trapèze, de Simpson et de Simpson 3/8. Dans chaque cas, n'utiliser qu'un seul sous-intervalle, c'est-à-dire $n = 1$.

- a) $\int_{0,5}^1 x^4 dx$
- b) $\int_0^{0,5} \frac{2}{x-4} dx$
- c) $\int_1^{1,5} x^2 \ln x dx$
- d) $\int_0^{\pi/4} e^{3x} \sin 2x dx$

6.3 L'approximation de $\int_0^2 f(x) dx$ avec $n = 1$ donne 4 avec la méthode du trapèze et 2 avec la méthode de Simpson. Déterminer la valeur de $f(1)$.

Réponses

- 6.2**
- a) 0,158203; 0,265625; 0,194010; 0,193866
 - b) -0,266667; -0,267857; -0,267064; -0,267063
 - c) 0,174331; 0,228074; 0,192245; 0,192253
 - d) 1,803915; 4,143260; 2,583696; 2,585789

6.3 $\frac{1}{2}$

Troisième partie

Algèbre linéaire

$$\begin{bmatrix} \cos 90^\circ & \sin 90^\circ \\ -\sin 90^\circ & \cos 90^\circ \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Tiré de [XKCD.com](http://xkcd.com)

7 Notions fondamentales d'algèbre linéaire

Objectifs du chapitre

- ▶ Utiliser les principaux opérateurs de l'arithmétique matricielle, dont la somme, le produit, la transposée et la trace.
- ▶ Exprimer un système d'équations linéaires sous forme matricielle et le résoudre par élimination gaussienne ainsi que par produit de matrices élémentaires.
- ▶ Vérifier si des éléments d'un espace vectoriel sont linéairement indépendants.
- ▶ Définir les propriétés de l'inverse d'une matrice.
- ▶ Définir les propriétés du déterminant d'une matrice.
- ▶ Calculer l'inverse d'une matrice à partir de matrices élémentaires ainsi que par la méthode de la matrice adjointe.
- ▶ Déterminer le rang d'une matrice.
- ▶ Établir les liens entre les concepts ci-dessus.

Ce chapitre propose une révision des éléments fondamentaux d'algèbre linéaire normalement étudiés au collège : matrice, déterminant, inverse, vecteur, espace vectoriel et relations avec les systèmes d'équations linéaires. Nous tâchons d'insister sur l'établissement de liens entre les différentes notions passées en revue. À ce titre, le [théorème 7.13](#) en fin de chapitre est particulièrement important puisqu'il unifie un grand nombre de résultats.

Le texte ne prétend à aucune exhaustivité ; le lecteur devrait donc consulter, au besoin, l'un des nombreux livres d'introduction à l'algèbre linéaire disponibles en librairie ou à la bibliothèque. Les éditions successives de [Anton \(2000\)](#) ont longtemps servi pour les examens professionnels d'actuariat et constituent toujours des références de choix.

Le langage R contient à peu près tous les outils nécessaires pour la manipulation et le traitement des vecteurs et matrices; voir?. Nous offrons une révision dans le code informatique de la [section 7.9](#).

7.1 Matrices

Une matrice est un tableau rectangulaire de nombres. Les matrices seront généralement représentées par des lettres majuscules et leurs éléments par des minuscules. Ainsi,

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}$$

est une matrice $m \times n$ composée des éléments a_{ij} , $i = 1, \dots, m$, $j = 1, \dots, n$. De manière plus compacte, nous écrirons parfois

$$\mathbf{A} = [a_{ij}]_{m \times n}$$

ou simplement

$$\mathbf{A} = [a_{ij}]$$

lorsque le contexte rend claires les dimensions de la matrice.

7.1.1 Matrice carrée

Lorsqu'une matrice compte autant de lignes que de colonnes ($m = n$), la matrice est dite *carrée*. Dans un tel cas, les éléments a_{ii} , $i = 1, \dots, n$ forment la *diagonale* de la matrice.

7.1.2 Matrice symétrique

Si $a_{ij} = a_{ji}$ pour tout i, j , alors la matrice carrée $\mathbf{A}$ est *symétrique*. Par exemple,

$$\begin{bmatrix} 2 & 3 & 4 & 5 \\ 3 & 1 & 0 & 6 \\ 4 & 0 & 2 & 7 \\ 5 & 6 & 7 & 1 \end{bmatrix}$$

constitue une matrice symétrique.

7.1.3 Matrices triangulaires et diagonales

Une matrice carrée dont tous les éléments sous la diagonale sont nuls est une matrice *triangulaire supérieure*. À l'inverse, une matrice carrée dont tous les éléments au-dessus de la diagonale sont nuls est une matrice *triangulaire inférieure*. Ainsi,

$$\begin{bmatrix} 2 & 3 & 4 & 5 \\ 0 & 1 & 0 & 6 \\ 0 & 0 & 2 & 7 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

est une matrice triangulaire supérieure, alors que

$$\begin{bmatrix} 2 & 0 & 0 & 0 \\ 3 & 1 & 0 & 0 \\ 4 & 0 & 2 & 0 \\ 5 & 6 & 7 & 1 \end{bmatrix}$$

est triangulaire inférieure.

Une matrice à la fois triangulaire inférieure et triangulaire supérieure ne comporte que des zéros à l'extérieur de la diagonale et est appelée une matrice *diagonale*, par exemple

$$\begin{bmatrix} 2 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

7.1.4 Matrice identité et matrice zéro

La matrice diagonale dont tous les éléments de la diagonale sont égaux à 1 est appelée *matrice identité*. Celle-ci joue un rôle important en algèbre matricielle. Elle est généralement notée **I** :

$$\mathbf{I}_{3 \times 3} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Finalement, la matrice constituée entièrement de 0 est appelée *matrice zéro* ou *matrice nulle*. Elle est généralement notée **0** :

$$\mathbf{0}_{3 \times 3} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

La matrice nulle est tout-à-fait différente du scalaire 0.

7.1.5 Vecteur ligne et vecteur colonne

Les matrices comptant une seule ligne ou une seule colonne portent un nom spécial. Une matrice $1 \times n$ est appelée *vecteur ligne* et une matrice $m \times 1$ est appelée *vecteur colonne* ou parfois simplement *vecteur*. Ces deux types de matrices sont généralement notées par des lettres minuscules et leurs éléments sont numérotés par un seul indice :

$$\mathbf{a} = [a_1 \quad a_2 \quad \cdots \quad a_n], \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix}.$$

7.2 Arithmétique matricielle

Cette section récapitule l'essentiel des règles d'arithmétique matricielle. Gardez à l'esprit que ces règles sont parfois fort différentes de celles de l'arithmétique des nombres, notamment au chapitre du produit matriciel.

7.2.1 Égalité, somme et différence

Soit $\mathbf{A} = [a_{ij}]$ et $\mathbf{B} = [b_{ij}]$ deux matrices de mêmes dimensions et c , une constante (scalaire). Alors,

1. $\mathbf{A} = \mathbf{B}$ si, et seulement si, $a_{ij} = b_{ij}$ pour tout i et tout j ;
2. $\mathbf{A} + \mathbf{B} = [a_{ij} + b_{ij}]$;
3. $\mathbf{A} - \mathbf{B} = [a_{ij} - b_{ij}]$;
4. $c\mathbf{A} = [ca_{ij}]$.

7.2.2 Produit

Le produit entre deux matrices, $\mathbf{AB}$, est défini seulement si le nombre de colonnes de la matrice $\mathbf{A}$ est égal au nombre de lignes de la matrice $\mathbf{B}$. Le résultat du produit est une matrice dont l'élément en position (i, j) correspond au produit scalaire entre la ligne i de la matrice $\mathbf{A}$ et la colonne j de la matrice $\mathbf{B}$. Autrement dit, soit $\mathbf{A}_{m \times p}$ et $\mathbf{B}_{p \times n}$ deux matrices. Alors le produit $\mathbf{AB}$ est défini et

$$\mathbf{AB} = \left[\sum_{k=1}^p a_{ik} b_{kj} \right]_{m \times n}.$$

Exemple 7.1. Soit les matrices

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 4 \\ 2 & 6 & 0 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 4 & 1 & 4 & 3 \\ 0 & -1 & 3 & 1 \\ 2 & 7 & 5 & 2 \end{bmatrix}.$$

La matrice $\mathbf{A}$ compte trois colonnes, soit autant que le nombre de lignes de la matrice $\mathbf{B}$. Le produit $\mathbf{AB}$ est donc défini. Le résultat sera une matrice $\mathbf{C}$ de deux lignes et quatre colonnes où

$$\begin{aligned} c_{11} &= \begin{bmatrix} 1 & 2 & 4 \end{bmatrix} \begin{bmatrix} 4 \\ 0 \\ 2 \end{bmatrix} \\ &= (1 \cdot 4) + (2 \cdot 0) + (4 \cdot 2) \\ &= 12, \\ c_{12} &= \begin{bmatrix} 1 & 2 & 4 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \\ 7 \end{bmatrix} \\ &= (1 \cdot 1) + (2 \cdot -1) + (4 \cdot 7) \\ &= 27, \end{aligned}$$

etc. Le résultat final est

$$\mathbf{C} = \begin{bmatrix} 12 & 27 & 30 & 13 \\ 8 & -4 & 26 & 12 \end{bmatrix}.$$

Vous remarquerez que le produit $\mathbf{BA}$ n'est pas défini, ce qui illustre que le produit matriciel n'est pas commutatif. $\square$

Théorème 7.1 (Propriétés de l'arithmétique matricielle). *Soit $\mathbf{A}$, $\mathbf{B}$ et $\mathbf{C}$ des matrices de dimensions telles que les opérations ci-dessous sont définies, et soit a et b , des constantes.*

1. $\mathbf{A} + \mathbf{B} = \mathbf{B} + \mathbf{A}$ (commutativité de l'addition)
2. $\mathbf{A} + (\mathbf{B} + \mathbf{C}) = (\mathbf{A} + \mathbf{B}) + \mathbf{C}$ (associativité de l'addition)
3. $\mathbf{AB} \neq \mathbf{BA}$ en général
4. $\mathbf{A}(\mathbf{BC}) = (\mathbf{AB})\mathbf{C}$ (associativité de la multiplication)
5. $\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{AB} + \mathbf{AC}$ (distributivité par la gauche)
6. $(\mathbf{B} + \mathbf{C})\mathbf{A} = \mathbf{BA} + \mathbf{CA}$ (distributivité par la droite)
7. $\mathbf{A}(\mathbf{B} - \mathbf{C}) = \mathbf{AB} - \mathbf{AC}$

$$8. (\mathbf{B} - \mathbf{C})\mathbf{A} = \mathbf{BA} - \mathbf{CA}$$

$$9. a(\mathbf{B} + \mathbf{C}) = a\mathbf{B} + a\mathbf{C}$$

$$10. a(\mathbf{B} - \mathbf{C}) = a\mathbf{B} - a\mathbf{C}$$

$$11. (a + b)\mathbf{C} = a\mathbf{C} + b\mathbf{C}$$

$$12. (a - b)\mathbf{C} = a\mathbf{C} - b\mathbf{C}$$

$$13. a(b\mathbf{C}) = (ab)\mathbf{C}$$

$$14. a(\mathbf{BC}) = (a\mathbf{B})\mathbf{C} = \mathbf{B}(a\mathbf{C})$$

Exemple 7.2. Soit $a = 5$, $b = 2$ et

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 4 & 3 \\ 2 & 1 \end{bmatrix}, \quad \mathbf{C} = \begin{bmatrix} 1 & 0 \\ 2 & 3 \end{bmatrix}.$$

Il est laissé en exercice de vérifier les propriétés du [théorème 7.1](#). Remarquez que $\mathbf{AB}$ est défini, mais pas $\mathbf{BA}$, ce qui vérifie la propriété 3, mais empêche de faire les opérations des propriétés 6 et 8. De plus, $\mathbf{BC} \neq \mathbf{CB}$. $\square$

7.2.3 Rôles de la matrice identité et de la matrice zéro

La matrice identité joue, en arithmétique matricielle, le rôle du nombre 1 en arithmétique usuelle. Ainsi,

$$\mathbf{A}_{m \times n} \mathbf{I}_n = \mathbf{I}_m \mathbf{A}_{m \times n} = \mathbf{A}.$$

De même, la matrice zéro joue un rôle similaire à celui du 0 en arithmétique.

Théorème 7.2 (Propriétés de la matrice zéro). *En supposant que les matrices sont de dimensions appropriées pour que les opérations soient définies, les propriétés ci-dessous sont valides en arithmétique matricielle.*

$$1. \mathbf{A} + \mathbf{0} = \mathbf{0} + \mathbf{A} = \mathbf{A}$$

$$2. \mathbf{A} - \mathbf{A} = \mathbf{0}$$

$$3. \mathbf{0} - \mathbf{A} = -\mathbf{A}$$

$$4. \mathbf{A}\mathbf{0} = \mathbf{0} \text{ et } \mathbf{0}\mathbf{A} = \mathbf{0}$$

L'exemple suivant illustre toutefois les limites aux similitudes entre les deux systèmes arithmétiques.

Exemple 7.3. Soit les matrices

$$\mathbf{A} = \begin{bmatrix} 0 & 1 \\ 0 & 2 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 1 & 1 \\ 3 & 4 \end{bmatrix}, \quad \mathbf{C} = \begin{bmatrix} 2 & 5 \\ 3 & 4 \end{bmatrix}, \quad \mathbf{D} = \begin{bmatrix} 3 & 7 \\ 0 & 0 \end{bmatrix}.$$

On vérifie aisément que

$$\mathbf{AB} = \mathbf{AC} = \begin{bmatrix} 3 & 4 \\ 6 & 8 \end{bmatrix},$$

d'où $\mathbf{AB} = \mathbf{AC}$ n'implique pas nécessairement $\mathbf{B} = \mathbf{C}$. De même, on vérifie que $\mathbf{AD} = \mathbf{0}$ même si $\mathbf{A} \neq \mathbf{0}$ et $\mathbf{D} \neq \mathbf{0}$. $\square$

7.2.4 Trace

La *trace* d'une matrice carrée $\mathbf{A}_{n \times n}$, notée $\text{tr}(\mathbf{A})$, est égale à la somme des éléments de la diagonale :

$$\text{tr}(\mathbf{A}) = \sum_{i=1}^n a_{ii}.$$

7.2.5 Transposée

La *transposée* d'une matrice $\mathbf{A}$ est la matrice obtenue en échangeant les lignes et les colonnes de $\mathbf{A}$. Elle est généralement notée $\mathbf{A}^\top$ ou $\mathbf{A}'$:

$$\mathbf{A} = [a_{ij}] \Rightarrow \mathbf{A}^\top = [a_{ji}].$$

Théorème 7.3 (Propriétés de la transposée). *Soit $\mathbf{A}$, $\mathbf{B}$ et $\mathbf{C}$ des matrices de dimensions telles que les opérations ci-dessous sont définies, et soit k une constante.*

1. $(\mathbf{A}^\top)^\top = \mathbf{A}$
2. $(\mathbf{A} + \mathbf{B})^\top = \mathbf{A}^\top + \mathbf{B}^\top$
3. $(k\mathbf{A})^\top = k\mathbf{A}^\top$
4. $(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top$
5. $\mathbf{A}^\top \mathbf{A}$ et $\mathbf{AA}^\top$ sont symétriques.

Démonstration. Nous ne démontrons que les propriétés 4 et 5. En premier lieu, soit $\mathbf{A}_{m \times p}$ et $\mathbf{B}_{p \times m}$ des matrices, $\mathbf{C} = (\mathbf{AB})^\top$ et $\mathbf{D} = \mathbf{B}_{m \times p}^\top \mathbf{A}_{p \times m}^\top$. L'élément c_{ij} de la matrice $\mathbf{C}$ est égal à l'élément en position (j, i) du produit $\mathbf{AB}$, d'où

$$c_{ij} = \sum_{k=1}^p a_{jk} b_{ki}.$$

De plus, puisque $\mathbf{A}^\top = [a_{ji}]$ et $\mathbf{B}^\top = [b_{ji}]$

$$d_{ij} = \sum_{k=1}^p b_{ki} a_{jk},$$

d'où $c_{ij} = d_{ij}$ pour tout $i = 1, \dots, m$ et $j = 1, \dots, m$ et, donc, $\mathbf{C} = \mathbf{D}$.

D'autre part, par la propriété 4,

$$\begin{aligned} (\mathbf{A}^\top \mathbf{A})^\top &= \mathbf{A}^\top (\mathbf{A}^\top)^\top \\ &= \mathbf{A}^\top \mathbf{A}, \end{aligned}$$

d'où $\mathbf{A}^\top \mathbf{A}$ est une matrice symétrique. On procède de même pour le second résultat de la propriété 5. $\square$



Vous rencontrerez certainement ce concept dans la suite de vos études en actuariat : la dérivée d'une matrice. Ce qui pourrait sembler bien intimidant au départ est en fait très simple : si $\mathbf{C} = [c_{ij}(x)]_{m \times n}$ est une matrice formée de fonctions différentiables de x , alors $\mathbf{C}' = [c'_{ij}(x)]_{m \times n}$. En d'autres termes, la dérivée d'une matrice est la matrice des dérivées.

7.3 Lien avec les systèmes d'équations linéaires

L'application de loin la plus fréquente des matrices est la représentation et la résolution de systèmes d'équations linéaires. Soit le système à trois équations et trois inconnues

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 &= b_1 \\ a_{21}x_1 + a_{22}x_2 + a_{23}x_3 &= b_2 \\ a_{31}x_1 + a_{32}x_2 + a_{33}x_3 &= b_3. \end{aligned}$$

On peut représenter ce système sous la forme d'une équation matricielle avec

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix},$$

soit

$$\mathbf{Ax} = \mathbf{b},$$

avec les définitions évidentes pour $\mathbf{A}$, $\mathbf{x}$ et $\mathbf{b}$.

Tout système d'équations linéaires comporte ou bien une seule solution, ou bien aucune solution, ou bien une infinité de solutions. La [figure 7.1](#) illustre ces trois possibilités pour un système à deux équations et deux inconnues, de même que pour un système à trois équations et trois inconnues.



Un système avec plus d'inconnues qu'il n'y a d'équations ($m < n$) ne peut avoir qu'aucune ou une infinité de solutions. (Il est, par exemple, impossible d'avoir un seul point commun entre deux plans.)

7.3.1 Résolution d'un système d'équations linéaires

De manière plus compacte et appropriée pour l'élimination gaussienne (voir plus bas), le système d'équation $\mathbf{Ax} = \mathbf{b}$ peut aussi être représenté sous forme d'une *matrice augmentée*

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & b_1 \\ a_{21} & a_{22} & a_{23} & b_2 \\ a_{31} & a_{32} & a_{33} & b_3 \end{bmatrix}.$$

Pour résoudre un système d'équations linéaires, il faut généralement remplacer le système donné par un autre système équivalent, mais plus simple à résoudre. Ce nouveau système est obtenu par diverses opérations :

1. multiplication d'une équation par une constante ;
2. échange de deux équations ;
3. addition d'un multiple d'une équation à une autre.

Puisque les lignes d'une matrice augmentée correspondent aux équations d'un système d'équations, les opérations ci-dessus se traduisent naturellement en opérations sur les lignes de la matrice augmentée :

1. multiplication d'une ligne par une constante ;
2. échange de deux lignes ;
3. addition d'un multiple d'une ligne à une autre.

Ces opérations sont appelées *opérations élémentaires sur les lignes*.

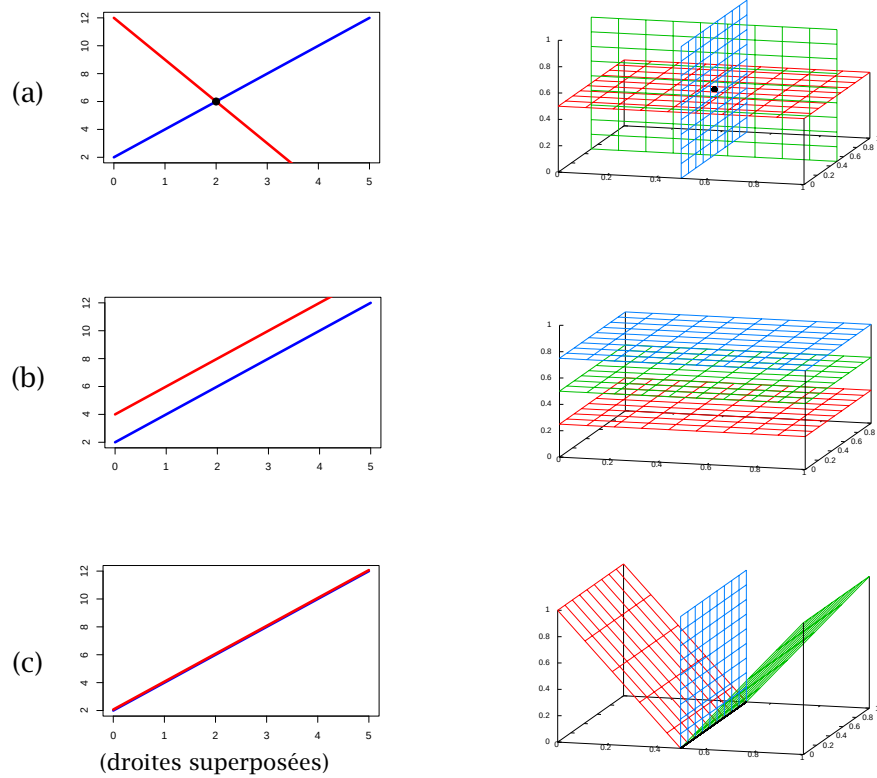


FIG. 7.1 – Solutions possibles de systèmes d'équations linéaires à deux et trois variables : (a) une seule solution; (b) aucune solution; (c) infinité de solutions. La ou les solutions sont illustrées par un point ou une droite en noir, lorsque possible.

Exemple 7.4. Soit le système d'équations

$$\begin{aligned}x + y + 2z &= 9 \\ 2x + 4y - 3z &= 1 \\ 3x + 6y - 5z &= 0.\end{aligned}$$

La procédure de résolution classique du système d'équations se trouve dans la colonne de gauche, alors que la procédure utilisant la matrice augmentée se trouve dans la colonne de droite.

Système d'équations original :

$$\begin{aligned}x + y + 2z &= 9 \\2x + 4y - 3z &= 1 \\3x + 6y - 5z &= 0.\end{aligned}$$

Matrice augmentée originale :

$$\begin{bmatrix}1 & 1 & 2 & 9 \\2 & 4 & -3 & 1 \\3 & 6 & -5 & 0\end{bmatrix}$$

Additionner -2 fois la première équation à la seconde et -3 fois la première à la troisième :

$$\begin{aligned}x + y + 2z &= 9 \\2y - 7z &= -17 \\3y - 11z &= -27.\end{aligned}$$

Additionner -2 fois la première ligne à la seconde et -3 fois la première à la troisième :

$$\begin{bmatrix}1 & 1 & 2 & 9 \\0 & 2 & -7 & -17 \\0 & 3 & -11 & -27\end{bmatrix}$$

Additionner $-\frac{3}{2}$ fois la seconde équation à la troisième :

$$\begin{aligned}x + y + 2z &= 9 \\2y - 7z &= -17 \\-\frac{1}{2}z &= -\frac{3}{2}.\end{aligned}$$

Additionner $-\frac{3}{2}$ fois la seconde ligne à la troisième :

$$\begin{bmatrix}1 & 1 & 2 & 9 \\0 & 2 & -7 & -17 \\0 & 0 & -\frac{1}{2} & -\frac{3}{2}\end{bmatrix}$$

À cette étape, nous pouvons déjà résoudre facilement le système d'équations par substitutions successives : la troisième équation nous donne $z = 3$, la seconde $2y - 7(3) = -17 \Rightarrow y = 2$ et la première, $x + 2 + 2(3) = 9 \Rightarrow x = 1$.

Nous pouvons aussi continuer les opérations élémentaires jusqu'à obtenir un système trivial.

Additionner -14 fois la troisième équation à la seconde et 4 fois la troisième à la première :

$$\begin{aligned}x + y &= 3 \\2y &= 4 \\-\frac{1}{2}z &= -\frac{3}{2}.\end{aligned}$$

Additionner -14 fois la troisième ligne à la seconde et 4 fois la troisième à la première :

$$\begin{bmatrix}1 & 1 & 0 & 3 \\0 & 2 & 0 & 4 \\0 & 0 & -\frac{1}{2} & -\frac{3}{2}\end{bmatrix}$$

Additionner $-\frac{1}{2}$ fois la seconde
équation à la première :

$$\begin{aligned}x &= 1 \\ 2y &= 4 \\ -\frac{1}{2}z &= -\frac{3}{2}.\end{aligned}$$

Additionner $-\frac{1}{2}$ fois la seconde
ligne à la première :

$$\begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 2 & 0 & 4 \\ 0 & 0 & -\frac{1}{2} & -\frac{3}{2} \end{bmatrix}$$

Ce dernier système d'équations mène à la même réponse que ci-dessus. $\square$

7.3.2 Élimination gaussienne

La procédure de la matrice augmentée utilisée pour résoudre le système d'équations de l'exemple 7.4 s'apparente à l'*élimination gaussienne*. Plus précisément, l'élimination gaussienne consiste à appliquer des opérations élémentaires à la matrice augmentée correspondant à un système d'équations linéaires jusqu'à ce que la partie gauche de celle-ci se trouve sous forme *échelonnée*, c'est-à-dire :

1. si une ligne n'est pas constituée uniquement de zéros, alors le premier élément non nul de cette ligne est un 1 ;
2. les lignes composées uniquement de zéros sont groupées au bas de la matrice ;
3. pour toutes les lignes adjacentes non composées uniquement de zéros, le premier 1 suivant des zéros apparaît plus à droite dans la ligne du dessous.

Exemple 7.5. Les matrices suivantes sont échelonnées (le symbole * représente un nombre quelconque) :

$$\begin{bmatrix} 1 & * & * & * \\ 0 & 1 & * & * \\ 0 & 0 & 1 & * \\ 0 & 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & * & * & * \\ 0 & 1 & * & * \\ 0 & 0 & 1 & * \\ 0 & 0 & 0 & 0 \end{bmatrix}, \begin{bmatrix} 1 & * & * & * \\ 0 & 1 & * & * \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \begin{bmatrix} 1 & * & * & * & * \\ 0 & 1 & * & * & * \\ 0 & 0 & 0 & 1 & * \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

$\square$

Si nous ajoutons l'exigence que le premier 1 suivant des zéros d'une colonne ne soit lui-même suivi que de zéros, alors la matrice résultante est sous forme *échelonnée réduite*. La procédure de résolution du système d'équations linéaires est alors appelée *élimination de Gauss-Jordan*.

Exemple 7.6. Les matrices suivantes sont sous forme échelonnée réduite :

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 & * \\ 0 & 1 & 0 & * \\ 0 & 0 & 1 & * \\ 0 & 0 & 0 & 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 & * & * \\ 0 & 1 & * & * \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 & * & 0 & * \\ 0 & 1 & * & 0 & * \\ 0 & 0 & 0 & 1 & * \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

□

Exemple 7.7. La matrice échelonnée réduite d'un système d'équations linéaires est la suivante :

$$\begin{bmatrix} 1 & 0 & 0 & 4 & -1 \\ 0 & 1 & 0 & 2 & 6 \\ 0 & 0 & 1 & 3 & 2 \end{bmatrix}.$$

Le système d'équations compte plus d'inconnues que d'équations. Il a donc une infinité de solutions. Le système correspondant à la matrice échelonnée réduite est

$$x_1 + 4x_4 = -1$$

$$x_2 + 2x_4 = 6$$

$$x_3 + 3x_4 = 2,$$

soit

$$x_1 = -1 - 4x_4$$

$$x_2 = 6 - 2x_4$$

$$x_3 = 2 - 3x_4.$$

Afin d'exprimer l'ensemble de solutions, nous avons recours à une (ou plusieurs, le cas échéant) *variable libre* égale à une valeur quelconque. Ici, nous choisirons tout naturellement de poser $x_4 = t$. Ainsi, nous obtenons la solution générale

$$x_1 = -1 - 4t, \quad x_2 = 6 - 2t, \quad x_3 = 2 - 3t, \quad x_4 = t.$$

□

L'élimination gaussienne et l'élimination de Gauss-Jordan se prêtent bien à la mise en oeuvre informatique de méthodes de résolution de grands systèmes d'équations linéaires. Un algorithme efficace prendra toutefois soin de minimiser les erreurs d'arrondi ainsi que le nombre d'opérations. Nous étudierons au [chapitre 9](#) la méthode la plus souvent retenue pour résoudre les systèmes d'équations linéaires.

7.3.3 Matrices élémentaires

Les opérations élémentaires réalisées sur une matrice peuvent être représentées comme un produit entre une *matrice élémentaire* et la matrice (dans cet ordre). Une matrice élémentaire $n \times n$ est le résultat d'une — et une seule — opération sur les lignes de la matrice identité I_n .

Par exemple, soit la matrice

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}.$$

Le produit suivant permet d'intervertir les deux lignes de A :

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 3 & 4 \\ 1 & 2 \end{bmatrix}.$$

Quant au produit suivant, il permet d'ajouter à la première ligne 2 fois la seconde :

$$\begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 7 & 10 \\ 3 & 4 \end{bmatrix}.$$

Les matrices les plus à gauche dans les produits ci-dessus sont des matrices élémentaires.

Les matrices élémentaires sont généralement notées E . Leur principale propriété est formalisée dans le théorème suivant.

Théorème 7.4. *Soit E une matrice obtenue en faisant une opération sur les lignes de la matrice identité. Alors EA est la matrice obtenue en faisant la même opération sur la matrice A .*

Exemple 7.8. Exprimons la solution de l'[exemple 7.4](#) à l'aide de matrices élémentaires. À chaque opération sur les lignes de la matrice augmentée correspond une matrice élémentaire.

1. Additionner -2 fois la première ligne à la seconde :

$$E_1 = \begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

2. Additionner -3 fois la première ligne à la troisième :

$$E_2 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -3 & 0 & 1 \end{bmatrix}.$$

3. Additionner $-\frac{3}{2}$ fois la seconde ligne à la troisième :

$$\mathbf{E}_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -\frac{3}{2} & 1 \end{bmatrix}.$$

Il est laissé en exercice de vérifier que

$$\mathbf{E}_3 \mathbf{E}_2 \mathbf{E}_1 \begin{bmatrix} 1 & 1 & 2 & 9 \\ 2 & 4 & -3 & 1 \\ 3 & 6 & -5 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 2 & 9 \\ 0 & 2 & -7 & -17 \\ 0 & 0 & -\frac{1}{2} & -\frac{3}{2} \end{bmatrix}.$$

□

Les matrices élémentaires seront utiles lorsque nous étudierons la décomposition LU au [chapitre 9](#).

7.3.4 Systèmes d'équations homogènes

Un système d'équations linéaires est dit *homogène* lorsque $\mathbf{b} = \mathbf{0}$, c'est-à-dire de la forme

$$\begin{aligned} a_{11}x_1 + \cdots + a_{1n}x_n &= 0 \\ a_{21}x_1 + \cdots + a_{2n}x_n &= 0 \\ &\vdots \\ a_{m1}x_1 + \cdots + a_{mn}x_n &= 0. \end{aligned}$$

Tout système d'équations homogène compte au moins une solution, la *solution triviale* $x_1 = \cdots = x_n = 0$. Par conséquent, de tels systèmes peuvent n'admettre que la solution triviale ou une infinité de solutions.

Théorème 7.5. *Tout système d'équations homogène composé de plus d'inconnues que d'équations ($m < n$) a une infinité de solutions.*

7.4 Lien avec les espaces vectoriels

De manière très générale, un *espace vectoriel* est un ensemble d'objets pour lequel sont définis un opérateur d'addition et un opérateur de produit par un scalaire, tous deux satisfaisant un certain nombre de conditions.

Définition 7.1 (Espace vectoriel). Soit V un ensemble non vide d'objets. Si les axiomes ci-dessous sont satisfaits pour tous objets $\mathbf{u}$, $\mathbf{v}$ et $\mathbf{w}$ de V et tous scalaires k et l , alors V est un espace vectoriel.

1. Si $\mathbf{u} \in V$ et $\mathbf{v} \in V$, alors $\mathbf{u} + \mathbf{v} \in V$.
2. $\mathbf{u} + \mathbf{v} = \mathbf{v} + \mathbf{u}$
3. $\mathbf{u} + (\mathbf{v} + \mathbf{w}) = (\mathbf{u} + \mathbf{v}) + \mathbf{w}$
4. Il existe un objet $\mathbf{0} \in V$ tel que $\mathbf{u} + \mathbf{0} = \mathbf{0} + \mathbf{u} = \mathbf{u}$ pour tout $\mathbf{u} \in V$.
5. Pour tout $\mathbf{u} \in V$, il existe un objet $-\mathbf{u}$ tel que $\mathbf{u} + (-\mathbf{u}) = (-\mathbf{u}) + \mathbf{u} = \mathbf{0}$.
6. Si k est un scalaire et $\mathbf{u} \in V$, alors $k\mathbf{u} \in V$.
7. $k(\mathbf{u} + \mathbf{v}) = k\mathbf{u} + k\mathbf{v}$
8. $(k + l)\mathbf{u} = k\mathbf{u} + l\mathbf{u}$
9. $k(l\mathbf{u}) = (kl)\mathbf{u}$
10. $1\mathbf{u} = \mathbf{u}$

Définition 7.2 (Produit scalaire). Soit $\mathbf{u}$, $\mathbf{v}$ et $\mathbf{w}$ des éléments d'un espace vectoriel V et k , un scalaire. Un produit scalaire dans V est une fonction $\langle \mathbf{u}, \mathbf{v} \rangle$ associant un nombre réel aux éléments $\mathbf{u}$ et $\mathbf{v}$ de V et respectant les axiomes suivants :

1. $\langle \mathbf{u}, \mathbf{v} \rangle = \langle \mathbf{v}, \mathbf{u} \rangle$ (symétrie)
2. $\langle \mathbf{u} + \mathbf{v}, \mathbf{w} \rangle = \langle \mathbf{u}, \mathbf{w} \rangle + \langle \mathbf{v}, \mathbf{w} \rangle$ (distributivité)
3. $\langle k\mathbf{u}, \mathbf{v} \rangle = k\langle \mathbf{u}, \mathbf{v} \rangle$ (multiplication par un scalaire)
4. $\langle \mathbf{v}, \mathbf{v} \rangle \geq 0$ et $\langle \mathbf{v}, \mathbf{v} \rangle = 0$ si, et seulement si, $\mathbf{v} = \mathbf{0}$.

Définition 7.3 (Norme). La norme (longueur) d'un vecteur $\mathbf{u}$, notée $\|\mathbf{u}\|$, est la racine carrée du produit scalaire du vecteur avec lui-même :

$$\|\mathbf{u}\| = \sqrt{\langle \mathbf{u}, \mathbf{u} \rangle}.$$

Définition 7.4 (Espace pré-hilbertien). Un espace pré-hilbertien (*inner product space*) est un espace vectoriel muni d'un produit scalaire.

Définition 7.5 (Orthogonalité). Deux éléments $\mathbf{u}$ et $\mathbf{v}$ d'un espace pré-hilbertien sont orthogonaux si $\langle \mathbf{u}, \mathbf{v} \rangle = 0$.

Définition 7.6 (Ensemble orthogonal et orthonormal). Un ensemble d'éléments d'un espace pré-hilbertien est dit orthogonal si toutes les paires d'éléments distincts sont orthogonales. Si, de plus, la norme de chaque élément est 1, l'ensemble est dit orthonormal.

Exemple 7.9. L'espace euclidien $\mathbb{R}^n$ est un espace pré-hilbertien pour lequel l'addition et le produit par un scalaire sont respectivement définis par

$$\mathbf{u} + \mathbf{v} = [u_i + v_i]$$

et

$$k\mathbf{v} = [kv_i],$$

où $\mathbf{u} = (u_1, \dots, u_n)$ et $\mathbf{v} = (v_1, \dots, v_n)$ sont deux vecteurs et k est une constante. De plus, le produit scalaire dans $\mathbb{R}^n$ est défini ainsi :

$$\langle \mathbf{u}, \mathbf{v} \rangle = \mathbf{u}^T \mathbf{v} = \sum_{i=1}^n u_i v_i.$$

Par conséquent, la norme d'un vecteur dans $\mathbb{R}^n$ est la racine carrée de la somme des composantes au carré ou, si l'on préfère :

$$\|\mathbf{u}\|^2 = \sum_{i=1}^n u_i^2.$$

□

Exemple 7.10. L'ensemble des variables aléatoires de second moment fini forme un espace pré-hilbertien communément noté $\mathcal{L}^2$. En effet, la somme $X + Y$ de deux variables aléatoires de second moment fini et le produit d'une variable aléatoire par une constante kX sont des variables aléatoires de second moment fini. De plus, le produit scalaire est défini ainsi dans cet espace :

$$\langle X, Y \rangle = E[XY].$$

La notion d'orthogonalité est donc liée à celle d'indépendance stochastique dans cet espace. □

On peut voir la matrice

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1m} \\ a_{21} & a_{22} & \cdots & a_{2m} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nm} \end{bmatrix}$$

comme la réunion dans un même objet des coordonnées (dans un espace à n dimensions) de m vecteurs $\mathbf{a}_1, \dots, \mathbf{a}_m$:

$$\mathbf{A} = [\mathbf{a}_1 \quad \mathbf{a}_2 \quad \cdots \quad \mathbf{a}_m],$$

où $\mathbf{a}_i = (a_{1i}, \dots, a_{ni})$. Les règles de l'arithmétique matricielle nous fournissent donc une arithmétique pour les espaces vectoriels.

La principale notion d'algèbre vectorielle qui nous sera utile ici est celle d'indépendance linéaire.

Soit $V = \{\mathbf{v}_1, \dots, \mathbf{v}_n\}$ un ensemble de vecteurs non nuls. Si la seule solution de l'équation vectorielle

$$k_1 \mathbf{v}_1 + \dots + k_n \mathbf{v}_n = \mathbf{0}$$

est la solution triviale $k_1 = 0, \dots, k_n = 0$, alors V est un ensemble *linéairement indépendant*. S'il existe une autre solution que la solution triviale à l'équation, alors V est un ensemble *linéairement dépendant*.

Lorsque $V = \{\mathbf{v}_1, \dots, \mathbf{v}_n\}$ est un ensemble de vecteurs linéairement indépendants, on dit que V forme une *base* pour l'espace vectoriel.

Exemple 7.11. Soit $\mathbf{v}_1 = (2, -1, 0, 3)$, $\mathbf{v}_2 = (1, 2, 5, -1)$ et $\mathbf{v}_3 = (7, -1, 5, 8)$. Ces trois vecteurs sont linéairement dépendants puisque $3\mathbf{v}_1 + \mathbf{v}_2 - \mathbf{v}_3 = \mathbf{0}$. Ils ne peuvent donc pas servir de base pour $\mathbb{R}^4$. $\square$

Exemple 7.12. Les vecteurs $\mathbf{i} = (1, 0, 0)$, $\mathbf{j} = (0, 1, 0)$ et $\mathbf{k} = (0, 0, 1)$ sont linéairement indépendants puisque

$$\begin{aligned} k_1 \mathbf{i} + k_2 \mathbf{j} + k_3 \mathbf{k} &= (k_1, 0, 0) + (0, k_2, 0) + (0, 0, k_3) \\ &= (k_1, k_2, k_3) \\ &= (0, 0, 0) \end{aligned}$$

seulement si $k_1 = k_2 = k_3 = 0$. Ces vecteurs forment une base pour l'espace $\mathbb{R}^3$. Puisque les vecteurs sont orthogonaux deux à deux et que leur norme est 1, $\mathbf{i}$, $\mathbf{j}$ et $\mathbf{k}$ forment aussi un ensemble orthonormal. $\square$

Pour établir le lien entre l'indépendance linéaire et les matrices, supposons un ensemble de trois vecteurs $V = \{\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3\}$, où $\mathbf{v}_i = (v_{1i}, v_{2i}, v_{3i})$. Les vecteurs sont linéairement indépendants si

$$k_1 \mathbf{v}_1 + k_2 \mathbf{v}_2 + k_3 \mathbf{v}_3 = \mathbf{0},$$

soit

$$k_1(v_{11}, v_{21}, v_{31}) + k_2(v_{12}, v_{22}, v_{32}) + k_3(v_{13}, v_{23}, v_{33}) = (0, 0, 0)$$

ou

$$(k_1 v_{11} + k_2 v_{12} + k_3 v_{13}, k_1 v_{21} + k_2 v_{22} + k_3 v_{23}, k_1 v_{31} + k_2 v_{32} + k_3 v_{33}) = (0, 0, 0)$$

ou encore, exprimé sous forme matricielle,

$$\begin{bmatrix} v_{11} & v_{12} & v_{13} \\ v_{21} & v_{22} & v_{23} \\ v_{31} & v_{32} & v_{33} \end{bmatrix} \begin{bmatrix} k_1 \\ k_2 \\ k_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}.$$

Ainsi, un ensemble de vecteurs est linéairement indépendant si la seule solution de l'équation homogène $\mathbf{Ax} = \mathbf{0}$, où les colonnes de $\mathbf{A}$ sont formées des coordonnées des vecteurs, est la solution triviale.

7.5 Inverse d'une matrice

En arithmétique usuelle, l'inverse d'un nombre x est le nombre y tel que $xy = 1$. Par habitude, nous écrivons simplement $y = 1/x$ ou $y = x^{-1}$. L'inverse d'une matrice est défini de manière similaire, sauf que l'opérateur de division n'existe pas en arithmétique matricielle. Prenez donc garde de ne *jamais* écrire une telle abomination :

$$\mathbf{AB} = \mathbf{C} \Rightarrow \mathbf{A} = \frac{\mathbf{C}}{\mathbf{B}}.$$

Si l'on veut isoler la matrice $\mathbf{A}$ dans l'équation $\mathbf{AB} = \mathbf{C}$, on ne peut que multiplier de part et d'autre par la matrice inverse de $\mathbf{B}$.

Définition 7.7 (Inverse d'une matrice). Soit $\mathbf{A}$ une matrice carrée. S'il existe une matrice $\mathbf{B}$ tel que

$$\mathbf{AB} = \mathbf{BA} = \mathbf{I},$$

alors $\mathbf{A}$ est inversible, $\mathbf{B}$ est la matrice inverse — ou simplement inverse — de $\mathbf{A}$ et cette dernière est notée $\mathbf{A}^{-1}$. S'il n'existe pas de matrice $\mathbf{B}$ satisfaisant l'égalité ci-dessus, alors $\mathbf{A}$ est dite singulière ou non-inversible.

Exemple 7.13. Soit l'équation $\mathbf{AB} = \mathbf{C}$. Pour isoler $\mathbf{A}$, il faut multiplier de part et d'autre de l'égalité par $\mathbf{B}^{-1}$. Attention, cependant : le produit matriciel n'étant pas commutatif, le produit doit se faire par la droite :

$$(\mathbf{AB})\mathbf{B}^{-1} = \mathbf{CB}^{-1} \Leftrightarrow \mathbf{A}(\mathbf{BB}^{-1}) = \mathbf{CB}^{-1} \Leftrightarrow \mathbf{A} = \mathbf{CB}^{-1}.$$

Pour isoler $\mathbf{B}$, on devra multiplier de part et d'autre de l'égalité par $\mathbf{A}^{-1}$, mais cette fois par la gauche :

$$\mathbf{A}^{-1}(\mathbf{AB}) = \mathbf{A}^{-1}\mathbf{C} \Leftrightarrow (\mathbf{A}^{-1}\mathbf{A})\mathbf{B} = \mathbf{A}^{-1}\mathbf{C} \Leftrightarrow \mathbf{B} = \mathbf{A}^{-1}\mathbf{C}.$$

□

Théorème 7.6. Soit $\mathbf{A}$ et $\mathbf{B}$ des matrices inversibles de mêmes dimensions. Alors on a les résultats suivants.

1. $(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$
2. $(\mathbf{A}^{-1})^T = (\mathbf{A}^T)^{-1}$

Démonstration. Pour le premier résultat, on doit vérifier que $(\mathbf{AB})(\mathbf{AB})^{-1} = \mathbf{I}$. Or,

$$\begin{aligned} (\mathbf{AB})(\mathbf{AB})^{-1} &= (\mathbf{AB})\mathbf{B}^{-1}\mathbf{A}^{-1} \\ &= \mathbf{A}(\mathbf{BB}^{-1})\mathbf{A}^{-1} \\ &= \mathbf{A}\mathbf{I}\mathbf{A}^{-1} \\ &= \mathbf{AA}^{-1} \\ &= \mathbf{I}. \end{aligned}$$

Pour le second résultat, il faut démontrer que $(\mathbf{A}^{-1})^T\mathbf{A}^T = \mathbf{A}^T(\mathbf{A}^{-1})^T = \mathbf{I}$. Or, par le résultat 4 du [théorème 7.3 \(page 163\)](#),

$$(\mathbf{A}^{-1})^T\mathbf{A}^T = (\mathbf{AA}^{-1})^T = \mathbf{I}^T = \mathbf{I}$$

et

$$\mathbf{A}^T(\mathbf{A}^{-1})^T = (\mathbf{A}^{-1}\mathbf{A})^T = \mathbf{I}^T = \mathbf{I},$$

puisque la matrice identité est symétrique. □

On a également le résultat suivant, utile lorsque l'on travaille avec les matrices élémentaires.

Théorème 7.7. Toute matrice élémentaire est inversible et son inverse est aussi une matrice élémentaire.

7.5.1 Calcul de l'inverse

La manière la plus usuelle de calculer l'inverse d'une matrice repose sur l'utilisation du déterminant, qui sera présenté à la [section 7.6](#). D'ici là, nous pouvons calculer l'inverse d'une matrice par une méthode similaire à l'élimination de Gauss-Jordan.

En effet, on peut démontrer que si la matrice $\mathbf{A}$ est inversible, alors on peut trouver des matrices élémentaires $\mathbf{E}_1, \dots, \mathbf{E}_k$ tel que

$$\mathbf{E}_k \cdots \mathbf{E}_1 \mathbf{A} = \mathbf{I}.$$

(Ce fait sera confirmé dans le [théorème 7.13](#).) Par conséquent,

$$\mathbf{A}^{-1} = \mathbf{E}_k \cdots \mathbf{E}_1 \mathbf{I},$$

ce qui signifie que l'on peut obtenir l'inverse de $\mathbf{A}$ en appliquant aux lignes de la matrice identité les opérations élémentaires nécessaires pour convertir $\mathbf{A}$ en matrice identité.

Exemple 7.14. Soit la matrice

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 5 & 3 \\ 1 & 0 & 8 \end{bmatrix}.$$

L'idée pour calculer l'inverse de cette matrice consiste à convertir $\mathbf{A}$ en la matrice identité par une suite d'opérations élémentaires et à appliquer simultanément ces opérations à la matrice identité. Pour ce faire, nous joignons côte à côte les matrices $\mathbf{A}$ et $\mathbf{I}$ dans une nouvelle matrice

$$[\mathbf{A}|\mathbf{I}] = \left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 2 & 5 & 3 & 0 & 1 & 0 \\ 1 & 0 & 8 & 0 & 0 & 1 \end{array} \right].$$

Suite aux opérations élémentaires, on obtiendra

$$[\mathbf{I}|\mathbf{A}^{-1}].$$

La procédure est la suivante :

1. Additionner -2 fois la première ligne à la seconde et -1 fois à la troisième :

$$\left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 0 & 1 & -3 & -2 & 1 & 0 \\ 0 & -2 & 5 & -1 & 0 & 1 \end{array} \right]$$

2. Additionner 2 fois la seconde ligne à la troisième, puis multiplier celle-ci par -1 :

$$\left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 0 & 1 & -3 & -2 & 1 & 0 \\ 0 & 0 & 1 & 5 & -2 & -1 \end{array} \right].$$

3. Additionner 3 fois la troisième ligne à la seconde et -3 fois à la première :

$$\left[\begin{array}{ccc|ccc} 1 & 2 & 0 & -14 & 6 & 3 \\ 0 & 1 & 0 & 13 & -5 & -3 \\ 0 & 0 & 1 & 5 & -2 & -1 \end{array} \right].$$

4. Additionner -2 fois la seconde ligne à la première :

$$\left[\begin{array}{ccc|ccc} 1 & 0 & 0 & -40 & 16 & 9 \\ 0 & 1 & 0 & 13 & -5 & -3 \\ 0 & 0 & 1 & 5 & -2 & -1 \end{array} \right].$$

Par conséquent,

$$\mathbf{A}^{-1} = \begin{bmatrix} -40 & 16 & 9 \\ 13 & -5 & -3 \\ 5 & -2 & -1 \end{bmatrix}.$$

□

Exemple 7.15. Soit la matrice

$$\mathbf{A} = \begin{bmatrix} 1 & 6 & 4 \\ 2 & 4 & -1 \\ -1 & 2 & 5 \end{bmatrix}.$$

Il est laissé en exercice de vérifier que la procédure d'élimination pour les deuxième et troisième lignes donne

$$\left[\begin{array}{ccc|ccc} 1 & 6 & 4 & 1 & 0 & 0 \\ 0 & -8 & -9 & -2 & 1 & 0 \\ 0 & 0 & 0 & -1 & 1 & 1 \end{array} \right].$$

Comme le côté gauche contient une ligne complète de zéros, la matrice est singulière. □

7.5.2 Résolution de systèmes d'équations par l'inverse

Outre l'élimination gaussienne et l'élimination de Gauss-Jordan, les systèmes d'équations linéaires de la forme $\mathbf{Ax} = \mathbf{b}$ peuvent être résolus avec l'inverse de $\mathbf{A}$, lorsqu'elle existe.

Théorème 7.8. *Si la matrice $\mathbf{A}$ est inversible, alors le système d'équations linéaires $\mathbf{Ax} = \mathbf{b}$ a une solution unique et celle-ci est donnée par*

$$\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}.$$

Le résultat ci-dessus ne peut évidemment servir que lorsque le système d'équations compte autant d'inconnues que d'équations, c'est-à-dire lorsque la matrice $\mathbf{A}$ est carrée. Dans le cas contraire, il faut utiliser les méthodes d'élimination pour résoudre le système d'équations.

7.5.3 Matrice orthogonale

Pour certaines matrices, le calcul de l'inverse se résume à une simple transposition. Ces matrices jouent un rôle suffisamment important en algèbre linéaire pour leur attribuer un nom spécial.

Définition 7.8 (Matrice orthogonale). Une matrice carrée $\mathbf{A}$ est dite orthogonale lorsque $\mathbf{A}^{-1} = \mathbf{A}^T$.

De la définition précédente découle qu'une matrice $\mathbf{A}$ est orthogonale si, et seulement si,

$$\mathbf{A}^T \mathbf{A} = \mathbf{A} \mathbf{A}^T = \mathbf{I}.$$

7.6 Déterminant

La fonction déterminant, notée $\det(\cdot)$, associe un nombre réel à une matrice carrée $\mathbf{A}$.

Il y a différentes façons de présenter le déterminant ; nous ne retiendrons que celle basée sur les cofacteurs. Pour ce faire, définissons tout d'abord le déterminant d'une matrice 2×2 , ainsi que son inverse. La justification de ce second résultat viendra plus loin.

Théorème 7.9. Soit $\mathbf{A} = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$. Alors

$$\det(\mathbf{A}) = \begin{vmatrix} a & b \\ c & d \end{vmatrix} = ad - bc$$

et

$$\mathbf{A}^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}.$$

Soit $\mathbf{A} = [a_{ij}]_{n \times n}$ une matrice carrée. Le *mineur de l'élément* a_{ij} , noté M_{ij} , est le déterminant de la sous-matrice résultante après avoir éliminé la ligne i et la colonne j de la matrice $\mathbf{A}$. De plus, le *cofacteur de l'élément* a_{ij} est $C_{ij} = (-1)^{i+j} M_{ij}$.

Exemple 7.16. Soit

$$\mathbf{A} = \begin{bmatrix} 3 & 1 & -4 \\ 2 & 5 & 6 \\ 1 & 4 & 8 \end{bmatrix}.$$

Une fois la première ligne et la première colonne éliminées de la matrice $\mathbf{A}$, on a

$$M_{11} = \begin{vmatrix} 5 & 6 \\ 4 & 8 \end{vmatrix} = 16,$$

d'où $C_{11} = (-1)^2 M_{11} = M_{11} = 16$. De même, après avoir éliminé la troisième ligne et la deuxième colonne, on obtient

$$M_{32} = \begin{vmatrix} 3 & -4 \\ 2 & 6 \end{vmatrix} = 26$$

et $C_{32} = (-1)^{3+2} M_{32} = -M_{32} = -26$. □

Le déterminant d'une matrice $\mathbf{A}$ est le produit scalaire entre les éléments d'une ligne ou d'une colonne quelconque de la matrice et les cofacteurs de cette ligne ou colonne.

Théorème 7.10. Soit $\mathbf{A}$ une matrice carrée $n \times n$. Alors, pour tout $1 \leq i \leq n$ et $1 \leq j \leq n$,

$$\begin{aligned} \det(\mathbf{A}) &= a_{i1}C_{i1} + a_{i2}C_{i2} + \cdots + a_{in}C_{in} \\ &= \sum_{j=1}^n a_{ij}C_{ij} \end{aligned}$$

ou

$$\begin{aligned} \det(\mathbf{A}) &= a_{1j}C_{1j} + a_{2j}C_{2j} + \cdots + a_{nj}C_{nj} \\ &= \sum_{i=1}^n a_{ij}C_{ij}. \end{aligned}$$

Exemple 7.17. Calculons le déterminant de la matrice de l'exemple 7.16. En faisant le calcul à partir de la première ligne, nous obtenons

$$\begin{aligned} \det(\mathbf{A}) &= 3(-1)^2 \begin{vmatrix} 5 & 6 \\ 4 & 8 \end{vmatrix} + (-1)^3 \begin{vmatrix} 2 & 6 \\ 1 & 8 \end{vmatrix} + (-4)(-1)^4 \begin{vmatrix} 2 & 5 \\ 1 & 4 \end{vmatrix} \\ &= 3 \begin{vmatrix} 5 & 6 \\ 4 & 8 \end{vmatrix} - \begin{vmatrix} 2 & 6 \\ 1 & 8 \end{vmatrix} - 4 \begin{vmatrix} 2 & 5 \\ 1 & 4 \end{vmatrix} \\ &= 3(16) - 10 - 4(3) \\ &= 26. \end{aligned}$$

La réponse est la même avec un développement selon la deuxième colonne :

$$\begin{aligned}\det(\mathbf{A}) &= - \begin{vmatrix} 2 & 6 \\ 1 & 8 \end{vmatrix} + 5 \begin{vmatrix} 3 & -4 \\ 1 & 8 \end{vmatrix} - 4 \begin{vmatrix} 3 & -4 \\ 2 & 6 \end{vmatrix} \\ &= -10 + 5(28) - 4(26) \\ &= 26.\end{aligned}$$

□

7.6.1 Propriétés du déterminant

Du [théorème 7.10](#), il est clair que si une matrice contient une ligne ou une colonne complète de zéros, alors son déterminant est nul.

De plus, si $\mathbf{A}$ est une matrice triangulaire (ou diagonale), alors son déterminant est égal au produit des éléments de la diagonale :

$$\det(\mathbf{A}) = \prod_{i=1}^n a_{ii}.$$

Le théorème suivant énonce quelques autres propriétés de la fonction déterminant.

Théorème 7.11. *Soit $\mathbf{A}$ une matrice $n \times n$ inversible.*

1. $\det(\mathbf{A}^\top) = \det(\mathbf{A})$
2. $\det(k\mathbf{A}) = k^n \det(\mathbf{A})$
3. $\det(\mathbf{A} + \mathbf{B}) \neq \det(\mathbf{A}) + \det(\mathbf{B})$ en général.
4. $\det(\mathbf{AB}) = \det(\mathbf{A}) \det(\mathbf{B})$
5. $\det(\mathbf{A}^{-1}) = \det(\mathbf{A})^{-1}$

7.6.2 Calcul de l'inverse avec la matrice adjointe

La manière de loin la plus usuelle de calculer l'inverse d'une matrice — à la main du moins — est intimement liée aux notions de cofacteur et de déterminant.

Avant toute chose, définissons $[C_{ij}]_{n \times n}$, la *matrice des cofacteurs* de la matrice $\mathbf{A}$. La transposée de cette matrice des cofacteurs est appelée la *matrice adjointe* de $\mathbf{A}$ et est notée $\text{adj}(\mathbf{A})$.

Exemple 7.18. Il est laissé en exercice de vérifier que la matrice adjointe de la matrice de l'exemple 7.16 est

$$\begin{bmatrix} 16 & -24 & 26 \\ -10 & 28 & -26 \\ 3 & -11 & 13 \end{bmatrix}.$$

□

Théorème 7.12. Si A est une matrice inversible, alors

$$A^{-1} = \frac{1}{\det(A)} \operatorname{adj}(A).$$

7.7 Rang

Le *rang* d'une matrice $A_{m \times n}$ correspond au nombre de lignes ou de colonnes linéairement indépendantes dans cette matrice. Évidemment,

$$\operatorname{rang}(A) \leq \min(m, n).$$

Lorsqu'il y a égalité, on dit de la matrice qu'elle est de rang complet.

7.8 Unification des résultats

Le théorème suivant unifie un grand nombre de résultats et de concepts présentés dans les sections précédentes. En ce sens, il fait le sommaire des plus importants résultats à connaître en algèbre linéaire.

Théorème 7.13. Soit A une matrice $n \times n$. Les énoncés suivants sont équivalents.

1. A est inversible.
2. $\det(A) \neq 0$
3. $\operatorname{rang}(A) = n$
4. La seule solution de $A\mathbf{x} = \mathbf{0}$ est la solution triviale.
5. Le système d'équations $A\mathbf{x} = \mathbf{b}$ a une solution pour tout vecteur $\mathbf{b}$ et celle-ci est unique.
6. A peut être exprimée comme un produit de matrices élémentaires.
7. Les vecteurs lignes de A sont linéairement indépendants.

8. Les vecteurs colonnes de A sont linéairement indépendants.
9. Les vecteurs lignes de A forment une base de $\mathbb{R}^n$.
10. Les vecteurs colonnes de A forment une base de $\mathbb{R}^n$.



Le code informatique de la [section 7.9](#) passe en revue les fonctions d'algèbre linéaire offertes dans R.

7.9 Code informatique

📄 Fichier d'accompagnement notions_fondamentales.R

```

12 ###
13 ### MATRICES ET VECTEURS
14 ###
15
16 ## La fonction 'diag' a plusieurs usages. Un premier consiste
17 ## à extraire la diagonale d'une matrice carrée.
18 (a <- matrix(1:9, nrow = 3))
19 diag(a)
20
21 ## Si la matrice passée en argument est rectangulaire 'm x n',
22 ## 'diag' retourne la liste des éléments a[i, i], i = 1, ...,
23 ## min(m, n).
24 (a <- matrix(1:12, nrow = 4))
25 diag(a)
26
27 ## Lorsque l'on passe un vecteur à la fonction 'diag', le
28 ## résultat est une matrice diagonale formée du vecteur.
29 diag(1:3)
30
31 ## Enfin, lorsque l'argument de 'diag' est un scalaire 'n', le
32 ## résultat est la matrice identité 'n x n'.
33 diag(3)
34
35 ## La fonction 'lower.tri' (respectivement 'upper.tri')
36 ## retourne une matrice de la même dimension que celle passée
37 ## en argument avec des valeurs TRUE (FALSE) aux positions des
38 ## éléments sous la diagonale et FALSE (TRUE) aux positions
39 ## au-dessus de la diagonale. La diagonale sera dans le
40 ## premier ou dans le second groupe selon que l'argument
41 ## 'diag' est TRUE ou FALSE.
42 (a <- matrix(1:9, nrow = 3))
43 lower.tri(a)

```

```

44 upper.tri(a, diag = TRUE)
45
46 ## On utilise les fonctions 'lower.tri' et 'upper.tri'
47 ## principalement pour extraire les éléments au-dessus ou
48 ## au-dessous de la diagonale d'une matrice carrée.
49 a[lower.tri(a)]
50 a[upper.tri(a)]
51
52 ## Bien que rarement nécessaires en R, on peut créer des
53 ## vecteurs ligne ou colonne avec les fonctions 'rbind' et
54 ## 'cbind', dans l'ordre.
55 rbind(1:3)
56 dim(rbind(1:3))
57 cbind(1:3)
58
59 ####
60 #### ARITHMÉTIQUE MATRICIELLE
61 ####
62
63 ## Le produit matriciel s'effectue en R avec l'opérateur
64 ## '%*%', et non avec '*', qui fait le produit élément par
65 ## élément (une opération qui n'est pas définie en
66 ## arithmétique matricielle usuelle, mais qui l'est dans R).
67 (a1 <- matrix(c(1, 2, 2, 6, 4, 0), nrow = 2))
68 (a2 <- matrix(c(4, 0, 2, 1, -1, 7, 4, 3, 5, 3, 1, 2), nrow = 3))
69 a1 %*% a2
70
71 ## Il n'y a pas d'opérateur pour calculer la trace d'une
72 ## matrice carrée. Il suffit de sommer les éléments de la
73 ## diagonale.
74 a
75 sum(diag(a))
76
77 ## La transposée est obtenue avec la fonction 't'.
78 t(a)
79
80 ####
81 #### DÉTERMINANT ET INVERSE
82 ####
83
84 ## Le déterminant d'une matrice est obtenu avec la fonction
85 ## 'det'.
86 A <- matrix(c(1, 2, 1, 2, 5, 0, 3, 3, 8), nrow = 3)
87 det(A)
88

```

```

89 ## Avec pour seul argument une matrice carrée, la fonction
90 ## 'solve' calcule l'inverse de cette matrice.
91 solve(A)
92
93 ## Avec deux arguments, une matrice 'A' et un vecteur 'b',
94 ## 'solve' calcule la solution du système d'équations 'Ax =
95 ## b', c'est-à-dire 'A^{-1} b'.
96 b <- 1:3
97 solve(A, b)
98
99 ## À noter qu'il est *beaucoup* plus rapide de calculer
100 ## directement la solution du système d'équations avec
101 ## 'solve(A, b)' que d'inverser la matrice et calculer la
102 ## solution par la suite avec 'solve(A) %*% b'. Pour en faire
103 ## la démonstration, on doit utiliser de grands objets.
104 A <- matrix(rnorm(500^2), nrow = 500)
105 b <- rnorm(500)
106 system.time(solve(A) %*% b)
107 system.time(solve(A, b))

```

7.10 Exercices

7.1 Pour chaque cas ci-dessous, supposer que la matrice donnée est une matrice augmentée d'un système d'équations linéaires exprimée sous forme échelonnée. Résoudre le système d'équations.

a)
$$\begin{bmatrix} 1 & -3 & 4 & 7 \\ 0 & 1 & 2 & 2 \\ 0 & 0 & 1 & 5 \end{bmatrix}$$

b)
$$\begin{bmatrix} 1 & 0 & 8 & -5 & 6 \\ 0 & 1 & 4 & -9 & 3 \\ 0 & 0 & 1 & 1 & 2 \end{bmatrix}$$

c)
$$\begin{bmatrix} 1 & 7 & -2 & 0 & -8 & -3 \\ 0 & 0 & 1 & 1 & 6 & 5 \\ 0 & 0 & 0 & 1 & 3 & 9 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

d)
$$\begin{bmatrix} 1 & -3 & 7 & 1 \\ 0 & 1 & 4 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

7.2 Résoudre chacun des systèmes d'équations suivants par l'élimination gaussienne et l'élimination de Gauss-Jordan.

a)
$$\begin{aligned} x_1 + x_2 + 2x_3 &= 8 \\ -x_1 - 2x_2 + 3x_3 &= 1 \\ 3x_1 - 7x_2 + 4x_3 &= 10 \end{aligned}$$

b)
$$\begin{aligned} 2x_1 + 2x_2 + 2x_3 &= 0 \\ -2x_1 + 5x_2 + 2x_3 &= 1 \\ 8x_1 + x_2 + 4x_3 &= -1 \end{aligned}$$

$$\begin{array}{ll}
 \text{c)} & \begin{array}{l} x - y + 2z - w = -1 \\ 2x + y - 2z - 2w = -2 \\ -x + 2y - 4z + w = 1 \\ 3x \qquad \qquad - 3w = -3 \end{array} & \text{d)} & \begin{array}{l} -2b + 3c = 1 \\ 3a + 6b - 3c = -2 \\ 6a + 6b + 3c = 5 \end{array}
 \end{array}$$

7.3 Résoudre chacun des systèmes d'équations suivants par l'élimination gaussienne et l'élimination de Gauss-Jordan.

$$\begin{array}{ll}
 \text{a)} & \begin{array}{l} 5x_1 - 2x_2 + 6x_3 = 0 \\ -2x_1 + x_2 + 3x_3 = 1 \end{array} & \text{b)} & \begin{array}{l} x_1 - 2x_2 + x_3 - 4x_4 = 1 \\ x_1 + 3x_2 + 7x_3 + 2x_4 = 2 \\ x_1 - 12x_2 - 11x_3 - 16x_4 = 5 \end{array} \\
 \text{c)} & \begin{array}{l} w + 2x - y = 4 \\ x - y = 3 \\ w + 3x - 2y = 7 \\ 2u + 4v + w + 7x = 7 \end{array}
 \end{array}$$

7.4 Résoudre les systèmes d'équations homogènes suivants.

$$\begin{array}{ll}
 \text{a)} & \begin{array}{l} 2x_1 + x_2 + 3x_3 = 0 \\ x_1 + 2x_2 = 0 \\ x_2 + x_3 = 0 \end{array} & \text{b)} & \begin{array}{l} 3x_1 + x_2 + x_3 + x_4 = 0 \\ 5x_1 - x_2 + x_3 - x_4 = 0 \end{array} \\
 \text{c)} & \begin{array}{l} 2x + 2y + 4z = 0 \\ w - y - 3z = 0 \\ 2w + 3x + y + z = 0 \\ -2w + x + 3y - 2z = 0 \end{array}
 \end{array}$$

7.5 Pour quelle valeur de a le système d'équations

$$\begin{array}{l}
 x + 2y - \qquad \qquad 3z = 4 \\
 3x - y + \qquad \qquad 5z = 2 \\
 4x + y + (a^2 - 14)z = a + 2
 \end{array}$$

n'aura-t-il aucune solution? Une seule solution? Une infinité de solutions?

7.6 Soit les matrices $\mathbf{A}_{4 \times 5}$, $\mathbf{B}_{4 \times 5}$, $\mathbf{C}_{5 \times 2}$, $\mathbf{D}_{4 \times 2}$ et $\mathbf{E}_{5 \times 4}$. Lesquelles des expressions matricielles suivantes sont définies? Pour les expressions définies, donner les dimensions de la matrice résultante.

- a) $\mathbf{BA}$ b) $\mathbf{AC} + \mathbf{D}$ c) $\mathbf{AE} + \mathbf{B}$ d) $\mathbf{AB} + \mathbf{B}$
 e) $\mathbf{E}(\mathbf{A} + \mathbf{B})$ f) $\mathbf{E}(\mathbf{AC})$ g) $\mathbf{E}^T \mathbf{A}$ h) $(\mathbf{A}^T + \mathbf{E})\mathbf{D}$

7.7 Soit les matrices

$$\mathbf{A} = \begin{bmatrix} 3 & 0 \\ -1 & 2 \\ 1 & 1 \end{bmatrix} \quad \mathbf{B} = \begin{bmatrix} 4 & -1 \\ 0 & 2 \end{bmatrix} \quad \mathbf{C} = \begin{bmatrix} 1 & 4 & 2 \\ 3 & 1 & 5 \end{bmatrix}$$

$$\mathbf{D} = \begin{bmatrix} 1 & 5 & 2 \\ -1 & 0 & 1 \\ 3 & 2 & 4 \end{bmatrix} \quad \mathbf{E} = \begin{bmatrix} 6 & 1 & 3 \\ -1 & 1 & 2 \\ 4 & 1 & 3 \end{bmatrix}.$$

Calculer les expressions suivantes (lorsque possible).

- a) $\mathbf{D} + \mathbf{E}$ b) $\mathbf{AB}$
 c) $(2\mathbf{D}^T - \mathbf{E})\mathbf{A}$ d) $(4\mathbf{B})\mathbf{C} + 2\mathbf{B}$
 e) $(-\mathbf{AC})^T + 5\mathbf{D}^T$ f) $(\mathbf{BA}^T - 2\mathbf{C})^T$
 g) $\mathbf{B}^T(\mathbf{CC}^T - \mathbf{A}^T\mathbf{A})$ h) $\mathbf{D}^T\mathbf{E}^T - (\mathbf{ED})^T$
 i) $\text{tr}(\mathbf{DD}^T)$ j) $\text{tr}(4\mathbf{E}^T - \mathbf{D})$

7.8 Démontrer que si $\mathbf{A}$ et $\mathbf{B}$ sont deux matrices $n \times n$, alors $\text{tr}(\mathbf{A} + \mathbf{B}) = \text{tr}(\mathbf{A}) + \text{tr}(\mathbf{B})$, où $\text{tr}(\mathbf{A}) = \sum_{i=1}^n a_{ii}$.

7.9 Une matrice $\mathbf{A}$ est dite *symétrique* si $\mathbf{A}^T = \mathbf{A}$ et *antisymétrique* si $\mathbf{A}^T = -\mathbf{A}$. Démontrer les énoncés suivants, étant donné une matrice carrée $\mathbf{A}$.

- a) $\mathbf{AA}^T$ et $\mathbf{A}^T\mathbf{A}$ sont symétriques.
 b) $\mathbf{A} - \mathbf{A}^T$ est antisymétrique.

7.10 Démontrer que si $\mathbf{A}^T\mathbf{A} = \mathbf{A}$, alors $\mathbf{A}$ est symétrique et $\mathbf{A} = \mathbf{A}^2$. Une matrice qui satisfait cette dernière égalité est dite *idempotente*.

7.11 Démontrer chacun des résultats ci-dessous si les éléments des matrices $\mathbf{A}$ et $\mathbf{B}$ sont des fonctions différentiables de x et que les dimensions des matrices sont appropriées.

- a) $\frac{d}{dx}(k\mathbf{A}) = k \frac{d}{dx}\mathbf{A}$
 b) $\frac{d}{dx}(\mathbf{A} + \mathbf{B}) = \frac{d}{dx}\mathbf{A} + \frac{d}{dx}\mathbf{B}$
 c) $\frac{d}{dx}(\mathbf{AB}) = \frac{d\mathbf{A}}{dx}\mathbf{B} + \mathbf{A} \frac{d\mathbf{B}}{dx}$

7.12 Lesquels des ensembles de vecteurs dans $\mathbb{R}^3$ suivants sont linéairement dépendants ?

- a) $(4, -1, 2), (-4, 10, 2)$
- b) $(-3, 0, 4), (5, -1, 2), (1, 1, 3)$
- c) $(8, -1, 3), (4, 0, 1)$
- d) $(-2, 0, 1), (3, 2, 5), (6, -1, 1), (7, 0, -2)$

7.13 Soit $\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3 \in \mathbb{R}^3$. Déterminer si les trois vecteurs se trouvent dans un plan.

- a) $\mathbf{v}_1 = (2, -2, 0), \mathbf{v}_2 = (6, 1, 4), \mathbf{v}_3 = (2, 0, -4)$
- b) $\mathbf{v}_1 = (-6, 7, 2), \mathbf{v}_2 = (3, 2, 4), \mathbf{v}_3 = (4, -1, 2)$

7.14 Soit la matrice

$$\mathbf{A} = \begin{bmatrix} a_{11} & 0 & \cdots & 0 \\ 0 & a_{22} & \cdots & 0 \\ \vdots & \vdots & & \vdots \\ 0 & 0 & \cdots & a_{nn} \end{bmatrix},$$

où $\prod_{i=1}^n a_{ii} \neq 0$. Démontrer que $\mathbf{A}$ est inversible et trouver son inverse.

7.15 Soit les matrices

$$\mathbf{A} = \begin{bmatrix} 3 & 4 & 1 \\ 2 & -7 & -1 \\ 8 & 1 & 5 \end{bmatrix} \quad \mathbf{B} = \begin{bmatrix} 8 & 1 & 5 \\ 2 & -7 & -1 \\ 3 & 4 & 1 \end{bmatrix} \quad \mathbf{C} = \begin{bmatrix} 3 & 4 & 1 \\ 2 & -7 & -1 \\ 2 & -7 & 3 \end{bmatrix}.$$

Trouver les matrices élémentaires $\mathbf{E}_1, \mathbf{E}_2, \mathbf{E}_3$ et $\mathbf{E}_4$ satisfaisant les équations suivantes.

- a) $\mathbf{E}_1 \mathbf{A} = \mathbf{B}$ b) $\mathbf{E}_2 \mathbf{B} = \mathbf{A}$ c) $\mathbf{E}_3 \mathbf{A} = \mathbf{C}$ d) $\mathbf{E}_4 \mathbf{C} = \mathbf{A}$

7.16 Utiliser la méthode des opérations élémentaires sur les lignes pour trouver l'inverse des matrices suivantes, si elle existe. Le cas échéant, vérifier votre réponse par multiplication de la matrice et de son inverse.

- a) $\begin{bmatrix} 3 & 4 & -1 \\ 1 & 0 & 3 \\ 2 & 5 & -4 \end{bmatrix}$ b) $\begin{bmatrix} -1 & 3 & -4 \\ 2 & 4 & 1 \\ -4 & 2 & -9 \end{bmatrix}$ c) $\begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \end{bmatrix}$

$$\text{d) } \begin{bmatrix} 2 & 6 & 6 \\ 2 & 7 & 6 \\ 2 & 7 & 7 \end{bmatrix} \quad \text{e) } \begin{bmatrix} 1 & 0 & 1 \\ -1 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

7.17 Trouver l'inverse des matrices suivantes, où $k, k_1, \dots, k_4$ sont des constantes non nulles.

$$\text{a) } \begin{bmatrix} k_1 & 0 & 0 & 0 \\ 0 & k_2 & 0 & 0 \\ 0 & 0 & k_3 & 0 \\ 0 & 0 & 0 & k_4 \end{bmatrix} \quad \text{b) } \begin{bmatrix} 0 & 0 & 0 & k_1 \\ 0 & 0 & k_2 & 0 \\ 0 & k_3 & 0 & 0 \\ k_4 & 0 & 0 & 0 \end{bmatrix} \quad \text{c) } \begin{bmatrix} k & 0 & 0 & 0 \\ 1 & k & 0 & 0 \\ 0 & 1 & k & 0 \\ 0 & 0 & 1 & k \end{bmatrix}$$

7.18 Soit la matrice

$$\mathbf{A} = \begin{bmatrix} 1 & 0 \\ -5 & 2 \end{bmatrix}.$$

- a) Trouver des matrices élémentaires $\mathbf{E}_1$ et $\mathbf{E}_2$ tel que $\mathbf{E}_2\mathbf{E}_1\mathbf{A} = \mathbf{I}$.
- b) Écrire $\mathbf{A}^{-1}$ comme le produit de deux matrices élémentaires.
- c) Écrire $\mathbf{A}$ comme le produit de deux matrices élémentaires.

7.19 Résoudre le système d'équations suivant par la méthode de la matrice inverse :

$$\begin{aligned} x_1 + 2x_2 + x_3 &= -1 \\ x_1 - x_2 + x_3 &= 3 \\ x_1 + x_2 &= 4. \end{aligned}$$

7.20 Soit la matrice et le vecteur

$$\mathbf{A} = \begin{bmatrix} 2 & 1 & 2 \\ 2 & 2 & -2 \\ 3 & 1 & 1 \end{bmatrix} \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}.$$

- a) Vérifier que l'équation $\mathbf{Ax} = \mathbf{x}$ peut s'écrire sous la forme $(\mathbf{I} - \mathbf{A})\mathbf{x} = \mathbf{0}$ et utiliser cette formule pour résoudre $\mathbf{Ax} = \mathbf{x}$ pour $\mathbf{x}$.
- b) Résoudre $\mathbf{Ax} = 4\mathbf{x}$.

7.21 Sans faire aucun calcul, déterminer si les systèmes d'équations homogènes ci-dessous admettent une solution autre que la solution triviale, puis déterminer si la matrice des coefficients est inversible.

$$\begin{array}{ll}
 \text{a)} & 2x_1 - x_2 - 3x_3 + x_4 = 0 \\
 & 5x_2 + 4x_3 + 3x_4 = 0 \\
 & x_3 + 2x_4 = 0 \\
 & 3x_4 = 0 \\
 \text{b)} & 5x_1 - x_2 + 4x_3 + x_4 = 0 \\
 & 2x_3 - x_4 = 0 \\
 & x_3 + x_4 = 0 \\
 & 7x_4 = 0
 \end{array}$$

7.22 Par simple inspection visuelle de la matrice

$$\mathbf{A} = \begin{bmatrix} -2 & 8 & 1 & 4 \\ 3 & 2 & 5 & 1 \\ 1 & 10 & 6 & 5 \\ 4 & -6 & 4 & -3 \end{bmatrix},$$

expliquer pourquoi $\det(\mathbf{A}) = 0$.

7.23 Déterminer si les matrices suivantes sont inversibles.

$$\begin{array}{ll}
 \text{a)} & \begin{bmatrix} 1 & 0 & -1 \\ 9 & -1 & 4 \\ 9 & 9 & -1 \end{bmatrix} \\
 \text{b)} & \begin{bmatrix} 4 & 2 & 8 \\ -2 & 1 & -4 \\ 3 & 1 & 6 \end{bmatrix} \\
 \text{c)} & \begin{bmatrix} \sqrt{2} & -\sqrt{7} & 0 \\ 3\sqrt{2} & -3\sqrt{7} & 0 \\ 5 & -9 & 0 \end{bmatrix} \\
 \text{d)} & \begin{bmatrix} -3 & 0 & 1 \\ 5 & 0 & 6 \\ 8 & 0 & 3 \end{bmatrix}
 \end{array}$$

7.24 Soit $\mathbf{A}$ une matrice 3×3 dont $\det(\mathbf{A}) = -7$. Trouver les déterminants suivants.

$$\text{a) } \det(3\mathbf{A}) \quad \text{b) } \det(\mathbf{A}^{-1}) \quad \text{c) } \det(2\mathbf{A}^{-1}) \quad \text{d) } \det((2\mathbf{A})^{-1})$$

7.25 Pour quelle(s) valeur(s) de k les matrices ci-dessous ne sont pas inversibles ?

$$\begin{array}{ll}
 \text{a)} & \begin{bmatrix} k-3 & -2 \\ -2 & k-2 \end{bmatrix} \\
 \text{b)} & \begin{bmatrix} 1 & 2 & 4 \\ 3 & 1 & 6 \\ k & 3 & 2 \end{bmatrix}
 \end{array}$$

7.26 Soit

$$\mathbf{A} = \begin{bmatrix} 4 & -1 & 1 & 6 \\ 0 & 0 & -3 & 3 \\ 4 & 1 & 0 & 14 \\ 4 & 1 & 3 & 2 \end{bmatrix}.$$

Trouver les valeurs suivantes.

- a) M_{13} et C_{13}
- b) M_{23} et C_{23}
- c) M_{22} et C_{22}
- d) M_{21} et C_{21}
- e) $\det(A)$

Réponses

- 7.1** a) $x_1 = -37, x_2 = -8, x_3 = 5$
 b) $x_1 = -10 + 13t, x_2 = -5 + 13t, x_3 = 2 - t, x_4 = t$
 c) $x_1 = -11 - 7s + 2t, x_2 = s, x_3 = -4 - 3t, x_4 = 9 - 3t, x_5 = t$
 d) pas de solution
- 7.2** a) $x_1 = 3, x_2 = 1, x_3 = 2$
 b) $x_1 = -\frac{1}{7} - \frac{3}{7}t, x_2 = \frac{1}{7} - \frac{4}{7}t, x_3 = t$
 c) $x_1 = t - 1, x_2 = 2s, x_3 = s, x_4 = t$
 d) pas de solution
- 7.3** a) $x_1 = 2 - 12t, x_2 = 5 - 27t, x_3 = t$
 b) pas de solution
 c) $u = -2s - 3t - 6, v = s, w = -t - 2, x = t + 3, y = t$
- 7.4** a) solution triviale
 b) $x_1 = -s, x_2 = -t - s, x_3 = 4s, x_4 = t$
 c) $w = t, x = -t, y = t, z = 0$
- 7.5** Aucune solution : $a = -4$. Une seule solution : $a \neq \pm 4$. Infinité de solutions : $a = 4$.
- 7.6** a) non défini b) 4×2 c) non défini d) non défini e) 5×5 f) 5×2 g) non défini h) 5×2
- 7.7** a) $\begin{bmatrix} 7 & 6 & 5 \\ -2 & 1 & 3 \\ 7 & 3 & 7 \end{bmatrix}$ b) $\begin{bmatrix} 12 & -3 \\ -4 & 5 \\ 4 & 1 \end{bmatrix}$ c) $\begin{bmatrix} -6 & -3 \\ 36 & 0 \\ 4 & 7 \end{bmatrix}$ d) non défini
 e) $\begin{bmatrix} 2 & -10 & 11 \\ 13 & 2 & 5 \\ 4 & -3 & 13 \end{bmatrix}$ f) $\begin{bmatrix} 10 & -6 \\ -14 & 2 \\ -1 & -8 \end{bmatrix}$ g) $\begin{bmatrix} 40 & 72 \\ 26 & 42 \end{bmatrix}$ h) $\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$ i) 61 j) 35

7.12 d) seulement

7.13 a) non b) oui

7.15 a) $\begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}$ b) $\begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}$ c) $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -2 & 0 & 1 \end{bmatrix}$ d) $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 2 & 0 & 1 \end{bmatrix}$

7.16 a) $\begin{bmatrix} \frac{3}{2} & -\frac{11}{10} & -\frac{6}{5} \\ -1 & 1 & 1 \\ -\frac{1}{2} & \frac{7}{10} & \frac{2}{5} \end{bmatrix}$ b) n'existe pas c) $\frac{1}{2} \begin{bmatrix} 1 & -1 & 1 \\ -1 & 1 & 1 \\ 1 & 1 & -1 \end{bmatrix}$
d) $\begin{bmatrix} \frac{7}{2} & 0 & -3 \\ -1 & 1 & 0 \\ 0 & -1 & 1 \end{bmatrix}$ e) $\frac{1}{2} \begin{bmatrix} 1 & -1 & 1 \\ 0 & 0 & 2 \\ 1 & 1 & -1 \end{bmatrix}$

7.17 a) $\begin{bmatrix} k_1^{-1} & 0 & 0 & 0 \\ 0 & k_2^{-1} & 0 & 0 \\ 0 & 0 & k_3^{-1} & 0 \\ 0 & 0 & 0 & k_4^{-1} \end{bmatrix}$ b) $\begin{bmatrix} 0 & 0 & 0 & k_1^{-1} \\ 0 & 0 & k_2^{-1} & 0 \\ 0 & k_3^{-1} & 0 & 0 \\ k_4^{-1} & 0 & 0 & 0 \end{bmatrix}$
c) $\begin{bmatrix} k^{-1} & 0 & 0 & 0 \\ -k^{-2} & k^{-1} & 0 & 0 \\ k^{-3} & -k^{-2} & k^{-1} & 0 \\ -k^{-4} & k^{-3} & -k^{-2} & k^{-1} \end{bmatrix}$

7.18 a) $E_1 = \begin{bmatrix} 1 & 0 \\ 5 & 1 \end{bmatrix}, E_2 = \begin{bmatrix} 1 & 0 \\ 0 & \frac{1}{2} \end{bmatrix}$

7.19 $x_1 = \frac{16}{3}, x_2 = -\frac{4}{3}, x_3 = -\frac{11}{3}$

7.21 a) solution triviale seulement, matrice inversible b) infinité de solutions, matrice singulière

7.23 a) oui b) non c) non d) non

7.24 a) -189 b) $-\frac{1}{7}$ c) $-\frac{8}{7}$ d) $-\frac{1}{56}$

7.25 a) $(5 \pm \sqrt{17})/2$ b) -1

7.26 a) $M_{13} = 0, C_{13} = 0$

b) $M_{23} = -96, C_{23} = 96$

c) $M_{22} = -48, C_{22} = -48$

d) $M_{21} = 72, C_{21} = -72$

e) -216

8 Valeurs propres, vecteurs propres et diagonalisation

Objectifs du chapitre

- ▶ Énoncer les définitions de valeur propre et de vecteur propre d'une matrice.
- ▶ Calculer les valeurs propres d'une matrice et les vecteurs propres correspondants.
- ▶ Déterminer si une matrice est diagonalisable ou non.
- ▶ Diagonaliser une matrice carrée.

Les valeurs et vecteurs propres sont des caractéristiques liées à une matrice carrée. Ces notions admettent une grande variété d'interprétations physiques différentes, chacune ancrée dans son domaine d'application : géométrie, résolution d'équations différentielles, mécanique quantique, finance, moteurs de recherche, etc.

Dans le cadre de ce cours, l'étude des valeurs et vecteurs propres demeurera plutôt collée aux définitions de base. Outre l'exemple résolu du chapitre, nous ne verrons qu'une seule application, celle que vous êtes la plus susceptible de rencontrer dans vos études en actuariat, soit la diagonalisation d'une matrice.



Visionnez la vidéo qui explique comment **Google utilise les vecteurs propres** [🔗](#) pour établir son classement des résultats de recherche.



8.1 Énoncé du problème

Depuis le milieu des années 1960, les systèmes postaux de tri acheminent une énorme proportion du courrier vers la bonne adresse tout à fait automatiquement. Pour y arriver, ils ont recours à la reconnaissance optique de caractères (*optical character recognition*, OCR), un procédé informatique par lequel une image est convertie en texte.

La reconnaissance optique de caractères repose généralement, après divers pré-traitements, sur le découpage de chaque caractère en un certain nombre de « pixels », chacun de ceux-ci pouvant être « blanc » ou « noir ». À titre d'exemple, la base de données d'apprentissage machine [Optical Recognition of Handwritten Digits](#)  (Dua et Graff, 2019) contient la représentation matricielle de taille 32×32 de 1 797 chiffres entre 0 et 9 écrits à la main. Le tout premier caractère de cette base de données est représenté à la [figure 8.1](#).

Afin de réduire la dimension des données, l'information est enregistrée dans la base de données sous forme du nombre de pixels noirs dans des carrés disjoints de taille 4×4 . Ces carrés forment la grille visible dans la [figure 8.1](#). L'objet `optd` du paquetage R **RSKC** (Kondo et collab., 2016) est une matrice de 1 797 lignes et de 64 colonnes qui contient les représentations compactes des chiffres de la base de données. Par exemple, celle du chiffre de la figure 8.1 est la suivante (présentée sous forme de matrice pour faciliter la comparaison avec l'image) :

```
> matrix(optd[1, ], nrow = 8, byrow = TRUE)
      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8]
[1,]    0    0    5   13    9    1    0    0
[2,]    0    0   13   15   10   15    5    0
[3,]    0    3   15    2    0   11    8    0
[4,]    0    4   12    0    0    8    8    0
[5,]    0    5    8    0    0    9    8    0
[6,]    0    4   11    0    1   12    7    0
[7,]    0    2   14    5   10   12    0    0
[8,]    0    0    6   13   10    0    0    0
```

Pour effectuer la reconnaissance optique de caractères, nous souhaitons classer, à l'aide d'un nombre restreint de caractéristiques, les différentes images de la base de données en fonction du chiffre auquel chacune est le plus susceptible de correspondre. Présenté autrement, ce problème consiste à réduire la dimension des données — actuellement de 64 pour chaque image — tout en conservant le plus d'information possible pour réaliser la classi-

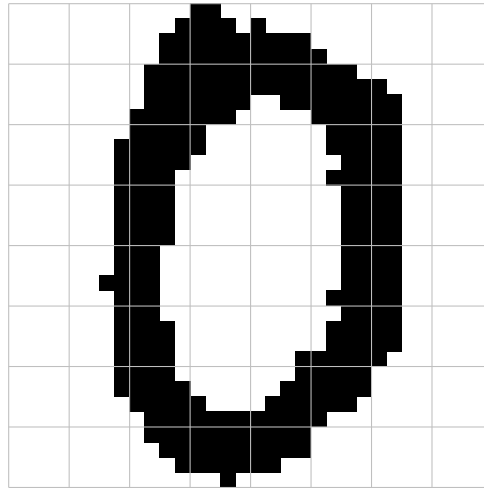


FIG. 8.1 – Représentation matricielle de 32 pixels de haut et 32 pixels de large d'un chiffre écrit à la main. La grille représente le regroupement des pixels en groupes de 4×4 .

fication. Cela peut sembler simple avec le caractère de la [figure 8.1](#), mais sauriez-vous identifier facilement le caractère de la [figure 8.2](#) tiré de la même base de données ?

Le contexte du présent problème est celui de l'analyse en composantes principales, une procédure statistique qui, comme nous le verrons, est intimement liée aux notions de valeur et de vecteur propre.

8.2 Définitions

En algèbre vectorielle, le produit par une matrice $\mathbf{A}$ peut être vu comme un opérateur permettant d'appliquer une transformation à un vecteur $\mathbf{x}$. Par exemple :

► La multiplication par un scalaire :

$$\begin{bmatrix} 3 & 0 \\ 8 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 3 \\ 6 \end{bmatrix} = 3 \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

► La rotation d'un angle θ :

$$\begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x \cos \theta - y \sin \theta \\ x \sin \theta + y \cos \theta \end{bmatrix} = \begin{bmatrix} w \\ z \end{bmatrix}.$$

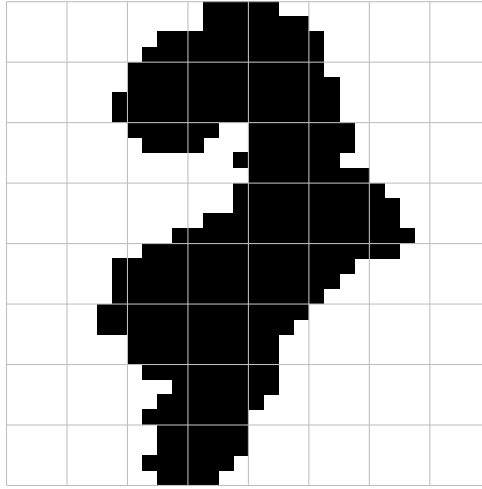
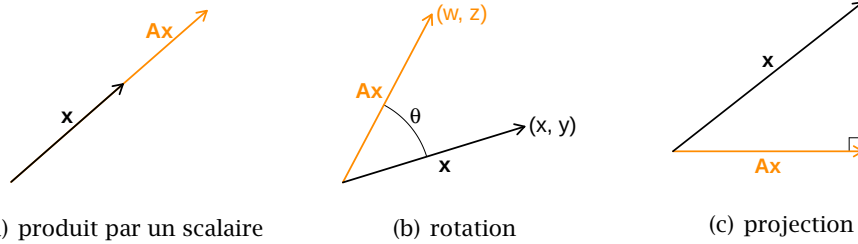


FIG. 8.2 – Représentation matricielle peu lisible d'un chiffre écrit à la main



(a) produit par un scalaire

(b) rotation

(c) projection

FIG. 8.3 – Illustrations de trois transformations d'un vecteur $\mathbf{x}$ correspondant à un produit matriciel $\mathbf{Ax}$.

► La projection sur un autre vecteur :

$$\begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \end{bmatrix}.$$

Voir la [figure 8.3](#) pour des illustrations des transformations ci-dessus.

Les définitions de valeur propre et de vecteur propre procèdent de cette interprétation du produit matriciel.

Définition 8.1. Soit $\mathbf{A}$ une matrice $n \times n$. Alors le vecteur non nul $\mathbf{x} \in \mathbb{R}^n$ est un *vecteur propre* de $\mathbf{A}$ si

$$\mathbf{Ax} = \lambda \mathbf{x}$$

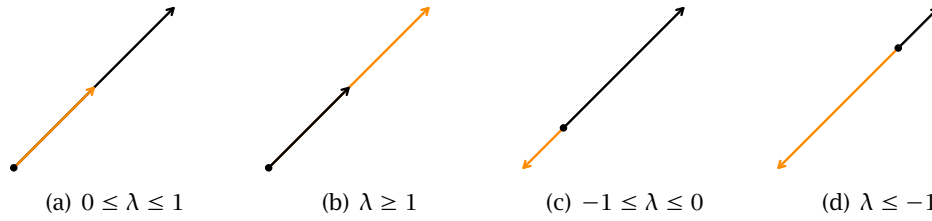


FIG. 8.4 - Illustration de la relation entre une matrice, une valeur propre de celle-ci et le vecteur propre associé. Un vecteur propre $\mathbf{x}$ d'une matrice $\mathbf{A}$ (en noir) est transformé en un multiple λ de lui-même lorsque multiplié par $\mathbf{A}$ (en orange).

pour une valeur de λ appelée *valeur propre* de $\mathbf{A}$. On dit que $\mathbf{x}$ est le vecteur propre *correspondant* à λ .

Ainsi, par définition, lorsqu'un vecteur propre $\mathbf{x}$ est multiplié par la matrice $\mathbf{A}$, il est transformé en un multiple de lui-même ; voir la [figure 8.4](#).

Exemple 8.1. Le vecteur $\mathbf{x} = (1, 2)$ est un vecteur propre de

$$\mathbf{A} = \begin{bmatrix} 3 & 0 \\ 8 & -1 \end{bmatrix}$$

correspondant à la valeur propre $\lambda = 3$ car $\mathbf{Ax} = 3\mathbf{x}$. En effet,

$$\begin{bmatrix} 3 & 0 \\ 8 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = 3 \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

□



Le qualificatif « propre » doit être pris dans le sens de « qui caractérise ». En anglais, les termes les plus souvent utilisés pour « valeur propre » et « vecteur propre » sont *eigenvalue* et *eigenvector*, dans l'ordre. Ils sont formés à partir du préfixe allemand *eigen* plutôt qu'à partir de son équivalent anglais *own*, probablement pour des raisons générales de viabilité phonétique.



8.3 Indice pour le problème

Nous aurons besoin d'un peu de notation pour résoudre notre problème. Soit $\mathbf{X} = (X_1, \dots, X_p)^\top$ un vecteur $p \times 1$ de variables aléatoires. (La coutume

de représenter les variables aléatoires par des lettres majuscules $\mathbf{v}$, ici, l'emporter sur celle suivie jusqu'à présent de représenter les vecteurs par des lettres minuscules.) Chaque *ligne* de la matrice optd correspond à un vecteur $\mathbf{X}$. Dans notre exemple, nous avons donc $p = 64$ et 1 797 réalisations de $\mathbf{X}$.

Pour diverses considérations sur lesquelles nous ferons l'impasse ici, et sans perte de généralité, les observations de $\mathbf{X}$ sont généralement normalisées de telle sorte que leur moyenne vaut 0 et leur variance, 1. La matrice de variance-covariance¹ de $\mathbf{X}$ est donc une matrice $\text{Var}[\mathbf{X}] = \Sigma$ avec des 1 sur la diagonale.

Nous avons établi précédemment vouloir réduire la dimension des données afin de faciliter leur classement. Nous considérons qu'une telle réduction est possible parce que les données comportent une certaine redondance, ou corrélation. Nous chercherons donc d'abord à éliminer cette corrélation. Une manière de « décorrélérer » les éléments d'un vecteur aléatoire consiste à les combiner linéairement. Nous voulons donc créer un nouveau vecteur aléatoire $\mathbf{Y}$ à l'aide de la transformation

$$\mathbf{Y} = \mathbf{W}\mathbf{X},$$

où $\mathbf{W}$ est une matrice $p \times p$ déterminée de telle sorte que

$$\begin{aligned}\text{Var}[\mathbf{Y}] &= \text{Var}[\mathbf{W}\mathbf{X}] \\ &= \mathbf{W} \text{Var}[\mathbf{X}] \mathbf{W}^T \\ &= \mathbf{W} \Sigma \mathbf{W}^T\end{aligned}$$

est une matrice diagonale.

Tel qu'expliqué dans la section précédente, la combinaison linéaire $\mathbf{Y} = \mathbf{W}\mathbf{X}$ correspond à un changement de base ou, de manière équivalente, à une rotation d'axes. Quant au rôle des valeurs propres dans la solution, nous ne le connaissons pas encore.

8.4 Calcul des valeurs propres

Pour trouver les valeurs propres d'une matrice $\mathbf{A}$, nous devons résoudre

$$\mathbf{A}\mathbf{x} = \lambda\mathbf{x}$$

1. Comme son nom le suggère, une matrice de variance-covariance contient sur la diagonale les variances des éléments de $\mathbf{X}$ et, hors de la diagonale, les covariances des éléments pris deux à deux. Par essence, une matrice de variance est symétrique.

ou, de manière équivalente,

$$(\lambda \mathbf{I} - \mathbf{A})\mathbf{x} = \mathbf{0}.$$

pour $\mathbf{x} \neq \mathbf{0}$. Par conséquent, λ est une valeur propre de $\mathbf{A}$ si, et seulement si, il existe une solution autre que la solution triviale à ce système d'équations homogène.

Or, par le [théorème 7.13](#), il existe une solution de $(\lambda \mathbf{I} - \mathbf{A})\mathbf{x} = \mathbf{0}$ autre que la solution triviale si, et seulement si, $\det(\lambda \mathbf{I} - \mathbf{A}) = 0$. Par conséquent :

► les valeurs propres de $\mathbf{A}$ sont les solutions de

$$\det(\lambda \mathbf{I} - \mathbf{A}) = 0;$$

► cette équation est l'*équation caractéristique* de $\mathbf{A}$;

► $\det(\lambda \mathbf{I} - \mathbf{A})$ est le *polynôme caractéristique* (de degré n) de $\mathbf{A}$.

Exemple 8.2. Soit

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 4 & -17 & 8 \end{bmatrix}.$$

Le polynôme caractéristique de $\mathbf{A}$ est

$$\begin{aligned} \det(\lambda \mathbf{I} - \mathbf{A}) &= \begin{vmatrix} \lambda & -1 & 0 \\ 0 & \lambda & -1 \\ -4 & 17 & \lambda - 8 \end{vmatrix} \\ &= \lambda[\lambda(\lambda - 8) + 17] - 4 \\ &= \lambda^3 - 8\lambda^2 + 17\lambda - 4. \end{aligned}$$

Les valeurs propres sont les racines de ce polynôme, c'est-à-dire les solutions de

$$\lambda^3 - 8\lambda^2 + 17\lambda - 4 = 0.$$

Or,

$$\lambda^3 - 8\lambda^2 + 17\lambda - 4 = (\lambda - 4)(\lambda^2 - 4\lambda + 1),$$

d'où les valeurs propres sont

$$\lambda_1 = 4$$

$$\lambda_2 = 2 + \sqrt{3}$$

$$\lambda_3 = 2 - \sqrt{3}.$$

□

Exemple 8.3. Soit la matrice triangulaire

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ 0 & a_{22} & a_{23} \\ 0 & 0 & a_{33} \end{bmatrix}.$$

On a

$$\begin{aligned} \det(\lambda \mathbf{I} - \mathbf{A}) &= \begin{vmatrix} \lambda - a_{11} & -a_{12} & -a_{13} \\ 0 & \lambda - a_{22} & -a_{23} \\ 0 & 0 & \lambda - a_{33} \end{vmatrix} \\ &= (\lambda - a_{11})(\lambda - a_{22})(\lambda - a_{33}), \end{aligned}$$

d'où $\lambda_1 = a_{11}$, $\lambda_2 = a_{22}$ et $\lambda_3 = a_{33}$. □

Le résultat de l'exemple précédent justifie un théorème.

Théorème 8.1. *Si $\mathbf{A}_{n \times n}$ est une matrice triangulaire (supérieure ou inférieure) ou diagonale, alors ses valeurs propres sont les éléments de la diagonale.*

De ce théorème, nous pouvons établir les faits suivants pour une matrice triangulaire :

1. la trace est égale à la somme des valeurs propres ;
2. le déterminant est égal au produit des valeurs propres.

Le théorème suivant confirme que ces résultats tiennent pour toute matrice carrée.

Théorème 8.2. *Soit $\mathbf{A}_{n \times n}$ une matrice dont les valeurs propres sont $\lambda_1, \dots, \lambda_n$ (égalités et valeurs complexes comprises). Alors*

$$\begin{aligned} \operatorname{tr}(\mathbf{A}) &= \sum_{i=1}^n \lambda_i \\ \det(\mathbf{A}) &= \prod_{i=1}^n \lambda_i. \end{aligned}$$

Démonstration. On peut démontrer que le polynôme caractéristique d'une matrice carrée $\mathbf{A}$ est toujours de la forme

$$\begin{aligned} \det(\lambda \mathbf{I} - \mathbf{A}) &= \lambda^n - c_1 \lambda^{n-1} + \dots + c_n \\ &= (\lambda - \lambda_1) \cdots (\lambda - \lambda_n), \end{aligned}$$

où

$$\begin{aligned}c_1 &= \lambda_1 + \cdots + \lambda_n \\ c_n &= (-1)^n \lambda_1 \cdots \lambda_n.\end{aligned}$$

(Pensez au cas $n = 2$, soit un polynôme de second degré; vous savez que le coefficient du terme de degré un est la somme des racines et que le terme constant est le produit des racines.) Il faut maintenant établir que $c_1 = \text{tr}(\mathbf{A})$ et que $c_n = (-1)^n \det(\mathbf{A})$ pour compléter la démonstration. Or, d'une part, il est facile de vérifier que le coefficient de λ^{n-1} dans le polynôme caractéristique provient du produit

$$(\lambda - a_{11}) \cdots (\lambda - a_{nn}) = \lambda^n - (a_{11} + \cdots + a_{nn})\lambda^{n-1} + \dots$$

dans le calcul du déterminant de $\lambda \mathbf{I} - \mathbf{A}$. Par conséquent,

$$\begin{aligned}\text{tr}(\mathbf{A}) &= \sum_{i=1}^n a_{ii} \\ &= c_1 \\ &= \sum_{i=1}^n \lambda_i.\end{aligned}$$

D'autre part, en posant $\lambda = 0$ dans le polynôme caractéristique, nous obtenons

$$\begin{aligned}\det(-\mathbf{A}) &= (-1)^n \det(\mathbf{A}) \\ &= c_n \\ &= (-1)^n \lambda_1 \cdots \lambda_n,\end{aligned}$$

d'où $\det(\mathbf{A}) = \prod_{i=1}^n \lambda_i$. □

Du théorème précédent, il découle que si au moins une valeur propre d'une matrice est nulle, alors son déterminant est nul et la matrice est singulière. Cette observation nous permet d'ajouter un énoncé au [théorème 7.13](#).

Théorème 8.3. *Soit $\mathbf{A}$ une matrice $n \times n$. Les énoncés suivants sont équivalents.*

1. $\mathbf{A}$ est inversible.
2. $\det(\mathbf{A}) \neq 0$
3. $\text{rang}(\mathbf{A}) = n$

4. La seule solution de $\mathbf{Ax} = \mathbf{0}$ est la solution triviale.
5. Le système d'équations $\mathbf{Ax} = \mathbf{b}$ a une solution pour tout vecteur $\mathbf{b}$ et celle-ci est unique.
6. $\mathbf{A}$ peut être exprimée comme un produit de matrices élémentaires.
7. Les vecteurs lignes de $\mathbf{A}$ sont linéairement indépendants.
8. Les vecteurs colonnes de $\mathbf{A}$ sont linéairement indépendants.
9. Les vecteurs lignes de $\mathbf{A}$ forment une base de $\mathbb{R}^n$.
10. Les vecteurs colonnes de $\mathbf{A}$ forment une base de $\mathbb{R}^n$.
11. $\lambda = 0$ n'est pas une valeur propre de $\mathbf{A}$.

Un dernier résultat : nous observons que si λ est une valeur propre de $\mathbf{A}$ et $\mathbf{x}$ est un vecteur propre correspondant, alors

$$\mathbf{A}^2\mathbf{x} = \mathbf{A}(\mathbf{Ax}) = \mathbf{A}(\lambda\mathbf{x}) = \lambda(\mathbf{Ax}) = \lambda(\lambda\mathbf{x}) = \lambda^2\mathbf{x},$$

d'où λ^2 est une valeur propre de $\mathbf{A}^2$ et $\mathbf{x}$ est un vecteur propre correspondant.

Théorème 8.4. Si λ est une valeur propre de la matrice $\mathbf{A}$ et $\mathbf{x}$ est un vecteur propre correspondant, alors

- λ^k est une valeur propre de $\mathbf{A}^k$, $k \in \mathbb{N}$;
- $\mathbf{x}$ est un vecteur propre correspondant.

Démonstration. Laissée en exercice. □



Vous savez que pour une fonction $y = f(x)$ quelconque, $y = f(x - c)$ correspond à une translation de la fonction de c unités vers la droite. Par analogie, l'équation caractéristique $\det(\lambda\mathbf{I} - \mathbf{A})$ fournit cette interprétation de la valeur propre : c'est le nombre de fois dont il faut « déplacer » la matrice $\mathbf{A}$ de $\mathbf{I}$ jusqu'à ce qu'elle devienne singulière.

8.5 Calcul des vecteurs propres

Les vecteurs propres de la matrice $\mathbf{A}$ correspondant à la valeur propre λ sont les vecteurs non nuls satisfaisant

$$\mathbf{Ax} = \lambda\mathbf{x}$$

ou, de manière équivalente,

$$(\lambda \mathbf{I} - \mathbf{A})\mathbf{x} = \mathbf{0}.$$

Or, puisque $\det(\lambda \mathbf{I} - \mathbf{A}) = 0$:

- le rang de $\lambda \mathbf{I} - \mathbf{A}$ est inférieur à n ;
- il existe une solution $(\lambda \mathbf{I} - \mathbf{A})\mathbf{x} = \mathbf{0}$ autre que la solution triviale ;
- le système d'équations homogène comporte une infinité de solutions.

Par conséquent, pour chaque valeur propre λ , il faut trouver une base de $\mathbb{R}^k$, où

$$k = n - \text{rang}(\lambda \mathbf{I} - \mathbf{A}).$$

Exemple 8.4. Soit

$$\mathbf{A} = \begin{bmatrix} 0 & 0 & -2 \\ 1 & 2 & 1 \\ 1 & 0 & 3 \end{bmatrix}.$$

L'équation caractéristique de $\mathbf{A}$ est

$$(\lambda - 2)^2(\lambda - 1) = 0,$$

d'où les valeurs propres sont $\lambda_1 = 1$ et $\lambda_2 = \lambda_3 = 2$.

1. Cas $\lambda_1 = 1$. Nous devons trouver $\mathbf{x} = (x_1, x_2, x_3)$ tel que

$$(\mathbf{I} - \mathbf{A})\mathbf{x} = \mathbf{0}$$

soit

$$\begin{bmatrix} 1 & 0 & 2 \\ -1 & -1 & -1 \\ -1 & 0 & -2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

ou, de manière équivalente,

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & -1 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}.$$

Par conséquent :

- la solution du système d'équations est $x_1 = -2x_3$, $x_2 = x_3$ et x_3 est une variable libre ;

- $(-2s, s, s)$ est un vecteur propre correspondant à $\lambda = 1$;
- en fait, tout vecteur de la forme $(-2, 1, 1)$ est un vecteur propre correspondant à $\lambda = 1$.

Ainsi, $(-2, 1, 1)$ est une base de vecteurs propres correspondant à $\lambda = 1$.

2. Cas $\lambda_2 = \lambda_3 = 2$. Nous avons

$$(2\mathbf{I} - \mathbf{A})\mathbf{x} = \mathbf{0}$$

soit

$$\begin{bmatrix} 2 & 0 & 2 \\ -1 & 0 & -1 \\ -1 & 0 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

ou, de manière équivalente,

$$\begin{bmatrix} 1 & 0 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}.$$

Par conséquent :

- la solution est $x_1 = -x_3$, x_2 et x_3 sont des variables libres ;
- $(s, t, -s)$ est un vecteur propre correspondant à $\lambda = 2$;
- on a

$$\begin{bmatrix} s \\ t \\ -s \end{bmatrix} = s \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix} + t \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}.$$

Ainsi $(1, 0, -1)$ et $(0, 1, 0)$ forment une base de vecteurs propres correspondant à $\lambda = 2$.

□



Le nombre de vecteurs propres dans la base est *au plus* la multiplicité de λ . Par conséquent, si toutes les valeurs propres d'une matrice sont différentes, alors elle compte n vecteurs propres différents.

8.6 Diagonalisation

La diagonalisation d'une matrice est la principale application des valeurs et vecteurs propres qu'un étudiant en actuariat est susceptible de rencontrer dans ses études. Elle joue un rôle dans diverses procédures statistiques, dont l'analyse en composantes principales, dans la résolution de systèmes d'équations différentielles ordinaires, ainsi que dans le calcul de l'exponentielle d'une matrice, tel qu'expliqué dans l'exemple suivant.

Exemple 8.5. L'exponentielle d'un nombre réel x est définie comme

$$\begin{aligned} e^x &= 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots \\ &= \sum_{n=0}^{\infty} \frac{x^n}{n!}. \end{aligned}$$

Par analogie, on définit l'exponentielle d'une matrice $\mathbf{A}$ ainsi :

$$\begin{aligned} e^{\mathbf{A}} &= \mathbf{I} + \mathbf{A} + \frac{\mathbf{A}^2}{2!} + \frac{\mathbf{A}^3}{3!} + \dots \\ &= \sum_{n=0}^{\infty} \frac{\mathbf{A}^n}{n!}, \end{aligned}$$

avec $\mathbf{A}^0 = \mathbf{I}$ par convention.

L'évaluation numérique de l'exponentielle d'une matrice de manière efficace, fiable et précise est chose difficile. Dans un article célèbre, [Moler et Van Loan \(1978\)](#) passent en revue pas moins de 19 manières différentes de le faire, pour conclure... qu'aucune méthode n'est uniformément meilleure que les autres !

Cela dit, le calcul de l'exponentielle d'une matrice demeure simple si celle-ci est diagonale. Pour les fins de la démonstration et sans perte de généralité, nous utiliserons une matrice diagonale $\mathbf{D}$ de dimension 2 :

$$\mathbf{D} = \begin{bmatrix} d_1 & 0 \\ 0 & d_2 \end{bmatrix}.$$

Ainsi

$$\mathbf{D}^n = \begin{bmatrix} d_1^n & 0 \\ 0 & d_2^n \end{bmatrix}$$

et, par conséquent,

$$\begin{aligned}
 e^{\mathbf{D}} &= \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} + \begin{bmatrix} d_1 & 0 \\ 0 & d_2 \end{bmatrix} + \frac{1}{2!} \begin{bmatrix} d_1^2 & 0 \\ 0 & d_2^2 \end{bmatrix} + \frac{1}{3!} \begin{bmatrix} d_1^3 & 0 \\ 0 & d_2^3 \end{bmatrix} + \dots \\
 &= \begin{bmatrix} 1 + d_1 + \frac{d_1^2}{2!} + \frac{d_1^3}{3!} + \dots & 0 \\ 0 & 1 + d_2 + \frac{d_2^2}{2!} + \frac{d_2^3}{3!} + \dots \end{bmatrix} \\
 &= \begin{bmatrix} e^{d_1} & 0 \\ 0 & e^{d_2} \end{bmatrix}.
 \end{aligned}$$

Le calcul de l'exponentielle d'une matrice diagonale se résume donc à calculer l'exponentielle des éléments de la diagonale.

Poussons l'idée une étape plus loin. Supposons que $\mathbf{A}$ est une matrice que nous pouvons écrire sous la forme

$$\mathbf{A} = \mathbf{P}\mathbf{D}\mathbf{P}^{-1},$$

où $\mathbf{D}$ est diagonale. Puisque

$$\begin{aligned}
 \mathbf{A}^2 &= (\mathbf{P}\mathbf{D}\mathbf{P}^{-1})(\mathbf{P}\mathbf{D}\mathbf{P}^{-1}) \\
 &= \mathbf{P}\mathbf{D}^2\mathbf{P}^{-1}, \\
 \mathbf{A}^3 &= (\mathbf{P}\mathbf{D}\mathbf{P}^{-1})(\mathbf{P}\mathbf{D}^2\mathbf{P}^{-1}) \\
 &= \mathbf{P}\mathbf{D}^3\mathbf{P}^{-1} \\
 &\vdots \\
 \mathbf{A}^n &= \mathbf{P}\mathbf{D}^n\mathbf{P}^{-1},
 \end{aligned}$$

alors

$$\begin{aligned}
 e^{\mathbf{A}} &= \sum_{n=0}^{\infty} \frac{\mathbf{P}\mathbf{D}^n\mathbf{P}^{-1}}{n!} \\
 &= \mathbf{P} \left(\sum_{n=0}^{\infty} \frac{\mathbf{D}^n}{n!} \right) \mathbf{P}^{-1} \\
 &= \mathbf{P}e^{\mathbf{D}}\mathbf{P}^{-1}.
 \end{aligned}$$

Par conséquent, si nous pouvons écrire une matrice $\mathbf{A}$ sous la forme $\mathbf{A} = \mathbf{P}\mathbf{D}\mathbf{P}^{-1}$ (ou, inversement, si $\mathbf{P}^{-1}\mathbf{A}\mathbf{P}$ est diagonale), alors le calcul de son exponentielle se résumera essentiellement au calcul de l'exponentielle de la matrice diagonale $\mathbf{D}$, calcul qui est simple, comme nous l'avons vu. $\square$

Définition 8.2. Une matrice carrée $\mathbf{A}$ est *diagonalisable* s'il existe une matrice inversible $\mathbf{P}$ telle que

$$\mathbf{P}^{-1}\mathbf{A}\mathbf{P}$$

est une matrice diagonale.

Nous établirons qu'une matrice est diagonalisable si elle possède n vecteurs propres linéairement indépendants.

Exemple 8.6. À l'exemple 8.4, nous avons obtenu que les vecteurs propres de la matrice $\mathbf{A}$ sont

$$\mathbf{p}_1 = \begin{bmatrix} -2 \\ 1 \\ 1 \end{bmatrix} \quad \mathbf{p}_2 = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix} \quad \mathbf{p}_3 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}.$$

Ces vecteurs propres sont linéairement indépendants. Vérification rapide :

$$\begin{vmatrix} -2 & 1 & 0 \\ 1 & 0 & 1 \\ 1 & -1 & 0 \end{vmatrix} = -1 \neq 0.$$

□

Supposons qu'une matrice $\mathbf{A}_{3 \times 3}$ possède trois vecteurs propres linéairement indépendants $\mathbf{p}_1, \dots, \mathbf{p}_3$ et soit la matrice $\mathbf{P}$ formée de ces trois vecteurs dans les colonnes :

$$\begin{aligned} \mathbf{P} &= [\mathbf{p}_1 \ \mathbf{p}_2 \ \mathbf{p}_3] \\ &= \begin{bmatrix} p_{11} & p_{12} & \dots & p_{1n} \\ p_{21} & p_{22} & \dots & p_{2n} \\ \vdots & \vdots & & \vdots \\ p_{n1} & p_{n2} & \dots & p_{nn} \end{bmatrix}. \end{aligned}$$

Alors,

$$\mathbf{A}\mathbf{P} = [\mathbf{A}\mathbf{p}_1 \ \mathbf{A}\mathbf{p}_2 \ \mathbf{A}\mathbf{p}_3]$$

et, par définition du vecteur propre,

$$\begin{aligned} \mathbf{A}\mathbf{P} &= [\lambda_1 \mathbf{p}_1 \ \lambda_2 \mathbf{p}_2 \ \lambda_3 \mathbf{p}_3] \\ &= [\mathbf{p}_1 \ \mathbf{p}_2 \ \mathbf{p}_3] \begin{bmatrix} \lambda_1 & 0 & \dots & 0 \\ 0 & \lambda_2 & \dots & 0 \\ \vdots & \vdots & & \vdots \\ 0 & 0 & \dots & \lambda_n \end{bmatrix} \\ &= \mathbf{P}\mathbf{D}, \end{aligned}$$

où $\mathbf{D}$ est une matrice diagonale. Or, puisque les colonnes de $\mathbf{P}$ sont linéairement indépendantes, $\mathbf{P}^{-1}$ existe et, par conséquent,

$$\mathbf{P}^{-1}\mathbf{A}\mathbf{P} = \mathbf{D} = \text{diag}(\lambda_1, \dots, \lambda_n),$$

d'où $\mathbf{A}$ est diagonalisable.

Nous pourrions faire le cheminement inverse, ce qui justifie le théorème suivant.

Théorème 8.5. *Soit $\mathbf{A}$ une matrice $n \times n$. Les énoncés suivants sont équivalents.*

1. $\mathbf{A}$ est diagonalisable.
2. $\mathbf{A}$ possède n vecteurs propres linéairement indépendants.

De plus, si $\mathbf{P}$ est la matrice qui diagonalise $\mathbf{A}$, alors

$$\mathbf{P}^{-1}\mathbf{A}\mathbf{P} = \begin{bmatrix} \lambda_1 & 0 & \dots & 0 \\ 0 & \lambda_2 & \dots & 0 \\ \vdots & \vdots & & \vdots \\ 0 & 0 & \dots & \lambda_n \end{bmatrix}.$$

La remarque qui suit l'exemple 8.4 établit que si toutes les valeurs propres sont distinctes (multiplicité 1), alors la matrice compte n vecteurs propres. Or, on peut démontrer que si les valeurs propres sont distinctes, alors les vecteurs propres sont linéairement indépendants. Il en découle le théorème suivant.

Théorème 8.6. *Si la matrice $\mathbf{A}_{n \times n}$ possède n valeurs propres distinctes, alors elle est diagonalisable.*



La contraposée du théorème précédent indique que si une matrice n'est pas diagonalisable, alors ses valeurs propres ne sont pas distinctes. En revanche, la matrice peut être diagonalisable même si ses valeurs propres ne sont pas distinctes.

Exemple 8.7. Les valeurs propres de la matrice

$$\mathbf{A} = \begin{bmatrix} 0 & 0 & -2 \\ 1 & 2 & 1 \\ 1 & 0 & 3 \end{bmatrix}$$

de l'exemple 8.4 sont $\lambda_1 = 1$ et $\lambda_2 = \lambda_3 = 2$. Néanmoins, les trois vecteurs propres forment une base de $\mathbb{R}^3$. Par conséquent,

$$\begin{aligned} \mathbf{P}^{-1}\mathbf{A}\mathbf{P} &= \begin{bmatrix} -1 & 0 & -1 \\ -1 & 0 & -2 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 0 & 0 & -2 \\ 1 & 2 & 1 \\ 1 & 0 & 3 \end{bmatrix} \begin{bmatrix} -2 & 1 & 0 \\ 1 & 0 & 1 \\ 1 & -1 & 0 \end{bmatrix} \\ &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{bmatrix}. \end{aligned}$$

□

Exemple 8.8. Soit

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 2 & 0 \\ -3 & 5 & 2 \end{bmatrix}.$$

Le polynôme caractéristique de $\mathbf{A}$ est

$$\det(\lambda\mathbf{I} - \mathbf{A}) = (\lambda - 1)(\lambda - 2)^2,$$

d'où les valeurs propres sont $\lambda_1 = 1$ et $\lambda_2 = \lambda_3 = 2$. Il est laissé en exercice de vérifier que

- le vecteur propre correspondant à $\lambda = 1$ est $\mathbf{p}_1 = (\frac{1}{8}, -\frac{1}{8}, 1)$;
- le vecteur propre correspondant à $\lambda = 2$ est $\mathbf{p}_2 = (0, 0, 1)$.

Puisque nous avons seulement deux vecteurs propres linéairement indépendants, la matrice $\mathbf{A}$ n'est pas diagonalisable. □

Exemple 8.9. Soit

$$\mathbf{A} = \begin{bmatrix} -1 & 2 & 4 & 0 \\ 0 & 3 & 1 & 7 \\ 0 & 0 & 5 & 8 \\ 0 & 0 & 0 & -2 \end{bmatrix}$$

Nous pouvons facilement établir que :

- la matrice $\mathbf{A}$ est triangulaire;
- ses valeurs propres sont $\lambda_1 = -1$, $\lambda_2 = 3$, $\lambda_3 = 5$ et $\lambda_4 = -2$;
- les valeurs propres sont distinctes;
- la matrice est diagonalisable.

□

Terminons par un résultat qui joue un rôle important en statistique. Certaines matrices sont diagonalisables à l'aide d'une matrice $\mathbf{P}$ orthogonale (définition 7.8), c'est-à-dire que $\mathbf{P}^{-1}\mathbf{A}\mathbf{P} = \mathbf{P}^T\mathbf{A}\mathbf{P}$ est diagonale. On dit d'une telle matrice qu'elle est *orthogonalement diagonalisable*. Le théorème suivant établit que toute matrice orthogonalement diagonalisable est symétrique, et vice versa.

Théorème 8.7. *Soit $\mathbf{A}$ une matrice carrée $n \times n$. Les énoncés suivants sont équivalents.*

1. $\mathbf{A}$ est orthogonalement diagonalisable.
2. $\mathbf{A}$ possède un ensemble orthonormal de n vecteurs propres.
3. $\mathbf{A}$ est symétrique.

Exemple 8.10. Soit la matrice symétrique

$$\mathbf{A} = \begin{bmatrix} 4 & 2 & 2 \\ 2 & 4 & 2 \\ 2 & 2 & 4 \end{bmatrix}.$$

Il est laissé en exercice de vérifier que les valeurs propres de la matrice sont $\lambda_1 = \lambda_2 = 2$ et $\lambda_3 = 8$. Vous pouvez ensuite vérifier que, d'une part,

$$\mathbf{v}_1 = \begin{bmatrix} -1/\sqrt{2} \\ 1/\sqrt{2} \\ 0 \end{bmatrix} \quad \text{et} \quad \mathbf{v}_2 = \begin{bmatrix} -1/\sqrt{6} \\ -1/\sqrt{6} \\ 2/\sqrt{6} \end{bmatrix}$$

forment une base orthonormale correspondant à $\lambda = 2$ et que, d'autre part,

$$\mathbf{v}_3 = \begin{bmatrix} 1/\sqrt{3} \\ 1/\sqrt{3} \\ 1/\sqrt{3} \end{bmatrix}$$

forme une base orthonormale correspondant à $\lambda = 8$. (La conversion d'une base de vecteur propre « standard » en une base orthonormale se fait par la **procédure de Gram-Schmidt** , qui est hors de la portée du présent ouvrage.)

Enfin, il est simple de vérifier que la matrice

$$\mathbf{P} = \begin{bmatrix} -1/\sqrt{2} & -1/\sqrt{6} & 1/\sqrt{3} \\ 1/\sqrt{2} & -1/\sqrt{6} & 1/\sqrt{3} \\ 0 & 2/\sqrt{6} & 1/\sqrt{3} \end{bmatrix},$$

formée à partir des vecteurs propres ci-dessus, diagonalise orthogonalement la matrice $\mathbf{A}$. □



La fonction `eigen` calcule les valeurs et vecteurs propres d'une matrice dans R. Étudiez attentivement les lignes 13-41 du fichier de script `valeurs_propres.R` reproduit à la section 8.10 pour en connaître davantage.



8.7 Indice pour le problème

Le [théorème 8.7](#) nous permet d'avancer de manière importante dans la résolution de notre problème. En effet, la matrice Σ étant symétrique, nous savons désormais que :

- ▶ la matrice diagonale $\text{Var}[\mathbf{Y}] = \mathbf{W}\Sigma\mathbf{W}^T$ existe et elle est formée des valeurs propres de Σ ;
- ▶ la matrice $\mathbf{W}$ est formée des vecteurs propres de Σ sur les lignes ;
- ▶ les vecteurs propres en question sont orthonormaux.

Supposons que $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p \geq 0$ sont les valeurs propres de Σ en ordre décroissant et que $\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_p$ sont les vecteurs propres correspondants. Pour des raisons qui deviendront claires sous peu, nous allons former les matrices $\text{Var}[\mathbf{Y}]$ et $\mathbf{W}$ avec les valeurs propres dans cet ordre :

$$\text{Var}[\mathbf{Y}] = \begin{bmatrix} \lambda_1 & 0 & \dots & 0 \\ 0 & \lambda_2 & \dots & 0 \\ \vdots & \vdots & & \vdots \\ 0 & 0 & \dots & \lambda_p \end{bmatrix} \quad \text{et} \quad \mathbf{W} = \begin{bmatrix} \mathbf{w}_1^T \\ \mathbf{w}_2^T \\ \vdots \\ \mathbf{w}_p^T \end{bmatrix}.$$

Le nouveau vecteur aléatoire

$$\mathbf{Y} = \mathbf{W}\mathbf{X} = \begin{bmatrix} \mathbf{w}_1^T \mathbf{X} \\ \mathbf{w}_2^T \mathbf{X} \\ \vdots \\ \mathbf{w}_p^T \mathbf{X} \end{bmatrix}$$

est appelé le vecteur des *composantes principales* de $\mathbf{X}$.

8.8 Formes quadratiques

Cette section offre une courte introduction aux formes quadratiques afin de présenter un théorème important pour l'analyse en composantes principales et, par conséquent, pour résoudre le problème de ce chapitre. Nous n'irons pas plus loin que le cadre de ce théorème, mais sachez que l'étude

des formes quadratiques se révèle en soi fort intéressante. Par exemple, les propriétés des formes quadratiques permettent de justifier les conditions — que vous connaissez — selon lesquelles une fonction dans $\mathbb{R}^3$ atteint un maximum, un minimum ou un point de selle en un point donné.

Une forme quadratique est une fonction de n variables $x_1, \dots, x_n$ de la forme

$$\sum_{i=1}^n a_{ii}x_i^2 + 2 \sum_{i=1}^{n-1} \sum_{j=i}^n a_{ij}x_i x_j = a_{11}x_1^2 + a_{22}x_2^2 + \dots + a_{nn}x_n^2 \\ + 2a_{12}x_1x_2 + 2a_{13}x_1x_3 + \dots + 2a_{n-1,n}x_{n-1}x_n.$$

Par exemple, une forme quadratique de deux variables x_1 et x_2 est

$$a_{11}x_1^2 + a_{22}x_2^2 + 2a_{12}x_1x_2,$$

alors qu'une forme quadratique de trois variables x_1 , x_2 et x_3 est

$$a_{11}x_1^2 + a_{22}x_2^2 + a_{33}x_3^2 + 2a_{12}x_1x_2 + 2a_{13}x_1x_3 + 2a_{23}x_2x_3.$$

Nous pouvons écrire les deux expressions ci-dessus sous forme de produits matriciels ainsi, dans l'ordre :

$$\begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} a_{11} & a_{12} \\ a_{12} & a_{22} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

et

$$\begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{12} & a_{22} & a_{23} \\ a_{13} & a_{23} & a_{33} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}.$$

Vous remarquerez que ces deux produits sont de la forme $\mathbf{x}^T \mathbf{A} \mathbf{x}$, où $\mathbf{x}$ est le vecteur colonne des variables et $\mathbf{A}$ est la matrice symétrique des coefficients. En fait, il s'agit là de la définition même d'une forme quadratique exprimée de manière matricielle : une fonction de la forme $\mathbf{x}^T \mathbf{A} \mathbf{x}$.



Certains auteurs n'incluent pas le facteur 2 devant les termes croisés dans la définition d'une forme quadratique, mais divisent alors par 2 les coefficients a_{ij} , $i \neq j$, dans la matrice $\mathbf{A}$.

Exemple 8.11. Voici quelques formes quadratiques et leurs représentations matricielles respectives.

$$\begin{aligned}
 2x^2 + 6xy - 7y^2 &= \begin{bmatrix} x & y \end{bmatrix} \begin{bmatrix} 2 & 3 \\ 3 & -7 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \\
 4x^2 - 5y^2 &= \begin{bmatrix} x & y \end{bmatrix} \begin{bmatrix} 4 & 0 \\ 0 & -5 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \\
 xy &= \begin{bmatrix} x & y \end{bmatrix} \begin{bmatrix} 0 & \frac{1}{2} \\ \frac{1}{2} & 0 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \\
 x^2 + 7y^2 - 3z^2 + 4xy - 2xz + 6yz &= \begin{bmatrix} x & y & z \end{bmatrix} \begin{bmatrix} 1 & 2 & -1 \\ 2 & 7 & 3 \\ -1 & 3 & -3 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix}
 \end{aligned}$$

□

Le théorème suivant contient deux résultats importants liant les formes quadratiques aux valeurs et vecteurs propres.

Théorème 8.8. Soit A une matrice symétrique $n \times n$ dont les valeurs propres sont, en ordre décroissant, $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$. Si le vecteur $\mathbf{x}$ est tel que $\|\mathbf{x}\| = 1$ selon le produit scalaire dans $\mathbb{R}^n$, alors :

- a) $\lambda_1 \geq \mathbf{x}^T A \mathbf{x} \geq \lambda_n$;
- b) $\mathbf{x}^T A \mathbf{x} = \lambda_n$ si $\mathbf{x}$ est le vecteur propre de A correspondant à λ_n et $\mathbf{x}^T A \mathbf{x} = \lambda_1$ si $\mathbf{x}$ est le vecteur propre de A correspondant à λ_1 .

Démonstration. Débutons par la partie a). Du [théorème 8.7](#), la matrice symétrique A compte n vecteurs propres qui forment une base orthonormale de $\mathbb{R}^n$. Supposons que l'ensemble $\{\mathbf{v}_1, \dots, \mathbf{v}_n\}$ constitue cette base où, par définition, $\langle \mathbf{v}_i, \mathbf{v}_i \rangle = 1$ et $\langle \mathbf{v}_i, \mathbf{v}_j \rangle = 0$ quand $i \neq j$. Le vecteur $\mathbf{x}$ peut donc s'exprimer comme une combinaison linéaire des vecteurs propres :

$$\mathbf{x} = k_1 \mathbf{v}_1 + \dots + k_n \mathbf{v}_n.$$

De plus, les vecteurs $\mathbf{v}_1, \dots, \mathbf{v}_n$ étant des vecteurs propres de A , nous avons que

$$\begin{aligned}
 A\mathbf{x} &= k_1 A\mathbf{v}_1 + \dots + k_n A\mathbf{v}_n \\
 &= \lambda_1 k_1 \mathbf{v}_1 + \dots + \lambda_n k_n \mathbf{v}_n.
 \end{aligned}$$

Or, $\|\mathbf{x}\|^2 = \langle \mathbf{x}, \mathbf{x} \rangle = k_1^2 \langle \mathbf{v}_1, \mathbf{v}_1 \rangle + \dots + k_n^2 \langle \mathbf{v}_n, \mathbf{v}_n \rangle = k_1^2 + \dots + k_n^2 = 1$ et, par conséquent,

$$\begin{aligned} \mathbf{x}^T \mathbf{A} \mathbf{x} &= \langle \mathbf{x}, \mathbf{A} \mathbf{x} \rangle \\ &= \lambda_1 \langle k_1 \mathbf{v}_1, k_1 \mathbf{v}_1 \rangle + \dots + \lambda_n \langle k_n \mathbf{v}_n, k_n \mathbf{v}_n \rangle \\ &= \lambda_1 k_1^2 + \dots + \lambda_n k_n^2 \\ &\leq \lambda_1 k_1^2 + \dots + \lambda_1 k_n^2 \\ &= \lambda_1 (k_1^2 + \dots + k_n^2) \\ &= \lambda_1. \end{aligned}$$

La démonstration que $\mathbf{x}^T \mathbf{A} \mathbf{x} \geq \lambda_n$ est similaire.

Pour la partie b), si $\mathbf{x}$ est le vecteur propre de $\mathbf{A}$ correspondant à λ_1 et que $\|\mathbf{x}\| = 1$, alors

$$\mathbf{x}^T \mathbf{A} \mathbf{x} = \lambda_1 \mathbf{x}^T \mathbf{x} = \lambda_1.$$

De la même manière, $\mathbf{x}^T \mathbf{A} \mathbf{x} = \lambda_n$ si $\mathbf{x}$ est le vecteur propre de $\mathbf{A}$ correspondant à λ_n . $\square$

Le [théorème 8.8](#) permet d'établir que, sous la contrainte

$$\|\mathbf{x}\| = (x_1^2 + x_2^2 + \dots + x_n^2)^{1/2} = 1,$$

la valeur maximale de la forme quadratique $\mathbf{x}^T \mathbf{A} \mathbf{x}$ est λ_1 (la plus grande valeur propre de $\mathbf{A}$) et la valeur minimale est λ_n (la plus petite valeur propre).

8.9 Solution du problème

Par le [théorème 8.8](#), nous savons maintenant que la composante principale $Y_1 = \mathbf{w}_1^T \mathbf{X}$ est la combinaison linéaire de $\mathbf{X}$ avec la plus grande variance, $Y_2 = \mathbf{w}_2^T \mathbf{X}$ celle avec la seconde plus grande variance, et ainsi de suite. Les premières composantes principales s'avèrent donc les plus intéressantes puisque ce sont celles qui conservent le plus d'information.

Pour réduire la dimension de nos données, il suffit de choisir les $m \ll p$ premières composantes principales. En fait, la k^e composante principale explique une proportion

$$\frac{\lambda_k}{\lambda_1 + \dots + \lambda_p}$$

de la variabilité totale qui se trouve dans le vecteur d'origine $\mathbf{X}$ et l'ensemble des m premières composantes principales en explique une proportion

$$\frac{\sum_{i=1}^m \lambda_i}{\sum_{i=1}^p \lambda_i}.$$

La [figure 8.5](#) présente les 1 797 points de la base de données *Optical Recognition of Handwritten Digits* en tenant compte des deux premières composantes principales seulement. Vous constaterez qu'il serait déjà possible d'établir un système de classification relativement fiable. Par exemple, l'immense majorité des points dans l'ensemble $\{(Y_1, Y_2); -8 \leq Y_1 \leq 10, -30 \leq Y_2 \leq -16\}$ représentent le chiffre 0 et ce chiffre est peu présent en dehors de cet ensemble. C'est pourquoi il était facile de reconnaître le chiffre 0 dans la [figure 8.1](#). Par contre, vous remarquerez que le chiffre 7 s'avère beaucoup plus difficile à distinguer des autres avec seulement deux composantes principales. Eh oui, c'était le chiffre représenté à la [figure 8.2](#)!



Vous pouvez recréer le graphique de la [figure 8.5](#) avec le code des lignes [44-64](#) du fichier de script `valeurs_propres.R`.

8.10 Code informatique

✓ Fichier d'accompagnement `valeurs_propres.R`

```

12 ###
13 ### VALEURS ET VECTEURS PROPRES
14 ###
15
16 ## Nous allons illustrer le calcul des valeurs et vecteurs
17 ## propres dans R avec une matrice utilisée dans les exemples
18 ## du chapitre.
19 (A <- matrix(c(0, 1, 1, 0, 2, 0, -2, 1, 3), nrow = 3))
20
21 ## La fonction 'eigen' calcule les valeurs propres et vecteurs
22 ## propres d'une matrice.
23 (e <- eigen(A))
24
25 ## Les vecteurs propres sont normalisés de sorte que leur
26 ## norme (longueur) soit toujours égale à 1. Pour vérifier les
27 ## résultats calculés algébriquement, comparer simplement les
28 ## valeurs relatives des coordonnées des vecteurs.
29 e$values[c(1, 2)]           # deux premières valeurs propres...
30 e$vectors[, c(1, 2)]        # ... et vecteurs correspondants
31 e$vectors[, 2]              # équivalent à (1, 0, -1)
32 e$vectors[, 2] * sqrt(2)    # avec norme de 2 plutôt que 1
33
34 e$values[3]                 # troisième valeur propre
35 e$vectors[, 3]              # vecteur équivalent à (-2, 1, 1)
36 e$vectors[, 3] * sqrt(6)    # avec norme de 6 plutôt que 1

```

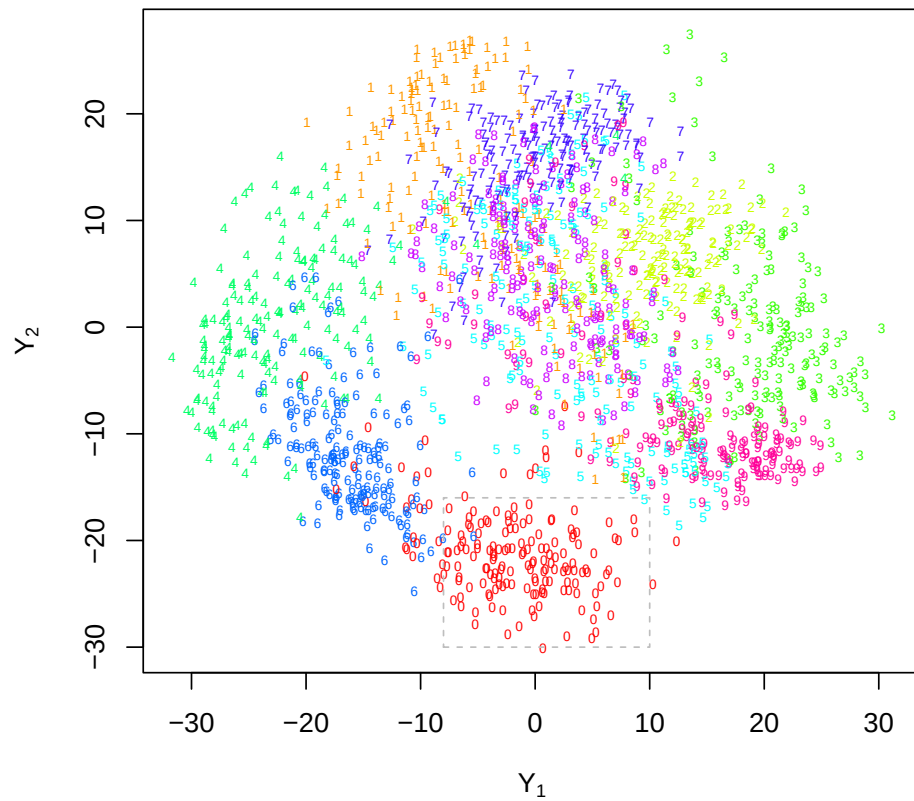


FIG. 8.5 – Représentation des deux premières composantes principales des données *Optical Recognition of Handwritten Digits*. Le rectangle pointillé représente l'ensemble $\{(Y_1, Y_2); -8 \leq Y_1 \leq 10, -30 \leq Y_2 \leq -16\}$

```

37
38 ## Si la matrice est symétrique, les vecteurs propres sont
39 ## orthonormaux.
40 (A <- matrix(c(4, 2, 2, 2, 4, 2, 2, 2, 4), nrow = 3))
41 crossprod(eigen(A)$vectors) # matrice identité
42
43 ###
44 ### ANALYSE EN COMPOSANTES PRINCIPALES
45 ###
46
47 ## Installation du paquetage RSKC pour obtenir les données
48 ## "Optical Recognition of Handwritten Digits". Exécuter une
49 ## seule fois!
50 #install.packages("RSKC")
51
52 ## Chargement du jeu de données 'optd' sans charger tout le
53 ## paquetage dans la session de travail.
54 data(optd, package = "RSKC")
55
56 ## Création du graphique des deux composantes principales.
57 optd.acp <- prcomp(optd, center = TRUE, scale. = FALSE)
58 optd.coord <- predict(optd.acp, newdata = optd)
59 plot(x = optd.coord[, 1], y = optd.coord[, 2], type = "n",
60      xlab = expression(Y[1]), ylab = expression(Y[2]))
61 text(x = optd.coord[, 1], y = optd.coord[, 2],
62      labels = rownames(optd), cex = 0.5,
63      col = rainbow(10)[as.numeric(rownames(optd)) + 1])
64 rect(-8, -30, 10, -16, lty = 2, border = "gray")

```

8.11 Exercices

8.1 Trouver l'équation caractéristique, les valeurs propres et les bases de vecteurs propres des matrices suivantes. Vérifier les réponses obtenues à l'aide de la fonction `eigen` de R.

a) $\begin{bmatrix} 3 & 0 \\ 8 & -1 \end{bmatrix}$

b) $\begin{bmatrix} 10 & -9 \\ 4 & -2 \end{bmatrix}$

c) $\begin{bmatrix} 4 & 0 & 1 \\ -2 & 1 & 0 \\ -2 & 0 & 1 \end{bmatrix}$

d) $\begin{bmatrix} 5 & 6 & 2 \\ 0 & -1 & -8 \\ 1 & 0 & -2 \end{bmatrix}$

$$\text{e) } \begin{bmatrix} 0 & 0 & 2 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & -2 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

8.2 Démontrer par induction que si λ est une valeur propre de la matrice A , alors λ^k est une valeur propre de A^k .

8.3 Trouver les valeurs et vecteurs propres de A^{25} si

$$A = \begin{bmatrix} -1 & -2 & -2 \\ 1 & 2 & 1 \\ -1 & -1 & 0 \end{bmatrix}.$$

8.4 Soit

$$(x - x_1)(x - x_2) \cdots (x - x_n) = x^n + c_1 x^{n-1} + \cdots + c_n.$$

Démontrer par induction les identités suivantes.

a) $c_1 = -\sum_{i=1}^n x_i$

b) $c_n = (-1)^n \prod_{i=1}^n x_i$

8.5 Démontrer que l'équation caractéristique d'une matrice $A_{2 \times 2}$ est

$$\lambda^2 - \text{tr}(A)\lambda + \det(A) = 0.$$

8.6 Démontrer que si λ est une valeur propre de la matrice inversible A et que $\mathbf{x}$ est un vecteur propre correspondant, alors λ^{-1} est une valeur propre de A^{-1} et $\mathbf{x}$ est un vecteur propre correspondant.

8.7 Trouver les valeurs propres et les bases de vecteurs propres de A^{-1} , où

$$A = \begin{bmatrix} -2 & 2 & 3 \\ -2 & 3 & 2 \\ -4 & 2 & 5 \end{bmatrix}$$

8.8 Démontrer que tout vecteur est un vecteur propre de la matrice identité correspondant à la valeur propre $\lambda = 1$.

8.9 Pour chacune des matrices A ci-dessous :

- i) trouver les valeurs propres de la matrice;
- ii) trouver le rang de la matrice $\lambda I - A$ pour chaque valeur propre λ ;
- iii) déterminer si la matrice est diagonalisable;
- iv) si la matrice est diagonalisable, trouver la matrice P qui diagonalise A et $P^{-1}AP$;
- v) vérifier les réponses en iv) avec la fonction `eigen` de R.

a) $\begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix}$

b) $\begin{bmatrix} 4 & 0 & 1 \\ 2 & 3 & 2 \\ 1 & 0 & 4 \end{bmatrix}$

c) $\begin{bmatrix} 3 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 1 & 2 \end{bmatrix}$

d) $\begin{bmatrix} -1 & 4 & -2 \\ -3 & 4 & 0 \\ -3 & 1 & 3 \end{bmatrix}$

e) $\begin{bmatrix} -2 & 0 & 0 & 0 \\ 0 & -2 & 5 & -5 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 3 \end{bmatrix}$

Réponses

8.1 a) $\lambda = 3$ avec base $(\frac{1}{2}, 1)$, $\lambda = -1$ avec base $(0, 1)$

b) $\lambda = 4$ avec base $(\frac{3}{2}, 1)$

c) $\lambda = 1$ avec base $(0, 1, 0)$, $\lambda = 2$ avec base $(-\frac{1}{2}, 1, 1)$, $\lambda = 3$ avec base $(-1, 1, 1)$

d) $\lambda = -4$ avec base $(-2, \frac{8}{3}, 1)$, $\lambda = 3$ avec base $(5, -2, 1)$

e) $\lambda = 1$ avec base $(0, 0, 0, 1)$ et $(2, 3, 1, 0)$, $\lambda = -2$ avec base $(-1, 0, 1, 0)$, $\lambda = -1$ avec base $(-2, 1, 1, 0)$

8.3 $\lambda = 1$ avec base $(-1, 1, 0)$ et $(-1, 0, 1)$, $\lambda = -1$ avec base $(2, -1, 1)$

8.7 $\lambda = 1$ avec base $(1, 0, 1)$, $\lambda = \frac{1}{2}$ avec base $(\frac{1}{2}, 1, 0)$, $\lambda = \frac{1}{3}$ avec base $(1, 1, 1)$

8.9 a) pas diagonalisable

b) $\mathbf{P} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \mathbf{P}^{-1}\mathbf{A}\mathbf{P} = \begin{bmatrix} 3 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 5 \end{bmatrix}$

c) pas diagonalisable

d) $\mathbf{P} = \begin{bmatrix} 1 & 2 & 1 \\ 1 & 3 & 3 \\ 1 & 3 & 4 \end{bmatrix}, \mathbf{P}^{-1}\mathbf{A}\mathbf{P} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{bmatrix}$

e) $\mathbf{P} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & -1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \mathbf{P}^{-1}\mathbf{A}\mathbf{P} = \begin{bmatrix} -2 & 0 & 0 & 0 \\ 0 & -2 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 3 \end{bmatrix}$

9 Méthodes de résolution de systèmes d'équations linéaires

Objectifs du chapitre

- ▶ Comparer la rapidité des méthodes usuelles de résolutions des systèmes d'équations linéaires.
- ▶ Calculer la solution d'un système d'équations linéaires par la décomposition LU .

L'utilisation des matrices de loin la plus fréquente en actuariat consiste à résoudre des systèmes d'équations linéaires du type

$$\mathbf{Ax} = \mathbf{b}.$$

Nous avons jusqu'à maintenant étudié diverses façons — toutes équivalentes d'un point de vue mathématique — d'obtenir la solution

$$\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}.$$

Nous pouvons maintenant nous demander laquelle est la plus efficace d'un point de vue informatique, surtout lorsque le système compte un grand nombre d'équations.

9.1 Comparaison du nombre d'opérations

Les principales méthodes de résolution d'un système d'équations linéaires sont :

1. l'élimination gaussienne avec substitution successive ;

Méthode	Nombre d'opérations
Élimination gaussienne	$n^3/3$
Élimination de Gauss-Jordan	$n^3/3$
Transformation de $[A I]$ en $[I A^{-1}]$	n^3
Calcul de $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$	n^3

TAB. 9.1 – Nombre approximatif de multiplications et divisions pour résoudre le système d'équations à n équations et n inconnues $\mathbf{Ax} = \mathbf{b}$

2. l'élimination de Gauss-Jordan ;
3. le calcul de $\mathbf{A}^{-1}$, puis de $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$.

Le calcul de $\mathbf{A}^{-1}$ peut quant à lui s'effectuer par transformation de $[A|I]$ en $[I|A^{-1}]$, ou par la méthode des cofacteurs.

L'élément décisif dans la comparaison des temps de calcul de ces diverses méthodes de calcul est le nombre d'opérations arithmétiques requis. Le [tableau 9.1](#) présente l'ordre de grandeur (et non le nombre exact) du nombre de multiplications et de divisions nécessaires pour obtenir une réponse avec chacune des méthodes ci-dessus. Concentrons-nous sur les multiplications et divisions, sachant que ces opérations coûtent plus cher en temps de calcul que les additions et les soustractions. Nous constatons que l'élimination gaussienne avec substitution successive et l'élimination de Gauss-Jordan sont les méthodes les plus rapides, leur avantage augmentant rapidement avec la taille du système d'équations.

9.2 Décomposition LU

Le principal inconvénient des méthodes d'élimination réside dans le fait qu'il faut connaître le vecteur des coefficients $\mathbf{b}$ au moment d'effectuer les calculs. Si l'on souhaite résoudre le système $\mathbf{Ax} = \mathbf{b}$ pour un nouveau vecteur $\mathbf{b}$, il faut répéter la procédure depuis le début.

Les principales routines de résolution de systèmes d'équations linéaires disponibles dans les divers outils informatiques (R, Maple, Matlab, Mathematica, etc.) reposent donc plutôt sur la technique de la décomposition LU , une variante de l'élimination gaussienne ne nécessitant pas de connaître d'avance le vecteur $\mathbf{b}$.

L'idée est très simple : si la matrice $\mathbf{A}$ peut être factorisée en un produit de matrices $n \times n$

$$\mathbf{A} = \mathbf{LU},$$

où $\mathbf{L}$ est une matrice triangulaire inférieure et $\mathbf{U}$ est une matrice triangulaire supérieure, alors nous obtenons le système d'équations linéaires

$$\mathbf{L}\mathbf{U}\mathbf{x} = \mathbf{b}. \quad (9.1)$$

Or, en posant

$$\mathbf{U}\mathbf{x} = \mathbf{y} \quad (9.2)$$

il est possible de réécrire (9.1) comme

$$\mathbf{L}\mathbf{y} = \mathbf{b}. \quad (9.3)$$

La solution $\mathbf{y}$ de ce dernier système d'équations est simple à obtenir par substitutions successives puisque $\mathbf{L}$ est triangulaire. De même, une fois $\mathbf{y}$ connu, le vecteur $\mathbf{x}$ est obtenu en résolvant (9.2), toujours par simples substitutions.

Exemple 9.1. Soit le système d'équations linéaires

$$\begin{bmatrix} 2 & 6 & 2 \\ -3 & -8 & 0 \\ 4 & 9 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \\ 3 \end{bmatrix}.$$

On peut démontrer que

$$\mathbf{A} = \begin{bmatrix} 2 & 6 & 2 \\ -3 & -8 & 0 \\ 4 & 9 & 2 \end{bmatrix} = \begin{bmatrix} 2 & 0 & 0 \\ -3 & 1 & 0 \\ 4 & -3 & 7 \end{bmatrix} \begin{bmatrix} 1 & 3 & 1 \\ 0 & 1 & 3 \\ 0 & 0 & 1 \end{bmatrix},$$

soit $\mathbf{A} = \mathbf{L}\mathbf{U}$ avec

$$\mathbf{L} = \begin{bmatrix} 2 & 0 & 0 \\ -3 & 1 & 0 \\ 4 & -3 & 7 \end{bmatrix}$$

et

$$\mathbf{U} = \begin{bmatrix} 1 & 3 & 1 \\ 0 & 1 & 3 \\ 0 & 0 & 1 \end{bmatrix}.$$

Nous avons donc $\mathbf{A}\mathbf{x} = \mathbf{L}\mathbf{U}\mathbf{x} = \mathbf{b}$. En posant $\mathbf{U}\mathbf{x} = \mathbf{y}$, le système d'équations peut se réécrire sous la forme $\mathbf{L}\mathbf{y} = \mathbf{b}$, soit

$$\begin{bmatrix} 2 & 0 & 0 \\ -3 & 1 & 0 \\ 4 & -3 & 7 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \\ 3 \end{bmatrix}.$$

Par substitution successive,

$$\begin{aligned}y_1 &= 1 \\y_2 &= 2 + 3y_1 = 5 \\y_3 &= \frac{3 - 4y_1 + 3y_2}{7} = 2.\end{aligned}$$

Pour trouver la solution du système d'équations original, il suffit maintenant de résoudre $\mathbf{U}\mathbf{x} = \mathbf{y}$, soit

$$\begin{bmatrix} 1 & 3 & 1 \\ 0 & 1 & 3 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 5 \\ 2 \end{bmatrix}.$$

Nous obtenons alors

$$\begin{aligned}x_3 &= 2 \\x_2 &= 5 - 3x_3 = -1 \\x_1 &= 1 - 3x_2 - x_3 = 2.\end{aligned}$$

□

L'essentiel des calculs dans la décomposition LU se trouve dans la factorisation de la matrice $\mathbf{A}$ en un produit de matrices triangulaires. Pour justifier la technique, supposons que l'on réduit la matrice $\mathbf{A}$ sous forme échelonnée par une série d'opérations élémentaires sur les lignes. Cela signifie que l'on peut trouver des matrices élémentaires $\mathbf{E}_1, \dots, \mathbf{E}_k$ tel que

$$\mathbf{E}_k \cdots \mathbf{E}_1 \mathbf{A} = \mathbf{U}.$$

Par le [théorème 7.7](#), l'inverse d'une matrice élémentaire existe et est également une matrice élémentaire, d'où

$$\mathbf{A} = \mathbf{E}_1^{-1} \cdots \mathbf{E}_k^{-1} \mathbf{U}$$

et donc

$$\mathbf{L} = \mathbf{E}_1^{-1} \cdots \mathbf{E}_k^{-1}.$$

Cette dernière matrice est triangulaire inférieure *à condition de ne pas échanger des lignes* lors de la réduction de $\mathbf{A}$ vers $\mathbf{U}$. Il existe un algorithme simple pour construire la matrice $\mathbf{L}$ sans devoir effectuer le produit des matrices élémentaires inverses; consultez [Anton \(2000, section 9.9\)](#).

Exemple 9.2. Reprenons la matrice

$$\mathbf{A} = \begin{bmatrix} 2 & 6 & 2 \\ -3 & -8 & 0 \\ 4 & 9 & 2 \end{bmatrix}$$

de l'exemple 9.1. Il est laissé en exercice de vérifier que les opérations élémentaires suivantes permettent de réduire la matrice $\mathbf{A}$ sous la forme

$$\mathbf{U} = \begin{bmatrix} 1 & 3 & 1 \\ 0 & 1 & 3 \\ 0 & 0 & 1 \end{bmatrix}.$$

1. Multiplier la ligne 1 par $\frac{1}{2}$.
2. Additionner 3 fois la ligne 1 à la ligne 2 et -4 fois la ligne 1 à la ligne 3.
3. Additionner 3 fois la ligne 2 à la ligne 3.
4. Multiplier la ligne 3 par $\frac{1}{7}$.

Les matrices élémentaires correspondant à ces opérations sont, dans l'ordre :

$$\begin{aligned} \mathbf{E}_1 &= \begin{bmatrix} \frac{1}{2} & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} & \mathbf{E}_2 &= \begin{bmatrix} 1 & 0 & 0 \\ 3 & 1 & 0 \\ -4 & 0 & 1 \end{bmatrix} \\ \mathbf{E}_3 &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 3 & 1 \end{bmatrix} & \mathbf{E}_4 &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & \frac{1}{7} \end{bmatrix}. \end{aligned}$$

Vous pouvez vérifier que $\mathbf{E}_4\mathbf{E}_3\mathbf{E}_2\mathbf{E}_1\mathbf{A} = \mathbf{U}$. Nous avons donc $\mathbf{A} = \mathbf{E}_1^{-1}\mathbf{E}_2^{-1}\mathbf{E}_3^{-1}\mathbf{E}_4^{-1}\mathbf{U} = \mathbf{LU}$ avec $\mathbf{L} = \mathbf{E}_1^{-1}\mathbf{E}_2^{-1}\mathbf{E}_3^{-1}\mathbf{E}_4^{-1}$. Or, les matrices élémentaires inverses sont :

$$\begin{aligned} \mathbf{E}_1^{-1} &= \begin{bmatrix} 2 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} & \mathbf{E}_2^{-1} &= \begin{bmatrix} 1 & 0 & 0 \\ -3 & 1 & 0 \\ 4 & 0 & 1 \end{bmatrix} \\ \mathbf{E}_3^{-1} &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -3 & 1 \end{bmatrix} & \mathbf{E}_4^{-1} &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 7 \end{bmatrix}, \end{aligned}$$

d'où

$$\mathbf{L} = \mathbf{E}_1^{-1}\mathbf{E}_2^{-1}\mathbf{E}_3^{-1}\mathbf{E}_4^{-1} = \begin{bmatrix} 2 & 0 & 0 \\ -3 & 1 & 0 \\ 4 & -3 & 7 \end{bmatrix}.$$

□

Le nombre d'opérations de la décomposition LU est du même ordre que les méthodes d'élimination. Par contre, on remarquera que la factorisation est tout à fait indépendante du vecteur $\mathbf{b}$. Une fois la factorisation connue, on peut donc résoudre plusieurs systèmes d'équations différents utilisant tous la même matrice de coefficients $\mathbf{A}$ sans devoir répéter une grande partie des calculs.



Plusieurs progiciels mathématiques recourent à LAPACK¹ pour les calculs d'algèbre linéaire numérique. Cette bibliothèque est elle-même basée sur LINPACK, dont les origines remontent aux années 1970. La bibliothèque est écrite en Fortran, un autre très ancien langage de programmation, toujours utilisé pour le calcul scientifique. D'ailleurs, au moins une particularité de ce langage demeure visible en R. En effet, c'est du Fortran que vient cette habitude de remplir les matrices par colonne.

9.3 Exercices

9.1 Résoudre le système d'équations

$$\begin{aligned} 3x_1 - 6x_2 - 3x_3 &= -3 \\ 2x_1 &+ 6x_3 = -22 \\ -4x_1 + 7x_2 + 4x_3 &= 3 \end{aligned}$$

par la décomposition LU sachant que

$$\begin{bmatrix} 3 & -6 & -3 \\ 2 & 0 & 6 \\ -4 & 7 & 4 \end{bmatrix} = \begin{bmatrix} 3 & 0 & 0 \\ 2 & 4 & 0 \\ -4 & -1 & 2 \end{bmatrix} \begin{bmatrix} 1 & -2 & -1 \\ 0 & 1 & 2 \\ 0 & 0 & 1 \end{bmatrix}.$$

9.2 Soit

$$\mathbf{E}_1 = \begin{bmatrix} 1 & 0 \\ 2 & 3 \end{bmatrix}$$

$$\mathbf{E}_2 = \begin{bmatrix} \frac{1}{3} & 0 \\ 0 & \frac{1}{3} \end{bmatrix}$$

et

$$\mathbf{A} = \mathbf{E}_1^{-1} \mathbf{E}_2^{-1} \begin{bmatrix} 1 & -2 \\ 0 & 1 \end{bmatrix}.$$

Résoudre le système d'équations

$$\mathbf{Ax} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

par la décomposition LU .

Réponses

9.1 $x_1 = -2, x_2 = 1, x_3 = -3$

9.2 $\mathbf{x} = (2, 1)$

A Planification d'une simulation avec R

Il existe de multiples façons de réaliser la mise en œuvre informatique d'une simulation, mais certaines sont plus efficaces que d'autres. Cette annexe passe en revue diverses façons de faire des simulations avec R à l'aide d'un exemple simple de nature statistique.

A.1 Contexte

Soit $X_1, \dots, X_n$ un échantillon aléatoire tiré d'une population distribuée selon une loi uniforme sur l'intervalle $(\theta - \frac{1}{2}, \theta + \frac{1}{2})$ et trois estimateurs sans biais du paramètre inconnu θ :

1. la moyenne arithmétique

$$\hat{\theta}_1 = \frac{1}{n} \sum_{i=1}^n X_i;$$

2. la médiane empirique

$$\hat{\theta}_2 = \begin{cases} X_{(\frac{n+1}{2})}, & n \text{ impair} \\ \frac{1}{2}(X_{(\frac{n}{2})} + X_{(\frac{n}{2}+1)}), & n \text{ pair}, \end{cases}$$

où $X_{(k)}$ est la k^e statistique d'ordre de l'échantillon aléatoire;

3. la mi-étendue

$$\hat{\theta}_3 = \frac{X_{(1)} + X_{(n)}}{2}.$$

À l'aide de la simulation, nous voulons, d'une part, vérifier si les trois estimateurs sont bel et bien sans biais et, d'autre part, déterminer lequel a la plus faible variance.

Pour ce faire, nous devons d'abord simuler un grand nombre N d'échantillons aléatoires de taille n d'une distribution $U(\theta - \frac{1}{2}, \theta + \frac{1}{2})$ pour une

valeur de θ choisie. Pour chaque échantillon, nous calculerons ensuite les trois estimateurs ci-dessus, puis la moyenne et la variance, par type d'estimateur, de tous les estimateurs obtenus. Si la moyenne des N estimateurs $\hat{\theta}_i$, $i = 1, 2, 3$ est près de θ , alors nous pourrions conclure que $\hat{\theta}_i$ est sans biais. De même, nous déterminerons lequel des trois estimateurs a la plus faible variance selon le classement des variances empiriques.

A.2 Première approche : avec une boucle

La façon la plus intuitive de mettre en œuvre cette étude de simulation en R consiste à utiliser une boucle `for`. Avec cette approche, il est nécessaire d'initialiser une matrice de 3 lignes et N colonnes (ou l'inverse) dans laquelle seront stockées les valeurs des trois estimateurs pour chaque simulation. Une fois la matrice remplie dans la boucle, il ne reste plus qu'à calculer la moyenne et la variance par ligne pour obtenir les résultats souhaités.

La [figure A.1](#) présente un exemple de code adéquat pour réaliser la simulation à l'aide d'une boucle.

Si l'on souhaite pouvoir exécuter le code de la [figure A.1](#) facilement à l'aide d'une seule expression, il suffit de placer l'ensemble du code dans une fonction. La fonction `simul1` de la [figure A.2](#) reprend le code de la [figure A.1](#), sans les commentaires.

```
> simul1(10000, 100, 0)
$biais
      Moyenne      Mediane  Mi-etendue
-9.838816e-06  2.921451e-04  1.350118e-04

$variances
      Moyenne      Mediane  Mi-etendue
8.518037e-04  2.473287e-03  4.800155e-05
```

A.3 Seconde approche : avec `sapply`

Dans R, il est plus élégant — et parfois plus efficace — de masquer les calculs itératifs avec les fonctions d'application `lapply` et `sapply` ([Goulet, 2020](#), chapitre 5). Rappelons que la syntaxe de ces fonctions est :

```
lapply(x, FUN, ...)
sapply(x, FUN, ...)
```



```
## Bonne habitude à prendre: stocker les constantes dans
## des variables faciles à modifier au lieu de les écrire
## explicitement dans le code.
size <- 100                # taille de chaque échantillon
nsimul <- 10000            # nombre de simulations
theta <- 0                 # la valeur du paramètre

## Les lignes ci-dessous éviteront de faire deux additions
## 'nsimul' fois.
a <- theta - 0.5           # borne inférieure de l'uniforme
b <- theta + 0.5           # borne supérieure de l'uniforme

## Initialisation de la matrice dans laquelle seront
## stockées les valeurs des estimateurs. Les noms donnés
## aux lignes de la matrice permettent de facilement
## identifier les estimateurs.
x <- matrix(0, nrow = 3, ncol = nsimul)
rownames(x) <- c("Moyenne", "Mediane", "Mi-etendue")

## Simulation comme telle.
for (i in 1:nsimul)
{
  u <- runif(size, a, b)
  x[, i] <- c(mean(u),      # moyenne
             median(u)     # médiane
             mean(range(u))) # mi-étendue
}

## Nous pouvons maintenant calculer la moyenne et la
## variance par ligne.
rowMeans(x) - theta        # vérification du biais
apply(x, 1, var)           # comparaison des variances
```

FIG. A.1 - Code pour la simulation utilisant une boucle for

```

simul1 <- function(nsimul, size, theta)
{
  a <- theta - 0.5
  b <- theta + 0.5

  x <- matrix(0, nrow = 3, ncol = nsimul)
  rownames(x) <- c("Moyenne", "Mediane", "Mi-etendue")

  for (i in 1:nsimul)
  {
    u <- runif(size, a, b)
    x[, i] <- c(mean(u), median(u), mean(range(u)))
  }

  list(biais = rowMeans(x) - theta,
       variances = apply(x, 1, var))
}

```

FIG. A.2 - Définition de la fonction simul1

Les fonctions appliquent la fonction FUN à tous les éléments de la liste ou du vecteur x et retournent les résultats sous forme de liste (`lapply`) ou, lorsque c'est possible, de vecteur ou de matrice (`sapply`). Il est important de noter que les valeurs successives de x seront passées comme *premier* argument à la fonction FUN. Le cas échéant, les autres arguments de FUN sont spécifiés dans le champ '...'.

Pour pouvoir utiliser ces fonctions dans le cadre d'une simulation comme celle dont il est question ici, il s'agit de définir une fonction qui fera tous les calculs pour une simulation, puis de la passer à `sapply` pour obtenir les résultats de N simulations. La [figure A.3](#) présente une première version d'une telle fonction. Vous remarquerez que l'argument i ne joue aucun rôle dans la fonction. L'expression ci-dessous fournit un exemple d'utilisation pour un petit (4) nombre de simulations.

```

> sapply(1:4, fun1, size = 10, a = -0.5, b = 0.5)
      [,1]      [,2]      [,3]
Moyenne -0.01007878 0.11501305 0.009009255
Mediane -0.03463497 0.17977776 -0.039299407
Mi-etendue -0.02082610 0.09114235 -0.027448889

```

```

fun1 <- function(i, size, a, b)
{
  u <- runif(size, a, b)
  c(Moyenne = mean(u),
    Mediane = median(u),
    "Mi-etendue" = mean(range(u)))
}

simul2 <- function(nsimul, size, theta)
{
  a <- theta - 0.5
  b <- theta + 0.5

  x <- sapply(1:nsimul, fun1, size, a, b)

  list(biais = rowMeans(x) - theta,
       variances = apply(x, 1, var))
}

```

FIG. A.3 - Définitions des fonction `fun1` et `simul2`

```

      [,4]
Moyenne  0.04510892
Mediane  0.13849000
Mi-etendue -0.01938738

```

Vous remarquerez que les résultats de chaque simulation se trouvent dans les colonnes de la matrice obtenue avec `sapply`.

Pour compléter l'analyse, nous englobons le tout dans une fonction `simul2`, dont le code se trouve à la [figure A.3](#).

```

> simul2(10000, 100, 0)
$biais
      Moyenne      Mediane  Mi-etendue
3.996721e-04 5.920967e-04 8.241174e-06

$variances
      Moyenne      Mediane  Mi-etendue
8.274470e-04 2.422062e-03 4.820046e-05

```

Il est généralement plus facile de déboguer le code avec cette approche puisque nous pouvons rapidement circonscrire un éventuel problème à `fun1` ou `simul2`.

A.4 Variante de la seconde approche

Une chose manque d'élégance dans la seconde approche : l'obligation d'inclure un argument factice dans la fonction `fun1`. La fonction `replicate` permet toutefois de passer outre cette contrainte. En effet, cette fonction exécute un nombre donné de fois une expression quelconque.

Les fonctions `fun2` et `simul3` de la [figure A.4](#) sont des versions légèrement modifiées de `fun1` et `simul2` pour utilisation avec `replicate`.

```
> simul3(10000, 100, 0)
$biais
      Moyenne      Mediane  Mi-etendue
1.103661e-04  5.702932e-04 -7.249573e-05

$variances
      Moyenne      Mediane  Mi-etendue
8.562141e-04  2.461136e-03  4.834617e-05
```

A.5 Gestion des fichiers

Pour un petit projet comme celui utilisé en exemple ici, il est simple et pratique de placer tout le code informatique dans un seul fichier de script. Pour un plus gros projet, cependant, il vaut souvent mieux avoir recours à plusieurs fichiers différents. Le présent auteur utilise pour sa part un fichier par fonction.

À des fins d'illustration, supposons que nous utilisons l'approche de la [section A.4](#) avec la fonction `replicate` et que le code des fonctions `fun2` et `simul3` est sauvegardé dans des fichiers `fun2.R` et `simul3.R`, dans l'ordre. Si nous créons un autre fichier, disons `go.R`, ne contenant que des expressions `source` pour lire les autres fichiers, il est alors possible de démarrer des simulations en exécutant ce seul fichier. Dans notre exemple, le fichier `go.R` contiendrait les lignes suivantes :

```
fun2 <- function(size, a, b)
{
  u <- runif(size, a, b)
  c(Moyenne = mean(u),
    Mediane = median(u),
    "Mi-etendue" = mean(range(u)))
}

simul3 <- function(nsimul, size, theta)
{
  a <- theta - 0.5
  b <- theta + 0.5

  x <- replicate(nsimul, fun2(size, a, b))

  list(biais = rowMeans(x) - theta,
       variances = apply(x, 1, var))
}
```

FIG. A.4 - Définitions des fonction fun2 et simul3

```
source("fun2.R")
source("simul3.R")
simul3(10000, 100, 0)
```

La simple commande ci-dessous exécutera alors une simulation complète.

```
> source("go.R")
```

A.6 Exécution en lot

Les utilisateurs plus avancés pourront vouloir exécuter leur simulation R en lot (*batch*) pour en accélérer le traitement. Dans ce mode, aucune interface graphique n'est démarrée et tous les résultats sont redirigés vers un fichier pour consultation ultérieure. Pour les simulations demandant un long temps de calcul, c'est très pratique.

On exécute R en lot depuis la ligne de commande (Invite de commande sous Windows, Terminal sous macOS ou Linux). Une fois placé dans le ré-

pertoire contenant les fichiers de script, il suffit d'entrer à la ligne de commande :

```
$ R CMD BATCH go.R
```

La sortie de cette commande (et donc tous les résultats des expressions R du fichier `go.R`) sera placée par défaut dans le fichier `go.Rout`. Sous Windows, le dossier d'installation de R peut ne pas se trouver dans la variable d'environnement `%PATH%`, auquel cas il faut spécifier le chemin d'accès complet du programme à la ligne de commande :

```
$ "c:\Program Files\R\R-(x.y.z)\bin\R" CMD BATCH go.R
```

A.7 Conclusion

Le nombre de simulations, N , et la taille de l'échantillon, n , ont tous deux un impact sur la qualité des résultats, mais de manière différente. Quand n augmente, la précision des estimateurs augmente. Ainsi, dans l'exemple ci-dessus, le biais et la variance des estimateurs de θ seront plus faibles. D'autre part, l'augmentation du nombre de simulations diminue l'impact des échantillons aléatoires individuels et, de ce fait, améliore la fiabilité des conclusions de l'étude.

D'ailleurs, les conclusions de l'étude de simulation sur le biais et la variance des trois estimateurs de la moyenne d'une loi uniforme sont les suivantes : les trois estimateurs sont sans biais et la mi-étendue a la plus faible variance. En effet, on peut démontrer mathématiquement que, pour n impair,

$$\begin{aligned}\text{Var}[\hat{\theta}_1] &= \frac{1}{12n} \\ \text{Var}[\hat{\theta}_2] &= \frac{1}{4n+2} \\ \text{Var}[\hat{\theta}_3] &= \frac{1}{2(n+1)(n+2)}\end{aligned}$$

et donc

$$\text{Var}[\hat{\theta}_3] \leq \text{Var}[\hat{\theta}_1] \leq \text{Var}[\hat{\theta}_2]$$

pour tout $n \geq 2$.

B Transformations de variables aléatoires

Cette annexe porte sur les transformations, ou fonctions, de variables aléatoires. En termes mathématiques, étant donné la distribution conjointe des variables aléatoires $X_1, \dots, X_n$, nous cherchons à déterminer la fonction de probabilité ou de densité de la variable aléatoire $Y = u(X_1, \dots, X_n)$.

Voici quelques exemples de transformations fréquemment rencontrés :

$$Y = X^2$$

$$Y = \frac{X - E[X]}{\sqrt{\text{Var}[X]}}$$

$$Y = \frac{X_1 + \dots + X_n}{n}$$

$$Y = F_X(X).$$

Il existe trois techniques principales pour déterminer la distribution de la transformation Y :

1. la technique de la fonction de répartition;
2. la technique du changement de variable;
3. la technique de la fonction génératrice des moments.

B.1 Technique de la fonction de répartition

C'est la technique la plus simple, mais pas toujours la plus facile d'emploi. En effet, la fonction de répartition de certaines lois de probabilité est compliquée, voire n'existe pas sous forme explicite (penser ici aux lois normale et gamma, par exemple).

L'idée consiste simplement à calculer la fonction de répartition de la transformation avec

$$\begin{aligned} F_Y(y) &= \Pr[Y \leq y] \\ &= \Pr[u(X_1, \dots, X_n) \leq y], \end{aligned}$$

puis à calculer la densité (ou la fonction de probabilité) par différenciation :

$$f_Y(y) = F'_Y(y).$$



Le domaine de définition de la transformation n'est pas nécessairement le même que celui des variables aléatoires de départ.

Exemple B.1. Soit

$$f_X(x) = \begin{cases} 6x(1-x), & 0 < x < 1 \\ 0, & \text{ailleurs,} \end{cases}$$

c'est-à-dire $X \sim \text{Bêta}(2, 2)$. Déterminons la densité de $Y = X^3$. Nous avons :

$$\begin{aligned} F_Y(y) &= \Pr[X^3 \leq y] \\ &= \Pr[X \leq y^{1/3}] \\ &= F_X(y^{1/3}) \\ &= \int_0^{y^{1/3}} 6x(1-x) dx \\ &= 3y^{2/3} - 2y, \quad 0 < y < 1, \end{aligned}$$

et donc

$$\begin{aligned} f_Y(y) &= F'_Y(y) \\ &= \begin{cases} 2y^{-1/3} - 2, & 0 < y < 1 \\ 0, & \text{ailleurs.} \end{cases} \end{aligned}$$

□

Exemple B.2. Soit X une variable aléatoire continue quelconque avec fonction de répartition $F_X(x)$ et $Y = aX + b$, où a et b sont des constantes réelles. Nous avons :

$$\begin{aligned} F_Y(y) &= \Pr[aX + b \leq y] \\ &= F_X\left(\frac{y-b}{a}\right) \end{aligned}$$

et, par conséquent,

$$f_Y(y) = \frac{1}{a} f_X\left(\frac{y-b}{a}\right).$$

La transformation $Y = X + b$ représente une *translation* de X , vers la droite si $b > 0$ et vers la gauche si $b < 0$. En assurance, b pourra être interprété comme une franchise.

La transformation $Y = aX$ n'est quant à elle qu'un *changement d'échelle* — par exemple un changement d'unité monétaire. Si $a > 1$ on a une dilatation, alors que le cas où $0 < a < 1$ est une contraction. $\square$

Exemple B.3. Soit X une variable aléatoire continue quelconque avec densité $f_X(x)$. Nous voulons déterminer la densité de $Y = |X|$. Premièrement, il convient de remarquer que Y est définie au plus sur les réels positifs, même si X est définie sur tout l'axe des réels. Par la technique de la fonction de répartition,

$$\begin{aligned} F_Y(y) &= \Pr[|X| \leq y] \\ &= \Pr[-y < X < y] \\ &= F_X(y) - F_X(-y) \end{aligned}$$

et

$$f_Y(y) = \begin{cases} f_X(y) + f_X(-y), & y > 0 \\ 0, & \text{ailleurs.} \end{cases}$$

Par exemple, soit $X \sim N(0, 1)$, c'est-à-dire

$$\begin{aligned} f_X(x) &= \phi(x) \\ &= \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}x^2}. \end{aligned}$$

La densité de $Y = |X|$ est alors

$$\begin{aligned} f_Y(y) &= \phi(y) + \phi(-y) \\ &= 2\phi(y) \\ &= \frac{2}{\sqrt{2\pi}} e^{-\frac{1}{2}y^2}, \quad y > 0. \end{aligned}$$

$\square$

Exemple B.4. Soit $Y = X_1 + X_2$, où X_1 et X_2 sont deux variables aléatoires indépendantes, chacune distribuée uniformément sur l'intervalle $(0, 1)$. Nous avons donc

$$f_{X_1 X_2}(x_1, x_2) = f_{X_1}(x_1) = f_{X_2}(x_2) = 1, \quad 0 < x_1 < 1, 0 < x_2 < 1.$$

Le domaine de définition de Y sera l'intervalle $(0, 2)$. Le domaine d'intégration étant un carré, il faut distinguer quatre cas.

1. Si $y \leq 0$, alors clairement $F_Y(y) = 0$.
2. Si $0 < y < 1$, alors

$$\begin{aligned} F_Y(y) &= \int_0^y \int_0^{y-x_2} dx_1 dx_2 \\ &= \frac{1}{2} y^2. \end{aligned}$$

3. Si $1 < y < 2$, alors

$$\begin{aligned} F_Y(y) &= (y-1)(1) + \int_{y-1}^1 \int_0^{y-x_2} dx_1 dx_2 \\ &= 1 - \frac{1}{2} (2-y)^2. \end{aligned}$$

4. Enfin, si $y \geq 2$, clairement $F_Y(y) = 1$.

Par conséquent, nous avons

$$F_Y(y) = \begin{cases} 0, & y \leq 0 \\ \frac{1}{2} y^2, & 0 < y \leq 1 \\ 1 - \frac{1}{2} (2-y)^2, & 1 < y < 2 \\ 1, & y \geq 2 \end{cases}$$

et

$$f_Y(y) = \begin{cases} 0, & y \leq 0 \\ y, & 0 < y \leq 1 \\ 2-y, & 1 < y < 2 \\ 0, & y \geq 2. \end{cases}$$

□

B.2 Technique du changement de variable univariée

Cette technique est étroitement liée au changement de variable en intégration. Il convient toutefois de faire une distinction entre les cas discret et continu.

B.2.1 Cas discret

Soit la transformation $Y = u(X)$. Dans le cas discret, il suffit généralement de faire la substitution :

$$\begin{aligned}\Pr[Y = y] &= \Pr[u(X) = y] \\ &= \Pr[X = u^{-1}(y)].\end{aligned}$$

Les probabilités ne changent donc pas, elles ne sont qu'affectées à d'autres valeurs.

Exemple B.5. Soit la variable aléatoire X avec fonction de probabilité

$$\Pr[X = x] = \begin{cases} 1/16, & x = 0 \\ 4/16, & x = 1 \\ 6/16, & x = 2 \\ 4/16, & x = 3 \\ 1/16, & x = 4. \end{cases}$$

Nous voulons déterminer la fonction de probabilité de $Y = (X - 2)^2$. Les valeurs possibles de Y sont $y = 0, 1$ et 4 . Nous avons :

$$\begin{aligned}\Pr[Y = y] &= \Pr[(X - 2)^2 = y] \\ &= \Pr[X = \pm\sqrt{y} + 2]\end{aligned}$$

et donc

$$\begin{aligned}\Pr[Y = 0] &= \Pr[X = 2] = \frac{6}{16} \\ \Pr[Y = 1] &= \Pr[X = 1] + \Pr[X = 3] = \frac{8}{16} \\ \Pr[Y = 4] &= \Pr[X = 0] + \Pr[X = 4] = \frac{2}{16}.\end{aligned}$$

□

B.2.2 Cas continu

Le cas continu est beaucoup plus délicat. Nous étudions toujours la transformation $Y = u(X)$ avec les hypothèses suivantes :

- la fonction $u(\cdot)$ est différentiable;
- la fonction $u(\cdot)$ est soit croissante, soit décroissante sur tout le domaine de $f_X(x)$.

Ainsi, l'inverse $u^{-1}(\cdot) = w(\cdot)$ de la fonction u existe et est différentiable.

Théorème B.1. Soit $f_X(x)$ la fonction de densité de probabilité en x d'une variable aléatoire X et $y = u(x)$ une fonction satisfaisant les hypothèses ci-dessus. Alors la densité de la variable aléatoire $Y = u(X)$ est

$$\begin{aligned} f_Y(y) &= f_X(u^{-1}(y)) |(u^{-1}(y))'| \\ &= f_X(w(y)) |w'(y)|, \end{aligned}$$

où $w(y) = u^{-1}(y)$ et en supposant $u'(x) \neq 0$.

Démonstration. Considérons le cas où u est une fonction croissante. Alors

$$\begin{aligned} \Pr[a < Y < b] &= \Pr[u^{-1}(a) < X < u^{-1}(b)] \\ &= \int_{w(a)}^{w(b)} f_X(x) dx \end{aligned}$$

puis, avec le changement de variable $y = u(x) \Leftrightarrow x = w(y)$ et donc $dx = w'(y) dy$

$$\Pr[a < Y < b] = \int_a^b f_X(w(y)) w'(y) dy,$$

d'où $f_Y(y) = f_X(w(y)) w'(y)$. Si u est décroissante, alors

$$\begin{aligned} f_Y(y) &= -f_X(w(y)) w'(y) \\ &= f_X(w(y)) |w'(y)| \end{aligned}$$

car $w'(y) < 0$. □

Exemple B.6. Soit $Y = -2 \ln(X)$ où $X \sim U(0, 1)$. En premier lieu, il importe de spécifier qu'au domaine $(0, 1)$ de la variable aléatoire X correspond le domaine $(0, \infty)$ pour la transformation Y . Nous avons $u(x) = -2 \ln(x)$,

d'où $w(y) = u^{-1}(y) = e^{-y/2}$ et $w'(y) = -\frac{1}{2}e^{-y/2}$. Par le [théorème B.1](#), nous obtenons

$$\begin{aligned} f_Y(y) &= f_X(e^{-y/2}) \left| -\frac{1}{2}e^{-y/2} \right| \\ &= \frac{1}{2} e^{-y/2} \\ &= \frac{1/2}{\Gamma(1)} y^{1-1} e^{-y/2}, \quad y > 0, \end{aligned}$$

soit $Y \sim \text{Gamma}(1, 1/2) \equiv \chi^2(2)$. □

Exemple B.7. Nous voulons déterminer la distribution de $Y = Z^2$, où $Z \sim N(0, 1)$. La variable aléatoire Z étant définie sur $\mathbb{R}$, la transformation $y = z^2$ n'est pas bijective. Le truc consiste ici à d'abord définir la transformation $X = |Z|$, de sorte que X soit définie sur les réels positifs seulement. De l'[exemple B.3](#), nous savons que

$$f_X(x) = \frac{2}{\sqrt{2\pi}} e^{-x^2/2}.$$

Par la suite, nous définissons $Y = X^2 = |Z|^2 = Z^2$, une transformation bijective de X dont le domaine de définition est $\mathbb{R}^+$. Nous avons alors $u(x) = x^2$ pour $x > 0$, soit $w(y) = \sqrt{y}$ et $w'(y) = \frac{1}{2}y^{-1/2}$, d'où

$$\begin{aligned} f_Y(y) &= f_X(\sqrt{y}) \left| \frac{y^{-1/2}}{2} \right| \\ &= \frac{y^{-1/2}}{\sqrt{2\pi}} e^{-y/2} \\ &= \frac{(\frac{1}{2})^{1/2}}{\Gamma(\frac{1}{2})} y^{-1/2} e^{-y/2}, \quad y > 0, \end{aligned}$$

soit $Y \sim \chi^2(1)$. □

B.3 Technique du changement de variable multivariée

Il s'agit simplement ici de généraliser les concepts étudiés à la section précédente à des transformations impliquant plusieurs variables aléatoires.

L'idée reste la même sinon qu'il faut s'assurer que la transformation compte autant de nouvelles variables que d'anciennes. Ainsi, si nous parlons de la distribution conjointe de deux variables aléatoires X_1 et X_2 , il faudra trouver la distribution conjointe de deux nouvelles variables aléatoires

$Y_1 = u_1(X_1, X_2)$ et $Y_2 = u_2(X_1, X_2)$. Plus souvent qu'autrement, seule la distribution de la variable Y_1 est d'intérêt. Il suffit alors de définir Y_2 comme une variable muette, par exemple $Y_2 = X_2$ (les textes anglais utilisent généralement l'expression *dummy variable*).

B.3.1 Cas discret

Si la transformation $Y_1 = u_1(X_1, X_2)$ et $Y_2 = u_2(X_1, X_2)$ est bijective, alors simplement

$$\begin{aligned}\Pr[Y_1 = y_1, Y_2 = y_2] &= \Pr[u_1(X_1, X_2) = y_1, u_2(X_1, X_2) = y_2] \\ &= \Pr[X_1 = w_1(y_1, y_2), X_2 = w_2(y_1, y_2)],\end{aligned}$$

où

$$\begin{aligned}w_1(y_1, y_2) &= u_1^{-1}(x_1, x_2) \\ w_2(y_1, y_2) &= u_2^{-1}(x_1, x_2).\end{aligned}$$

Les fonctions de probabilité marginales sont alors obtenues en sommant :

$$\begin{aligned}\Pr[Y_1 = y_1] &= \sum_{y_2=-\infty}^{\infty} \Pr[Y_1 = y_1, Y_2 = y_2] \\ \Pr[Y_2 = y_2] &= \sum_{y_1=-\infty}^{\infty} \Pr[Y_1 = y_1, Y_2 = y_2].\end{aligned}$$

Exemple B.8. Soit $X_1 \sim \text{Poisson}(\lambda_1)$, $X_2 \sim \text{Poisson}(\lambda_2)$ et X_1 et X_2 sont indépendantes. Posons $Y = X_1 + X_2$.

Le calcul de la distribution de Y requiert une variable muette. Définissons $Y_2 = X_2$. Nous avons alors :

$$\begin{aligned}Y_1 &= X_1 + X_2 & \Leftrightarrow & & X_1 &= Y_1 - Y_2 \\ Y_2 &= X_2 & & & X_2 &= Y_2.\end{aligned}$$

Le domaine de Y_1 est donc $0, 1, 2, \dots$, alors que celui de Y_2 est $0, 1, \dots, Y_1$. Ainsi,

$$\begin{aligned}\Pr[Y_1 = y_1, Y_2 = y_2] &= \Pr[X_1 + X_2 = y_1, X_2 = y_2] \\ &= \Pr[X_1 = y_1 - y_2, X_2 = y_2] \\ &= \Pr[X_1 = y_1 - y_2] \Pr[X_2 = y_2] \\ &= \frac{\lambda_1^{y_1-y_2} e^{-\lambda_1}}{(y_1 - y_2)!} \frac{\lambda_2^{y_2} e^{-\lambda_2}}{y_2!}\end{aligned}$$

et donc

$$\begin{aligned}
 \Pr[Y_1 = y_1] &= \sum_{y_2=0}^{y_1} \frac{\lambda_1^{y_1-y_2} \lambda_2^{y_2} e^{-(\lambda_1+\lambda_2)}}{(y_1 - y_2)! y_2!} \\
 &= \frac{e^{-(\lambda_1+\lambda_2)}}{y_1!} \sum_{y_2=0}^{y_1} \frac{y_1!}{(y_1 - y_2)! y_2!} \lambda_1^{y_1-y_2} \lambda_2^{y_2} \\
 &= \frac{e^{-(\lambda_1+\lambda_2)}}{y_1!} \sum_{y_2=0}^{y_1} \binom{y_1}{y_2} \lambda_1^{y_1-y_2} \lambda_2^{y_2} \\
 &= \frac{e^{-(\lambda_1+\lambda_2)}}{y_1!} (\lambda_1 + \lambda_2)^{y_1},
 \end{aligned}$$

soit $Y \sim \text{Poisson}(\lambda_1 + \lambda_2)$. □

B.3.2 Cas continu

Généralisons le [théorème B.1](#) afin de trouver la distribution conjointe (et éventuellement les distributions marginales) de $Y_1 = u_1(X_1, X_2)$ et $Y_2 = u_2(X_1, X_2)$. Nous supposons que :

- toutes les premières dérivées partielles de u_1 et u_2 existent sur le domaine de X_1 et X_2 ;
- la transformation est bijective.

Ces hypothèses garantissent que les fonctions inverses $w_1 = u_1^{-1}$ et $w_2 = u_2^{-1}$ existent.

Théorème B.2. Soit $f_{X_1 X_2}(x_1, x_2)$ la fonction de densité de probabilité conjointe en (x_1, x_2) des variables aléatoires X_1 et X_2 et $y_1 = u_1(x_1, x_2)$, $y_2 = u_2(x_1, x_2)$ des fonctions satisfaisant les hypothèses ci-dessus. Alors la densité conjointe des variables aléatoires $Y_1 = u_1(X_1, X_2)$ et $Y_2 = u_2(X_1, X_2)$ est

$$f_{Y_1 Y_2}(y_1, y_2) = f_{X_1 X_2}(w_1(y_1, y_2), w_2(y_1, y_2)) |J|,$$

où

$$J = \begin{vmatrix} \frac{\partial x_1}{\partial y_1} & \frac{\partial x_1}{\partial y_2} \\ \frac{\partial x_2}{\partial y_1} & \frac{\partial x_2}{\partial y_2} \end{vmatrix}$$

est appelé le Jacobien de la transformation.

Exemple B.9. Soit les variables aléatoires stochastiquement indépendantes $X_1 \sim \text{Gamma}(\alpha, 1)$ et $X_2 \sim \text{Gamma}(\eta, 1)$. Nous allons démontrer que les variables aléatoires $Y_1 = X_1 + X_2$ et $Y_2 = X_1 / (X_1 + X_2)$ sont indépendantes et trouver leur densité marginale respective.

Tout d'abord, nous avons

$$f_{X_1 X_2}(x_1, x_2) = \frac{1}{\Gamma(\alpha)\Gamma(\eta)} x_1^{\alpha-1} x_2^{\eta-1} e^{-x_1-x_2}, \quad x_1 > 0, x_2 > 0.$$

De plus,

$$\begin{aligned} Y_1 &= X_1 + X_2 \\ Y_2 &= \frac{X_1}{X_1 + X_2} \end{aligned} \quad \Leftrightarrow \quad \begin{aligned} X_1 &= Y_1 Y_2 \\ X_2 &= Y_1 (1 - Y_2) \end{aligned}$$

et donc

$$\begin{aligned} \frac{\partial x_1}{\partial y_1} &= y_2 & \frac{\partial x_1}{\partial y_2} &= y_1 \\ \frac{\partial x_2}{\partial y_1} &= 1 - y_2 & \frac{\partial x_2}{\partial y_2} &= -y_1, \end{aligned}$$

d'où le Jacobien de la transformation est

$$J = \begin{vmatrix} y_2 & y_1 \\ 1 - y_2 & -y_1 \end{vmatrix} = -y_1.$$

Le domaine de Y_1 est $\mathbb{R}^+$ alors que celui de Y_2 est limité à l'intervalle $(0, 1)$. Par le [théorème B.2](#),

$$\begin{aligned} f_{Y_1 Y_2}(y_1, y_2) &= f_{X_1 X_2}(y_1 y_2, y_1 (1 - y_2)) | -y_1 | \\ &= \frac{1}{\Gamma(\alpha)\Gamma(\eta)} y_1 (y_1 y_2)^{\alpha-1} (y_1 (1 - y_2))^{\eta-1} e^{-y_1 y_2 - y_1 (1 - y_2)} \\ &= \frac{1}{\Gamma(\alpha)\Gamma(\eta)} y_1^{\alpha+\eta-1} y_2^{\alpha-1} (1 - y_2)^{\eta-1} e^{-y_1} \\ &= g(y_1) h(y_2) \end{aligned}$$

où $g(\cdot)$ et $h(\cdot)$ sont des fonctions quelconques. Ceci démontre que Y_1 et Y_2 sont indépendantes. De plus,

$$\begin{aligned} f_{Y_1}(y_1) &= \int_0^1 f_{Y_1 Y_2}(y_1, y_2) dy_2 \\ &= y_1^{\alpha+\eta-1} e^{-y_1} \int_0^1 \frac{1}{\Gamma(\alpha)\Gamma(\eta)} y_2^{\alpha-1} (1 - y_2)^{\eta-1} dy_2 \\ &= \frac{1}{\Gamma(\alpha + \eta)} y_1^{\alpha+\eta-1} e^{-y_1}, \quad y_1 > 0 \end{aligned}$$

et

$$\begin{aligned} f_{Y_2}(y_1, y_2) &= \frac{f_{Y_1 Y_2}(y_1, y_2)}{f_{Y_1}(y_1, y_2)} \\ &= \frac{\Gamma(\alpha + \eta)}{\Gamma(\alpha)\Gamma(\eta)} y_2^{\alpha-1} (1 - y_2)^{\eta-1}, \quad 0 < y_2 < 1. \end{aligned}$$

Nous avons donc $Y_1 = X_1 + X_2 \sim \text{Gamma}(\alpha + \eta, 1)$ et $Y_2 = X_1 / (X_1 + X_2) \sim \text{Bêta}(\alpha, \eta)$. $\square$

B.4 Technique de la fonction génératrice des moments

Cette technique s'avère tout spécialement puissante pour déterminer la distribution (marginale) d'une combinaison linéaire de variables aléatoires indépendantes. La technique repose sur le théorème suivant.

Théorème B.3. *Soit $X_1, \dots, X_n$ des variables aléatoires indépendantes et $Y = X_1 + \dots + X_n$. Alors*

$$M_Y(t) = \prod_{i=1}^n M_{X_i}(t).$$

Il est laissé en exercice de refaire les exemples B.8 et B.9 (distribution de Y_1 seulement) à l'aide de la technique de la fonction génératrice des moments.

C Solutions des exercices

Chapitre 1

1.1 Dans tous les cas, le générateur de nombres aléatoires est

$$\begin{aligned}x_i &= (ax_{i-1} + c) \bmod m \\ &= (ax_{i-1} + c) - \left\lfloor \frac{ax_{i-1} + c}{m} \right\rfloor m\end{aligned}$$

où $m = 64$ et $x_0 = 19$. Les suites ont été générées avec la fonction `rand` de la [figure C.1](#).

a) On obtient

```
> rand(n = 5, a = 29, c = 17, m = 64, seed = 19)
[1] 56 41 54 47 36
```

b) On obtient

```
> rand(n = 5, a = 9, c = 1, m = 64, seed = 19)
[1] 44 13 54 39 32
```

```
rand <- function(n, a, c, m, seed)
{
  x <- numeric(n + 1)
  x[1] <- seed
  for (i in seq(n))
    x[i + 1] <- (a * x[i] + c) %% m
  x[-1]
}
```

FIG. C.1 – Code de la fonction `rand`

c) On obtient

```
> rand(n = 5, a = 13, c = 0, m = 64, seed = 19)
[1] 55 11 15 3 39
```

d) On obtient

```
> rand(n = 5, a = 11, c = 0, m = 64, seed = 19)
[1] 17 59 9 35 1
```

1.2 Nous pouvons utiliser de nouveau la fonction `rand` de la [figure C.1](#). Le graphique de la [figure C.2](#) a été créé avec les commandes ci-dessous. Les points sont alignés sur 17 lignes.

```
> x <- rand(n = 500, a = 17, c = 0, m = 2^13 - 1,
+          seed = 19)
> plot(x[-length(x)], x[-1], xlab = expression(x[i]),
+      ylab = expression(x[i + 1]), pch = 19)
```

Comme le montre la [figure C.3](#), le générateur avec $a = 85$ semble préférable au précédent puisque les points sont plus uniformément répartis sur le graphique. Les nombres utilisés dans ce dernier ont été générés avec l'expression suivante.

```
> x <- rand(n = 500, a = 85, c = 0, m = 2^13 - 1,
+          seed = 19)
```

1.3 a) On obtient environ 200 points alignés sur 10 lignes.

```
> u <- rand(n = 20002, a = 65539, c = 0, m = 2^31,
+          seed = 19)/2^31
> mat <- matrix(c(u[1:20000], u[2:20001], u[3:20002]),
+             ncol = 3)
> mat <- mat[(0.5 <= mat[,2]) & (mat[,2] <= 0.51),
+           c(1, 3)]
> plot(mat, xlab = expression(u[i]),
+      ylab = expression(u[i + 2]))
```

b) Les expressions suivantes permettent de générer le graphique interactif en trois dimensions.

```
> library(rgl)
> u <- rand(n = 1002, a = 65539, c = 0, m = 2^31,
```

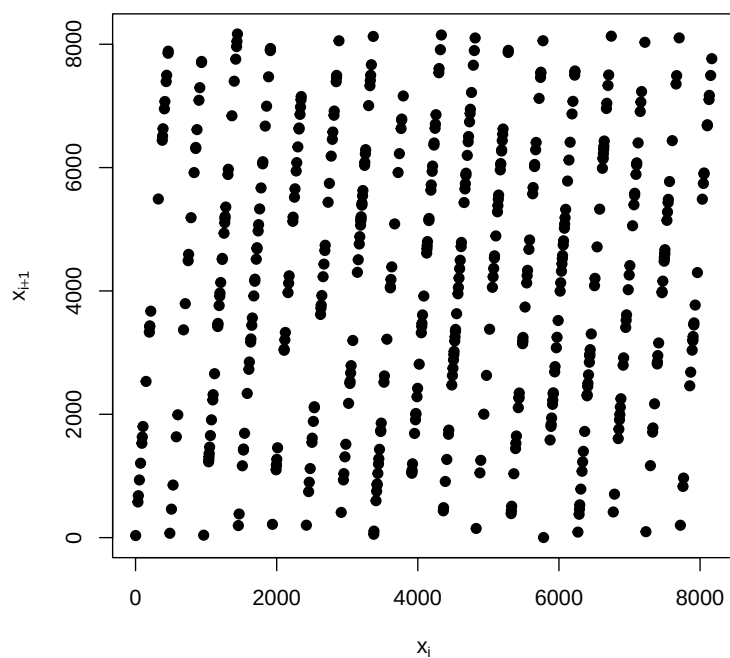


FIG. C.2 - Paires de valeurs du générateur congruentiel multiplicatif avec $m = 2^{31} - 1$ et $a = 17$

```
+ seed = 19)/2^31
> plot3d(u[1:1000], u[2:1001], u[3:1002])
```

Chapitre 2

2.1 a) Tout d'abord, on voit que

$$\cos(2\pi U_2) \in (-1, 1)$$

$$\sin(2\pi U_2) \in (-1, 1)$$

et

$$(-2 \log U_1)^{1/2} \in (0, \infty).$$

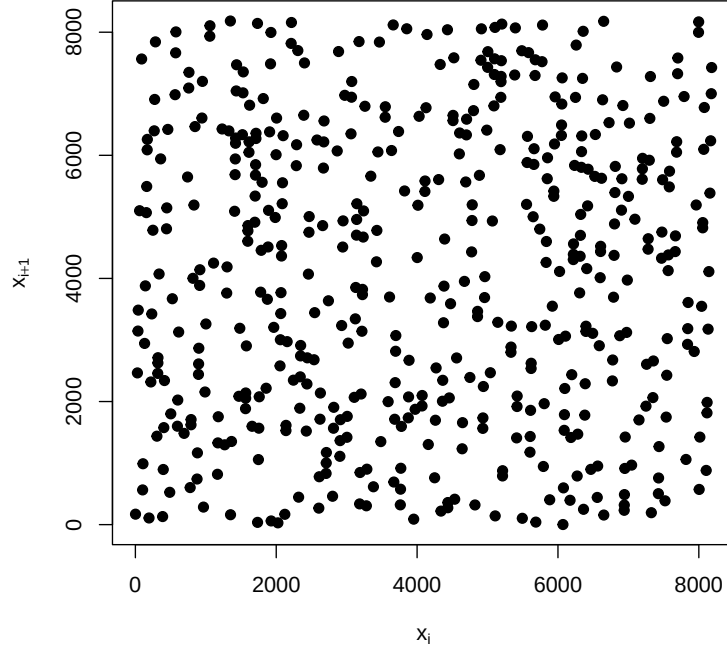


FIG. C.3 - Paires de valeurs du générateur congruentiel multiplicatif avec $m = 2^{31} - 1$ et $a = 85$

Par conséquent, $X_1 \in (-\infty, \infty)$ et $X_2 \in (-\infty, \infty)$. On vérifie la bijectivité de façon heuristique avec quelques valeurs de u_1 et u_2 .

b) On a

$$X_1^2 = (-2 \log U_1) \cos^2(2\pi U_2)$$

$$X_2^2 = (-2 \log U_1) \sin^2(2\pi U_2).$$

Or, $X_1^2 + X_2^2 = -2 \log U_1$ car $\sin^2(x) + \cos^2(x) = 1$. Ainsi, $U_1 = \exp\{-(X_1^2 + X_2^2)/2\}$. Ensuite, $\tan(2\pi U_2) = X_2/X_1$ par définition de la tangente. Par conséquent, $U_2 = (2\pi)^{-1} \arctan X_2/X_1$.

c) Soit les fonctions

$$x_1(u_1, u_2) = (-2 \log u_1)^{1/2} \cos(2\pi u_2)$$

$$x_2(u_1, u_2) = (-2 \log u_1)^{1/2} \sin(2\pi u_2)$$

et

$$u_1(x_1, x_2) = e^{-(x_1^2 + x_2^2)/2}$$

$$u_2(x_1, x_2) = \frac{1}{2\pi} \arctan \frac{x_2}{x_1}.$$

Les variables aléatoires U_1 et U_2 sont indépendantes, donc leur fonction de densité de probabilité conjointe est le produit des densités marginales :

$$f_{U_1, U_2}(u_1, u_2) = 1, \quad 0 < u_1 < 1, 0 < u_2 < 1.$$

La densité conjointe de X_1 et X_2 est, par définition d'une transformation,

$$f_{X_1, X_2}(x_1, x_2) = f_{U_1, U_2}(x_1(u_1, u_2), x_2(u_1, u_2)) |\det(J)|,$$

où

$$J = \begin{bmatrix} \frac{\partial u_1}{\partial x_1} & \frac{\partial u_1}{\partial x_2} \\ \frac{\partial u_2}{\partial x_1} & \frac{\partial u_2}{\partial x_2} \end{bmatrix} = \begin{bmatrix} -x_1 e^{-(x_1^2 + x_2^2)/2} & -x_2 e^{-(x_1^2 + x_2^2)/2} \\ -\frac{1}{2\pi} \frac{x_2}{x_1^2 + x_2^2} & \frac{1}{2\pi} \frac{x_1}{x_1^2 + x_2^2} \end{bmatrix}.$$

Or,

$$|\det(J)| = \frac{1}{2\pi} e^{-(x_1^2 + x_2^2)/2}$$

$$= \frac{1}{\sqrt{2\pi}} e^{-x_1^2/2} \cdot \frac{1}{\sqrt{2\pi}} e^{-x_2^2/2},$$

d'où

$$f_{X_1, X_2}(x_1, x_2) = \frac{1}{\sqrt{2\pi}} e^{-x_1^2/2} \cdot \frac{1}{\sqrt{2\pi}} e^{-x_2^2/2}.$$

Par conséquent, X_1 et X_2 sont deux variables aléatoires $N(0, 1)$ indépendantes.

- d) Les expressions R ci-dessous permettent de superposer un histogramme des données simulées et la fonction de densité de probabilité d'une loi normale.

```
> u1 <- runif(500)
> u2 <- runif(500)
> x1 <- outer(u1, u2, function(x, y)
```

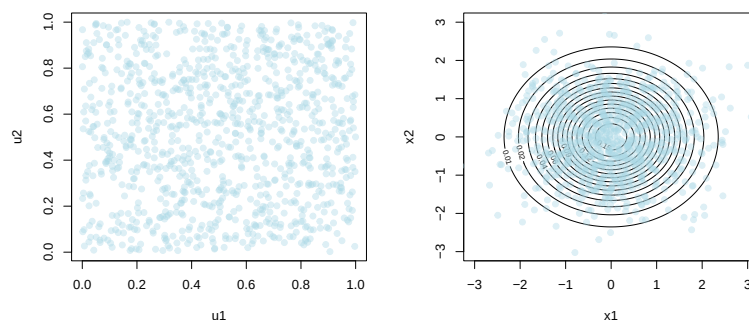


FIG. C.4 - Démonstration graphique du fonctionnement de la transformation de Box-Muller

```
+          sqrt((-2 * log(x))) * cos(2 * pi * y))
> x2 <- outer(u1, u2, function(x, y)
+          sqrt((-2 * log(x))) * sin(2 * pi * y))
> hist(x1, prob = TRUE)
> curve(dnorm(x), add = TRUE)
```

La [figure C.4](#) illustre d'une autre façon que la transformation de Box-Muller fonctionne bel et bien. Le graphique de gauche contient plusieurs couples de points (u_1, u_2) , où chaque composante provient d'une distribution uniforme sur l'intervalle $(0, 1)$.

Chacun de ces points a été transformé en un point (x_1, x_2) selon la transformation de Box-Muller, puis placé dans le graphique de droite. Nous avons superposé le nuage de points ainsi obtenu aux lignes de niveau d'une distribution normale bivariable (avec paramètre $\rho = 0$). Vous observerez que la répartition et la densité du nuage de points correspondent effectivement aux lignes de niveau.

2.2 Deux suggestions :

1. Simuler deux nombres indépendants x_1 et x_2 d'une loi exponentielle de paramètre λ et poser $y = x_1 - x_2$.
2. La fonction de répartition de la distribution de Laplace est

$$F_Y(y) = \begin{cases} \frac{1}{2}e^{\lambda x}, & x < 0 \\ 1 - \frac{1}{2}e^{-\lambda x}, & x \geq 0, \end{cases}$$

d'où

$$F_Y^{-1}(u) = \begin{cases} \frac{1}{\lambda} \ln(2u) & u < 0,5 \\ -\frac{1}{\lambda} \ln(2(1-u)) & u \geq 0,5. \end{cases}$$

Il est donc aussi possible d'utiliser la méthode de l'inverse qui revient, ici, à l'algorithme suivant : simuler un nombre x issu d'une loi exponentielle de paramètre λ et un nombre u issu d'une loi uniforme sur $(0, 1)$, puis poser $y = x$ si $u < 1/2$, ou $y = -x$ sinon.

2.3 Soit $X \sim \text{Exponentielle}(\lambda)$ et $K = \lfloor X \rfloor$. Par conséquent :

$$\begin{aligned} \Pr[K = 0] &= \Pr[\lfloor X \rfloor = 0] \\ &= \Pr[0 \leq X < 1] \\ &= F_X(1) \\ &= 1 - e^{-\lambda}, \\ \Pr[K = 1] &= \Pr[1 \leq X < 2] \\ &= F_X(2) - F_X(1) \\ &= (1 - e^{-2\lambda}) - (1 - e^{-\lambda}) \\ &= (e^{-\lambda} - e^{-2\lambda}) \\ &= (1 - e^{-\lambda})e^{-\lambda}, \end{aligned}$$

soit, de manière générale,

$$\begin{aligned} \Pr[K = k] &= \Pr[k \leq X < k+1] \\ &= (1 - e^{-\lambda})(e^{-\lambda})^k, \quad k = 0, 1, 2, \dots \end{aligned}$$

La forme de cette fonction de masse de probabilité est celle d'une loi géométrique de paramètre $p = 1 - e^{-\lambda}$.

2.4 Voir la fonction R de la [figure C.5](#). Les expressions suivantes permettent de tracer les graphiques pour vérifier la validité de l'algorithme.

```
> hist(rgamma2(1000, 5), prob = TRUE)
> curve(dgamma(x, 5, 1), add = TRUE)
```

2.5 Deux suggestions.

1. Si $X \sim \text{Gamma}(\alpha, 1)$, alors $Y = X/\lambda \sim \text{Gamma}(\alpha, \lambda)$. Nous pouvons donc générer un nombre x d'une loi $\text{Gamma}(\alpha, 1)$ avec l'algorithme de l'[exercice 2.4](#), puis poser $y = x/\lambda$.

```

rgamma2 <- function(nsim, alpha)
{
  x <- numeric(nsim)
  i <- 0
  while (i < nsim)
  {
    u <- runif(2)
    v <- (alpha - 1/(6 * alpha)) * u[1] /
          ((alpha - 1) * u[2])

    if ((2 * (u[2] - 1)/(alpha - 1) +
          v + 1/v <= 2) |
        (2 * log(u[2])/(alpha - 1) -
          log(v) + v <= 1))
      x[i <- i + 1] <- (alpha - 1) * v
  }
  x
}

```

FIG. C.5 – Fonction de simulation d'une distribution $\text{Gamma}(\alpha, 1)$ avec $\alpha > 1$

2. Si α est entier, nous pouvons également générer α nombres $x_1, \dots, x_\alpha$ d'une distribution Exponentielle(λ) et poser $y = \sum_{i=1}^{\alpha} x_i$.

2.6 a) Vous aurez reconnu dans cette combinaison de variables aléatoires un mélange d'une exponentielle et d'une gamma. Par la loi des probabilités totales,

$$\begin{aligned}
 f_X(x) &= \int_0^\infty f(x|\theta)u(\theta) d\theta \\
 &= \frac{\lambda^\alpha}{\Gamma(\alpha)} \int_0^\infty \theta^{\alpha+1-1} e^{-(\lambda+x)\theta} d\theta \\
 &= \frac{\lambda^\alpha}{\Gamma(\alpha)} \frac{\Gamma(\alpha+1)}{(\lambda+x)^{\alpha+1}} \\
 &= \frac{\alpha\lambda^\alpha}{(\lambda+x)^{\alpha+1}}.
 \end{aligned}$$

- b) Pour générer un nombre d'une distribution de Pareto de paramètres α et λ avec le résultat en a), il faut d'abord obtenir un nombre θ d'une distribution gamma de mêmes paramètres, puis obtenir un nombre x d'une distribution exponentielle de paramètre θ .

```
| > rexp(1, rgamma(1, alpha, lambda))
```

2.7 La fonction de répartition de la Pareto translatée(α, λ) est

$$F(x) = \int_{\lambda}^x \frac{\alpha \lambda^{\alpha}}{y^{\alpha+1}} dy$$

$$= \begin{cases} 0, & x \leq \lambda \\ 1 - \left(\frac{\lambda}{x}\right)^{\alpha}, & x > \lambda \end{cases}$$

et son inverse est

$$F^{-1}(y) = \begin{cases} \lambda, & y = 0 \\ \frac{\lambda}{(1-y)^{1/\alpha}}, & 0 < y < 1. \end{cases}$$

Par conséquent, si $U \sim U(0, 1)$, alors

$$X = \frac{\lambda}{(1-U)^{1/\alpha}} \sim \text{Pareto translatée}(\alpha, \lambda).$$

Les trois premières valeurs retournées par le générateur

$$x_n = (65x_{n-1} + 1) \bmod 2048$$

avec une amorce de 12 sont 781, 1 614 et 463. La division de ces nombres par 2 048 fournit des nombres dans l'intervalle (0, 1) :

$$0,3813 \quad 0,7881 \quad 0,2261.$$

Finalement, les observations de la Pareto(2, 1 000) sont les suivantes.

```
| > 1000/sqrt(1 - c(0.3813, 0.7881, 0.2261))
| [1] 1271.335 2172.374 1136.731
```

Remarque : puisque $1 - U \sim U(0, 1)$ si $U \sim U(0, 1)$, les nombres issus de la transformation $\lambda(1 - U)^{-1/\alpha}$ seraient tout aussi distribués selon une Pareto translatée. Les réponses seraient toutefois différentes.

2.8 Nous avons la transformation

$$\begin{aligned} X_1 &= \sqrt{U_1} \cos(2\pi U_2) \\ X_2 &= \sqrt{U_1} \sin(2\pi U_2) \end{aligned} \Leftrightarrow \begin{aligned} U_1 &= X_1^2 + X_2^2 \\ U_2 &= \frac{1}{2\pi} \arctan \frac{X_2}{X_1}. \end{aligned}$$

Cette transformation associe les points de l'espace $\{(u_1, u_2) : 0 < u_1 < 1, 0 < u_2 < 1\}$ à ceux de l'espace $\{(x_1, x_2) : x_1^2 + x_2^2 < 1 \setminus (0, 0)\}$. Cela se vérifie aisément en examinant les limites de l'espace de départ :

$$\begin{array}{lll}
 u_1 > 0 & \Rightarrow & x_1^2 + x_2^2 > 0 \\
 u_1 < 1 & \Rightarrow & x_1^2 + x_2^2 < 1 \\
 u_2 > 0 & \Rightarrow & \frac{x_2}{x_1} > 0 \\
 u_2 < 1 & \Rightarrow & \frac{x_2}{x_1} < 0.
 \end{array}$$

Les troisième et quatrième inégalités définissent les quadrants I et III, puis II et IV de $\mathbb{R}^2$, respectivement. Vous remarquerez que le point $(0, 0)$, qui a probabilité zéro, ne se trouve pas dans l'espace image.

Le Jacobien de la transformation est

$$\begin{aligned}
 J &= \begin{vmatrix} \frac{\partial u_1}{\partial x_1} & \frac{\partial u_1}{\partial x_2} \\ \frac{\partial u_2}{\partial x_1} & \frac{\partial u_2}{\partial x_2} \end{vmatrix} \\
 &= \begin{vmatrix} 2x_1 & 2x_2 \\ -\frac{1}{2\pi} \frac{x_2}{x_1^2 + x_2^2} & \frac{1}{2\pi} \frac{x_1}{x_1^2 + x_2^2} \end{vmatrix} \\
 &= \frac{1}{\pi}.
 \end{aligned}$$

La fonction de densité de probabilité conjointe de X_1 et X_2 est donc

$$\begin{aligned}
 f_{X_1, X_2}(x_1, x_2) &= f_{U_1, U_2}(u_1, u_2) |J| \\
 &= \frac{1}{\pi}, \quad -1 < x_1 < 1, -\sqrt{1 - x_1^2} < x_2 < \sqrt{1 - x_1^2},
 \end{aligned}$$

soit une distribution uniforme sur le disque unité.

Le résultat peut évidemment servir à simuler des points uniformément répartis sur un disque de rayon 1 centré en $(0, 0)$. La [figure C.6](#) illustre d'ailleurs cette transformation. Les points (u_1, u_2) dans le graphique de gauche sont tirés aléatoirement sur le carré $(0, 1) \times (0, 1)$. Le graphique de droite montre que suite à la transformation ci-dessus, les points (x_1, x_2) sont distribués uniformément sur un disque de rayon 1 centré en $(0, 0)$.

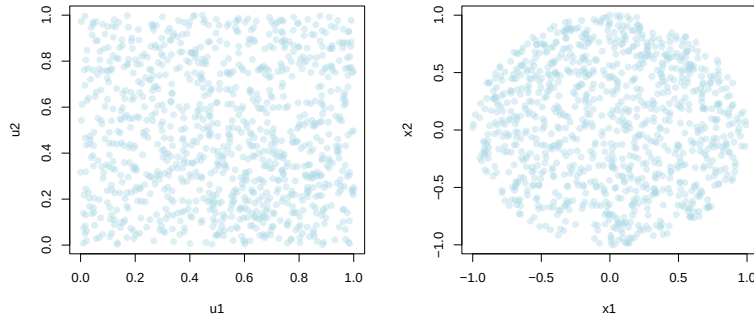


FIG. C.6 - Démonstration graphique du fonctionnement de la transformation de l'exercice 2.8

2.9 a) Nous avons

$$f_{Y_1}(y_1) = \frac{1}{\Gamma(\alpha)} y_1^{\alpha-1} e^{-y_1}, \quad y_1 > 0,$$

$$f_{Y_2}(y_2) = \frac{1}{\Gamma(\beta)} y_2^{\beta-1} e^{-y_2}, \quad y_2 > 0$$

et

$$f_{Y_1, Y_2}(y_1, y_2) = \frac{1}{\Gamma(\alpha)\Gamma(\beta)} y_1^{\alpha-1} y_2^{\beta-1} e^{-(y_1+y_2)}, \quad y_1 > 0, y_2 > 0.$$

Soit $X_1 = Y_1/(Y_1 + Y_2)$ et $X_2 = Y_1 + Y_2$ (le choix de X_2 étant justifié par l'exposant de la distribution conjointe de Y_1 et Y_2). Nous cherchons la distribution conjointe de X_1 et X_2 , $f_{X_1, X_2}(x_1, x_2)$ à partir de la transformation

$$\begin{aligned} x_1 &= \frac{y_1}{y_1 + y_2} & \Leftrightarrow & \quad y_1 = x_1 x_2 \\ x_2 &= y_1 + y_2 & & \quad y_2 = x_2 - x_1 x_2. \end{aligned}$$

Cette transformation associe de manière évidente les points de l'espace $\{(y_1, y_2) : y_1 > 0, y_2 > 0\}$ à ceux de l'espace $\{(x_1, x_2) : 0 < x_1 < 1, x_2 > 0\}$.

Le Jacobien de la transformation est

$$\begin{aligned} J &= \begin{vmatrix} \frac{\partial y_1}{\partial x_1} & \frac{\partial y_1}{\partial x_2} \\ \frac{\partial y_2}{\partial x_1} & \frac{\partial y_2}{\partial x_2} \end{vmatrix} \\ &= \begin{vmatrix} x_2 & x_1 \\ -x_2 & 1 - x_1 \end{vmatrix} \\ &= x_2. \end{aligned}$$

La fonction de densité de probabilité conjointe de X_1 et X_2 est donc

$$\begin{aligned} f_{X_1, X_2}(x_1, x_2) &= f_{Y_1, Y_2}(y_1, y_2) |J| \\ &= \frac{1}{\Gamma(\alpha)\Gamma(\beta)} x_1^{\alpha-1} (1-x_1)^{\beta-1} x_2^{\alpha+\beta-1} e^{-x_2} \\ &= \left[\frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)} x_1^{\alpha-1} (1-x_1)^{\beta-1} \right] \left[\frac{1}{\Gamma(\alpha+\beta)} x_2^{\alpha+\beta-1} e^{-x_2} \right], \end{aligned}$$

pour $0 < x_1 < 1$, $x_2 > 0$, d'où X_1 et X_2 sont indépendantes, $X_1 \sim \text{Bêta}(\alpha, \beta)$ et $X_2 \sim \text{Gamma}(\alpha + \beta)$ (un résultat connu).

b) La conversion du résultat en un algorithme est très simple :

1. Générer y_1 d'une distribution $\text{Gamma}(\alpha, 1)$.
2. Générer y_2 d'une distribution $\text{Gamma}(\beta, 1)$.
3. Poser $x = y_1 / (y_1 + y_2)$.

Cet algorithme suppose évidemment qu'une source de nombres provenant d'une loi gamma est disponible.

La [figure C.7](#) illustre le fonctionnement de cette transformation. Dans le graphique de gauche, nous avons un nuage de points (y_1, y_2) tirés de manière indépendante de deux distributions gamma de paramètre d'échelle égal à 1. Nous avons superposé ce nuage de points aux courbes de niveaux de la distribution conjointe des deux lois gamma.

Dans le graphique de droite, les points $x = y_1 / (y_1 + y_2)$ résultant de la transformation sont placés en abscisse. Vous constaterez que la répartition et la densité de ces points correspond à la densité de la loi bêta également représentée sur le graphique.

c) Dans R :

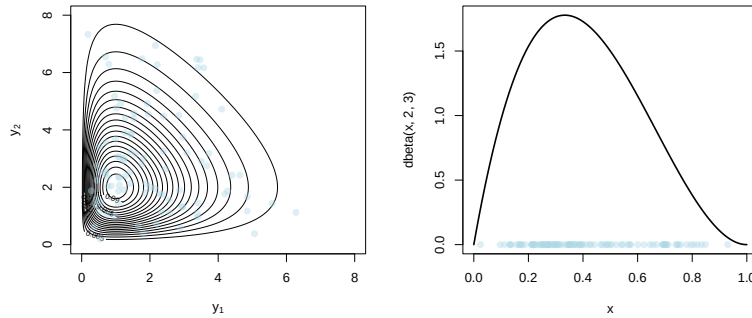


FIG. C.7 – Démonstration graphique du fonctionnement de la transformation de l'exercice 2.9

```
| > (y <- rgamma(1, alpha, 1))/(y + rgamma(1, beta, 1))
```

2.10 a) Nous avons

$$\begin{aligned}
 \Pr \left[U \leq \frac{f_X(y)}{c g_Y(y)} \right] &= \int_{-\infty}^{\infty} \Pr \left[U \leq \frac{f_X(y)}{c g_Y(y)} \middle| Y = y \right] g_Y(y) dy \\
 &= \int_{-\infty}^{\infty} \frac{f_X(y)}{c g_Y(y)} g_Y(y) dy \\
 &= \frac{1}{c} \int_{-\infty}^{\infty} f_X(y) dy \\
 &= \frac{1}{c}.
 \end{aligned}$$

b) Les essais étant indépendants, la distribution du nombre d'essais avant d'obtenir un succès (accepter un nombre y) est géométrique de paramètre $1/c$, c'est-à-dire que

$$\Pr[Z = z] = \left(\frac{1}{c} \right) \left(1 - \frac{1}{c} \right)^{z-1}, \quad z = 1, 2, \dots,$$

où Z représente le nombre d'essais avant d'accepter un nombre.

c) Nous savons que $E[Z] = 1/(1/c) = c$.

2.11 a) Posons

$$c g_Y(x) = M, \quad a < x < b,$$

soit $Y \sim U(a, b)$ et $c = M(b - a)$. L'algorithme d'acceptation-rejet est donc le suivant :

1. Simuler deux nombres indépendants u_1 et u_2 d'une loi $U(0, 1)$.
 2. Poser $y = a + (b - a)u_1$.
 3. Si $u_2 \leq f_X(y)/M$, poser $x = y$. Sinon, retourner à l'étape 1.
- b) L'efficacité est

$$\frac{1}{c} = \frac{1}{M(b-a)}.$$

- 2.12** a) Il est facile de démontrer que le mode M d'une distribution bêta de paramètres α et β se trouve en

$$x = \frac{\alpha - 1}{\alpha + \beta - 2}.$$

Par conséquent, l'efficacité de l'algorithme d'acceptation-rejet décrit à l'exercice 2.11 et consistant à borner la densité par un rectangle de hauteur M est

$$\begin{aligned} \frac{1}{M} &= \frac{1}{f((\alpha - 1)/(\alpha + \beta - 2))} \\ &= \frac{\Gamma(\alpha)\Gamma(\beta)}{\Gamma(\alpha + \beta)} \left(\frac{\alpha - 1}{\alpha + \beta - 2} \right)^{1-\alpha} \left(\frac{\beta - 1}{\alpha + \beta - 2} \right)^{1-\beta}. \end{aligned}$$

Avec $\alpha = 3$ et $\beta = 2$, l'efficacité est 9/16.

- b) Nous avons obtenu $c = 1,2$ dans l'exemple 2.5, soit une efficacité de $1/c = 5/6$. Cet algorithme est évidemment plus efficace puisque la surface de l'enveloppe de la densité bêta est nettement plus petite.
- c) Utilisons l'algorithme développé à l'exemple 2.5. Une première mise en œuvre de l'algorithme en R est fournie dans le code informatique de la section 2.9. La figure C.8 en propose une autre. Vous pouvez vérifier l'exactitude la fonction `rbeta.ar2` avec les expressions suivantes.

```
> x <- rbeta.ar2(10000)
> hist(x, prob = TRUE)
> curve(dbeta(x, 3, 2), add = TRUE)
```

- 2.13** a) Vous reconnaitrez l'algorithme d'acceptation-rejet de l'exercice 2.11.
- b) Vous devez simuler deux observations d'une loi Bêta(2, 3) dont la fonction de densité de probabilité est

$$f(x) = 12x(1 - x)^2, \quad 0 < x < 1.$$


```

rbeta.ar2 <- function(n)
{
  g <- function(x)
    ifelse(x < 0.8, 2.5 * x, 10 - 10 * x)
  Ginv <- function(y)
    ifelse(y < 0.8, sqrt(0.8 * y),
           1 - sqrt(0.2 - 0.2 * y))
  x <- numeric(n)
  i <- 0
  while(i < n)
  {
    y <- Ginv(runif(1))
    if(1.2 * g(y) * runif(1) <=
       dbeta(y, shape1 = 3, shape2 = 2))
      x[i <- i + 1] <- y
  }
  x
}

```

FIG. C.8 – Code R de la fonction `rbeta.ar2`

Le mode de cette densité se trouve en $x = 1/3$ (voir la solution de l'exercice 2.12) et la valeur de ce mode est $M = f(1/3) = 16/9$. Pour obtenir le résultat de l'appel de la fonction `simul`, il faut s'assurer d'utiliser les nombres uniformes dans le bon ordre. Quatre itérations de la boucle `repeat` seront nécessaires. Voici leurs résultats.

1. On a $u = 0,72$, puis $(16/9)(0,88) > f(0,72)$, donc u est rejeté.
2. On a $u = 0,76$, puis $(16/9)(0,89) > f(0,76)$, donc u est rejeté.
3. On a $u = 0,46$, puis $(16/9)(0,17) < f(0,46)$, donc u est accepté : $x_1 = 0,46$.
4. On a $u = 0,33$, puis $(16/9)(0,51) < f(0,33)$, donc u est accepté : $x_2 = 0,33$.

Le résultat est donc le vecteur $\mathbf{x} = (0,46, 0,33)$.

- 2.14** a) Si $0 \leq x \leq 1$, $e^{-1} < e^{-x} < 1$, d'où $x^{\alpha-1}e^{-x} \leq x^{\alpha-1}$. De même, puisque $0 < \alpha < 1$, $x^{\alpha-1} < 1$ pour $x > 1$, d'où $x^{\alpha-1}e^{-x} \leq e^{-x}$ pour $x > 1$.

b) Il faut borner la densité $f_X(x) = x^{\alpha-1}e^{-x}/\Gamma(\alpha)$, $x > 0$ et $0 < \alpha < 1$.

Du résultat en a) :

$$f_X(x) \leq \begin{cases} x^{\alpha-1}/\Gamma(\alpha), & 0 \leq x \leq 1 \\ e^{-x}/\Gamma(\alpha), & x > 1. \end{cases}$$

Posons

$$cg_Y(x) = \begin{cases} x^{\alpha-1}/\Gamma(\alpha), & 0 \leq x \leq 1 \\ e^{-x}/\Gamma(\alpha), & x > 1. \end{cases}$$

L'aire totale sous la fonction $cg_Y(x)$ est

$$\int_0^1 \frac{x^{\alpha-1}}{\Gamma(\alpha)} dx + \int_1^\infty \frac{e^{-x}}{\Gamma(\alpha)} dx = \frac{1}{\Gamma(\alpha)} \left(\frac{1}{\alpha} + \frac{1}{e} \right),$$

d'où

$$g_Y(x) = \begin{cases} \frac{x^{\alpha-1}}{(1/\alpha) + (1/e)}, & 0 \leq x \leq 1 \\ \frac{e^{-x}}{(1/\alpha) + (1/e)}, & x > 1, \end{cases}$$

$$G_Y(x) = \begin{cases} \frac{e}{\alpha + e} x^\alpha, & 0 \leq x \leq 1 \\ 1 - \frac{e^{-x}}{(1/\alpha) + (1/e)}, & x > 1, \end{cases}$$

et

$$G_Y^{-1}(x) = \begin{cases} \left(\frac{\alpha + e}{e} x \right)^{1/\alpha}, & 0 \leq x \leq e/(\alpha + e) \\ -\ln[((1/\alpha) + (1/e))(1 - x)], & e/(\alpha + e) < x \leq 1. \end{cases}$$

Or,

$$\frac{f_X(x)}{cg_Y(x)} = \begin{cases} e^{-x}, & 0 \leq x \leq 1 \\ x^{\alpha-1}, & x > 1. \end{cases}$$

Il découle de ce précède l'algorithme de simulation suivant.

1. Simuler deux nombres u_1 et u_2 d'une $U(0, 1)$.
2. Poser $y = G_Y^{-1}(u_1)$.
3. Si

$$u_2 \leq \begin{cases} e^{-y}, & 0 \leq y \leq 1 \\ y^{\alpha-1}, & y > 1, \end{cases}$$

alors poser $x = y$. Sinon, retourner à l'étape 1.

2.15 Vous devez simuler des observations de la fonction de densité de probabilité $f_X(x) = x^{\alpha-1}e^{-x}/\Gamma(\alpha)$ avec $\alpha \geq 1$. Or, vous savez que

$$f_X(x) \leq \frac{\alpha^\alpha}{\Gamma(\alpha)} e^{1-\alpha} \frac{1}{\alpha} e^{-x/\alpha}, \quad x > 0,$$

d'où $f_X(x) \leq c g_Y(x)$ avec

$$c = \frac{\alpha^\alpha}{\Gamma(\alpha)} e^{1-\alpha}$$

et

$$g_Y(x) = \frac{1}{\alpha} e^{-x/\alpha}.$$

Ainsi, $Y \sim \text{Exponentielle}(1/\alpha)$. Soit y une observation de la variable aléatoire Y et u une observation d'une loi $U(0, 1)$. Selon l'algorithme d'acceptation-rejet, on accepte la valeur y comme observation d'une loi $\text{Gamma}(\alpha, 1)$ avec $\alpha \geq 1$ si

$$\begin{aligned} u &\leq \frac{f_X(y)}{c g_Y(y)} = y^{\alpha-1} \frac{e^{-y(1-1/\alpha)}}{\alpha^{\alpha-1} e^{-(\alpha-1)}} \\ &\Leftrightarrow \\ u^{1/(\alpha-1)} &\leq \left(\frac{y}{\alpha}\right) \frac{e^{-y/\alpha}}{e^{-1}} \\ &\Leftrightarrow \\ \ln u &\leq (\alpha-1) \left[\ln\left(\frac{y}{\alpha}\right) - \frac{y}{\alpha} + 1 \right] \\ &\Leftrightarrow \\ -\ln u &> (\alpha-1) \left[\frac{y}{\alpha} - \ln\left(\frac{y}{\alpha}\right) - 1 \right]. \end{aligned}$$

Or, tant la distribution de $-\ln U$ que celle de Y/α est une exponentielle de moyenne 1, d'où l'algorithme donné dans l'énoncé.

2.16 Il faut déterminer le nombre d'observations provenant de chacune des k distributions $F_1, \dots, F_k$ sachant que la probabilité qu'une observation provienne de la distribution j est p_j . La distribution conjointe du nombre d'observations de chaque distribution est une multinomiale de paramètres $p_1, \dots, p_k$ dont la fonction de masse de probabilité est :

$$\Pr[X_1 = x_1, \dots, X_k = x_k] = \frac{n!}{x_1! \cdots x_k!} \prod_{j=1, \dots, k} p_j^{x_j},$$

où $n = x_1 + \dots + x_k$. La fonction `rmultinom` de R permet de simuler des valeurs $x_1, \dots, x_k$ pour n et $p_1, \dots, p_k$ données. C'est la stratégie employée par la fonction `rmixture` de **actuar**.

2.17 Dans chacun des cas ci-dessous, nous considérons que l'objet `n` contient le nombre de valeurs à simuler.

- a) Il s'agit de simuler des observations d'un mélange discret d'une Exponentielle(2) et d'une Gamma(2, 1/3). Nous pouvons procéder en suivant la procédure explicitée à la [section 2.10](#) ou avec la fonction `rmixture` du paquetage **actuar**.

```
> n1 <- rbinom(1, n, 0.3)
> sample(c(rexp(n1, 2),
+         rgamma(n - n1, 2, scale = 3)))

> rmixture(n, c(0.3, 0.7),
+         expression(rexp(2), rgamma(2, scale = 3)))
```

- b) Le mélange discret est, ici, celui d'une distribution dégénérée en $x = 0$ et d'une Poisson de paramètre $\lambda = 2$. La simulation « manuelle » est en tous points similaire à ce que nous avons fait en a). Cependant, à moins de créer une fonction de « simulation » d'une distribution dégénérée, nous ne pouvons utiliser directement la fonction `rmixture`.

```
> n1 <- rbinom(1, n, 0.5647)
> sample(c(rep(0, n1), rpois(n - n1, 2)))
```

Le paquetage **actuar** fournit néanmoins une fonction pour simuler des observations d'un tel mélange. En effet, celui-ci est simplement une distribution de Poisson avec une probabilité en $x = 0$ de $p_0 = 0,5647 + 0,4353e^{-2} = 0,6236$, une distribution autrement connue sous le nom de Poisson zéro modifiée. Les fonctions avec racine `zmpois` de **actuar** permettent de travailler avec cette distribution.

```
> rzmpois(n, 2, p0 = 0.6236)
```

- c) Nous devons simuler des observations de la distribution Poisson-inverse gaussienne qui, par ailleurs, est définie par le mélange continu présenté dans la question. Nous pouvons procéder « manuellement » à l'aide de la fonction `rinvgauss` du paquetage **actuar**, ou encore plus directement avec la fonction `rpoisinvgauss` du même paquetage.

```
| > rpois(n, rinvgauss(n, 2, 1))
```

```
| > rpoisinvgauss(n, 2, 1)
```

- d) Nous devons simuler des observations d'une distribution Poisson composée. Le paquetage **actuar** fournit la fonction `rcomppois` pour ce faire.

```
| > sapply(rpois(n, 2),  
| +       function(n) sum(rlnorm(n, 7, 1)))
```

```
| > rcomppois(n, 2, rlnorm(7, 1))
```

- e) Il faut cette fois simuler des observations d'une distribution binomiale négative composée. Le paquetage **actuar** fournit la fonction `rcompound` pour ce cas général.

```
| > sapply(rnbinom(n, 2, 0.3),  
| +       function(n) sum(rlnorm(n, 7, 1)))
```

```
| > rcompound(n, rnbinom(2, 0.3), rlnorm(7, 1))
```

Chapitre 3

3.1 On a

$$\begin{aligned}\theta &= \int_0^1 \ln(5u + 4) dx \\ &= E[\ln(5U + 4)],\end{aligned}$$

où $U \sim U(0, 1)$, ou encore simplement

$$\theta = E[\ln(X)],$$

où $X \sim U(4, 9)$. Une approximation de θ est

$$\hat{\theta} = \frac{1}{n} \sum_{i=1}^n \ln(x_i),$$

où $x_1, \dots, x_n$ est un échantillon aléatoire d'une distribution $U(4, 9)$. Voici le résultat d'une évaluation avec R.

```
> mean(log(runif(1e6, 4, 9)))
[1] 1.846345
```

3.2 Posons

$$\begin{aligned}\theta &= \int_0^1 \int_0^1 e^{2xy} \ln(3x + y^2) dx dy \\ &= E[e^{2XY} \ln(3X + Y^2)],\end{aligned}$$

où $X \sim U(0, 1)$, $Y \sim U(0, 1)$ et X et Y sont indépendantes. Ainsi, leur densité conjointe est uniforme sur $(0, 1) \times (0, 1)$. Une approximation de θ est

$$\hat{\theta} = \frac{1}{n} \sum_{i=1}^n e^{2x_i y_i} \ln(3x_i + y_i^2)$$

où $x_1, \dots, x_n$ et $y_1, \dots, y_n$ sont deux échantillons aléatoires indépendants d'une distribution $U(0, 1)$. Voici le résultat d'une évaluation avec R.

```
> x <- runif(1e6)
> y <- runif(1e6)
> mean(exp(2 * x * y) * log(3 * x + y^2))
[1] 1.204604
```

- 3.3 a) Soit le changement de variable $u = e^{-x/2} \Leftrightarrow x = -\ln u^2$, d'où $-2du = e^{-x/2} dx$. On a donc

$$\begin{aligned}\theta &= \int_0^1 2(-\ln u^2)^2 \sin(-\pi \ln u^2) du \\ &= 2E[(-\ln U^2)^2 \sin(-\pi \ln U^2)],\end{aligned}$$

où $U \sim U(0, 1)$. Une estimation de θ est

$$\hat{\theta} = \frac{2}{n} \sum_{i=1}^n (-\ln u_i^2)^2 \sin(-\pi \ln u_i^2),$$

où $u_1, \dots, u_n$ est un échantillon aléatoire d'une distribution $U(0, 1)$. Voici le résultat d'une évaluation avec R.

```
> u <- runif(1e6)
> 2 * mean((-log(u^2))^2 * sin(pi * (-log(u^2))))
[1] -0.06134769
```

- b) On remarque que la fonction à intégrer contient, à une constante près, la fonction de densité de probabilité d'une loi Gamma(3, 1/2). Ainsi,

$$\begin{aligned}\theta &= 16 \int_0^\infty \sin(\pi x) \frac{(1/2)^3}{\Gamma(3)} x^2 e^{-x/2} dx \\ &= 16E[\sin(\pi X)],\end{aligned}$$

où $X \sim \text{Gamma}(3, 1/2)$. Une estimation de θ est donc

$$\hat{\theta} = \frac{16}{n} \sum_{i=1}^n \sin(\pi x_i)$$

où $x_1, \dots, x_n$ est un échantillon aléatoire d'une Gamma(3, 1/2). Voici le résultat d'une évaluation avec R.

```
> 16 * mean(sin(pi * rgamma(1e6, 3, 0.5)))
[1] -0.0758969
```

- 3.4** Vous pouvez effectuer l'estimation avec R simplement avec les expressions suivantes.

```
> x <- replicate(10000, rnorm(25))
> mean(apply(x, 2, function(x) sort(x)[5]))
[1] -0.905777
```

Chapitre 4

La notation x_b signifie que le nombre x est en base b . On omet généralement b pour les nombres en base 10.

- 4.1** L'algorithme de conversion des nombres décimaux en une base b se résume essentiellement à ceci pour la partie entière :

1. les chiffres du nombre en base b sont obtenus de droite à gauche en prenant le reste de divisions par b ;
2. on divise par b d'abord le nombre décimal d'origine, puis la partie entière de la division précédente, jusqu'à ce que celle-ci soit égale à 0.

On a donc les résultats suivants.

a) Conversion en base 6 :

$$119 \div 6 = 19 \text{ reste } 5$$

$$19 \div 6 = 3 \text{ reste } 1$$

$$3 \div 6 = 0 \text{ reste } 3,$$

$$\text{d'où } 119 \equiv 315_6.$$

Conversion en binaire :

$$119 \div 2 = 59 \text{ reste } 1$$

$$59 \div 2 = 29 \text{ reste } 1$$

$$29 \div 2 = 14 \text{ reste } 1$$

$$14 \div 2 = 7 \text{ reste } 0$$

$$7 \div 2 = 3 \text{ reste } 1$$

$$3 \div 2 = 1 \text{ reste } 1$$

$$1 \div 2 = 0 \text{ reste } 1,$$

$$\text{d'où } 119 \equiv 1110111_2.$$

b) Conversion en base 6 :

$$343 \div 6 = 57 \text{ reste } 1$$

$$57 \div 6 = 9 \text{ reste } 3$$

$$9 \div 6 = 1 \text{ reste } 3$$

$$1 \div 6 = 0 \text{ reste } 1,$$

$$\text{d'où } 343 \equiv 1331_6.$$

Conversion en binaire :

$$343 \div 2 = 171 \text{ reste } 1$$

$$171 \div 2 = 85 \text{ reste } 1$$

$$85 \div 2 = 42 \text{ reste } 1$$

$$42 \div 2 = 21 \text{ reste } 0$$

$$21 \div 2 = 10 \text{ reste } 1$$

$$10 \div 2 = 5 \text{ reste } 0$$

$$5 \div 2 = 2 \text{ reste } 1$$

$$2 \div 2 = 1 \text{ reste } 0$$

$$1 \div 2 = 0 \text{ reste } 1,$$

$$\text{d'où } 343 \equiv 101010111_2.$$

c) Conversion en base 6 :

$$96 \div 6 = 16 \text{ reste } 0$$

$$16 \div 6 = 2 \text{ reste } 4$$

$$2 \div 6 = 0 \text{ reste } 2,$$

$$\text{d'où } 96 \equiv 240_6.$$

Conversion en binaire :

$$96 \div 2 = 48 \text{ reste } 0$$

$$48 \div 2 = 24 \text{ reste } 0$$

$$24 \div 2 = 12 \text{ reste } 0$$

$$12 \div 2 = 6 \text{ reste } 0$$

$$6 \div 2 = 3 \text{ reste } 0$$

$$3 \div 2 = 1 \text{ reste } 1$$

$$1 \div 2 = 0 \text{ reste } 1,$$

$$\text{d'où } 96 \equiv 1100000_2.$$

d) Conversion en base 6 :

$$43 \div 6 = 7 \text{ reste } 1$$

$$7 \div 6 = 1 \text{ reste } 1$$

$$1 \div 6 = 0 \text{ reste } 1,$$

$$\text{d'où } 43 \equiv 111_6.$$

Conversion en binaire :

$$43 \div 2 = 21 \text{ reste } 1$$

$$21 \div 2 = 10 \text{ reste } 1$$

$$10 \div 2 = 5 \text{ reste } 0$$

$$5 \div 2 = 2 \text{ reste } 1$$

$$2 \div 2 = 1 \text{ reste } 0$$

$$1 \div 2 = 0 \text{ reste } 1,$$

$$\text{d'où } 43 \equiv 101011_2.$$

4.2 On fait les deux premières conversions à l'aide de la définition d'un nombre hexadécimal, puis les deux dernières à l'aide de l'algorithme de conversion des nombres en base b vers la base 10.

a) $A1B_{16} = 10 \times 16^2 + 1 \times 16 + 11 = 2\,587$

b) $12A_{16} = 1 \times 16^2 + 2 \times 16 + 10 = 298$

c) $B41_{16} = (11 \times 16 + 4) \times 16 + 1 = 2\,881$

d) $BAFFE_{16} = (((((11 \times 16 + 10) \times 16) + 15) \times 16) + 15) \times 16 + 14 = 765\,950$

4.3 La généralisation de l'algorithme de conversion des nombres en base b vers la base 10 à la conversion d'un nombre

$$x = x_{m-1}x_{m-2} \cdots x_1x_0$$

en base $[b_{m-1} \dots b_0]$ vers la base 10 est la suivante (nombre entiers seulement) :

1. Poser $x = 0$.
2. Pour $i = m - 1, m - 2, \dots, 0$, faire les étapes suivantes.
 - i) Trouver d_i , le nombre décimal correspondant au symbole x_i .
 - ii) Poser $x = xb_{i-1} + d_i$, avec $b_{-1} = 1$.

Cet algorithme permet de trouver les formules demandées.

- a) On trouve la position de l'élément a_{ijk} dans l'ordre de la liste des éléments du tableau en convertissant le nombre $[i - 1 \ j - 1 \ k - 1]$ de la base $[I \ J \ K]$ à la base 10, puis en additionnant 1. À l'aide de l'algorithme ci-dessus, on obtient

$$[((i - 1) \times J + j - 1) \times K + k - 1] + 1 = k + K(j - 1 + J(i - 1))$$

- b) Dans l'ordre R, on convertit le nombre $[k-1 \ j-1 \ i-1]$ exprimé dans la base $[K \ J \ I]$ en base 10. On obtient alors

$$[((k-1) \times J + j-1) \times I + i-1] + 1 = i + I(j-1 + J(k-1)).$$

- 4.4 a) Par la valeur du bit fort, on sait que $\boxed{1} \boxed{0000000}$ est un nombre négatif. Pour retrouver sa valeur absolue, on soustrait d'abord 1 (ce qui donne 01111111) et on inverse les bits : 10000000. Or $10000000_2 = 2^7 = 128$. Le nombre décimal correspondant à $\boxed{1} \boxed{0000000}$ en notation en complément à deux est donc -128 .

- b) Tout d'abord, $15_{10} = 1111_2$ et $5_{10} = 101_2$.

- i) Selon la notation « naturelle », on aurait pour 15 et -5 les représentations sur 7 bits suivantes, dans l'ordre :

$$\boxed{0} \boxed{0001111} \quad \text{et} \quad \boxed{1} \boxed{0000101}.$$

Or, en additionnant en binaire, on obtient

$$\begin{array}{r} 00001111 \\ + 10000101 \\ \hline 10010100 \end{array}$$

et $10010100_2 = -20_{10} \neq 15 + (-5)$.

- ii) En notation en complément à deux, on a plutôt les représentations suivantes pour 15 et -5 :

$$\boxed{0} \boxed{0001111} \quad \text{et} \quad \boxed{1} \boxed{1111011}.$$

En additionnant en binaire et en ignorant le neuvième bit qui apparaîtrait à gauche suite à la retenue, on obtient :

$$\begin{array}{r} 00001111 \\ + 11111011 \\ \hline 00001010 \end{array}$$

Toujours en complément à deux, $0001010_2 = 10_{10} = 15 + (-5)$. On voit donc que l'addition en complément à deux donne directement le bon résultat.

- c) Voir les tableaux C.1 et C.2. La troisième colonne des tableaux est obtenue par l'opération $-(2^n - x)$, où $n = 8$ ou $n = 16$ et x est la valeur dans la seconde colonne.
- d) De la partie c), l'ensemble des nombres admissibles pour le type `int` est

$$\{-2^{15}, \dots, 2^{15} - 1\} = \{-32\,768, \dots, 32\,767\}.$$

TAB. C.1 – Séquences des nombres en notation en complément à deux sur 8 bits

Bits	Valeur brute	Valeur en complément à deux
0000 0000	0	0
0000 0001	1	1
0000 0010	2	2
0111 1110	126	126
0111 1111	127	127
1000 0000	128	−128
1000 0001	129	−127
1000 0010	130	−126
1111 1110	254	−2
1111 1111	255	−1

TAB. C.2 – Séquences des nombres en notation en complément à deux sur 16 bits

Bits	Valeur brute	Valeur en complément à deux
0000 0000 0000 0000	0	0
0000 0000 0000 0001	1	1
0000 0000 0000 0010	2	2
0111 1111 1111 1110	32 766	32 766
0111 1111 1111 1111	32 767	32 767
1000 0000 0000 0000	32 768	−32 768
1000 0000 0000 0001	32 769	−32 767
1000 0000 0000 0010	32 770	−32 766
1111 1111 1111 1110	65 534	−2
1111 1111 1111 1111	65 535	−1

Par analogie, on trouve que l'ensemble des entiers admissibles pour le type `long` est

$$\{-2^{31}, \dots, 2^{31} - 1\} = \{-2\,147\,483\,648, \dots, 2\,147\,483\,647\}.$$

(C'est le type de nombre qui était utilisé par YouTube pour comptabiliser le nombre de visionnements d'une vidéo jusqu'à ce que *Gangnam Style* excède 2 147 483 647. Le nombre de visionnements est désormais stocké sous forme d'entier sur 64 bits (`long long`), ce qui a fait passer le maximum à $2^{63} - 1 = 9\,223\,372\,036\,854\,775\,808$.)

4.5 Voir la [section 4.5](#). Les calculs sont exactement les mêmes que pour les nombres en double précision.

4.6 Dans les égalités ci-dessous, le côté droit est en binaire.

a) Premièrement, $1234 \equiv 10011010010_2$. On a donc

$$\begin{aligned} -1\,234 &= (-1)^1 \times 2^{10} \times 1,0011010010 \\ &= (-1)^1 \times 2^{137-127} \times 1,0011010010. \end{aligned}$$

Or, puisque $137 \equiv 10001001_2$, on a la représentation en simple précision

$$\boxed{1} \boxed{10001001} \boxed{001101001000000000000000}$$

b) On a $55 \equiv 110111_2$, d'où

$$\begin{aligned} 55 &= (-1)^0 \times 2^5 \times 1,10111 \\ &= (-1)^0 \times 2^{132-127} \times 1,10111. \end{aligned}$$

Or, puisque $132 \equiv 10000100_2$, on a la représentation en simple précision

$$\boxed{0} \boxed{10000100} \boxed{101110000000000000000000}$$

c) On a $8\,191 \equiv 111111111111_2$ et $139 \equiv 10001011_2$, d'où

$$\begin{aligned} 8\,191 &= (-1)^0 \times 2^{12} \times 1,111111111111 \\ &= (-1)^0 \times 2^{139-127} \times 1,111111111111 \\ &= \boxed{0} \boxed{10001011} \boxed{111111111111000000000000}. \end{aligned}$$

d) On a $10 \equiv 1010_2$ et $130 \equiv 10000010_2$, d'où

$$\begin{aligned} -10 &= (-1)^1 \times 2^3 \times 1,010 \\ &= (-1)^1 \times 2^{130-127} \times 1,010 \\ &= \boxed{1} \boxed{10000010} \boxed{010000000000000000000000}. \end{aligned}$$

Le nombre précédent est

0 00111101 100100000111111111111111,

soit

$$2^{-66}(1 + 2^{-1} + 2^{-4} + 2^{-9} - 2^{-23}) \equiv 2,120\,229\,185 \times 10^{-20}.$$

c) Le nombre suivant est

0 10000100 100100001000000000000001,

soit

$$2^5(1 + 2^{-1} + 2^{-4} + 2^{-9} + 2^{-23}) \equiv 50,062\,503\,815.$$

Le nombre précédent est

0 10000100 100100000111111111111111,

soit

$$2^5(1 + 2^{-1} + 2^{-4} + 2^{-9} - 2^{-23}) \equiv 50,062\,496\,185.$$

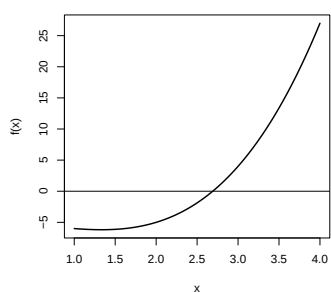
On remarque que les nombres sont beaucoup plus éloignés les uns des autres ici qu'en a).

Chapitre 5

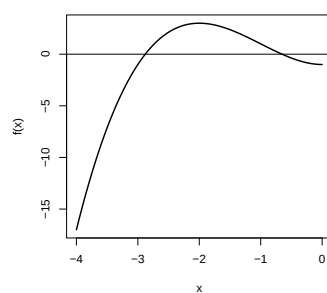
5.2 Les solutions suivantes utilisent nos fonctions de résolution d'équations à une variable. Les valeurs intermédiaires sont affichées pour montrer la convergence. La [figure C.9](#) contient les graphiques des cinq fonctions pour les intervalles mentionnés dans l'énoncé.

a) La fonction n'a qu'une seule racine dans $[1, 4]$.

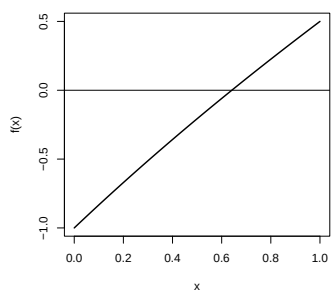
```
> f1 <- function(x) x^3 - 2 * x^2 - 5
> f1p <- function(x) 3 * x^2 - 4 * x
> bisection(f1, lower = 1, upper = 4, echo = TRUE)
[1] 1.000 4.000 2.500 -1.875
[1] 2.5000 4.0000 3.2500 8.2031
[1] 2.5000 3.2500 2.8750 2.2324
[1] 2.500000 2.875000 2.687500 -0.034424
[1] 2.6875 2.8750 2.7812 1.0432
[1] 2.68750 2.78125 2.73438 0.49078
```



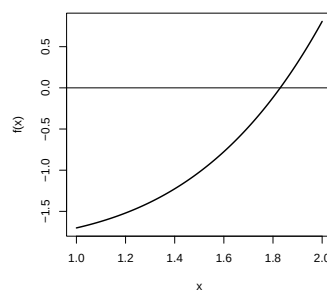
(a) $f(x) = x^3 - 2x^2 - 5$



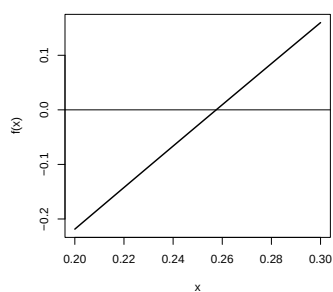
(b) $f(x) = x^3 + 3x^2 - 1$



(c) $f(x) = x - 2^{-x}$



(d) $f(x) = e^x + 2^{-x} + 2 \cos x - 6$



(e) $f(x) = e^x - x^2 + 3x - 2$

FIG. C.9 – Fonctions de l'exercice 5.2

```

[1] 2.68750 2.73438 2.71094 0.22481
[1] 2.687500 2.710938 2.699219 0.094355
[1] 2.687500 2.699219 2.693359 0.029757
[1] 2.6875000 2.6933594 2.6904297 -0.0023855
[1] 2.690430 2.693359 2.691895 0.013673
[1] 2.6904297 2.6918945 2.6911621 0.0056403
[1] 2.6904297 2.6911621 2.6907959 0.0016266
[1] 2.69042969 2.69079590 2.69061279 -0.00037968
[1] 2.6906128 2.6907959 2.6907043 0.0006234
[1] 2.69061279 2.69070435 2.69065857 0.00012185
[1] 2.69061279 2.69065857 2.69063568 -0.00012892
[1] 2.6906e+00 2.6907e+00 2.6906e+00 -3.5365e-06
[1] 2.6906e+00 2.6907e+00 2.6907e+00 5.9155e-05
[1] 2.6906e+00 2.6907e+00 2.6906e+00 2.7809e-05
[1] 2.6906e+00 2.6906e+00 2.6906e+00 1.2136e-05
$root
[1] 2.6906

$nb.iter
[1] 21
> nr(f1, f1p, start = 2.5, echo = TRUE)
[1] 2.5
[1] 2.7143
[1] 2.691
[1] 2.6906
$root
[1] 2.6906

$nb.iter
[1] 3
> secante(f1, start0 = 1, start1 = 4, echo = TRUE)
[1] 1.5455
[1] 1.9969
[1] 4.1051
[1] 2.2947
[1] 2.4787
[1] 2.7514
[1] 2.6831
[1] 2.6904

```



```
[1] 2.6906
$root
[1] 2.6906

$nb.iter
[1] 9
```

- b) La fonction possède deux racines dans $[-4, 0]$. La convergence se fera vers l'une ou l'autre selon la position de la ou des valeurs de départ par rapport à l'extremum de la fonction dans l'intervalle. Ici, il s'agit d'un maximum $x = -2$. Ainsi, avec des valeurs de départ inférieures au maximum, on trouve la première racine :

```
> f2 <- function(x) x^3 + 3 * x^2 - 1
> f2p <- function(x) 3 * x^2 + 6 * x
> bisection(f2, lower = -3, upper = -2.8, TRUE)
$root
[1] -2.9

$nb.iter
[1] 1

> nr(f2, f2p, start = -3, TRUE)
$root
[1] -2.8889

$nb.iter
[1] 0

> secante(f2, start0 = -3, start1 = -2, TRUE)
$root
[1] -2.75

$nb.iter
[1] 0
```

Pour trouver la seconde racine, on utilise des valeurs de départ supérieures au maximum :

```
> bisection(f2, lower = -1, upper = 0.5, TRUE)
$root
[1] -0.625
```

```
$nb.iter
[1] 2
> nr(f2, f2p, start = -1, TRUE)
$root
[1] -0.66667

$nb.iter
[1] 0
> secante(f2, start0 = -2, start1 = -1, TRUE)
$root
[1] -0.63636

$nb.iter
[1] 1
```

On remarquera que les deux valeurs de départ de la méthode de la sécante n'ont pas à se trouver de part et d'autre de la racine.

- c) La fonction n'a qu'une seule racine dans $[0, 1]$ et elle est légèrement supérieure à 0,6.

```
> f3 <- function(x) x - 2^(-x)
> f3p <- function(x) 1 + log(2) * 2^(-x)
> bisection(f3, lower = 0.6, upper = 0.65, TRUE)
$root
[1] 0.625

$nb.iter
[1] 1
> nr(f3, f3p, start = 0.6, TRUE)
$root
[1] 0.641

$nb.iter
[1] 0
> secante(f3, start0 = 0.6, start1 = 0.65, TRUE)
$root
[1] 0.64122
```

```
$nb.iter  
[1] 0
```

- d) La fonction n'a qu'une seule racine dans $[1, 2]$ et elle est légèrement supérieure à 1,8.

```
> f4 <- function(x)  
+   exp(x) + 2^(-x) + 2 * cos(x) - 6  
> f4p <- function(x)  
+   exp(x) - 2^(-x) * log(2) - 2 * sin(x)  
> bisection(f4, lower = 1.8, upper = 1.85, TRUE)  
$root  
[1] 1.825  
  
$nb.iter  
[1] 1  
> nr(f4, f4p, start = 1.8, TRUE)  
$root  
[1] 1.8301  
  
$nb.iter  
[1] 0  
> secante(f4, start0 = 1.8, start1 = 1.85, TRUE)  
$root  
[1] 1.8289  
  
$nb.iter  
[1] 0
```

- e) Encore ici, la fonction n'a qu'une seule racine dans l'intervalle mentionné et elle se situe autour de 0,25.

```
> f5 <- function(x) exp(x) - x^2 + 3 * x - 2  
> f5p <- function(x) exp(x) - 2 * x + 3  
> bisection(f5, lower = 0.24, upper = 0.26, TRUE)  
$root  
[1] 0.25  
  
$nb.iter  
[1] 1
```

TAB. C.3 – Valeurs successives de la méthode de bisection pour l'exercice 5.3

n	a_n	b_n	x_n	$f(x_n)$
1	0	2	1	-1
2	1,00	2,00	1,50	0,25
3	1,0000	1,5000	1,2500	-0,4375
4	1,250000	1,500000	1,375000	-0,109375
5	1,37500000	1,50000000	1,43750000	0,06640625
6	1,37500000	1,43750000	1,40625000	-0,02246094
7	1,40625000	1,43750000	1,42187500	0,02172852
8	1,4062500000	1,4218750000	1,4140625000	-0,0004272461
9	1,41406250	1,42187500	1,41796875	0,01063538
10	1,41406250	1,41796875	1,41601562	0,00510025

```

> nr(f5, f5p, start = 0.26, TRUE)
$root
[1] 0.25753

$nb.iter
[1] 0

> secante(f5, start0 = 0.24, start1 = 0.26, TRUE)
$root
[1] 0.25753

$nb.iter
[1] 0

```

5.3 On trouve la racine de $f(x) = x^2 - 2$ dans l'intervalle $[0, 2]$ par la méthode de bisection avec un maximum de dix itérations. Le [tableau C.3](#) contient les valeurs successives de a , b , $x = (a + b)/2$ et $f(x)$. On obtient donc une réponse de 1,41601562, alors que la vraie réponse est le nombre irrationnel $\sqrt{2} = 1,414214$.

5.4 On a que

$$\begin{aligned}
 x_n - x_{n-1} &= \sum_{k=1}^n \frac{1}{k} - \sum_{k=1}^{n-1} \frac{1}{k} \\
 &= \frac{1}{n},
 \end{aligned}$$

d'où $\lim_{n \rightarrow \infty} (x_n - x_{n-1}) = \lim_{n \rightarrow \infty} n^{-1} = 0$. Or, $\sum_{k=1}^n k^{-1}$ est la série harmonique qui est connue pour diverger. On peut, par exemple, justifier ceci par le fait que l'intégrale

$$\int_1^{\infty} \frac{1}{x} dx$$

diverge elle-même. Cet exercice illustre donc que le critère $|x_n - x_{n-1}| < \varepsilon$ peut être satisfait même pour une série divergente.

- 5.5 a) On considère la fonction $g(x) = 2^{-x}$ dans l'intervalle $[0, 1]$. En premier lieu, $1/2 \leq 2^{-x} \leq 1$ pour $0 \leq x \leq 1$, donc $g(x) \in [0, 1]$. De plus, $g'(x) = -2^{-x}(\ln 2)$, d'où

$$|g'(x)| = \frac{\ln 2}{2^x} \leq \ln 2 < 1, \quad 0 \leq x \leq 1.$$

Par le [théorème 5.1](#), la fonction g possède un point fixe unique dans l'intervalle $[0, 1]$.

- b) Le graphique de la fonction g se trouve à la [figure C.10](#). On constate que la fonction a effectivement un point fixe unique dans l'intervalle $[0, 1]$ et que celui-ci se trouve près de $x = 0,6$. On peut donc utiliser cette valeur comme point de départ de l'algorithme du point fixe :

```
> pointfixe(function(x) 2^(-x), start = 0.6)
$fixed.point
[1] 0.64119

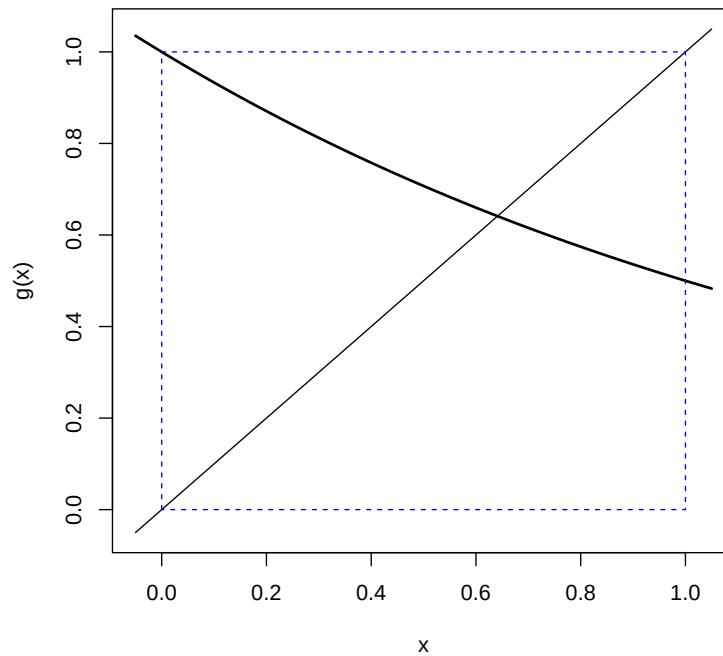
$nb.iter
[1] 15
```

On remarque que la réponse est indépendante de la valeur de départ :

```
> pointfixe(function(x) 2^(-x), start = 0)
$fixed.point
[1] 0.64119

$nb.iter
[1] 18

> pointfixe(function(x) 2^(-x), start = 1)
$fixed.point
[1] 0.64119
```

FIG. C.10 - Fonction $g(x) = 2^{-x}$ dans $[0, 1]$

```

$nb.iter
[1] 17
> pointfixe(function(x) 2^(-x), start = 2)
$fixed.point
[1] 0.64119

$nb.iter
[1] 18

```

5.6 Les cinq fonctions sont les suivantes :

$$g_1(x) = x - x^3 - 4x^2 + 10$$

$$g_2(x) = \left(\frac{10}{x} - 4x\right)^{1/2}$$

$$\begin{aligned}
 g_3(x) &= \frac{1}{2}(10 - x^3)^{1/2} \\
 g_4(x) &= \left(\frac{10}{4 + x} \right)^{1/2} \\
 g_5(x) &= x - \frac{x^3 + 4x^2 - 10}{3x^2 + 8x}.
 \end{aligned}$$

Soit $f(x) = x^3 + 4x^2 - 10$. On peut réécrire l'équation $f(x) = 0$ de différentes façons :

- a) $x = x - f(x)$;
- b) $x^3 = 10 - 4x^2 \Rightarrow x^2 = 10/x - 4x \Rightarrow x = (10/x - 4x)^{1/2}$;
- c) $4x^2 = 10 - x^3 \Rightarrow x^2 = (10 - x^3)/4 \Rightarrow x = \frac{1}{2}(10 - x^3)^{1/2}$;
- d) $4x^2 + x^3 = 10 \Rightarrow x^2 = 10/(4 + x) \Rightarrow x = (10/(4 + x))^{1/2}$;
- e) $x = x - f(x)/f'(x)$.

Les fonctions g_1 à g_5 correspondent, dans l'ordre, au côté droit de chacune des équations ci-dessus. On remarquera que la fonction g_5 est celle préconisée par la méthode de Newton-Raphson — et celle qui converge le plus rapidement.

- 5.7** La fonction $g(x) = 4x^2 - 14x + 14$ est une parabole ayant deux points fixes. On observe graphiquement que l'un se trouve en $x = 2$ et que $g(1,5) = g(2)$. Or, pour toute valeur de départ $x_0 < 1,5$ de l'algorithme du point fixe, on constate que les valeurs de $x_1, x_2, \dots$ se retrouveront de plus en plus loin sur la branche droite de la parabole. Il en va de même pour tout $x_0 > 2$. La procédure diverge donc pour de telles valeurs de départ. En revanche, il est facile de vérifier graphiquement que la procédure converge pour toute valeur de départ dans l'intervalle $[1,5, 2]$. Si $x_0 = 1,5$, on obtient le point fixe $x = 2$ après une seule itération.

- 5.8** En premier lieu, on remarque que les fonctions g_1, g_2 et g_3 sont développées à partir de l'équation $x^3 = 21$ pour calculer $\sqrt[3]{21}$ par la méthode du point fixe. La [figure C.11](#) présente ces trois fonctions. On a

$$\begin{aligned}
 g'_1(x) &= \frac{20}{21} - \frac{2}{x^3} \\
 g'_2(x) &= \frac{2}{3} - \frac{14}{x^3} \\
 g'_3(x) &= 1 - \frac{2x^5 - 84x^3 + 21x^2 + 441}{(x^2 - 21)^2}.
 \end{aligned}$$

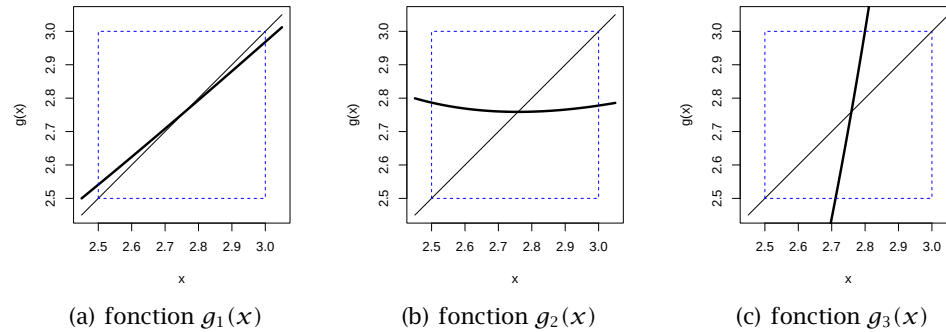


FIG. C.11 – Fonctions de l'exercice 5.8

Considérons l'intervalle $[2,5, 3]$ autour du point fixe. On a que $|g'_2(x)| < |g'_1(x)|$ pour tout x dans cet intervalle, donc la procédure du point fixe converge plus rapidement pour g_2 (la fonction de la méthode de Newton-Raphson). Cela se vérifie aisément sur les graphiques de la figure C.11. Toujours à l'aide des graphiques, on voit que la procédure diverge avec la fonction g_3 .

- 5.9** On considère la fonction $g(x) = 2^{-x}$ dans l'intervalle $[\frac{1}{3}, 1]$. En premier lieu, on a $g(x) \in [\frac{1}{2}, 1/\sqrt[3]{2}] \subset [\frac{1}{3}, 1]$ pour $x \in [\frac{1}{3}, 1]$. De plus, on a $g'(x) = -2^{-x}(\ln 2)$. Puisque $\ln 2 < 1$, on a aussi que $|g'(x)| = g(x)(\ln 2) \in [\frac{1}{3}, 1]$ pour $x \in [\frac{1}{3}, 1]$.

Par le théorème du point fixe, la fonction g possède donc un point fixe unique dans l'intervalle $[\frac{1}{3}, 1]$. La valeur de ce point fixe est

```
> pointfixe(function(x) 2^(-x), start = 2/3)
$fixed.point
[1] 0.64119

$nb.iter
[1] 14
```

- 5.10** La condition $|g'(x)| \leq k < 1$ du théorème 5.1 assure l'unicité d'un point fixe. Cette condition est nécessaire pour que la fonction $g(x)$ ne puisse repasser au-dessus de la droite $y = x$ après l'avoir déjà croisée une première fois pour créer un point fixe. Or, il est seulement nécessaire de limiter la croissance de la fonction — soit les valeurs positives de sa pente. Peu importe la vitesse de décroissance de la fonction dans l'intervalle $[a, b]$ (mais toujours sujet à ce que $g(x) \in [a, b]$), la fonc-

tion ne peut croiser la droite $y = x$ qu'une seule fois. La condition $g'(x) \leq k < 1$ est donc suffisante pour assurer l'unicité du point fixe dans $[a, b]$.

5.11 La distance entre deux points (x_1, y_1) et (x_2, y_2) de $\mathbb{R}^2$ est

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}.$$

Par conséquent, la distance entre le point (x, x^2) et le point $(1, 0)$ est une fonction de x

$$d(x) = \sqrt{(x - 1)^2 + (x^2 - 0)^2} = \sqrt{x^4 + x^2 - 2x + 1}.$$

On cherche la valeur de minimisant $d(x)$ ou, de manière équivalente, $d^2(x)$. On doit donc résoudre

$$\frac{d}{dx} d^2(x) = f(x) = 4x^3 + 2x - 2 = 0$$

à l'aide de la méthode de Newton-Raphson. Cela requiert également

$$f'(x) = 12x^2 + 2.$$

Le point sur la courbe $y = x^2$ le plus près du point $(1, 0)$ est donc la limite de la suite

$$x_n = x_{n-1} - \frac{4x_{n-1}^3 + 2x_{n-1} - 2}{12x_{n-1}^2 + 2}, \quad n = 1, 2, \dots,$$

avec x_0 une valeur de départ « près » de la solution. On obtient

```
> nr(function(x) 4 * x^3 + 2 * x - 2,
+   function(x) 12 * x^2 + 2, start = 0.6)
$root
[1] 0.58975

$nb.iter
[1] 2
```

On voit à la [figure C.12](#) que la solution, disons le point $(x^*, (x^*)^2)$, est la projection du point $(1, 0)$ sur la courbe $y = x^2$; autrement dit, la droite passant par $(x^*, (x^*)^2)$ et $(1, 0)$ est perpendiculaire à la tangente de $y = x^2$ en $x = x^*$.

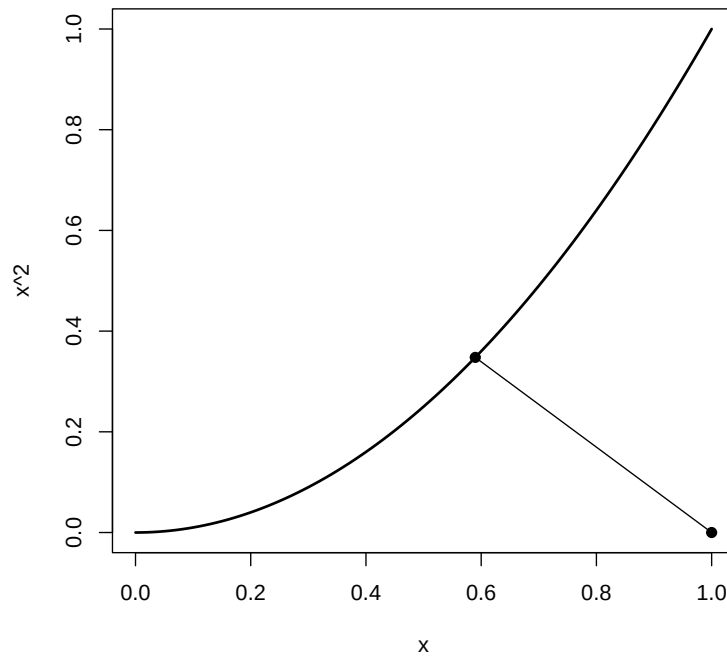


FIG. C.12 - Point de $y = x^2$ le plus près du point (1,0)

5.12 La fonction devra utiliser la méthode de Newton-Raphson pour trouver la racine de $f(i) = 0$, où

$$f(i) = \sum_{t=0}^n \frac{CF_t}{(1+i)^t}$$

et

$$f'(i) = - \sum_{t=0}^n \frac{t CF_t}{(1+i)^{t+1}}.$$

La fonction `tri` de la [figure C.13](#) calcule le taux de rendement interne d'un vecteur de flux financiers en utilisant la fonction de Newton-Raphson `nr`.

```

tri <- function(CF, erreur.max = 1E-6)
{
  if (!any(CF < 0))
    stop("au moins un flux financier doit être
         négatif")

  t <- 0:(length(CF) - 1)

  f <- function(i) sum(CF/(1 + i)^t)
  fp <- function(i) sum((-CF * t)/((1 + i)^(t + 1)))

  nr(f, fp, start = 0.05, TOL = erreur.max)$root
}

```

FIG. C.13 - Fonction R de calcul du taux de rendement interne d'une série de flux financiers

- 5.13** a) On veut estimer par la méthode du maximum de vraisemblance le paramètre θ d'une loi Gamma(3, $1/\theta$), dont la fonction de densité de probabilité est

$$f(x; \theta) = \frac{1}{2\theta^3} x^2 e^{-x/\theta}, \quad x > 0.$$

La fonction de log-vraisemblance d'un échantillon aléatoire $x_1, \dots, x_n$ tiré de cette loi est

$$\begin{aligned}
 l(\theta) &= \sum_{i=1}^n \ln f(x_i; \theta) \\
 &= \sum_{i=1}^n \left(2 \ln x_i - \frac{x_i}{\theta} - \ln 2 - 3 \ln \theta \right) \\
 &= 2 \sum_{i=1}^n \ln x_i - \frac{1}{\theta} \sum_{i=1}^n x_i - n \ln 2 - 3n \ln \theta.
 \end{aligned}$$

On cherche le maximum de la fonction $l(\theta)$, soit la valeur de θ tel que

$$l'(\theta) = \frac{1}{\theta^2} \sum_{i=1}^n x_i - \frac{3n}{\theta} = 0.$$

```

emv.gamma <- function(x, erreur.max = 1E-6)
{
  n <- length(x)
  xsum <- sum(x)
  f <- function(theta) xsum/theta^2 -
    (3 * n)/theta
  fp <- function(theta) -2 * xsum / theta^3 +
    (3 * n)/theta^2
  nr(f, fp, xsum/n/3, TOL = erreur.max)$root
}

```

FIG. C.14 – Fonction R d'estimation du paramètre d'échelle d'une loi gamma par le maximum de vraisemblance

Résoudre cette équation à l'aide de la méthode de Newton-Raphson requiert également

$$l''(\theta) = -\frac{2}{\theta^3} \sum_{i=1}^n x_i + \frac{3n}{\theta^2}.$$

La [figure C.14](#) présente une fonction R pour effectuer le calcul à l'aide de notre fonction de Newton-Raphson `nr`. On remarquera que la somme des valeurs de l'échantillon — qui ne change pas durant la procédure itérative — est calculée une fois pour toute dès le début de la fonction. De plus, on utilise comme valeur de départ l'estimateur des moments de θ , $\bar{x}/3$.

b) On a, par exemple,

```

> x <- rgamma(20, shape = 3, scale = 1000)
> emv.gamma(x)
[1] 835.51

```

5.14 On cherche à résoudre l'équation

$$a_{10|i}^{(12)} = \frac{1 - (1 + i)^{-10}}{i^{(12)}} = 8$$

ou, de manière équivalente,

$$f(i) = \frac{1 - (1 + i)^{-10}}{12[(1 + i)^{1/12} - 1]} - 8 = 0.$$

Pour résoudre le problème à l'aide de la méthode du point fixe, on peut considérer la fonction

$$g_1(i) = i - f(i),$$

mais, comme le graphique de la [figure C.15\(a\)](#) le montre, la procédure itérative divergera. En posant

$$\frac{1 - (1 + i)^{-10}}{8} = 12[(1 + i)^{1/12} - 1],$$

puis en isolant i du côté droit de l'équation, on obtient une alternative

$$g_2(i) = \left(\frac{97 - (1 + i)^{-10}}{96} \right)^{12} - 1.$$

Cette fonction satisfait les hypothèses pour l'existence et l'unicité d'un point fixe, mais sa pente est toutefois près de 1, donc la convergence sera lente; voir la [figure C.15\(b\)](#). Enfin, on peut considérer la fonction

$$g_3(i) = i - \frac{f(i)}{f'(i)}$$

avec

$$f'(i) = \frac{120(1 + i)^{-11}[(1 + i)^{1/12} - 1] - [1 - (1 + i)^{-10}](1 + i)^{-11/12}}{144[(1 + i)^{1/12} - 1]^2}.$$

On remarquera que cette dernière est la fonction utilisée dans la méthode de Newton-Raphson et sa pente est presque nulle, d'où une convergence très rapide; voir la [figure C.15\(c\)](#). On obtient les résultats suivants :

```
> f <- function(i)
+   (1 - (1 + i)^(-10))/(12 * ((1 + i)^(1/12) - 1)) -
+   8
> fp <- function(i)
+   (120 * (1 + i)^(-11) * ((1 + i)^(1/12) - 1) -
+   (1 - (1 + i)^(-10)) * (1 + i)^(-11/12)) /
+   (144 * ((1 + i)^(1/12) - 1)^2)
> g2 <- function(i) ((97 - (1 + i)^(-10))/96)^12 - 1
> g3 <- function(i) i - f(i)/fp(i)
> pointfixe(g2, start = 0.05)
```

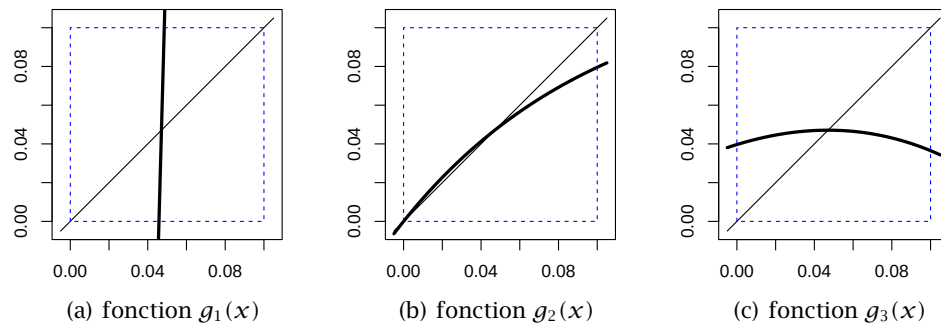


FIG. C.15 - Trois fonctions pour résoudre $a_{\frac{12}{10}i}^{(12)} = 8$ par la méthode du point fixe

```
$fixed.point
[1] 0.047081

$nb.iter
[1] 40
> pointfixe(g3, start = 0.05)
$fixed.point
[1] 0.047081

$nb.iter
[1] 2
```

5.15 Dans tous les cas la fonction R appropriée est `uniroot`. Il s'agit de déterminer un ou des intervalles contenant une racine.

a) La fonction ne compte qu'une racine dans l'intervalle $1 \leq x \leq 4$.

```
> f <- function(x) x^3 - 2 * x^2 - 5
> uniroot(f, lower = 1, upper = 4)
```

b) Comme un simple graphique le démontre, il y a deux racines dans l'intervalle. Pour calculer chacune, nous pouvons utiliser `uniroot` sur deux sous-intervalles distincts.

```
> f <- function(x) x^3 + 3 * x^2 - 1
> curve(f, xlim = c(-4, 0))
```

```
> uniroot(f, lower = -4, upper = -1)
> uniroot(f, lower = -1, upper = 0)
```

c) Il y a une racine dans l'intervalle.

```
> f <- function(x) x - 2^(-x)
> uniroot(f, lower = 0, upper = 1)
```

d) Il y a une racine dans l'intervalle.

```
> f <- function(x) exp(x) + 2^(-x) + 2 * cos(x) - 6
> uniroot(f, lower = 1, upper = 2)
```

e) Il y a une racine dans l'intervalle.

```
> f <- function(x) exp(x) - x^2 + 3 * x - 2
> uniroot(f, lower = 0, upper = 1)
```

5.16 On a

```
> dpareto <- function(x, alpha, lambda)
+   (alpha * lambda^alpha)/(x + lambda)^(alpha + 1)
> f <- function(par, x)
+   -sum(log(dpareto(x, par[1], par[2])))
> optim(c(1, 1000), f, x = x)
```

ou, en utilisant l'Astuce Ripley ([section 5.12](#)) consistant à estimer le logarithme des paramètres pour éviter les soucis de convergence :

```
> dpareto <- function(x, logAlpha, logLambda)
+ {
+   alpha <- exp(logAlpha)
+   lambda <- exp(logLambda)
+   (alpha * lambda^alpha)/(x + lambda)^(alpha+1)
+ }
> optim(c(log(2), log(1000)), f, x = x)
> exp(optim(c(log(2), log(1000)), f, x = x)$par)
```

Chapitre 6

6.1 On fait le cas de la méthode Simpson seulement, la procédure à suivre dans les autres cas étant tout à fait similaire.

On souhaite développer la formule (6.6) à partir de l'approximation sur un sous intervalle donnée par l'équation (6.5) pour l'intervalle $[x_0, x_2]$. De manière plus générale, nous avons

$$\int_{x_{2j}}^{x_{2(j+1)}} f(x) dx \approx \frac{h}{3} [f(x_{2j}) + 4f(x_{2j+1}) + f(x_{2(j+1)})], \quad j = 0, \dots, n-1,$$

d'où

$$\begin{aligned} \int_a^b f(x) dx &\approx \sum_{j=0}^{n-1} \int_{x_{2j}}^{x_{2(j+1)}} f(x) dx \\ &= \frac{h}{3} \sum_{j=0}^{n-1} [f(x_{2j}) + 4f(x_{2j+1}) + f(x_{2(j+1)})] \\ &= \frac{h}{3} \left[f(x_0) + \sum_{j=1}^{n-1} f(x_{2j}) + 4 \sum_{j=0}^{n-1} f(x_{2j+1}) \right. \\ &\quad \left. + \sum_{j=0}^{n-2} f(x_{2(j+1)}) + f(x_{2n}) \right] \\ &= \frac{h}{3} \left[f(a) + 2 \sum_{j=1}^{n-1} f(x_{2j}) + 4 \sum_{j=0}^{n-1} f(x_{2j+1}) + f(b) \right] \end{aligned}$$

puisque, par définition, $f(x_0) = f(a)$ et $f(x_{2n}) = f(b)$.

6.2 Puisque $n = 1$, on peut utiliser directement les formules (6.1), (6.3), (6.5) et (6.7).

a) Point milieu :

$$\int_{0,5}^1 x^4 dx \approx 2(0,25)(0,75)^4 = 0,158203$$

Trapèze :

$$\int_{0,5}^1 x^4 dx \approx \frac{0,5}{2} [(0,5)^4 + 1] = 0,265625$$

Simpson :

$$\int_{0,5}^1 x^4 dx \approx \frac{0,25}{3} [(0,5)^4 + 4(0,75)^4 + 1] = 0,194010$$

Simpson 3/8 :

$$\int_{0,5}^1 x^4 dx \approx \frac{3(0,5/3)}{8} [(0,5)^4 + 3(2/3)^4 + 3(5/6)^4 + 1] = 0,193866$$

b) Point milieu :

$$\int_0^{0,5} \frac{2}{x-4} dx \approx 2(0,25)(-2/3,75) = -0,266667$$

Trapèze :

$$\int_0^{0,5} \frac{2}{x-4} dx \approx \frac{0,5}{2}[(-2/4) + (-2/3,5)] = -0,267857$$

Simpson :

$$\begin{aligned} \int_0^{0,5} \frac{2}{x-4} dx &\approx \frac{0,25}{3}[(-2/4) + 4(-2/3,75) + (-2/3,5)] \\ &= -0,267064 \end{aligned}$$

Simpson 3/8 :

$$\begin{aligned} \int_0^{0,5} \frac{2}{x-4} dx &\approx \frac{3(0,5/3)}{8}[(-2/4) + 3(-12/23) + 3(12/22) + (-2/3,5)] \\ &= -0,267063 \end{aligned}$$

c) Point milieu :

$$\int_1^{1,5} x^2 \ln x dx \approx 2(0,25)(1,75)^2 \ln 1,75 = 0,174331$$

Trapèze :

$$\int_1^{1,5} x^2 \ln x dx \approx \frac{0,5}{2}[0 + (1,5)^2 \ln 1,5] = 0,228074$$

Simpson :

$$\begin{aligned} \int_1^{1,5} x^2 \ln x dx &\approx \frac{0,25}{3}[0 + 4(1,25)^2 \ln 1,25 + (1,5)^2 \ln 1,5] \\ &= 0,192245 \end{aligned}$$

Simpson 3/8 :

$$\begin{aligned} \int_1^{1,5} x^2 \ln x dx &\approx \frac{3(0,5/3)}{8}[0 + (7/6)^2 \ln(7/6) + (4/3)^2 \ln(4/3) \\ &\quad + (1,5)^2 \ln 1,5] = 0,192253 \end{aligned}$$

d) Point milieu :

$$\int_0^{\pi/4} e^{3x} \sin 2x \, dx \approx 2(\pi/8)e^{3\pi/8} \sin(\pi/4) = 1,803915$$

Trapèze :

$$\int_0^{\pi/4} e^{3x} \sin 2x \, dx \approx \frac{\pi/4}{2} [0 + e^{3\pi/4} \sin(\pi/2)] = 4,143260$$

Simpson :

$$\begin{aligned} \int_0^{\pi/4} e^{3x} \sin 2x \, dx &\approx \frac{\pi/8}{3} [0 + 4e^{3\pi/8} \sin(\pi/4) + e^{3\pi/4} \sin(\pi/2)] \\ &= 2,583696 \end{aligned}$$

Simpson 3/8 :

$$\begin{aligned} \int_0^{\pi/4} e^{3x} \sin 2x \, dx &\approx \frac{3(\pi/12)}{8} [0 + 3e^{3\pi/12} \sin(\pi/6) \\ &\quad + 3e^{3\pi/6} \sin(\pi/3) + e^{3\pi/4} \sin(\pi/2)] \\ &= 2,585789 \end{aligned}$$

6.3 Avec la méthode du trapèze et $n = 1$, on a $h = 2 - 0 = 2$ et l'approximation de l'intégrale est

$$f(0) + f(2) = 4.$$

Avec la méthode de Simpson, $h = (2 - 0)/2 = 1$ et l'approximation est

$$\frac{1}{3} [f(0) + 4f(1) + f(2)] = 2.$$

En remplaçant la première équation dans la seconde et en résolvant, on trouve facilement $f(1) = 1/2$.

Chapitre 7

7.1 a) On a une solution unique. Le système d'équations correspondant est

$$x_1 - 3x_2 + 4x_3 = 7$$

$$x_2 + 2x_3 = 2$$

$$x_3 = 5.$$

Par substitution successive, on obtient $x_3 = 5$, $x_2 = 2 - 2(5) = -8$ et $x_1 = 7 + (3)(-8) - 4(5) = -37$.

- b) Le système est sous-déterminé puisqu'il compte plus d'inconnues que d'équations. Il y a donc une infinité de solutions. Le système d'équations correspondant est

$$\begin{aligned}x_1 + 8x_3 - 5x_4 &= 6 \\x_2 + 4x_3 - 9x_4 &= 3 \\x_3 + x_4 &= 2.\end{aligned}$$

Posons la variable libre $x_4 = t$. Ainsi, la solution générale du système d'équations est $x_3 = 2 - t$, $x_2 = 3 - 4(2 - t) + 9t = -5 + 13t$ et $x_1 = 6 - 8(2 - t) + 5t = -10 + 13t$.

- c) Nous avons à l'origine un système d'équations à quatre équations et cinq inconnues. La matrice échelonnée contient une ligne complète de zéros, ce qui indique que la quatrième équation était une combinaison linéaire des trois autres. Ne reste donc qu'un système sous-déterminé à trois équations et cinq inconnues :

$$\begin{aligned}x_1 + x_2 - 2x_3 - 8x_5 &= -3 \\x_3 + x_4 + 6x_5 &= 5 \\x_4 + 3x_5 &= 9.\end{aligned}$$

Ce système a une infinité de solutions et il faudra, pour les exprimer, poser deux variables libres. Les candidates sont les variables correspondant aux colonnes sans un 1 sur la diagonale. Posons $x_2 = s$ et $x_5 = t$. Nous obtenons alors $x_4 = 9 - 3t$, $x_3 = 5 - (9 - 3t) - 6t = -4 - 3t$ et $x_1 = -3 - 7s + 2(-4 - 3t) + 8t = -11 - 7s + 2t$.

- d) La dernière ligne de la matrice échelonnée correspond à l'équation impossible à satisfaire

$$0x_1 + 0x_2 + 0x_3 = 1,$$

d'où le système n'a pas de solution.

- 7.2** a) Solution pour l'élimination gaussienne seulement. La matrice augmentée est

$$\begin{bmatrix} 1 & 1 & 2 & 8 \\ -1 & -2 & 3 & 1 \\ 3 & -7 & 4 & 10 \end{bmatrix}.$$

Les opérations élémentaires à effectuer sur cette matrice pour l'exprimer sous forme échelonnée sont, dans l'ordre :

1. additionner la première ligne à la seconde ;
2. additionner -3 fois la première ligne à la troisième ;
3. multiplier la deuxième ligne par -1 ;
4. additionner 10 fois la deuxième ligne à la troisième ;
5. diviser la troisième ligne par -52 .

On obtient alors la matrice échelonnée

$$\begin{bmatrix} 1 & 1 & 2 & 8 \\ 0 & 1 & -5 & -9 \\ 0 & 0 & 1 & 2 \end{bmatrix},$$

d'où $x_3 = 2$, $x_2 = -9 + 5(2) = 1$ et $x_1 = 8 - 1 - 2(2) = 3$.

- b) Solution pour l'élimination de Gauss-Jordan seulement. La matrice augmentée est

$$\begin{bmatrix} 2 & 2 & 2 & 0 \\ -2 & 5 & 2 & 1 \\ 8 & 1 & 4 & -1 \end{bmatrix}.$$

Les opérations élémentaires à effectuer sur cette matrice pour l'exprimer sous forme échelonnée réduite sont les suivantes :

1. multiplier la première ligne par $\frac{1}{2}$;
2. additionner 2 fois la première ligne à la deuxième ;
3. additionner -8 fois la première ligne à la troisième ;
4. additionner la deuxième ligne à la troisième ;
5. multiplier la deuxième ligne par $\frac{1}{7}$;
6. additionner -1 fois la deuxième ligne à la première.

On obtient alors la matrice échelonnée réduite

$$\begin{bmatrix} 1 & 0 & 3/7 & -1/7 \\ 0 & 1 & 4/7 & 1/7 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

En posant $x_3 = t$, on a la solution générale $x_2 = \frac{1}{7} - \frac{4}{7}t$ et $x_1 = -\frac{1}{7} - \frac{3}{7}t$.

- c) Solution pour l'élimination gaussienne seulement. La matrice augmentée est

$$\begin{bmatrix} 1 & -1 & 2 & -1 & -1 \\ 2 & 1 & -2 & -2 & -2 \\ -1 & 2 & -4 & 1 & 1 \\ 3 & 0 & 0 & -3 & -3 \end{bmatrix}.$$

Les opérations élémentaires à effectuer sur cette matrice pour l'exprimer sous forme échelonnée sont, dans l'ordre :

1. additionner -2 fois la première ligne à la seconde ;
2. additionner la première ligne à la troisième ;
3. additionner -3 fois la première ligne à la quatrième ;
4. additionner -1 fois la deuxième ligne à la quatrième ;
5. multiplier la deuxième ligne par $\frac{1}{3}$;
6. additionner -1 fois la deuxième ligne à la troisième.

On obtient alors la matrice échelonnée

$$\begin{bmatrix} 1 & -1 & 2 & -1 & -1 \\ 0 & 1 & -2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix},$$

d'où $z = s$, $w = t$, $y = 2s$ et $x = -1 + 2s - 2s + t = -1 + t$.

- d) En échangeant les première et troisième équations, on a la matrice augmentée

$$\begin{bmatrix} 6 & 6 & 3 & 5 \\ 3 & 6 & -3 & -2 \\ 0 & -2 & 3 & 1 \end{bmatrix}.$$

Après les opérations élémentaires

1. multiplier la première ligne par $\frac{1}{6}$;
2. additionner -3 fois la première ligne à la deuxième ;
3. multiplier la deuxième ligne par $\frac{1}{3}$;
4. additionner 2 fois la deuxième ligne à la troisième,

on obtient la matrice échelonnée

$$\begin{bmatrix} 1 & 1 & 1/2 & 5/6 \\ 0 & 1 & -3/2 & -3/2 \\ 0 & 0 & 0 & -2 \end{bmatrix}.$$

La troisième équation étant impossible à satisfaire, le système n'a pas de solution.

- 7.3** La procédure de réduction des matrices augmentées sous forme échelonnée est similaire à celle utilisée dans les solutions de l'exercice 7.2. On ne donne, ici que les résultats de cette procédure.

a) La matrice échelonnée est

$$\begin{bmatrix} 1 & -2/5 & 6/5 & 0 \\ 0 & 1 & 27 & 5 \end{bmatrix}$$

et la matrice échelonnée réduite est

$$\begin{bmatrix} 1 & 0 & 12 & 2 \\ 0 & 1 & 27 & 5 \end{bmatrix}.$$

Par conséquent, la solution générale est $x_3 = t$, $x_2 = 5 - 27t$ et $x_1 = 2 - 12t$.

b) La matrice échelonnée réduite est

$$\begin{bmatrix} 1 & -2 & 1 & -4 & 1 \\ 0 & 1 & 6/5 & 6/5 & 1/5 \\ 0 & 0 & 0 & 0 & 6 \end{bmatrix}.$$

Ce système n'a donc pas de solution.

c) La matrice échelonnée est

$$\begin{bmatrix} 1 & 2 & 1/2 & 7/2 & 0 & 7/2 \\ 0 & 0 & 1 & 2 & -1 & 4 \\ 0 & 0 & 0 & 1 & -1 & 3 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

et la matrice échelonnée réduite est

$$\begin{bmatrix} 1 & 2 & 0 & 0 & 3 & -6 \\ 0 & 0 & 1 & 0 & 1 & -2 \\ 0 & 0 & 0 & 1 & -1 & 3 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

Par conséquent, la solution générale est $v = s$, $y = t$, $x = 3 + t$, $w = -2 - t$ et $u = -6 - 3t - 2s$.

7.4 a) Après quelques opérations élémentaires sur les lignes de la matrice augmentée correspondant à ce système d'équations, on obtient

$$\begin{bmatrix} 2 & 1 & 3 & 0 \\ 0 & 3 & -3 & 0 \\ 0 & 0 & 6 & 0 \end{bmatrix},$$

d'où $x_3 = x_2 = x_1 = 0$, soit la solution triviale.

- b) La somme des deux équations donne une nouvelle équation $4x_1 + x_3 = 0$, d'où $x_3 = -4x_1$. De la première équation, on a également $x_2 = -3x_1 - x_3 - x_4 = x_1 - x_4$. En posant $x_1 = -s$ et $x_4 = t$, on a la solution générale $x_3 = 4s$ et $x_2 = -s - t$. Il y a évidemment de multiples autres façons d'exprimer la solution générale en utilisant des variables libres différentes.
- c) La matrice échelonnée de ce système d'équations est

$$\begin{bmatrix} 1 & 0 & -1 & -3 \\ 0 & 1 & 1 & 7/3 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

On a donc $z = 0$ et, en posant $y = t$, $x = -t$, $w = t$.

- 7.5** Après des opérations élémentaires sur la matrice augmentée du système d'équations, on obtient

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 0 & -7 & 14 & -10 \\ 0 & 0 & a^2 - 16 & a - 4 \end{bmatrix}.$$

On constate alors que le système d'équations n'a pas de solution si $a^2 - 16 = 0$ et $a - 4 \neq 0$, donc lorsque $a = -4$. En revanche, si $a^2 - 16 = 0$ et $a - 4 = 0$, soit lorsque $a = 4$, le système a une infinité de solutions. Finalement, si $a^2 - 16 \neq 0 \Leftrightarrow |a| \neq 4$, le système a une solution unique.

- 7.6** La somme (ou la différence) entre deux matrices est définie si les dimensions des matrices sont identiques. Les dimensions du résultat seront les mêmes. Quant au produit, il est défini si le nombre de colonnes de la première matrice est égal au nombre de lignes de la seconde ; le résultat est une matrice dont le nombre de lignes est égal à celui de la première matrice et le nombre de colonnes à celui de la seconde matrice.
- a) Le produit **BA** n'est pas défini.
- b) Le résultat de **AC** est une matrice 4×2 , donc l'opération **AC + D** est définie et le résultat est une matrice 4×2 .
- c) Le résultat de **AE** est une matrice 4×4 , donc l'opération **AE + B** n'est pas définie.
- d) Le produit **AB** n'est pas défini.
- e) La somme **A + B** est définie, tout comme le produit **E(A + B)**. Le résultat est une matrice 5×5 .

- f) Le résultat du produit $\mathbf{AC}$ est une matrice 4×2 , donc $\mathbf{E}(\mathbf{AC})$ est une matrice 5×2 .
- g) Puisque $\mathbf{E}^T$ est une matrice 4×5 , le produit $\mathbf{E}^T \mathbf{A}$ n'est pas défini.
- h) Le résultat de $\mathbf{A}^T + \mathbf{E}$ est une matrice 5×4 , donc $(\mathbf{A}^T + \mathbf{E})\mathbf{D}$ est une matrice 5×2 .

7.7 a) On a

$$\mathbf{D} + \mathbf{E} = \begin{bmatrix} 1 & 5 & 2 \\ -1 & 0 & 1 \\ 3 & 2 & 4 \end{bmatrix} + \begin{bmatrix} 6 & 1 & 3 \\ -1 & 1 & 2 \\ 4 & 1 & 3 \end{bmatrix} = \begin{bmatrix} 7 & 6 & 5 \\ -2 & 1 & 3 \\ 7 & 3 & 7 \end{bmatrix}$$

b) On a

$$\mathbf{AB} = \begin{bmatrix} 3 & 0 \\ -1 & 2 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 4 & -1 \\ 0 & 2 \end{bmatrix} = \begin{bmatrix} 12 & -3 \\ -4 & 5 \\ 4 & 1 \end{bmatrix}.$$

c) On a

$$\begin{aligned} (2\mathbf{D}^T - \mathbf{E})\mathbf{A} &= \left(2 \begin{bmatrix} 1 & -1 & 3 \\ 5 & 0 & 2 \\ 2 & 1 & 4 \end{bmatrix} - \begin{bmatrix} 6 & 1 & 3 \\ -1 & 1 & 2 \\ 4 & 1 & 3 \end{bmatrix} \right) \begin{bmatrix} 3 & 0 \\ -1 & 2 \\ 1 & 1 \end{bmatrix} \\ &= \left(\begin{bmatrix} 2 & -2 & 6 \\ 10 & 0 & 4 \\ 4 & 2 & 8 \end{bmatrix} - \begin{bmatrix} 6 & 1 & 3 \\ -1 & 1 & 2 \\ 4 & 1 & 3 \end{bmatrix} \right) \begin{bmatrix} 3 & 0 \\ -1 & 2 \\ 1 & 1 \end{bmatrix} \\ &= \begin{bmatrix} -4 & -3 & 3 \\ 11 & -1 & 2 \\ 0 & 1 & 5 \end{bmatrix} \begin{bmatrix} 3 & 0 \\ -1 & 2 \\ 1 & 1 \end{bmatrix} \\ &= \begin{bmatrix} -6 & -3 \\ 36 & 0 \\ 4 & 7 \end{bmatrix}. \end{aligned}$$

- d) On a $(4\mathbf{B})\mathbf{C} + 2\mathbf{B} = 2\mathbf{B}(2\mathbf{C} + \mathbf{I})$. Or la somme entre parenthèses n'est pas définie puisque la matrice $\mathbf{C}$ n'est pas carrée.

e) On a

$$\begin{aligned}
 (-\mathbf{AC})^T + 5\mathbf{D}^T &= \left(- \begin{bmatrix} 3 & 0 \\ -1 & 2 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 4 & 2 \\ 3 & 1 & 5 \end{bmatrix} \right)^T + 5 \begin{bmatrix} 1 & 5 & 2 \\ -1 & 0 & 1 \\ 3 & 2 & 4 \end{bmatrix}^T \\
 &= \left(- \begin{bmatrix} 3 & 12 & 6 \\ 5 & -2 & 8 \\ 4 & 5 & 7 \end{bmatrix} \right)^T + \begin{bmatrix} 5 & -5 & 15 \\ 25 & 0 & 10 \\ 10 & 5 & 20 \end{bmatrix} \\
 &= \begin{bmatrix} -3 & -5 & -4 \\ -12 & 2 & -5 \\ -6 & -8 & -7 \end{bmatrix} + \begin{bmatrix} 5 & -5 & 15 \\ 25 & 0 & 10 \\ 10 & 5 & 20 \end{bmatrix} \\
 &= \begin{bmatrix} 2 & -10 & 11 \\ 13 & 2 & 5 \\ 4 & -3 & 13 \end{bmatrix}.
 \end{aligned}$$

f) On a

$$\begin{aligned}
 (\mathbf{BA}^T - 2\mathbf{C})^T &= \left(\begin{bmatrix} 4 & -1 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} 3 & -1 & 1 \\ 0 & 2 & 1 \end{bmatrix} - \begin{bmatrix} 2 & 8 & 4 \\ 6 & 2 & 10 \end{bmatrix} \right)^T \\
 &= \left(\begin{bmatrix} 12 & -6 & 3 \\ 0 & 4 & 2 \end{bmatrix} - \begin{bmatrix} 2 & 8 & 4 \\ 6 & 2 & 10 \end{bmatrix} \right)^T \\
 &= \begin{bmatrix} 10 & -6 \\ -14 & 2 \\ -1 & -8 \end{bmatrix}
 \end{aligned}$$

g) On a

$$\mathbf{CC}^T = \begin{bmatrix} 1 & 4 & 2 \\ 3 & 1 & 5 \end{bmatrix} \begin{bmatrix} 1 & 3 \\ 4 & 1 \\ 2 & 5 \end{bmatrix} = \begin{bmatrix} 21 & 17 \\ 17 & 35 \end{bmatrix}$$

et

$$\mathbf{A}^T\mathbf{A} = \begin{bmatrix} 3 & -1 & 1 \\ 0 & 2 & 1 \end{bmatrix} \begin{bmatrix} 3 & 0 \\ -1 & 2 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 11 & -1 \\ -1 & 5 \end{bmatrix},$$

d'où

$$\mathbf{B}^T(\mathbf{CC}^T - \mathbf{A}^T\mathbf{A}) = \begin{bmatrix} 4 & 0 \\ -1 & 2 \end{bmatrix} \begin{bmatrix} 10 & 18 \\ 18 & 30 \end{bmatrix} = \begin{bmatrix} 40 & 72 \\ 26 & 42 \end{bmatrix}$$

- h) On a $\mathbf{D}^T \mathbf{E}^T - (\mathbf{ED})^T = (\mathbf{ED})^T - (\mathbf{ED})^T = \mathbf{0}_{3 \times 3}$.
- i) Pour calculer la trace, il suffit de connaître les éléments de la diagonale. Or l'élément d_{ii} de $\mathbf{DD}^T$ est la somme des carrés des éléments de la ligne i de la matrice $\mathbf{D}$. Par conséquent, $\text{tr}(\mathbf{DD}^T) = (1 + 25 + 4) + (1 + 0 + 1) + (9 + 4 + 16) = 61$.
- j) Encore une fois, on se concentre seulement sur les éléments de la diagonale de $4\mathbf{E}^T - \mathbf{D}$. La transposée ne joue donc aucun rôle, ici. Ainsi, $\text{tr}(4\mathbf{E}^T - \mathbf{D}) = (24 - 1) + (4 - 0) + (12 - 4) = 35$.

7.8 Puisque $\text{tr}(\mathbf{A}) = \sum_{i=1}^n a_{ii}$, alors

$$\begin{aligned} \text{tr}(\mathbf{A}) + \text{tr}(\mathbf{B}) &= \sum_{i=1}^n a_{ii} + \sum_{i=1}^n b_{ii} \\ &= \sum_{i=1}^n (a_{ii} + b_{ii}) \\ &= \text{tr}(\mathbf{A} + \mathbf{B}). \end{aligned}$$

- 7.9** a) On a que $(\mathbf{AA}^T)^T = (\mathbf{A}^T)^T \mathbf{A}^T = \mathbf{AA}^T$, d'où $\mathbf{AA}^T$ est symétrique. On procède de même pour le second résultat.
- b) On a que $(\mathbf{A} - \mathbf{A}^T)^T = \mathbf{A}^T - \mathbf{A} = -(\mathbf{A} - \mathbf{A}^T)$, d'où $\mathbf{A} - \mathbf{A}^T$ est antisymétrique.
- 7.10** Tout d'abord, $\mathbf{A}^T = (\mathbf{A}^T \mathbf{A})^T = \mathbf{A}^T \mathbf{A} = \mathbf{A}$, d'où $\mathbf{A}^T \mathbf{A}$ est symétrique. De plus, si $\mathbf{A}$ est symétrique, alors $\mathbf{A} = \mathbf{A}^T \mathbf{A} = \mathbf{AA} = \mathbf{A}^2$. La matrice $\mathbf{A}$ est donc idempotente.
- 7.11** Les résultats a) et b) découlent directement des propriétés de l'arithmétique matricielle et de celles des dérivées. Quant au résultat c), en supposant que

$$\mathbf{A} = [a_{ij}(x)]_{m \times n} \quad \mathbf{B} = [b_{ij}(x)]_{n \times m},$$

nous avons

$$\begin{aligned} \frac{d}{dx}(\mathbf{AB}) &= \frac{d}{dx} \left[\sum_{k=1}^n a_{ik}(x) b_{kj}(x) \right]_{m \times m} \\ &= \left[\sum_{k=1}^n (a'_{ik}(x) b_{kj}(x) + a_{ik}(x) b'_{kj}(x)) \right] \\ &= \left[\sum_{k=1}^n a'_{ik}(x) b_{kj}(x) \right] + \left[\sum_{k=1}^n a_{ik}(x) b'_{kj}(x) \right] \\ &= \frac{d\mathbf{A}}{dx} \mathbf{B} + \mathbf{A} \frac{d\mathbf{B}}{dx}. \end{aligned}$$

- 7.12** a) Deux vecteurs de $\mathbb{R}^3$ sont nécessairement linéairement indépendants à moins que l'un ne soit un multiple de l'autre. Ce n'est pas le cas ici.
- b) Trois vecteurs sont linéairement dépendants si l'un est une combinaison linéaire des deux autres. Si c'est le cas, le déterminant de la matrice formée des coordonnées des vecteurs sera nul. Or, ici,

$$\begin{vmatrix} -3 & 5 & 1 \\ 0 & -1 & 1 \\ 4 & 2 & 3 \end{vmatrix} = 39 \neq 0.$$

Les vecteurs sont donc linéairement indépendants.

- c) Le second vecteur n'est pas un multiple du premier, donc les vecteurs sont linéairement indépendants.
- d) Un ensemble de quatre vecteurs de $\mathbb{R}^3$ est nécessairement linéairement dépendant.
- 7.13** Les trois vecteurs se trouvent dans un plan si seulement deux vecteurs sont linéairement indépendants (ceux-ci engendrent un plan dont fait alors partie le troisième).

- a) Soit la matrice

$$\mathbf{A} = \begin{bmatrix} 2 & 6 & 2 \\ -2 & 1 & 0 \\ 0 & 4 & -4 \end{bmatrix}.$$

En additionnant la première et la deuxième ligne, on obtient

$$\begin{bmatrix} 2 & 6 & 2 \\ 0 & 7 & 2 \\ 0 & 4 & -4 \end{bmatrix}.$$

La troisième ligne n'étant pas un multiple de la deuxième, on constate que la seule solution du système d'équations homogène $\mathbf{Ax} = \mathbf{0}$ est la solution triviale. Par conséquent, les trois vecteurs sont linéairement indépendants et ne se trouvent pas dans un plan. On pourrait également vérifier que $\det(\mathbf{A}) \neq 0$.

- b) Soit la matrice

$$\mathbf{A} = \begin{bmatrix} -6 & 3 & 4 \\ 7 & 2 & -1 \\ 2 & 4 & 2 \end{bmatrix}.$$

À la suite d'opérations élémentaires sur les lignes, on obtient

$$\begin{bmatrix} -6 & 3 & 4 \\ 0 & 3 & 2 \\ 0 & 0 & 0 \end{bmatrix}.$$

Il n'y a donc que deux vecteurs linéairement indépendants formant un plan dans cet ensemble.

- 7.14** On peut interpréter la condition $\prod_{i=1}^n a_{ii} \neq 0$ comme le fait que le déterminant de la matrice est non nul, ou comme le fait qu'il n'y a aucun zéro sur la diagonale et, par conséquent, aucune ligne de zéros dans la matrice. Dans un cas comme dans l'autre, cela signifie que l'inverse existe. En utilisant la technique consistant à transformer la matrice $[A|I]$ en $[I|A^{-1}]$, on voit immédiatement que

$$A^{-1} = \begin{bmatrix} a_{11}^{-1} & 0 & \cdots & 0 \\ 0 & a_{22}^{-1} & \cdots & 0 \\ \vdots & \vdots & & \vdots \\ 0 & 0 & \cdots & a_{nn}^{-1} \end{bmatrix}.$$

- 7.15** Une matrice élémentaire est le résultat d'une opération élémentaire sur la matrice identité.

- a) On souhaite inverser les première et troisième ligne de la matrice A . Par conséquent,

$$E_1 = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}.$$

- b) Même chose que ci-dessus.
c) La matrice C est obtenue à partir de la matrice A en additionnant -2 fois la première ligne à la troisième ligne. Par conséquent,

$$E_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -2 & 0 & 1 \end{bmatrix}.$$

- d) De façon similaire, on obtient la matrice A en additionnant 2 fois la première ligne de la matrice C à la troisième ligne. On a donc

$$E_4 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 2 & 0 & 1 \end{bmatrix}.$$

7.16 Par des opérations élémentaire sur les lignes, on cherche à transformer la matrice $[A|I]$ en $[I|A^{-1}]$. Ci-dessous, L_i identifie la ligne i d'une matrice, le symbole $\leftarrow$ signifie qu'une ligne est remplacée et le symbole $\leftrightarrow$ que deux lignes sont échangées.

a) Matrice de départ
$$\left[\begin{array}{ccc|ccc} 3 & 4 & -1 & 1 & 0 & 0 \\ 1 & 0 & 3 & 0 & 1 & 0 \\ 2 & 5 & -4 & 0 & 0 & 1 \end{array} \right]$$

$L_1 \leftrightarrow L_2$
$$\left[\begin{array}{ccc|ccc} 1 & 0 & 3 & 0 & 1 & 0 \\ 3 & 4 & -1 & 1 & 0 & 0 \\ 2 & 5 & -4 & 0 & 0 & 1 \end{array} \right]$$

$L_2 \leftarrow -3L_1 + L_2$
 $L_3 \leftarrow -2L_1 + L_3$
$$\left[\begin{array}{ccc|ccc} 1 & 0 & 3 & 0 & 1 & 0 \\ 0 & 4 & -10 & 1 & -3 & 0 \\ 0 & 5 & -10 & 0 & -2 & 1 \end{array} \right]$$

$L_3 \leftarrow -L_2 + L_3$
 $L_2 \leftrightarrow L_3$
$$\left[\begin{array}{ccc|ccc} 1 & 0 & 3 & 0 & 1 & 0 \\ 0 & 1 & 0 & -1 & 1 & 1 \\ 0 & 4 & -10 & 1 & -3 & 0 \end{array} \right]$$

$L_3 \leftarrow -4L_2 + L_3$
$$\left[\begin{array}{ccc|ccc} 1 & 0 & 3 & 0 & 1 & 0 \\ 0 & 1 & 0 & -1 & 1 & 1 \\ 0 & 0 & -10 & 5 & -7 & -4 \end{array} \right]$$

$L_3 \leftarrow -L_3/10$
 $L_1 \leftarrow -3L_3 + L_1$
$$\left[\begin{array}{ccc|ccc} 1 & 0 & 0 & \frac{3}{2} & -\frac{11}{10} & -\frac{6}{5} \\ 0 & 1 & 0 & -1 & 1 & 1 \\ 0 & 0 & 1 & -\frac{1}{2} & \frac{7}{10} & \frac{2}{5} \end{array} \right]$$

b) Matrice de départ
$$\left[\begin{array}{ccc|ccc} -1 & 3 & -4 & 1 & 0 & 0 \\ 2 & 4 & 1 & 0 & 1 & 0 \\ -4 & 2 & -9 & 0 & 0 & 1 \end{array} \right]$$

$L_2 \leftarrow 2L_1 + L_2$
 $L_3 \leftarrow -4L_1 + L_3$
$$\left[\begin{array}{ccc|ccc} -1 & 3 & -4 & 1 & 0 & 0 \\ 0 & 10 & -7 & 2 & 1 & 0 \\ 0 & -10 & 7 & -4 & 0 & 1 \end{array} \right]$$

$L_3 \leftarrow L_2 + L_3$
$$\left[\begin{array}{ccc|ccc} -1 & 3 & -4 & 1 & 0 & 0 \\ 0 & 10 & -7 & 2 & 1 & 0 \\ 0 & 0 & 0 & -2 & 1 & 1 \end{array} \right]$$

L'inverse n'existe pas puisque l'on obtient une ligne de zéros du côté gauche de la matrice augmentée.

c)

Matrice de départ

$$\left[\begin{array}{ccc|ccc} 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 \end{array} \right]$$

$$L_3 \leftarrow -L_1 + L_3$$

$$\left[\begin{array}{ccc|ccc} 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & -1 & -1 & 0 & 1 \end{array} \right]$$

$$L_3 \leftarrow -L_2 + L_3$$

$$L_3 \leftarrow -L_3/2$$

$$\left[\begin{array}{ccc|ccc} 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & \frac{1}{2} & \frac{1}{2} & -\frac{1}{2} \end{array} \right]$$

$$L_2 \leftarrow -L_3 + L_2$$

$$L_1 \leftarrow -L_3 + L_1$$

$$\left[\begin{array}{ccc|ccc} 1 & 0 & 0 & \frac{1}{2} & -\frac{1}{2} & \frac{1}{2} \\ 0 & 1 & 0 & -\frac{1}{2} & \frac{1}{2} & \frac{1}{2} \\ 0 & 0 & 1 & \frac{1}{2} & \frac{1}{2} & -\frac{1}{2} \end{array} \right]$$

d) Matrice de départ

$$\left[\begin{array}{ccc|ccc} 2 & 6 & 6 & 1 & 0 & 0 \\ 2 & 7 & 6 & 0 & 1 & 0 \\ 2 & 7 & 7 & 0 & 0 & 1 \end{array} \right]$$

$$L_2 \leftarrow -L_1 + L_2$$

$$L_3 \leftarrow -L_1 + L_3$$

$$\left[\begin{array}{ccc|ccc} 2 & 6 & 6 & 1 & 0 & 0 \\ 0 & 1 & 0 & -1 & 1 & 0 \\ 0 & 1 & 1 & -1 & 0 & 1 \end{array} \right]$$

$$L_1 \leftarrow L_1/2$$

$$L_3 \leftarrow -L_2 + L_3$$

$$\left[\begin{array}{ccc|ccc} 1 & 3 & 3 & \frac{1}{2} & 0 & 0 \\ 0 & 1 & 0 & -1 & 1 & 0 \\ 0 & 0 & 1 & 0 & -1 & 1 \end{array} \right]$$

$$L_1 \leftarrow -3L_3 + L_1$$

$$\left[\begin{array}{ccc|ccc} 1 & 3 & 0 & \frac{1}{2} & 3 & -3 \\ 0 & 1 & 0 & -1 & 1 & 0 \\ 0 & 0 & 1 & 0 & -1 & 1 \end{array} \right]$$

$$L_1 \leftarrow -3L_2 + L_1$$

$$\left[\begin{array}{ccc|ccc} 1 & 0 & 0 & \frac{7}{2} & 0 & -3 \\ 0 & 1 & 0 & -1 & 1 & 0 \\ 0 & 0 & 1 & 0 & -1 & 1 \end{array} \right]$$

e) Matrice de départ

$$\left[\begin{array}{ccc|ccc} 1 & 0 & 1 & 1 & 0 & 0 \\ -1 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 \end{array} \right]$$

$$\begin{array}{ll}
L_2 \leftrightarrow L_3 & \left[\begin{array}{ccc|ccc} 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 2 & 1 & 1 & 0 \end{array} \right] \\
L_3 \leftarrow L_1 + L_3 & \\
\\
L_3 \leftarrow -L_2 + L_3 & \left[\begin{array}{ccc|ccc} 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & \frac{1}{2} & \frac{1}{2} & -\frac{1}{2} \end{array} \right] \\
L_3 \leftarrow L_3/2 & \\
\\
L_1 \leftarrow -L_3 + L_1 & \left[\begin{array}{ccc|ccc} 1 & 0 & 0 & \frac{1}{2} & -\frac{1}{2} & \frac{1}{2} \\ 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & \frac{1}{2} & \frac{1}{2} & -\frac{1}{2} \end{array} \right]
\end{array}$$

7.17 a) Tel que démontré à l'exercice 7.14, l'inverse est

$$\begin{bmatrix} k_1^{-1} & 0 & 0 & 0 \\ 0 & k_2^{-1} & 0 & 0 \\ 0 & 0 & k_3^{-1} & 0 \\ 0 & 0 & 0 & k_4^{-1} \end{bmatrix}.$$

b) Pour obtenir l'inverse de cette matrice par la méthode utilisée à l'exercice 7.16, il suffit d'échanger les lignes de bas en haut, puis de diviser celles-ci par k_4 , k_3 , k_2 et k_1 , respectivement. On trouve alors que l'inverse est

$$\begin{bmatrix} 0 & 0 & 0 & k_4^{-1} \\ 0 & 0 & k_3^{-1} & 0 \\ 0 & k_2^{-1} & 0 & 0 \\ k_1^{-1} & 0 & 0 & 0 \end{bmatrix}.$$

c) On utilise la méthode de l'exercice 7.16.

$$\text{Matrice de départ} \quad \left[\begin{array}{cccc|cccc} k & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & k & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & k & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & k & 0 & 0 & 0 & 1 \end{array} \right]$$

$$\begin{array}{l}
L_1 \leftarrow L_1/k \\
L_2 \leftarrow -L_1 + L_2 \\
L_2 \leftarrow L_2/k
\end{array}$$

$$\left[\begin{array}{cccc|cccc} 1 & 0 & 0 & 0 & k^{-1} & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & -k^{-2} & k^{-1} & 0 & 0 \\ 0 & 1 & k & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & k & 0 & 0 & 0 & 1 \end{array} \right]$$

$$\begin{array}{l} L_3 \leftarrow -L_2 + L_3 \\ L_3 \leftarrow L_3/k \end{array} \quad \left[\begin{array}{cccc|cccc} 1 & 0 & 0 & 0 & k^{-1} & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & -k^{-2} & k^{-1} & 0 & 0 \\ 0 & 0 & 1 & 0 & k^{-3} & -k^{-2} & k^{-1} & 0 \\ 0 & 0 & 1 & k & 0 & 0 & 0 & 1 \end{array} \right]$$

$$\begin{array}{l} L_4 \leftarrow -L_3 + L_4 \\ L_4 \leftarrow L_4/k \end{array} \quad \left[\begin{array}{cccc|cccc} 1 & 0 & 0 & 0 & k^{-1} & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & -k^{-2} & k^{-1} & 0 & 0 \\ 0 & 0 & 1 & 0 & k^{-3} & -k^{-2} & k^{-1} & 0 \\ 0 & 0 & 0 & 1 & -k^{-4} & k^{-3} & -k^{-2} & k^{-1} \end{array} \right]$$

7.18 a) Les opérations élémentaires à effectuer pour transformer la matrice **A** en matrice identité sont, dans l'ordre :

1. additionner 5 fois la première ligne à la deuxième;
2. multiplier la deuxième ligne par $\frac{1}{2}$.

Les matrices élémentaires correspondantes sont

$$\mathbf{E}_1 = \begin{bmatrix} 1 & 0 \\ 5 & 1 \end{bmatrix} \quad \text{et} \quad \mathbf{E}_2 = \begin{bmatrix} 1 & 0 \\ 0 & \frac{1}{2} \end{bmatrix}.$$

b) De la partie a), on a que

$$\mathbf{A}^{-1} = \mathbf{E}_2 \mathbf{E}_1 = \begin{bmatrix} 1 & 0 \\ \frac{5}{2} & \frac{1}{2} \end{bmatrix}.$$

c) Du [théorème 7.7](#) des notes de cours, une matrice élémentaire est inversible et son inverse est aussi une matrice élémentaire. Par conséquent, on peut isoler **A** dans l'équation $\mathbf{E}_2 \mathbf{E}_1 \mathbf{A} = \mathbf{I}$ pour obtenir $\mathbf{A} = \mathbf{E}_1^{-1} \mathbf{E}_2^{-1}$. Or,

$$\mathbf{E}_1^{-1} = \begin{bmatrix} 1 & 0 \\ -5 & 1 \end{bmatrix} \quad \text{et} \quad \mathbf{E}_2^{-1} = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix},$$

d'où

$$\mathbf{A} = \begin{bmatrix} 1 & 0 \\ -5 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}.$$

7.19 On a le système d'équations linéaires $\mathbf{Ax} = \mathbf{b}$ avec

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 1 \\ 1 & -1 & 1 \\ 1 & 1 & 0 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} -1 \\ 3 \\ 4 \end{bmatrix}.$$

Par la même technique que les exercices précédents, on trouve

$$\mathbf{A}^{-1} = \frac{1}{3} \begin{bmatrix} -1 & 1 & 3 \\ 1 & -1 & 0 \\ 2 & 1 & -3 \end{bmatrix}.$$

Par conséquent,

$$\mathbf{x} = \mathbf{A}^{-1}\mathbf{b} = \frac{1}{3} \begin{bmatrix} 16 \\ -4 \\ -11 \end{bmatrix}.$$

7.20 a) On a $\mathbf{Ax} = \mathbf{x} \Leftrightarrow \mathbf{x} - \mathbf{Ax} = \mathbf{0} \Leftrightarrow (\mathbf{I} - \mathbf{A})\mathbf{x} = \mathbf{0}$. On veut par la suite résoudre ce système d'équations homogène, où

$$\mathbf{I} - \mathbf{A} = \begin{bmatrix} -1 & -1 & -2 \\ -2 & -1 & 2 \\ -3 & -1 & 0 \end{bmatrix}.$$

Or, on peut vérifier que le déterminant de cette matrice est non nul, ce qui signifie que le système d'équations n'admet que la solution triviale $x_1 = x_2 = x_3 = 0$. On verra au [chapitre 8](#) que cela signifie que 1 n'est pas une valeur propre de la matrice $\mathbf{A}$.

b) On procède comme ci-dessus en résolvant cette fois le système d'équations linéaires homogène $(4\mathbf{I} - \mathbf{A})\mathbf{x} = \mathbf{0}$. Or,

$$4\mathbf{I} - \mathbf{A} = \begin{bmatrix} 2 & -1 & -2 \\ -2 & 2 & 2 \\ -3 & -1 & 3 \end{bmatrix}.$$

Après quelques opérations élémentaires sur les lignes de cette matrice, on obtient

$$\begin{bmatrix} 2 & -1 & -2 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

Le système d'équations a donc une infinité de solutions. En posant $x_3 = t$, on a $x_2 = 0$ et $x_1 = t$. Autrement dit, tout vecteur de la forme

$$\begin{bmatrix} t & 0 & t \end{bmatrix}$$

est une solution de l'équation $\mathbf{Ax} = 4\mathbf{x}$. Encore une fois, on verra au chapitre 8 que 4 est une valeur propre de la matrice $\mathbf{A}$ et que $(t, 0, t)$ est un vecteur propre correspondant.

7.21 Tel que vu au théorème 7.13 des notes de cours, les concepts d'inversibilité et de solution d'un système d'équations linéaires homogène sont liés : une matrice carrée $\mathbf{A}$ est inversible si, et seulement si, $\mathbf{Ax} = \mathbf{0}$ n'admet que la solution triviale.

- a) Les quatre équations sont linéairement indépendantes, donc la matrice des coefficients est inversible et la seule solution du système est la solution triviale.
- b) Puisque le coefficient de x_2 est 0 dans la seconde équation, les équations sont linéairement dépendantes (il serait possible d'éliminer complètement une ligne du système). Par conséquent, la matrice des coefficients est singulière et il existe une infinité de solutions pour le système d'équations homogène. Vu autrement, la résolution de ce système par substitution successive mène à l'équation $5x_1 - x_2 = 0$, qui compte une infinité de solutions.

7.22 On voit que la troisième ligne de la matrice est la somme des deux premières. Les lignes n'étant pas linéairement indépendantes, le déterminant est nul.

- 7.23**
- a) Les lignes (et les colonnes) sont linéairement indépendantes, donc la matrice est inversible.
 - b) La troisième colonne est un multiple de la première. Par conséquent, les colonnes ne sont pas linéairement indépendantes et la matrice est singulière.
 - c) Le déterminant est clairement nul, donc la matrice est singulière.
 - d) Idem.

7.24 On utilise les propriétés du déterminant énoncées dans le théorème 7.11 des notes de cours.

- a) $\det(3\mathbf{A}) = 3^3 \det(\mathbf{A}) = -189$
- b) $\det(\mathbf{A}^{-1}) = 1 / \det(\mathbf{A}) = -\frac{1}{7}$
- c) $\det(2\mathbf{A}^{-1}) = 2^3 / \det(\mathbf{A}) = -\frac{8}{7}$
- d) $\det((2\mathbf{A})^{-1}) = 2^{-3} / \det(\mathbf{A}) = -\frac{1}{56}$

7.25 Une matrice n'est pas inversible si son déterminant est nul.

- a) Le déterminant de la matrice est $k^2 - 5k + 2$, donc la matrice est singulière lorsque $k = (5 \pm \sqrt{17})/2$.
- b) Le déterminant de la matrice est $8k + 8$, donc la matrice est singulière lorsque $k = -1$.

7.26 a) On a

$$M_{13} = \begin{vmatrix} 0 & 0 & 3 \\ 4 & 1 & 14 \\ 4 & 1 & 2 \end{vmatrix} = 0,$$

donc $C_{13} = 0$.

b) On a

$$M_{23} = \begin{vmatrix} 4 & -1 & 6 \\ 4 & 1 & 14 \\ 4 & 1 & 2 \end{vmatrix} = 4(-12) - 4(-8) + 4(-20) = -96$$

et $C_{23} = (-1)^5(-96) = 96$.

c) On a

$$M_{22} = \begin{vmatrix} 4 & 1 & 6 \\ 4 & 0 & 14 \\ 4 & 3 & 2 \end{vmatrix} = 4(-42) - 4(-16) + 4(14) = -48$$

et $C_{22} = (-1)^4(-48) = -48$.

d) On a

$$M_{21} = \begin{vmatrix} -1 & 1 & 6 \\ 1 & 0 & 14 \\ 1 & 3 & 2 \end{vmatrix} = -(-12) - 3(-20) = 72$$

et $C_{21} = (-1)^3(72) = -72$.

- e) En développant par la deuxième ligne, on a $\det(\mathbf{A}) = -3C_{23} + 3C_{24}$.
Or, de ce qui précède, $C_{23} = 96$ et $C_{24} = M_{24} = 24$. Par conséquent,
 $\det(\mathbf{A}) = -216$.

Chapitre 8

8.1 Dans tous les cas, la matrice mentionnée dans l'énoncé est notée $\mathbf{A}$.

a) On a

$$\begin{aligned}\det(\lambda \mathbf{I} - \mathbf{A}) &= \begin{vmatrix} \lambda - 3 & 0 \\ -8 & \lambda + 1 \end{vmatrix} \\ &= (\lambda - 3)(\lambda + 1).\end{aligned}$$

Les valeurs propres sont donc $\lambda_1 = 3$ et $\lambda_2 = -1$. D'une part, la forme échelonnée du système d'équations $(3\mathbf{I} - \mathbf{A})\mathbf{x} = \mathbf{0}$ est

$$\begin{bmatrix} 0 & 0 \\ 1 & -\frac{1}{2} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix},$$

soit $x_1 = s/2$ et $x_2 = s$. Une base de vecteurs propres correspondant à $\lambda = 3$ est donc $(\frac{1}{2}, 1)$. D'autre part, le système d'équations $(-\mathbf{I} - \mathbf{A})\mathbf{x} = \mathbf{0}$ se réduit à

$$\begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix},$$

soit $x_1 = 0$ et $x_2 = s$. Une base de vecteurs propres correspondant à $\lambda = -1$ est donc $(0, 1)$. Vérification :

```
> m <- matrix(c(3, 8, 0, -1), nrow = 2)
> eigen(m)
eigen() decomposition
$values
[1] 3 -1

$vectors
      [,1] [,2]
[1,] 0.4472136 0
[2,] 0.8944272 1
```

Remarquez que les vecteurs propres obtenus avec `eigen` sont normalisés de sorte que leur norme soit toujours égale à 1.

b) On a

$$\begin{aligned}\det(\lambda \mathbf{I} - \mathbf{A}) &= \begin{vmatrix} \lambda - 10 & 9 \\ -4 & \lambda + 2 \end{vmatrix} \\ &= (\lambda - 10)(\lambda + 2) + 36 \\ &= (\lambda - 4)^2.\end{aligned}$$

La matrice compte donc une seule valeur propre : $\lambda = 4$. Le système d'équations $(4\mathbf{I} - \mathbf{A})\mathbf{x} = \mathbf{0}$ se réduit à

$$\begin{bmatrix} 1 & -\frac{3}{2} \\ 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix},$$

soit $x_1 = 3s/2$ et $x_2 = s$. Une base de vecteurs propres correspondant à $\lambda = 4$ est donc $(\frac{3}{2}, 1)$. Vérification :

```
> m <- matrix(c(10, 4, -9, -2), nrow = 2)
> eigen(m)
eigen() decomposition
$values
[1] 4 4

$vectors
      [,1]      [,2]
[1,] -0.8320503 0.8320503
[2,] -0.5547002 0.5547002
```

c) On a

$$\begin{aligned} \det(\lambda\mathbf{I} - \mathbf{A}) &= \begin{vmatrix} \lambda - 4 & 0 & -1 \\ 2 & \lambda - 1 & 0 \\ 2 & 0 & \lambda - 1 \end{vmatrix} \\ &= (\lambda - 4)(\lambda - 1)^2 + 2(\lambda - 1) \\ &= (\lambda - 1)(\lambda - 2)(\lambda - 3). \end{aligned}$$

Les valeurs propres sont donc $\lambda_1 = 1$, $\lambda_2 = 2$ et $\lambda_3 = 3$. En premier lieu, le système d'équations $(\mathbf{I} - \mathbf{A})\mathbf{x} = \mathbf{0}$ se réduit à

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix},$$

soit $x_1 = x_3 = 0$ et $x_2 = s$. Une base de vecteurs propres correspondant à $\lambda = 1$ est donc $(0, 1, 0)$. Deuxièmement, le système d'équations $(2\mathbf{I} - \mathbf{A})\mathbf{x} = \mathbf{0}$ se réduit à

$$\begin{bmatrix} 1 & 0 & \frac{1}{2} \\ 0 & 1 & -1 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix},$$

soit $x_1 = -s/2$, $x_2 = s$ et $x_3 = s$. Une base de vecteurs propres correspondant à $\lambda = 2$ est donc $(-\frac{1}{2}, 1, 1)$. Finalement, le système d'équations $(3\mathbf{I} - \mathbf{A})\mathbf{x} = \mathbf{0}$ se réduit à

$$\begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & -1 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix},$$

soit $x_1 = -s$, $x_2 = s$ et $x_3 = s$. Une base de vecteurs propres correspondant à $\lambda = 2$ est donc $(-1, 1, 1)$. Vérification :

```
> m <- matrix(c(4, -2, -2, 0, 1, 0, 1, 0, 1),
+             nrow = 3)
> eigen(m)
eigen() decomposition
$values
[1] 3 2 1

$vectors
      [,1]      [,2] [,3]
[1,] 0.5773503 -0.3333333 0
[2,] -0.5773503 0.6666667 1
[3,] -0.5773503 0.6666667 0
```

d) On a

$$\begin{aligned} \det(\lambda \mathbf{I} - \mathbf{A}) &= \begin{vmatrix} \lambda - 5 & -6 & -2 \\ 0 & \lambda + 1 & 8 \\ -1 & 0 & \lambda + 2 \end{vmatrix} \\ &= (\lambda - 5)(\lambda + 1)(\lambda + 2) - 2(\lambda + 1) + 48 \\ &= \lambda^3 - 2\lambda^2 - 15\lambda + 36 \\ &= (\lambda - 3)^2(\lambda + 4). \end{aligned}$$

Les valeurs propres sont donc $\lambda_1 = \lambda_2 = 3$ et $\lambda_3 = -4$. En premier lieu, le système d'équations $(3\mathbf{I} - \mathbf{A})\mathbf{x} = \mathbf{0}$ se réduit à

$$\begin{bmatrix} 1 & 0 & -5 \\ 0 & 1 & 2 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix},$$

soit $x_1 = 5s$, $x_2 = -2s$ et $x_3 = s$. Une base de vecteurs propres correspondant à $\lambda = 3$ est donc $(5, -2, 1)$. Deuxièmement, le système

d'équations $(-4\mathbf{I} - \mathbf{A})\mathbf{x} = \mathbf{0}$ se réduit à

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & -\frac{8}{3} \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix},$$

soit $x_1 = -2s$, $x_2 = 8s/3$ et $x_3 = s$. Une base de vecteurs propres correspondant à $\lambda = -4$ est donc $(-2, \frac{8}{3}, 1)$. Vérification :

```
> m <- matrix(c(5, 0, 1, 6, -1, 0, 2, -8, -2),
+             nrow = 3)
> eigen(m)
eigen() decomposition
$values
[1] -4  3  3

$vectors
      [,1]      [,2]      [,3]
[1,] 0.5746958 -0.9128709 0.9128709
[2,] -0.7662610 0.3651484 -0.3651484
[3,] -0.2873479 -0.1825742 0.1825742
```

e) On a

$$\begin{aligned} \det(\lambda\mathbf{I} - \mathbf{A}) &= \begin{vmatrix} \lambda & 0 & -2 & 0 \\ -1 & \lambda & -1 & 0 \\ 0 & -1 & \lambda + 2 & 0 \\ 0 & 0 & 0 & \lambda - 1 \end{vmatrix} \\ &= (\lambda - 1) \begin{vmatrix} \lambda & 0 & -2 \\ -1 & \lambda & -1 \\ 0 & -1 & \lambda + 2 \end{vmatrix} \\ &= (\lambda - 1)(\lambda^3 + 2\lambda^2 - \lambda - 2) \\ &= (\lambda - 1)^2(\lambda + 1)(\lambda + 2). \end{aligned}$$

Les valeurs propres sont donc $\lambda_1 = \lambda_2 = 1$, $\lambda_3 = -1$ et $\lambda_4 = -2$. En premier lieu, le système d'équations $(\mathbf{I} - \mathbf{A})\mathbf{x} = \mathbf{0}$ se réduit à

$$\begin{bmatrix} 1 & 0 & -2 & 0 \\ 0 & 1 & -3 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix},$$

soit $x_1 = 2s$, $x_2 = 3s$, $x_3 = s$ et $x_4 = t$. Or, $(2s, 3s, s, t) = s(2, 3, 1, 0) + t(0, 0, 0, 1)$. Une base de vecteurs propres correspondant à $\lambda = 1$ est donc composée des vecteurs $(2, 3, 1, 0)$ et $(0, 0, 0, 1)$. Deuxièmement, le système d'équations $(-\mathbf{I} - \mathbf{A})\mathbf{x} = \mathbf{0}$ se réduit à

$$\begin{bmatrix} 1 & 0 & 2 & 0 \\ 0 & 1 & -1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix},$$

soit $x_1 = -2s$, $x_2 = s$, $x_3 = s$ et $x_4 = 0$. Une base de vecteurs propres correspondant à $\lambda = -1$ est donc $(-2, 1, 1, 0)$. Finalement, le système d'équations $(-2\mathbf{I} - \mathbf{A})\mathbf{x} = \mathbf{0}$ se réduit à

$$\begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix},$$

soit $x_1 = -s$, $x_2 = 0$, $x_3 = s$ et $x_4 = 0$. Une base de vecteurs propres correspondant à $\lambda = -2$ est donc $(-1, 0, 1, 0)$. Vérification :

```
> m <- matrix(c(0, 1, 0, 0, 0, 0, 1, 0,
+               2, 1, -2, 0, 0, 0, 0, 1),
+             nrow = 4)
> eigen(m)
eigen() decomposition
$values
[1] -2 -1  1  1

$vectors
      [,1]      [,2] [,3]      [,4]
[1,] -7.071068e-01  0.8164966    0 -0.5345225
[2,] -2.055734e-16 -0.4082483    0 -0.8017837
[3,]  7.071068e-01 -0.4082483    0 -0.2672612
[4,]  0.000000e+00  0.0000000    1  0.0000000
```

8.2 Premièrement, l'énoncé est clairement vrai pour $k = 1$. Nous supposons par la suite qu'il est vrai pour $k = n$, soit que si $\mathbf{Ax} = \lambda\mathbf{x}$, alors $\mathbf{A}^n\mathbf{x} = \lambda^n\mathbf{x}$. Ainsi,

$$\mathbf{A}^{n+1}\mathbf{x} = \mathbf{A}(\mathbf{A}^n\mathbf{x}) = \mathbf{A}(\lambda^n\mathbf{x}) = \lambda^n(\mathbf{Ax}) = \lambda^n(\lambda\mathbf{x}) = \lambda^{n+1}\mathbf{x},$$

d'où l'énoncé est vrai pour $k = n + 1$. Ceci complète la preuve.

8.3 Nous allons utiliser les résultats de l'exercice 8.2. Tout d'abord,

$$\begin{aligned}\det(\lambda \mathbf{I} - \mathbf{A}) &= \begin{vmatrix} \lambda + 1 & 2 & 2 \\ -1 & \lambda - 2 & -1 \\ 1 & 1 & \lambda \end{vmatrix} \\ &= (\lambda + 1)(\lambda - 1)^2,\end{aligned}$$

d'où les valeurs propres de $\mathbf{A}$ sont $\lambda_1 = \lambda_2 = 1$ et $\lambda_3 = -1$. Par conséquent, les valeurs propres de $\mathbf{A}^{25}$ sont $\lambda_1 = \lambda_2 = 1^{25} = 1$ et $\lambda_3 = (-1)^{25} = -1$. Toujours par le résultat de l'exercice 8.2, les vecteurs propres de $\mathbf{A}$ correspondant à $\lambda = 1$ et $\lambda = -1$ sont également des vecteurs propres de $\mathbf{A}^{25}$. Or, le système d'équations $(\mathbf{I} - \mathbf{A})\mathbf{x} = \mathbf{0}$ se réduit à

$$\begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix},$$

soit $x_1 = -s - t$, $x_2 = s$ et $x_3 = t$. Puisque $(-s - t, s, t) = s(-1, 1, 0) + t(-1, 0, 1)$, une base de vecteurs propres correspondant à $\lambda = 1$ est composée de $(-1, 1, 0)$ et $(-1, 0, 1)$. D'autre part, le système d'équations $(-\mathbf{I} - \mathbf{A})\mathbf{x} = \mathbf{0}$ se réduit à

$$\begin{bmatrix} 1 & 0 & -2 \\ 0 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix},$$

soit $x_1 = 2s$, $x_2 = -s$ et $x_3 = s$. Une base de vecteurs propres correspondant à $\lambda = -1$ est donc $(2, -1, 1)$.

8.4 Nous pouvons démontrer les deux résultats simultanément. Tout d'abord, les résultats sont clairement vrais pour $n = 1$, c'est-à-dire $c_1 = c_n = -x_1$. Supposons ensuite que les résultats sont vrais pour $n = k$, soit

$$\prod_{i=1}^k (x - x_i) = x^k + \left(- \sum_{i=1}^k x_i \right) x^{k-1} + \dots + \prod_{i=1}^k x_i.$$

Si $n = k + 1$, alors

$$\begin{aligned}
 \prod_{i=1}^{k+1} (x - x_i) &= \left[\prod_{i=1}^k (x - x_i) \right] (x - x_{k+1}) \\
 &= \left[x^k + \left(- \sum_{i=1}^k x_i \right) x^{k-1} + \cdots + \prod_{i=1}^k x_i \right] (x - x_{k+1}) \\
 &= x^{k+1} + \left(- \sum_{i=1}^k x_i - x_{k+1} \right) x^k + \cdots + x_{k+1} \prod_{i=1}^k x_i \\
 &= x^{k+1} + \left(- \sum_{i=1}^{k+1} x_i \right) x^k + \cdots + \prod_{i=1}^{k+1} x_i.
 \end{aligned}$$

Les résultats sont donc vrais pour $n = k + 1$. Ceci complète la preuve.

8.5 Soit

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}.$$

Le polynôme caractéristique de cette matrice est

$$\begin{aligned}
 \det(\lambda \mathbf{I} - \mathbf{A}) &= \begin{vmatrix} \lambda - a_{11} & -a_{12} \\ -a_{21} & \lambda - a_{22} \end{vmatrix} \\
 &= (\lambda - a_{11})(\lambda - a_{22}) - a_{12}a_{21} \\
 &= \lambda^2 - (a_{11} + a_{22})\lambda + (a_{11}a_{22} - a_{12}a_{21}) \\
 &= \lambda^2 - \operatorname{tr}(\mathbf{A})\lambda + \det(\mathbf{A}),
 \end{aligned}$$

d'où l'équation caractéristique est $\lambda^2 - \operatorname{tr}(\mathbf{A})\lambda + \det(\mathbf{A}) = 0$.

8.6 Par définition $\mathbf{A}\mathbf{x} = \lambda\mathbf{x}$. En multipliant de part et d'autre (par la gauche) par $\mathbf{A}^{-1}$, on obtient $\mathbf{x} = \lambda\mathbf{A}^{-1}\mathbf{x}$, soit $\mathbf{A}^{-1}\mathbf{x} = \lambda^{-1}\mathbf{x}$. Par conséquent, λ^{-1} est une valeur propre de $\mathbf{A}^{-1}$ et $\mathbf{x}$ est un vecteur propre correspondant.

8.7 Utilisons le résultat de l'exercice 8.6 pour éviter de calculer l'inverse de la matrice. Le polynôme caractéristique de la matrice $\mathbf{A}$ est

$$\begin{aligned}
 \det(\lambda \mathbf{I} - \mathbf{A}) &= \begin{vmatrix} \lambda + 2 & -2 & -3 \\ 2 & \lambda - 3 & -2 \\ 4 & -2 & \lambda - 5 \end{vmatrix} \\
 &= \lambda^3 - 6\lambda^2 + 11\lambda - 6 \\
 &= (\lambda - 1)(\lambda - 2)(\lambda - 3),
 \end{aligned}$$

d'où les valeurs propres de $\mathbf{A}$ sont $\lambda_1 = 1$, $\lambda_2 = 2$ et $\lambda_3 = 3$. Par conséquent, les valeurs propres de $\mathbf{A}^{-1}$ sont $\lambda_1 = 1$, $\lambda_2 = \frac{1}{2}$ et $\lambda_3 = \frac{1}{3}$. Toujours par le résultat de l'exercice 8.6, les vecteurs propres de $\mathbf{A}$ correspondant à $\lambda = 1$, $\lambda = 2$ et $\lambda = 3$ sont également des vecteurs propres de $\mathbf{A}^{-1}$ correspondant à $\lambda_1 = 1$, $\lambda_2 = \frac{1}{2}$ et $\lambda_3 = \frac{1}{3}$. Or, le système d'équations $(\mathbf{I} - \mathbf{A})\mathbf{x} = \mathbf{0}$ se réduit à

$$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix},$$

soit $x_1 = s$, $x_2 = 0$ et $x_3 = s$. Une base de vecteurs propres correspondant à $\lambda = 1$ est donc $(1, 0, 1)$. D'autre part, le système d'équations $(2\mathbf{I} - \mathbf{A})\mathbf{x} = \mathbf{0}$ se réduit à

$$\begin{bmatrix} 1 & -\frac{1}{2} & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix},$$

soit $x_1 = s/2$, $x_2 = s$ et $x_3 = 0$. Une base de vecteurs propres correspondant à $\lambda = 2$ (ou $\lambda = \frac{1}{2}$) est donc $(\frac{1}{2}, 1, 0)$. Finalement, le système d'équations $(3\mathbf{I} - \mathbf{A})\mathbf{x} = \mathbf{0}$ se réduit à

$$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & -1 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix},$$

soit $x_1 = s$, $x_2 = s$ et $x_3 = s$. Une base de vecteurs propres correspondant à $\lambda = 3$ (ou $\lambda = \frac{1}{3}$) est donc $(1, 1, 1)$.

8.8 Le résultat découle simplement du fait que l'équation $\mathbf{I}\mathbf{x} = \mathbf{x}$ est vraie pour tout vecteur $\mathbf{x}$.

8.9 a) Le polynôme caractéristique est $\lambda^2 - 4\lambda + 4 = (\lambda - 2)^2$, donc les valeurs propres sont $\lambda_1 = \lambda_2 = 2$. Par conséquent,

$$2\mathbf{I} - \mathbf{A} = \begin{bmatrix} 0 & 0 \\ -1 & 0 \end{bmatrix},$$

d'où $\text{rang}(2\mathbf{I} - \mathbf{A}) = 1$. Puisque la matrice $\mathbf{A}$ ne possède qu'un seul vecteur propre, elle n'est pas diagonalisable. Vérification :

```

> m <- matrix(c(2, 1, 0, 2), nrow = 2)
> eigen(m)
eigen() decomposition
$values
[1] 2 2

$vectors
      [,1]      [,2]
[1,]    0 4.440892e-16
[2,]    1 -1.000000e+00

```

- b) Le polynôme caractéristique est $(\lambda - 3)(\lambda^2 - 8\lambda + 15) = (\lambda - 3)^2(\lambda - 5)$, donc les valeurs propres sont $\lambda_1 = \lambda_2 = 3$ et $\lambda_3 = 5$. La forme échelonnée de la matrice $3\mathbf{I} - \mathbf{A}$ est

$$\begin{bmatrix} 1 & 0 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix},$$

d'où $\text{rang}(3\mathbf{I} - \mathbf{A}) = 1$. La base de vecteurs propres correspondant à $\lambda = 3$ est composée des vecteurs $(-1, 0, 1)$ et $(0, 1, 0)$. D'autre part, la forme échelonnée de la matrice $5\mathbf{I} - \mathbf{A}$ est

$$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & -2 \\ 0 & 0 & 0 \end{bmatrix}$$

d'où $\text{rang}(5\mathbf{I} - \mathbf{A}) = 2$. La base de vecteurs propres correspondant à $\lambda = 5$ est $(1, 2, 1)$. Bien que les valeurs propres de la matrice $\mathbf{A}$ ne sont pas toutes distinctes, les vecteurs propres $(1, 0, -1)$, $(0, 1, 0)$ et $(1, 2, 1)$ sont linéairement indépendants. Par conséquent, la matrice

$$\mathbf{P} = \begin{bmatrix} -1 & 0 & 1 \\ 0 & 1 & 2 \\ 1 & 0 & 1 \end{bmatrix}$$

diagonalise $\mathbf{A}$ et

$$\mathbf{P}^{-1}\mathbf{A}\mathbf{P} = \begin{bmatrix} 3 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 5 \end{bmatrix}.$$

Vérification :

```

> m <- matrix(c(4, 2, 1, 0, 3, 0, 1, 2, 4), nrow = 3)
> eigen(m)
eigen() decomposition
$values
[1] 5 3 3

$vectors
      [,1] [,2]      [,3]
[1,] 0.4082483 0 -0.7071068
[2,] 0.8164966 1 0.0000000
[3,] 0.4082483 0 0.7071068

```

- c) La matrice étant triangulaire, nous savons que les valeurs propres sont $\lambda_1 = 3$ et $\lambda_2 = \lambda_3 = 2$. La forme échelonnée de la matrice $3\mathbf{I} - \mathbf{A}$ est

$$\begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix},$$

d'où $\text{rang}(3\mathbf{I} - \mathbf{A}) = 2$. La base de vecteurs propres correspondant à $\lambda = 3$ est $(1, 0, 0)$. D'autre part, la forme échelonnée de la matrice $2\mathbf{I} - \mathbf{A}$ est

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

d'où $\text{rang}(2\mathbf{I} - \mathbf{A}) = 2$. La base de vecteurs propres correspondant à $\lambda = 2$ est $(0, 0, 1)$. Par conséquent, la matrice $\mathbf{A}$ n'a pas trois vecteurs linéairement indépendants, donc elle n'est pas diagonalisable.

```

> m <- matrix(c(3, 0, 0, 0, 2, 1, 0, 0, 2), nrow = 3)
> eigen(m)
eigen() decomposition
$values
[1] 3 2 2

$vectors
      [,1] [,2]      [,3]
[1,] 1 0 0.000000e+00
[2,] 0 0 4.440892e-16
[3,] 0 1 -1.000000e+00

```

- d) Le polynôme caractéristique est $\lambda^3 - 6\lambda^2 + 11\lambda - 6 = (\lambda - 1)(\lambda - 2)(\lambda - 3)$, donc les valeurs propres sont $\lambda_1 = 1$, $\lambda_2 = 2$ et $\lambda_3 = 3$. Les valeurs propres étant distinctes, la matrice est diagonalisable. Or, la forme échelonnée de la matrice $\mathbf{I} - \mathbf{A}$ est

$$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & -1 \\ 0 & 0 & 0 \end{bmatrix},$$

d'où $\text{rang}(\mathbf{I} - \mathbf{A}) = 2$. La base de vecteurs propres correspondant à $\lambda = 1$ est $(1, 1, 1)$. Deuxièmement, la forme échelonnée de la matrice $2\mathbf{I} - \mathbf{A}$ est

$$\begin{bmatrix} 1 & 0 & -\frac{2}{3} \\ 0 & 1 & -1 \\ 0 & 0 & 0 \end{bmatrix}$$

d'où $\text{rang}(2\mathbf{I} - \mathbf{A}) = 2$ et la base de vecteurs propres correspondant à $\lambda = 2$ est $(2, 3, 3)$. Enfin, la forme échelonnée de la matrice $3\mathbf{I} - \mathbf{A}$ est

$$\begin{bmatrix} 1 & 0 & -\frac{1}{4} \\ 0 & 1 & -\frac{3}{4} \\ 0 & 0 & 0 \end{bmatrix}$$

d'où $\text{rang}(3\mathbf{I} - \mathbf{A}) = 2$ et la base de vecteurs propres correspondant à $\lambda = 3$ est $(1, 3, 4)$. Par conséquent,

$$\mathbf{P} = \begin{bmatrix} 1 & 2 & 1 \\ 1 & 3 & 3 \\ 1 & 3 & 4 \end{bmatrix}$$

diagonalise $\mathbf{A}$ et

$$\mathbf{P}^{-1}\mathbf{A}\mathbf{P} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{bmatrix}.$$

Vérification :

```
> m <- matrix(c(-1, -3, -3, 4, 4, 1, -2, 0, 3),
+             nrow = 3)
> eigen(m)
eigen() decomposition
$values
```

```

[1] 3 2 1

$vector
      [,1]      [,2]      [,3]
[1,] 0.1961161 0.4264014 -0.5773503
[2,] 0.5883484 0.6396021 -0.5773503
[3,] 0.7844645 0.6396021 -0.5773503

```

- e) La matrice est triangulaire : les valeurs propres sont donc $\lambda_1 = \lambda_2 = -2$ et $\lambda_3 = \lambda_4 = 3$. La forme échelonnée de la matrice $-2\mathbf{I} - \mathbf{A}$ est

$$\begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix},$$

d'où $\text{rang}(-2\mathbf{I} - \mathbf{A}) = 2$ et la base de vecteurs propres correspondant à $\lambda = -2$ est composée des vecteurs $(1, 0, 0, 0)$ et $(0, 1, 0, 0)$. D'autre part, la forme échelonnée de la matrice $3\mathbf{I} - \mathbf{A}$ est

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & -1 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

d'où $\text{rang}(3\mathbf{I} - \mathbf{A}) = 2$ et la base de vecteurs propres correspondant à $\lambda = 3$ est composée des vecteurs $(0, 1, 1, 0)$ et $(0, -1, 0, 1)$. Les valeurs propres de la matrice $\mathbf{A}$ ne sont pas toutes distinctes, mais les vecteurs propres $(1, 0, 0, 0)$, $(0, 1, 0, 0)$, $(0, 1, 1, 0)$ et $(0, -1, 0, 1)$ sont linéairement indépendants. Par conséquent, la matrice

$$\mathbf{P} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & -1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

diagonalise $\mathbf{A}$ et

$$\mathbf{P}^{-1}\mathbf{A}\mathbf{P} = \begin{bmatrix} -2 & 0 & 0 & 0 \\ 0 & -2 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 3 \end{bmatrix}.$$

Vérification :

```

> m <- matrix(c(-2, 0, 0, 0, 0, -2, 0, 0,
+               0, 5, 3, 0, 0, -5, 0, 3),
+             nrow = 4)
> eigen(m)
eigen() decomposition
$values
[1] 3 3 -2 -2

$vectors
      [,1]      [,2] [,3] [,4]
[1,] 0.0000000 0.0000000 1 0
[2,] 0.7071068 -0.7071068 0 1
[3,] 0.7071068 0.0000000 0 0
[4,] 0.0000000 0.7071068 0 0

```

Chapitre 9

9.1 La matrice

$$\mathbf{A} = \begin{bmatrix} 3 & -6 & -3 \\ 2 & 0 & 6 \\ -4 & 7 & 4 \end{bmatrix}$$

est exprimée sous la forme $\mathbf{A} = \mathbf{LU}$, où

$$\mathbf{L} = \begin{bmatrix} 3 & 0 & 0 \\ 2 & 4 & 0 \\ -4 & -1 & 2 \end{bmatrix}$$

et

$$\mathbf{U} = \begin{bmatrix} 1 & -2 & -1 \\ 0 & 1 & 2 \\ 0 & 0 & 1 \end{bmatrix}.$$

Ainsi, le système d'équations $\mathbf{Ax} = \mathbf{b}$ (où $\mathbf{b} = (-3, -22, 3)^T$) peut être exprimé sous la forme $\mathbf{LUx} = \mathbf{b}$, soit $\mathbf{Ly} = \mathbf{b}$ et $\mathbf{Ux} = \mathbf{y}$. On résoud tout

d'abord $\mathbf{L}\mathbf{y} = \mathbf{b}$ par simple substitution successive. On trouve

$$\begin{aligned} y_1 &= -1 \\ y_2 &= \frac{-22 - 2y_1}{4} = -5 \\ y_3 &= \frac{3 + 4y_1 + y_2}{2} = -3. \end{aligned}$$

Par la suite, on résoud de même $\mathbf{U}\mathbf{x} = \mathbf{y}$, ce qui donne

$$\begin{aligned} x_3 &= -3 \\ x_2 &= -5 - 2x_3 = 1 \\ x_1 &= -1 + 2x_2 + x_3 = -2. \end{aligned}$$

9.2 Cherchons tout d'abord des matrices triangulaires inférieure et supérieure $\mathbf{L}$ et $\mathbf{U}$, respectivement, tel que $\mathbf{A} = \mathbf{LU}$. On nous donne dans l'énoncé

$$\mathbf{A} = \mathbf{E}_1^{-1}\mathbf{E}_2^{-1} \begin{bmatrix} 1 & -2 \\ 0 & 1 \end{bmatrix},$$

d'où

$$\mathbf{U} = \begin{bmatrix} 1 & -2 \\ 0 & 1 \end{bmatrix},$$

et $\mathbf{L} = \mathbf{E}_1^{-1}\mathbf{E}_2^{-1}$. Or,

$$\begin{aligned} \mathbf{E}_1^{-1} &= \frac{1}{3} \begin{bmatrix} 3 & 0 \\ -2 & 1 \end{bmatrix} \\ \mathbf{E}_2^{-1} &= \begin{bmatrix} 3 & 0 \\ 0 & 3 \end{bmatrix} \\ &= 3\mathbf{I} \end{aligned}$$

et, par conséquent,

$$\mathbf{L} = \begin{bmatrix} 3 & 0 \\ -2 & 1 \end{bmatrix}.$$

Pour résoudre par décomposition LU le système d'équations $\mathbf{Ax} = \mathbf{LUx} = \mathbf{b}$, où $\mathbf{b} = (0, 1)$, on pose $\mathbf{Ux} = \mathbf{y}$ et résout d'abord $\mathbf{Ly} = \mathbf{b}$ par substitution. On a donc le système d'équations

$$\begin{bmatrix} 3 & 0 \\ -2 & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix},$$

dont la solution est $y_1 = 0$ et $y_2 = 1$. Par la suite, on a

$$\begin{bmatrix} 1 & -2 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix},$$

d'où, finalement, $x_1 = 2$ et $x_2 = 1$.

Bibliographie

- ANSI. 1986, *ANSI X3.4-1986: Coded Character Set. 7-Bit American Standard Code for Information Interchange*, American National Standards Institute, New York. Cité à la page 94.
- Anton, H. 2000, *Elementary Linear Algebra*, 8^e éd., Wiley, ISBN 0-4711705-5-0. Cité aux pages 157 et 224.
- Burden, R. L. et J. D. Faires. 1988, *Numerical Analysis*, 4^e éd., PWS-Kent, Boston, ISBN 0-5349158-5-X. Cité à la page 88.
- Burden, R. L. et J. D. Faires. 2011, *Numerical Analysis*, 9^e éd., Brooks/Cole, ISBN 978-0-5387335-1-9. Cité aux pages 145 et 150.
- Unicode Consortium, T. 2007, *The Unicode Standard, Version 5.0.0*, Addison-Wesley, Boston, ISBN 0-32148091-0. Cité à la page 95.
- Devroye, L. 1986, *Non-Uniform Random Variate Generation*, Springer-Verlag, ISBN 0-38796305-7. URL <http://luc.devroye.org/rnbookindex.html>. Cité à la page 17.
- Dua, D. et C. Graff. 2019, «UCI machine learning repository», URL <http://archive.ics.uci.edu/ml>. Cité à la page 194.
- Dutang, C., V. Goulet et M. Pigeon. 2008, «actuar: An R package for actuarial science», *Journal of Statistical Software*, vol. 25, n^o 7. URL <http://www.jstatsoft.org/v25/i07>. Cité à la page 32.
- Goulet, V. 2020, *Programmer avec R*, Document libre sous licence Creative Commons BY-SA 4.0. URL <https://vigou3.gitlab.io/programmer-avec-r>. Cité aux pages 15 et 230.
- IEEE. 2003, *754-1985 IEEE Standard for Binary Floating-Point Arithmetic*, IEEE, Piscataway, NJ. Cité à la page 83.

- ISO. 1998, *ISO/IEC 8859-1:1998. Information Technology – 8-bit Single-Byte Coded Graphic Character Sets – Part 1: Latin Alphabet No. 1*, International Organization for Standardization, Geneva. Cité à la page 94.
- Kernighan, B. W. et P. J. Plauger. 1978, *The Elements of Programming Style*, 2^e éd., McGraw-Hill, ISBN 0-07034207-5. Cité à la page 72.
- Knuth, D. E. 1997, *The Art of Computer Programming*, vol. 2, Seminumerical Algorithms, Addison-Wesley, ISBN 0-20189684-2. Cité aux pages 7 et 88.
- Kondo, Y., M. Salibian-Barrera et R. Zamar. 2016, «RSKC: An R package for a robust and sparse K-means clustering algorithm», *Journal of Statistical Software*, vol. 72, n° 5, doi :10.18637/jss.v072.i05, p. 1-26. Cité à la page 194.
- McCullough, B. D. et D. A. Heiser. 2008a, «Microsoft Excel's 'Not The Wichmann-Hill' random number generators», *Computational statistics and data analysis*, vol. 52, n° 10, p. 4587-4593. Cité à la page 11.
- McCullough, B. D. et D. A. Heiser. 2008b, «On the accuracy of statistical procedures in Microsoft Excel 2007», *Computational statistics and data analysis*, vol. 52, n° 10, p. 4570-4578. Cité à la page 11.
- Microsoft Office Blog. 2009, «Function improvements in Excel 2010», URL <http://blogs.office.com/b/microsoft-excel/archive/2009/09/10/function-improvements-in-excel-2010.aspx>. Cité à la page 11.
- Moler, C. et C. Van Loan. 1978, «Nineteen dubious ways to compute the exponential of a matrix», *SIAM Review*, vol. 20, p. 801-836. Cité à la page 205.
- Monahan, J. F. 2001, *Numerical Methods of Statistics*, Cambridge University Press, Cambridge, UK, ISBN 0-52179168-5. Cité aux pages 88 et 90.
- Ripley, B. D. 2005, «Internationalization features of R 2.1.0», *R News*, vol. 5, n° 1, p. 2-7. URL <https://cran.r-project.org/doc/Rnews/>. Cité à la page 95.
- Sylvester, J. J. 1865, «On a special class of questions on the theory of probabilities», *Birmingham British Assoc. Rept.*, p. 8-9. Cité à la page 3.

Weisstein, E. W. «Sylvester's four-point problem», From MathWorld – A Wolfram Web Resource. URL <http://mathworld.wolfram.com/SylvestersFour-PointProblem.html>. Cité à la page 3.

Wheeler, B. 2013, *SuppDists: Supplementary Distributions*. URL <https://cran.r-project.org/package=SuppDists>, R package version 1.1-9. Cité à la page 32.

Wikipedia. 2012, «IEEE 754-2008 — Wikipedia, the free encyclopedia», URL https://en.wikipedia.org/wiki/IEEE_floating-point_standard. Cité à la page 83.

Wikipedia. 2013, « Complément à deux — Wikipédia, l'encyclopédie libre », URL https://fr.wikipedia.org/wiki/Complément_à_deux. Cité à la page 98.

Ce document a été produit avec le système de mise en page $\text{\LaTeX}$. Le texte principal est composé en Lucida Bright OT 11 points, les mathématiques en Lucida Bright Math OT, le code informatique en Lucida Grande Mono DK et les titres en Fira Sans. Des icônes proviennent de la police Font Awesome. Les graphiques ont été réalisés avec R.

